Stat 221: Lecture Notes

Contents

Reference works **5**

I Local methods **6**

1 Optimization **7**

1.1 Local models and convexity . . . 7

1.1.1 Gradients, Hessians, and local models . . . 7

1.1.2 Convexity, strong convexity, and smoothness . . . 9

1.2 Gradient descent and line search . . . 12

1.2.1 Descent methods . . . 12

1.2.2 Step-size selection . . . 12

1.2.3 Gradient descent on quadratics . . . 16

1.3 Geometry and conditioning . . . 18

1.3.1 Conditioning . . . 18

1.3.2 Preconditioning . . . 19

1.4 Second-order methods and structured linear algebra . . . 20

1.4.1 Newton's method . . . 20

1.4.2 Quasi-Newton updates . . . 22

1.4.3 Structure-preserving positive-definite geometry . . . 22

1.4.4 Solves, log-determinants, and Gaussian quadratics . . . 23

1.4.5 Local Gaussian geometry and damping . . . 23

1.4.6 Conjugate gradient for SPD systems . . . 24

1.4.7 Orthogonality and conjugacy in conjugate gradient . . . 26

1.4.8 Convergence guarantees for conjugate gradient . . . 27

1.5 Stochastic optimization . . . 29

1.5.1 Stochastic gradients and mini-batches . . . 29

1.5.2 Step sizes and convergence . . . 30
1.5.3 Averaging and practical guidance . . . 32
1.6 Constrained optimization and reparameterization . . . 33
1.6.1 Projections and projected gradient descent . . . 34
1.6.2 Affine constraints . . . 34
1.6.3 Simplex and positive-semidefinite projections . . . 35
1.6.4 Reparameterization for strict constraints . . . 37
1.6.5 KKT conditions and equality-constrained Newton . . . 38
1.6.6 Barrier methods . . . 41

2 Sampling **44**
2.1 Monte Carlo methods . . . 44
2.1.1 Monte Carlo estimation . . . 44
2.1.2 Basic sampling constructions . . . 44
2.1.3 Importance sampling . . . 46
2.1.4 Working on the log scale . . . 47
2.1.5 Transformed densities and Jacobians . . . 49
2.1.6 Proposal quality and effective sample size . . . 50
2.1.7 Variance reduction . . . 51
2.2 Markov chains, invariant distributions, and ergodicity . . . 52
2.2.1 Transition kernels and iterated kernels . . . 53
2.2.2 Invariant distributions and Monte Carlo averages . . . 53
2.2.3 Reversibility and detailed balance . . . 54
2.2.4 Autocorrelation and effective sample size . . . 54
2.2.5 Ergodicity, mixing time, and burn-in . . . 55
2.3 Markov chain Monte Carlo . . . 57
2.3.1 Metropolis–Hastings . . . 57
2.3.2 Gibbs sampling . . . 59
2.3.3 Multi-chain diagnostic workflow . . . 63
2.3.4 Split-$\hat{R}$ and effective sample size . . . 65
2.3.5 Langevin diffusion and the unadjusted Langevin algorithm . . . 66
2.3.6 MALA, stochastic gradients, and the road to HMC . . . 70

II Augmented-state methods **74**

3 Augmented-state optimization **75**
3.1 Momentum-based methods . . . 75
3.1.1 Ill-conditioning and zig-zagging . . . 75
3.1.2 Heavy-ball momentum . . . 76
3.1.3 Acceleration by look-ahead gradients . . . 76
3.1.4 Averaging with dual states . . . 77
3.1.5 Adaptive moment methods . . . 77
3.1.6 Scope-limited tuning guidance . . . 79
3.2 Simulated annealing . . . 80
3.2.1 Boltzmann targets and temperature flattening . . . 80
3.2.2 Metropolis updates and simulated annealing . . . 81
3.2.3 Cooling schedules and global guarantees . . . 82
3.2.4 Practical schedules and proposal design . . . 82
3.3 Expectation–maximization (EM) . . . 83
3.3.1 Free energy, the Q-function, and coordinate ascent . . . 84
3.3.2 Worked example: a two-coin mixture . . . 86
3.3.3 Stable responsibilities and regularized updates . . . 87
3.3.4 Initialization, local maxima, and the bridge to VI . . . 88

4 Augmented-state sampling **89**
4.1 Hamiltonian Monte Carlo . . . 89
4.1.1 Momentum augmentation and Hamiltonian dynamics . . . 91
4.1.2 Exact Hamiltonian transitions . . . 93
4.1.3 Leapfrog integration and Metropolis correction . . . 93
4.1.4 Mass-matrix adaptation and trajectory design . . . 95
4.1.5 Divergences and reparameterization . . . 97
4.2 The No-U-Turn Sampler . . . 97
4.2.1 The U-turn stopping criterion . . . 98
4.2.2 Symmetric tree expansion and candidate selection . . . 98
4.2.3 Why the transition is still valid . . . 100
4.2.4 Warm-up and practical tuning . . . 100
4.3 Tempering methods . . . 101
4.3.1 From annealing to temperature ladders . . . 102
4.3.2 Simulated tempering . . . 102
4.3.3 Parallel tempering . . . 104

4.3.4 Choosing the ladder in practice . . . 105
4.4 Sequential Monte Carlo and particle filters . . . 107
4.4.1 State-space models and filtering recursions . . . 108
4.4.2 Kalman filtering as a tractable baseline . . . 108
4.4.3 Particle approximations and weight degeneracy . . . 110
4.4.4 Resampling and the bootstrap filter . . . 112
4.4.5 Likelihood increments and parameter learning . . . 114

III Learned maps **116**

5 Variational inference **117**
5.1 Variational inference and the ELBO . . . 117
5.1.1 From EM free energy to the ELBO . . . 117
5.1.2 ELBO–KL identity . . . 117
5.1.3 Reconstruction–regularization decomposition . . . 118
5.1.4 What forward KL tends to prefer . . . 119
5.2 Mean-field variational inference . . . 119
5.2.1 Factorized families and conditional updates . . . 119
5.2.2 Coordinate ascent updates . . . 120
5.2.3 Worked examples: mixture assignments and factor analysis . . . 121
5.2.4 Strengths and limitations of mean-field . . . 123
5.3 Stochastic and amortized variational inference . . . 123
5.3.1 From mean-field factors to shared encoders . . . 123
5.3.2 Reparameterization . . . 125
5.3.3 Minibatch ELBO optimization . . . 126
5.3.4 Score-function gradients . . . 127

6 Diffusion models **128**
6.1 Bridges and infinitesimal dynamics . . . 128
6.1.1 Conditional and marginal probability paths . . . 128
6.1.2 Exact distributional posterior learning . . . 129
6.1.3 Stochastic interpolants and Gaussian bridges . . . 131
6.1.4 Velocity, Endpoint Means, and Score Recovery . . . 132
6.1.5 Density evolution and common-marginal dynamics . . . 134
6.2 Regression and reverse-time simulation . . . 136

6.2.1 Reverse-time sampling as regression . . . 136
6.2.2 Conditional and marginal flow matching . . . 137
6.2.3 Bridge regression training . . . 138
6.2.4 Reverse-time simulation . . . 138
6.2.5 Discrete-time ELBO connection . . . 138
6.3 Using the sampling toolkit with diffusion models . . . 141
6.3.1 Efficient sampling for the same target law . . . 141
6.3.2 Guidance as posterior tilting . . . 143
6.3.3 A sample set as a distributional approximation . . . 144

Reference works

These notes are intended to be self-contained, but the following references are used repeatedly across the main units. For convex optimization, duality, KKT conditions, line search, and Newton's method, the main background reference is Boyd and Vandenberghe (2004, Chs. 2–3, 5, and 9–11). For conjugate-gradient methods and their optimization interpretation, we also use Nocedal and Wright (2006, Ch. 5).

For Monte Carlo sampling, importance sampling, Metropolis–Hastings, Gibbs sampling, and sequential Monte Carlo, we primarily use Liu (2002, Chs. 2, 5–6, and §§3.4.3–3.4.4). For Hamiltonian Monte Carlo and the geometry of typical sets, the two references most closely aligned with the notes are Betancourt (2017, §3 and App. A.1) and Neal (2011, §§3.2–4.2). For theorem-level results on HMC irreducibility, Harris recurrence, and geometric ergodicity, see Durmus et al. (2019, Thms. 1–2 and 9). For the No-U-Turn sampler and dual-averaged adaptation, see Hoffman and Gelman (2014, §3 and §3.2). For modern multi-chain diagnostics, especially rank-normalized split-$\hat{R}$ and bulk/tail effective sample size, see Vehtari et al. (2021, §§3.2–4.5). For stochastic gradient Langevin dynamics, see Welling and Teh (2011, §§2–3); for replica-exchange tempering and ladder-design heuristics, see Earl and Deem (2005, §§2.1–2.3). For quantitative convergence of ULA to its biased limit under log-Sobolev assumptions, and for explicit bias bounds for that limit, see Vempala and Wibisono (2019, Thm. 1 and §7.1).

For constant-step stochastic-gradient stationary-law heuristics, see Mandt et al. (2015, pp. 1–2); for clean convergence guarantees for SGD under convex and Polyak–Łojasiewicz assumptions, we use Garrigos and Gower (2024, Thms. 5.3 and 5.10). For expectation–maximization as a lower-bound method and for population/local analyses of the EM map, we use Balakrishnan et al. (2014, §2.1 and §3). For mean-field variational families, coordinate-ascent VI, and conjugate exponential-family updates, we use Blei et al. (2017, §§2.3–2.4 and 4.1–4.2). For variational autoencoders and the reparameterization trick, the main references are Kingma and Welling (2014, §§2.2–2.4) and Rezende et al. (2014, §§3–4). For practical failure modes of amortized VI, especially posterior collapse and bottleneck-induced representation effects, see He et al. (2019, §§3–4) and Burgess et al. (2018, §§4.2–5).

For the diffusion unit, Albergo et al. (2025, Def. 1 and §2) supplies the stochastic-interpolant and common-marginal viewpoint, Lipman et al. (2023, Thm. 2 and §§3.1–4) supplies the core conditional-flow-matching regression result, and Holderrieth and Erives (2026, Thms. 11–12, 17 and §4.1) supplies the bridge-dynamics, transport, and score/velocity identities. For guidance, inverse-problem conditioning, and modern sampler design inside that same framework, we repeatedly use Lai et al. (2025, §§8.1–8.4 and §9.2.4); for the discrete-time ELBO viewpoint and its relation to continuous-time constructions, see also Kingma et al. (2021, §3.5, eqs. (11)–(12)).

Part I

Local methods

1 Optimization

Optimization starts from a simple task: given an objective $f : \mathbb{R}^d \to \mathbb{R}$, choose x so that $f(x)$ is small. The first questions are local. If we stand at a point x, which direction decreases f, how far should we move, and what extra information is carried by curvature? This unit develops the gradients, Hessians, curvature bounds, and step-size rules that answer those questions, then extends the same ideas to noisy gradients, structured linear algebra, and feasibility constraints. Later the same local information will reappear in sampling, augmentation, and diffusion.

1.1 Local models and convexity

Much of modern statistics and machine learning can be phrased as choosing a parameter vector $x \in \mathbb{R}^d$ to minimize an objective. To decide how to move from a current point, we first need a local model of how the objective changes nearby. Gradients record first-order change, Hessians record curvature, and convexity and smoothness inequalities tell us when these local objects control behavior away from the current point. We begin with these Euclidean definitions and then turn them into descent guarantees that will drive the algorithms in the rest of the unit.

1.1.1 Gradients, Hessians, and local models

Much of modern statistics and machine learning can be phrased as choosing a parameter vector $x \in \mathbb{R}^d$ to minimize an objective:

$$\min_{x \in \mathbb{R}^d} f(x), \tag{1}$$

This unconstrained problem is the core primitive behind many later constructions: penalties, constraints, stochastic approximations, and even some sampling algorithms are all variations on it. Many statistical procedures reduce to minimizing a loss plus regularization; see Boyd and Vandenberghe (2004, §9.1).

Local analysis begins with a concrete question: if we perturb the current point x in direction v, what is the first-order change in f? The differential $Df(x)[v]$ answers exactly that question, and it is linear in the direction v. That linearity is what makes the gradient useful: one vector can summarize all directional slopes at once. In Euclidean coordinates every linear functional on $\mathbb{R}^d$ can be written uniquely in the form

$$v \mapsto \langle a, v \rangle \qquad \text{for some } a \in \mathbb{R}^d \text{ and all } v \in \mathbb{R}^d,$$

so we represent first-order information by a vector. For a differentiable function f, the gradient is defined by the identity

$$Df(x)[v] = \langle \nabla f(x), v \rangle \qquad \text{for all } v \in \mathbb{R}^d.$$

Thus the gradient is not an extra construction; it is simply the Euclidean way of writing the linear approximation of the objective. For example, if $f(x) = a^\top x$ then $\nabla f(x) = a$, while for $f(x) = \frac{1}{2}\|x\|_2^2$ we have $\nabla f(x) = x$. For now we stay in the unconstrained setting and develop the derivative calculus itself. The multiplier language belongs to constrained optimization, where the new issue is feasibility rather than local change in f. In convex optimization this first-order information becomes especially powerful because the local model controls the function globally.

We will use regularized logistic regression as a running example throughout the optimization chapter. It is simple enough to differentiate explicitly, but rich enough to show how curvature, conditioning, and regularization interact.

Example 1.1 (Logistic regression (with ridge regularization))**.** Let data $(x_i, y_i)_{i=1}^n$ satisfy $x_i \in \mathbb{R}^d$ and $y_i \in \{0, 1\}$. In logistic regression we model

$$y_i \mid x_i \sim \text{Bernoulli}(p_i(\theta)), \qquad p_i(\theta) = \sigma(x_i^\top \theta) = \frac{1}{1 + e^{-x_i^\top \theta}}. \tag{2}$$

The log-likelihood is

$$\begin{aligned} \ell(\theta) &= \log p(y_1, \dots, y_n \mid x_1, \dots, x_n, \theta) \\ &= \sum_{i=1}^n \Big[y_i x_i^\top \theta - \log\big(1 + e^{x_i^\top \theta}\big) \Big]. \end{aligned} \tag{3}$$

Equivalently, the (convex) negative log-likelihood is

$$L(\theta) = -\ell(\theta) = \sum_{i=1}^n \Big[\log\big(1 + e^{x_i^\top \theta}\big) - y_i x_i^\top \theta \Big]. \tag{4}$$

A common regularized objective (ridge) is

$$s(\theta) = L(\theta) + \frac{m}{2}\|\theta\|_2^2, \qquad m > 0. \tag{5}$$

The objective (5) is a clean example of a smooth convex optimization problem with a finite-sum structure (a sum over data points). It also lets us see, explicitly, how curvature and conditioning depend on the data matrix X and on regularization.

Lemma 1.2 (Gradient and Hessian for logistic regression)**.** *Let $X \in \mathbb{R}^{n \times d}$ have rows $x_i^\top$, and let $p(\theta) \in \mathbb{R}^n$ have entries $p_i(\theta) = \sigma(x_i^\top \theta)$. Then*

$$\nabla L(\theta) = X^\top \big(p(\theta) - y\big), \qquad \nabla^2 L(\theta) = X^\top W(\theta) X, \tag{6}$$

where $W(\theta) = \operatorname{diag}(p_i(\theta)(1 - p_i(\theta)))$. Moreover, since $0 < p(1-p) \le 1/4$ for all $p \in (0, 1)$,

$$0 \preceq W(\theta) \preceq \tfrac{1}{4} I \quad \Rightarrow \quad 0 \preceq X^\top W(\theta) X \preceq \tfrac{1}{4} X^\top X. \tag{7}$$

For the ridge objective $s(\theta) = L(\theta) + \frac{m}{2}\|\theta\|_2^2$,

$$\nabla^2 s(\theta) = X^\top W(\theta) X + mI. \tag{8}$$

Proof. Write $z_i(\theta) = x_i^\top \theta$. The derivative of $\log(1 + e^z)$ is $\frac{e^z}{1+e^z} = \sigma(z)$, so differentiating (4) gives

$$\nabla L(\theta) = \sum_{i=1}^n \big(\sigma(z_i(\theta)) - y_i\big) x_i = X^\top (p(\theta) - y).$$

Since $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, another derivative yields

$$\nabla^2 L(\theta) = \sum_{i=1}^n p_i(\theta)(1 - p_i(\theta))\, x_i x_i^\top = X^\top W(\theta) X.$$

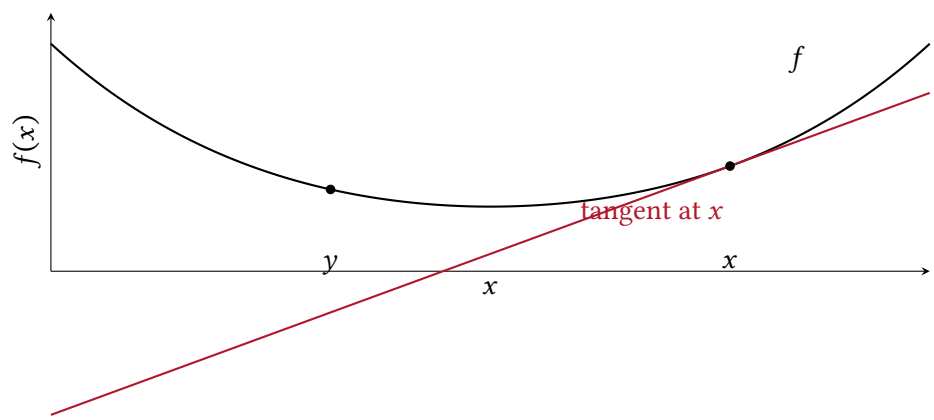

Figure 1: Convexity first-order condition: a convex function lies above every tangent line/hyperplane. In one dimension, (9) reads $f(y) \geq f(x) + f'(x)(y - x)$.

The bound $p(1-p) \leq 1/4$ is maximized at $p = 1/2$, which gives (7). Finally, $\nabla^2 \left(\frac{m}{2}\|\theta\|_2^2\right) = mI$, so (8) follows by additivity. ■

The Hessian for logistic regression is a weighted Gram matrix $X^\top W X$. When a datum is "confident" (i.e., $p_i(\theta)$ close to 0 or 1), its weight $p_i(1-p_i)$ is small, so it contributes little curvature; the maximum curvature occurs near the decision boundary ($p_i \approx 1/2$). Adding ridge regularization shifts all eigenvalues up by m, which improves conditioning and helps algorithms like gradient descent and Newton's method behave more predictably.

1.1.2 Convexity, strong convexity, and smoothness

Convex optimization is tractable because local derivative information controls global behavior. The definitions and inequalities below formalize the properties we will use repeatedly; see Boyd and Vandenberghe (2004, Ch. 9).

Definition 1.3 (Convex function (first-order condition))**.** Let $S \subseteq \mathbb{R}^d$ be convex, and let $f : S \to \mathbb{R}$ be differentiable. We say f is *convex* if for all $x, y \in S$,

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x). \tag{9}$$

Geometrically, (9) says the graph of f lies above every tangent hyperplane. Convexity does not require differentiability (one can state analogous conditions using subgradients), but in these notes we mostly assume differentiability to keep the calculus clean.

Definition 1.4 (m-strong convexity (via the Hessian))**.** Let $f : S \to \mathbb{R}$ be twice differentiable. We say f is *m-strongly convex* on S if there exists $m > 0$ such that

$$\nabla^2 f(x) \succeq mI \qquad \text{for all } x \in S. \tag{10}$$

Here (and throughout), for symmetric matrices A, B we write $A \succeq B$ (equivalently $B \preceq A$) to mean $A - B$ is positive semidefinite, i.e. $x^\top (A - B)x \geq 0$ for all $x \in \mathbb{R}^d$.

Definition 1.5 (M-smoothness (Lipschitz gradient and Hessian bounds))**.** Let $f : S \to \mathbb{R}$ be differentiable. We say f has *M-Lipschitz gradient* (is *M-smooth*) if

$$\|\nabla f(x) - \nabla f(y)\|_2 \le M\|x - y\|_2 \qquad \text{for all } x, y \in S. \tag{11}$$

If f is twice differentiable, then (11) is equivalent to the operator-norm bound

$$\|\nabla^2 f(x)\|_{\mathrm{op}} \le M \qquad \text{for all } x \in S. \tag{12}$$

For convex twice differentiable functions this is the same as $0 \preceq \nabla^2 f(x) \preceq MI$. More generally, the one-sided upper bound $\nabla^2 f(x) \preceq MI$ is weaker than (11), but it is still enough for the descent lemma below.

Theorem 1.6 (Quadratic bounds from Hessian bounds)**.** *Let $f : S \to \mathbb{R}$ be twice differentiable on a convex set S.*

- *If $\nabla^2 f(z) \preceq MI$ for all $z \in S$, then for all $x, y \in S$,*

$$f(y) \le f(x) + \nabla f(x)^\top (y - x) + \frac{M}{2}\|y - x\|_2^2.$$

- *If $\nabla^2 f(z) \succeq mI$ for all $z \in S$ (i.e., f is m-strongly convex on S), then for all $x, y \in S$,*

$$f(y) \ge f(x) + \nabla f(x)^\top (y - x) + \frac{m}{2}\|y - x\|_2^2.$$

In particular, if both bounds hold, then for all $x, y \in S$,

$$f(x) + \nabla f(x)^\top (y - x) + \frac{m}{2}\|y - x\|_2^2 \le f(y) \le f(x) + \nabla f(x)^\top (y - x) + \frac{M}{2}\|y - x\|_2^2. \tag{13}$$

Proof. Fix $x, y \in S$ and define $\phi(t) = f(x + t(y - x))$ for $t \in [0, 1]$. By the chain rule,

$$\phi'(t) = \nabla f(x + t(y - x))^\top (y - x), \qquad \phi''(t) = (y - x)^\top \nabla^2 f(x + t(y - x))(y - x).$$

If $\nabla^2 f \preceq MI$ on S, then $\phi''(t) \le M\|y - x\|_2^2$; if $\nabla^2 f \succeq mI$ on S, then $\phi''(t) \ge m\|y - x\|_2^2$. Applying Taylor's theorem to $\phi(1)$ around $t = 0$ with Lagrange remainder gives

$$\phi(1) = \phi(0) + \phi'(0) + \tfrac{1}{2}\phi''(\xi)$$

for some $\xi \in (0, 1)$. Plugging in the appropriate bound on $\phi''(\xi)$ yields the corresponding inequality; when both Hessian bounds hold we obtain (13). ∎

The upper bound in Theorem 1.6 is often called the *descent lemma* (it is a quantitative version of "f is well-approximated by its tangent for small steps"). The lower bound is the analytic content of strong convexity. Together they are the basic calculus inequalities that let us prove convergence rates and justify step-size rules.

Geometrically, the inequalities in (13) say that, on S, the function f is sandwiched between two quadratic bowls around any point x. The lower bound (parameter m) prevents f from being "too flat" and is what

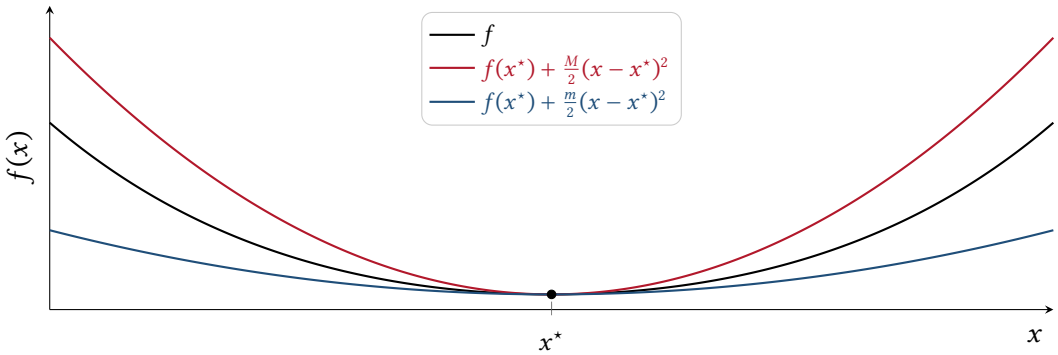

Figure 2: If $mI \preceq \nabla^2 f \preceq MI$ on a region containing $x^\star$, then f is "sandwiched" between two quadratics around the minimizer: $f(x^\star) + \frac{m}{2}\|x - x^\star\|^2 \le f(x) \le f(x^\star) + \frac{M}{2}\|x - x^\star\|^2$.

enables linear rates for first-order methods. The upper bound (parameter M) prevents f from having "too much curvature" and is what makes fixed (or backtracked) step sizes stable.

Corollary 1.7 (Uniqueness of the minimizer)**.** *If f is m-strongly convex on S, then f has at most one minimizer on S.*

Proof. Suppose $x^\star, y^\star \in S$ are two distinct minimizers, and define

$$\phi(t) = f\big((1-t)x^\star + ty^\star\big), \qquad t \in [0,1].$$

Then

$$\phi''(t) = (y^\star - x^\star)^\top \nabla^2 f\big((1-t)x^\star + ty^\star\big)(y^\star - x^\star) \ge m\|y^\star - x^\star\|_2^2 > 0,$$

so ϕ is strictly convex on $[0,1]$. Therefore

$$f\left(\frac{x^\star + y^\star}{2}\right) = \phi\left(\frac{1}{2}\right) < \frac{\phi(0) + \phi(1)}{2} = f(x^\star),$$

which contradicts the minimality of $x^\star$ and $y^\star$. ■

Corollary 1.8 (Strong convexity links gradients, suboptimality, and distance)**.** *Assume $f : \mathbb{R}^d \to \mathbb{R}$ is differentiable and m-strongly convex, and let $x^\star = \arg\min_{x \in \mathbb{R}^d} f(x)$. Then:*

(a) $\nabla f(x^\star) = 0$,

(b) for all $x \in \mathbb{R}^d$, $f(x) - f(x^\star) \le \frac{1}{2m}\|\nabla f(x)\|_2^2$,

(c) for all $x \in \mathbb{R}^d$, $\|x - x^\star\|_2 \le \frac{1}{m}\|\nabla f(x)\|_2$.

Proof. (a) is the first-order optimality condition for unconstrained differentiable convex minimization.

(b) Apply the left inequality in Theorem 1.6 and minimize the resulting quadratic lower bound over $y \in \mathbb{R}^d$: for any fixed x,

$$f(y) \ge f(x) + \nabla f(x)^\top (y - x) + \frac{m}{2}\|y - x\|_2^2.$$

The right-hand side, viewed as a function of y, is minimized at $\hat{y} = x - \frac{1}{m}\nabla f(x)$. Plugging in $\hat{y}$ gives

$$f(y) \geq f(x) - \frac{1}{2m}\|\nabla f(x)\|_2^2 \qquad \forall y,$$

and choosing $y = x^\star$ yields (b).

(c) Strong convexity implies (see Theorem 1.6 with $x = x^\star$ and $\nabla f(x^\star) = 0$)

$$f(x) \geq f(x^\star) + \frac{m}{2}\|x - x^\star\|_2^2.$$

Combine this with (b) to obtain $\frac{m}{2}\|x - x^\star\|_2^2 \leq \frac{1}{2m}\|\nabla f(x)\|_2^2$, i.e. $\|x - x^\star\|_2 \leq \frac{1}{m}\|\nabla f(x)\|_2$. ■

For unconstrained smooth strongly convex objectives, these simple inequalities are extremely useful: they let us translate between three common notions of "being close to optimum" (small function gap, small gradient norm, and small distance to $x^\star$). We will use them to turn a decrease in f into a geometric (linear) convergence rate.

1.2 Gradient descent and line search

With the local model from Section 1.1 in place, we can turn derivative information into an algorithm. The two design questions are the search direction and the step size. Gradient descent answers the first question by moving opposite the gradient, while fixed-step and line-search rules answer the second.

1.2.1 Descent methods

With this Euclidean setup in place, we can describe the generic update template behind many first-order algorithms. Many iterative methods for (1) have the form

$$x^{(k+1)} = x^{(k)} + t^{(k)}\Delta x^{(k)}, \qquad k = 0, 1, 2, \dots, \tag{14}$$

where $t^{(k)} > 0$ is a step size (learning rate) and $\Delta x^{(k)}$ is a descent direction chosen so that taking a small step decreases f.

Definition 1.9 (Descent direction)**.** Let f be differentiable at x. A direction d is a descent direction for f at x if $\nabla f(x)^\top d < 0$.

Lemma 1.10 (Why $\nabla f(x)^\top d < 0$ matters)**.** *If $\nabla f(x)^\top d < 0$, then for all sufficiently small $t > 0$,*

$$f(x + td) < f(x).$$

Proof. Consider $\phi(t) = f(x + td)$. Then $\phi'(0) = \nabla f(x)^\top d < 0$, so $\phi(t) < \phi(0)$ for all small enough $t > 0$. ■

1.2.2 Step-size selection

The remaining design choice in Algorithm 1 is the step size. There are two standard regimes. If a smoothness bound is available, a fixed step size gives a clean baseline and leads to explicit rates. If that information is unavailable or too conservative, a line search chooses a step size adaptively from local function values.

Algorithm 1 Generic descent method

Input: Objective f, initial point $x^{(0)}$, rule for choosing a descent direction, rule for choosing a step size, stopping criterion

Output: Final iterate x

```
k ← 0
while stopping criterion not met do
    Choose Δx^(k) such that ∇f(x^(k))^⊤ Δx^(k) < 0.
    Choose a step size t^(k) > 0.
    x^(k+1) ← x^(k) + t^(k) Δx^(k).
    k ← k + 1
end while
return x^(k)
```

Exact line search chooses the step size by solving

$$t^{(k)} = \arg\min_{s>0} f\left(x^{(k)} + s\Delta x^{(k)}\right). \tag{15}$$

This yields the best decrease along the ray, but it is not always tractable. More importantly, in many problems we want a step-size rule that works robustly without solving a difficult one-dimensional optimization at every iteration.

Backtracking line search (Armijo) chooses a step size by repeatedly shrinking until a *sufficient decrease* condition holds. Fix parameters $\alpha \in (0, 1/2)$ and $\beta \in (0, 1)$ and require the Armijo condition

$$f(x + t\Delta x) \leq f(x) + \alpha t \nabla f(x)^\top \Delta x, \tag{16}$$

to hold. See Boyd and Vandenberghe (2004, §9.2).

Algorithm 2 Backtracking line search (Armijo)

Input: Differentiable f, point x, descent direction Δx, parameters $\alpha \in (0, 1/2)$, $\beta \in (0, 1)$

Output: Step size t

```
t ← 1
while f(x + tΔx) > f(x) + αt∇f(x)^⊤ Δx do
    t ← βt
end while
return t
```

Typical parameter choices are:

- α controls how much decrease we demand; typical values are $\alpha \in (0.01, 0.3)$.
- β controls how aggressively we shrink; typical values are $\beta \in (0.1, 0.8)$ (smaller β means bigger reductions per backtracking step).

Backtracking is most natural in deterministic optimization where we can reliably evaluate $f(x + t\Delta x)$. In stochastic settings (e.g. mini-batch training), function values are noisy, so line search is less common; one typically uses fixed or scheduled learning rates, possibly with averaging or momentum. For the running logistic regression example in Example 1.1, this means that once we choose a direction $\Delta\theta$, we can evaluate the full objective $s(\theta + t\Delta\theta)$ exactly and shrink t until the sufficient-decrease test passes.

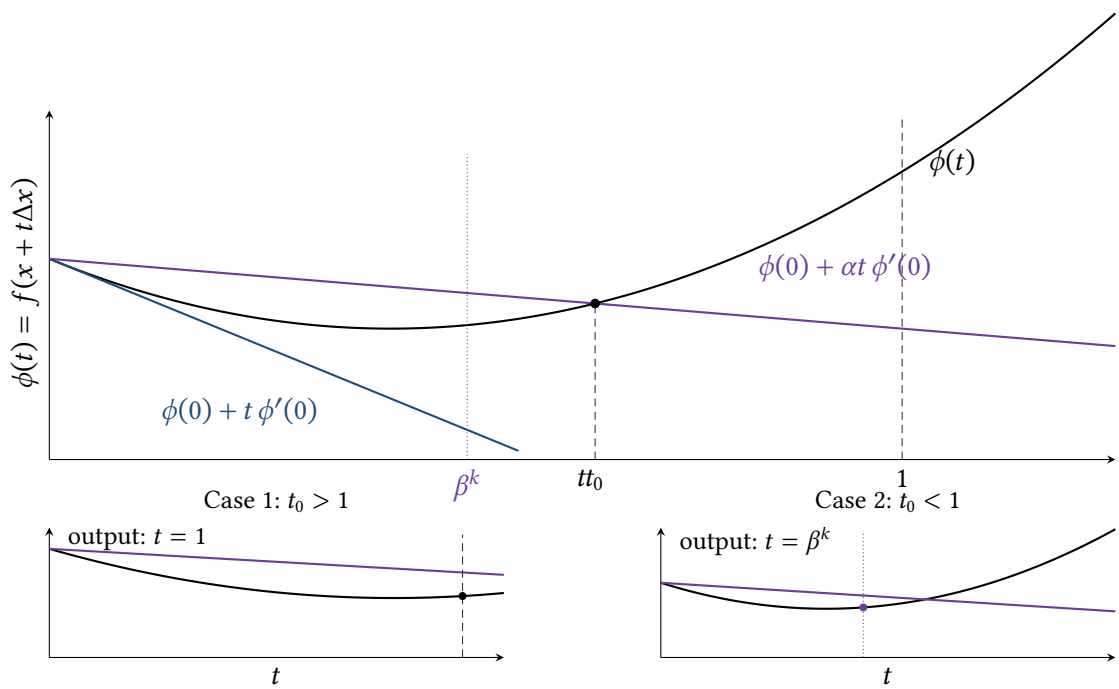

Figure 3: Geometry of the Armijo (sufficient decrease) condition for $\phi(t) = f(x + t\Delta x)$. The Armijo line $\phi(0) + \alpha t\, \phi'(0)$ intersects ϕ at t_0. Backtracking starts at $t = 1$ and returns $t = \beta^k$ (possibly $t = 1$) such that $\phi(t) \le \phi(0) + \alpha t\, \phi'(0)$.

Backtracking line search is a cheap way to find a step size that is "small enough" without knowing global smoothness constants. The Armijo condition compares the actual decrease $f(x + t\Delta x) - f(x)$ to the first-order prediction $t\nabla f(x)^\top \Delta x$. If f is reasonably well-approximated by its tangent for small steps, repeated shrinking will eventually make the inequality hold (see Boyd and Vandenberghe (2004, §9.2)).

When that scale is unknown, backtracking gives a local replacement. We focus on gradient descent with a backtracking line search (see Boyd and Vandenberghe (2004, §9.3)). Algorithm 3 is the specialization of Algorithm 1 with direction $\Delta x^{(k)} = -\nabla f(x^{(k)})$ and step size chosen by Algorithm 2. The analysis is modular: smoothness implies that *some* step sizes satisfy Armijo (Lemma 1.11), which implies backtracking never shrinks "too much" (Proposition 1.12); strong convexity then links the gradient norm to suboptimality (Corollary 1.8), yielding a linear rate (Theorem 1.13). In this subsection we work with unconstrained objectives $f : \mathbb{R}^d \to \mathbb{R}$, so every trial point $x - t\nabla f(x)$ is feasible.

The update has the form

$$x^{(k+1)} = x^{(k)} - t^{(k)}\nabla f(x^{(k)}), \tag{17}$$

where $t^{(k)}$ is the step size returned by backtracking.

Lemma 1.11 (A simple sufficient step size for the Armijo condition)**.** *Assume $f : \mathbb{R}^d \to \mathbb{R}$ is M-smooth and take $\alpha \in (0, 1/2)$. Then for any x and any step size $t \in (0, 1/M]$,*

$$f(x - t\nabla f(x)) \le f(x) - \alpha t\|\nabla f(x)\|_2^2. \tag{18}$$

Algorithm 3 Gradient descent with backtracking line search

Input: Differentiable $f : \mathbb{R}^d \to \mathbb{R}$, initial point $x^{(0)}$, parameters $\alpha \in (0, 1/2)$, $\beta \in (0, 1)$, stopping criterion
Output: Final iterate x

```
k ← 0
while stopping criterion not met do
    g^(k) ← ∇f(x^(k))
    Δx^(k) ← −g^(k)
    t^(k) ← BacktrackingLineSearch(f, x^(k), Δx^(k), α, β)        ▷ Algorithm 2
    x^(k+1) ← x^(k) + t^(k) Δx^(k)
    k ← k + 1
end while
return x^(k)
```

Proof. By the upper bound in Theorem 1.6 (the "descent lemma"), with $y = x - t\nabla f(x)$,

$$f(x - tg) \le f(x) + \nabla f(x)^\top(-tg) + \frac{M}{2}\|-tg\|_2^2 = f(x) - t\|g\|_2^2 + \frac{Mt^2}{2}\|g\|_2^2,$$

where $g = \nabla f(x)$. If $t \le 1/M$ then $-t + (Mt^2/2) \le -t/2$, so

$$f(x - tg) \le f(x) - \frac{t}{2}\|g\|_2^2 \le f(x) - \alpha t\|g\|_2^2,$$

since $\alpha < 1/2$. ■

Proposition 1.12 (Backtracking returns a step size bounded away from zero)**.** *Assume $f : \mathbb{R}^d \to \mathbb{R}$ is M-smooth and backtracking is run with $\alpha \in (0, 1/2)$ and $\beta \in (0, 1)$. Then the returned step size t satisfies*

$$t \ge \min\left\{1, \frac{\beta}{M}\right\}. \tag{19}$$

Proof. If backtracking accepts $t = 1$, then (19) holds. Otherwise, at least one shrink occurs, so the final accepted t has the form $t = \beta^j$ with $j \ge 1$ and the previous trial value t/β was rejected. By Lemma 1.11, any $s \le 1/M$ satisfies the Armijo condition, so a rejected step must satisfy $t/\beta > 1/M$. Thus $t > \beta/M$. Since this case only occurs when $t < 1$, we also have $\beta/M < 1$, so $t \ge \min\{1, \beta/M\}$. ■

Theorem 1.13 (Linear convergence of GD with backtracking for strongly convex and smooth f)**.** *Assume $f : \mathbb{R}^d \to \mathbb{R}$ is twice differentiable and satisfies*

$$mI \preceq \nabla^2 f(x) \preceq MI \qquad \forall x \in \mathbb{R}^d,$$

with $0 < m \le M$. Let $(x^{(k)})$ be the gradient descent iterates (17) with backtracking parameters $\alpha \in (0, 1/2)$ and $\beta \in (0, 1)$, and let $x^\star = \arg\min_{x \in \mathbb{R}^d} f(x)$. Then

$$f(x^{(k)}) - f(x^\star) \le c^k\big(f(x^{(0)}) - f(x^\star)\big), \qquad c = 1 - \min\left\{2\alpha m, \frac{2\alpha\beta m}{M}\right\}. \tag{20}$$

Proof. By the Armijo condition with $\Delta x^{(k)} = -\nabla f(x^{(k)})$,

$$f(x^{(k+1)}) \le f(x^{(k)}) - \alpha t^{(k)} \|\nabla f(x^{(k)})\|_2^2.$$

By Proposition 1.12, $t^{(k)} \ge \min\{1, \beta/M\}$. By Corollary 1.8(b), $\|\nabla f(x^{(k)})\|_2^2 \ge 2m\big(f(x^{(k)}) - f(x^\star)\big)$. Combining,

$$f(x^{(k+1)}) - f(x^\star) \le \left(1 - 2\alpha m \min\left\{1, \frac{\beta}{M}\right\}\right)\big(f(x^{(k)}) - f(x^\star)\big).$$

Since $2\alpha m \min\{1, \beta/M\} = \min\{2\alpha m, 2\alpha\beta m/M\}$, this yields (20) by iteration. ■

It is useful to express the rate in terms of the condition number $\kappa = M/m$. When κ is large (poor conditioning), the contraction factor in (20) is approximately

$$c \approx 1 - \frac{2\alpha\beta}{\kappa},$$

so convergence can be slow. Geometrically, the level sets are long narrow valleys and vanilla gradient descent tends to "zig-zag" (see Boyd and Vandenberghe (2004, §9.3 and Ex. 9.3.2)). This motivates changing the geometry of the update via preconditioning.

1.2.3 Gradient descent on quadratics

With step-size rules in hand, quadratics provide the cleanest exact benchmark for how first-order methods behave under conditioning. When a smoothness scale is available in advance, constant-step gradient descent gives a simple baseline, and for strongly convex quadratics the rate can be computed exactly from the spectrum. If we know a smoothness constant M in advance, we can choose a constant step size without a line search. Consider the update

$$x^{(k+1)} = x^{(k)} - t\,\nabla f(x^{(k)}), \qquad t > 0. \tag{21}$$

Theorem 1.14 (Sublinear convergence for smooth convex objectives)**.** *Assume f is convex and M-smooth, and let $x^\star \in \arg\min f$. If we run (21) with step size $t = 1/M$, then for every $k \ge 1$,*

$$f(x^{(k)}) - f(x^\star) \le \frac{M}{2k}\|x^{(0)} - x^\star\|_2^2. \tag{22}$$

Proof. Write $x_k = x^{(k)}$ and $g_k = \nabla f(x_k)$. By the smoothness upper bound in Theorem 1.6 with $y = x_k - t g_k$,

$$f(x_{k+1}) \le f(x_k) - t\|g_k\|_2^2 + \frac{Mt^2}{2}\|g_k\|_2^2 = f(x_k) - \left(t - \frac{Mt^2}{2}\right)\|g_k\|_2^2.$$

With $t = 1/M$ this becomes

$$f(x_k) - f(x_{k+1}) \ge \frac{1}{2M}\|g_k\|_2^2. \tag{23}$$

Next expand the squared distance to $x^\star$:

$$\|x_{k+1} - x^\star\|_2^2 = \|x_k - x^\star\|_2^2 - 2t\, g_k^\top (x_k - x^\star) + t^2\|g_k\|_2^2.$$

By convexity, $g_k^\top(x_k - x^\star) \geq f(x_k) - f(x^\star)$. Also (23) implies $t^2\|g_k\|_2^2 \leq 2t\,(f(x_k) - f(x_{k+1}))$. Combine these two bounds to get

$$\|x_{k+1} - x^\star\|_2^2 \leq \|x_k - x^\star\|_2^2 - 2t\,(f(x_k) - f(x^\star)) + 2t\,(f(x_k) - f(x_{k+1})) = \|x_k - x^\star\|_2^2 - 2t\,(f(x_{k+1}) - f(x^\star)).$$

Rearranging,

$$f(x_{k+1}) - f(x^\star) \leq \frac{1}{2t}\Big(\|x_k - x^\star\|_2^2 - \|x_{k+1} - x^\star\|_2^2\Big).$$

Sum from $k = 0$ to $K - 1$ to obtain

$$\sum_{k=0}^{K-1}\big(f(x_{k+1}) - f(x^\star)\big) \leq \frac{1}{2t}\|x_0 - x^\star\|_2^2.$$

Finally, (23) implies $f(x_{k+1}) \leq f(x_k)$, so

$$K\big(f(x_K) - f(x^\star)\big) \leq \sum_{k=0}^{K-1}\big(f(x_{k+1}) - f(x^\star)\big) \leq \frac{1}{2t}\|x_0 - x^\star\|_2^2.$$

With $t = 1/M$ this yields (22). ■

The rate (22) is *sublinear* in k: to drive the suboptimality below ε we need $k = O(1/\varepsilon)$ iterations.

Corollary 1.15 (Linear convergence under strong convexity with a fixed step size)**.** *If, in addition, f is m-strongly convex, then with step size $t = 1/M$,*

$$f(x^{(k)}) - f(x^\star) \leq \left(1 - \frac{m}{M}\right)^k \big(f(x^{(0)}) - f(x^\star)\big). \tag{24}$$

Proof. With $t = 1/M$, (23) gives

$$f(x_{k+1}) \leq f(x_k) - \frac{1}{2M}\|\nabla f(x_k)\|_2^2.$$

By Corollary 1.8(b), $\|\nabla f(x_k)\|_2^2 \geq 2m(f(x_k) - f(x^\star))$. Combine and rearrange to get

$$f(x_{k+1}) - f(x^\star) \leq \left(1 - \frac{m}{M}\right)\big(f(x_k) - f(x^\star)\big),$$

then iterate. ■

Geometric rates of the form c^k are often referred to as *linear convergence* (since $\log(f(x^{(k)}) - f(x^\star))$ decays roughly linearly in k).

For general smooth strongly convex objectives, the bound in Corollary 1.15 already shows a geometric rate. For quadratics we can compute the rate exactly via eigenvalues, and in particular identify the optimal constant step size and its dependence on the condition number.

Proposition 1.16 (Gradient descent on a strongly convex quadratic)**.** *Let $A \in \mathbb{R}^{d\times d}$ be symmetric positive definite with eigenvalues in $[m, M]$, and let*

$$f(x) = \frac{1}{2}x^\top Ax - b^\top x.$$

Run gradient descent with a constant step size $t > 0$,

$$x_{k+1} = x_k - t\nabla f(x_k) = x_k - t(Ax_k - b).$$

Then $x^\star = A^{-1}b$ and the error $e_k := x_k - x^\star$ satisfies

$$e_{k+1} = (I - tA)e_k, \qquad e_k = (I - tA)^k e_0.$$

Moreover, if $0 < t < 2/M$ then

$$\|e_k\|_2 \le \rho(t)^k \|e_0\|_2, \qquad \rho(t) := \max\{|1 - tm|, |1 - tM|\} \in (0, 1),$$

and the function gap contracts as

$$f(x_k) - f(x^\star) = \frac{1}{2}\|e_k\|_A^2 \le \frac{M}{2}\rho(t)^{2k}\|e_0\|_2^2.$$

The choice $t^\star = \frac{2}{m+M}$ minimizes $\rho(t)$, yielding

$$\rho(t^\star) = \frac{M - m}{M + m} = \frac{\kappa - 1}{\kappa + 1}, \qquad \kappa := M/m.$$

Proof. Since $\nabla f(x) = Ax - b$ and $x^\star = A^{-1}b$, we have

$$e_{k+1} = x_{k+1} - x^\star = x_k - x^\star - t(Ax_k - b) = e_k - tAe_k = (I - tA)e_k,$$

and iterating gives $e_k = (I - tA)^k e_0$.

Diagonalize $A = Q\Lambda Q^\top$ with Q orthogonal and $\Lambda = \mathrm{diag}(\lambda_1, \dots, \lambda_d)$. Then $I - tA = Q(I - t\Lambda)Q^\top$, so

$$\|e_k\|_2 = \|(I - tA)^k e_0\|_2 \le \|(I - tA)^k\|_2 \, \|e_0\|_2 = \max_i |1 - t\lambda_i|^k \|e_0\|_2.$$

The map $\lambda \mapsto |1 - t\lambda|$ is convex, so its maximum over $\lambda \in [m, M]$ occurs at an endpoint, giving $\max_i |1 - t\lambda_i| = \max\{|1 - tm|, |1 - tM|\} = \rho(t)$. If $0 < t < 2/M$ then $|1 - t\lambda_i| < 1$ for every $\lambda_i \in [m, M]$, hence $\rho(t) \in (0, 1)$.

Finally, completing the square gives $f(x) - f(x^\star) = \frac{1}{2}(x - x^\star)^\top A(x - x^\star) = \frac{1}{2}\|e\|_A^2$, and since $A \preceq MI$ we have $\|e\|_A^2 \le M\|e\|_2^2$. The minimizer of $\rho(t) = \max\{|1 - tm|, |1 - tM|\}$ occurs when the two absolute values are equal, i.e. when $1 - tm = -(1 - tM)$. This gives $t^\star = 2/(m + M)$ and $\rho(t^\star) = (M - m)/(M + m)$. ■

When M is unknown (or too conservative), a line search such as backtracking is a robust way to pick step sizes automatically; we analyze this next.

1.3 Geometry and conditioning

The next issue is not whether a direction is locally correct, but how the surrounding geometry scales that direction. Condition numbers summarize how anisotropic the local quadratic model is, and preconditioning changes the geometry of the update to compensate. These same ideas about geometry control return later in gradient-based MCMC, HMC, and diffusion models.

1.3.1 Conditioning

Convergence speed is strongly influenced by how anisotropic the curvature of f is. A standard way to quantify this is the *condition number* of the Hessian.

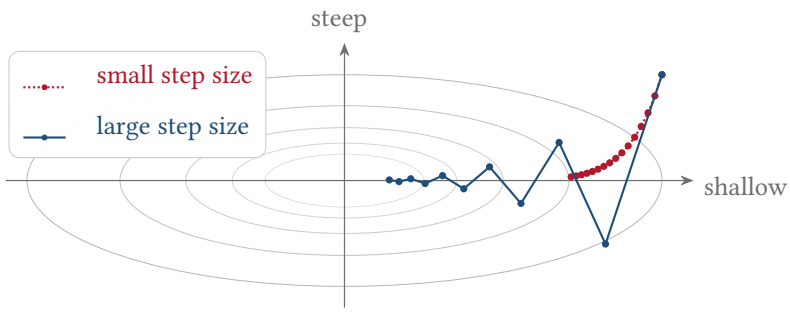

Figure 4: Ill-conditioned geometry: elongated level sets lead to slow, zig-zagging gradient descent. Small step sizes are stable but can be very slow; larger step sizes can oscillate in the steep direction (and must be controlled, e.g. by backtracking).

Definition 1.17 (Condition number)**.** If f is twice differentiable and $\nabla^2 f(x)$ is positive definite, define the (local) condition number

$$\kappa(x) = \frac{\lambda_{\max}(\nabla^2 f(x))}{\lambda_{\min}(\nabla^2 f(x))}. \tag{25}$$

If $mI \preceq \nabla^2 f(x) \preceq MI$ holds over a region of interest, then $\kappa(x) \leq M/m$ on that region and we refer to $\kappa = M/m$ as a global condition number bound.

Example 1.18 (Bounding κ for ridge logistic regression)**.** For $s(\theta)$ in Example 1.1, (7)–(8) imply

$$mI \preceq \nabla^2 s(\theta) \preceq \left(m + \tfrac{1}{4}\lambda_{\max}(X^\top X)\right)I,$$

so we can take

$$M = m + \tfrac{1}{4}\lambda_{\max}(X^\top X), \qquad \kappa \leq \frac{M}{m}.$$

Geometrically, think of the level sets of f near the optimum as ellipses. When κ is large, the ellipses are very elongated, so a single step size cannot simultaneously be "right" in both the steep and flat directions. Vanilla gradient descent can therefore zig-zag and make slow progress; preconditioning (or second-order methods) aims to reduce the effective condition number.

1.3.2 Preconditioning

One way to mitigate poor conditioning is to change the geometry of the update. Instead of (17), consider

$$x^{(k+1)} = x^{(k)} - t^{(k)} A \nabla f(x^{(k)}), \tag{26}$$

where $A \succ 0$ is a chosen matrix (a *preconditioner*). Equivalently, (26) corresponds to a linear change of variables that aims to make level sets of f "more spherical" so that one step size works well in all directions. Choosing a good A typically requires problem-specific structure; see Boyd and Vandenberghe (2004, §9.4).

If A is positive definite, the direction $-A\nabla f(x)$ is the steepest-descent direction when distance is measured in the norm $\|u\|_{A^{-1}} = \sqrt{u^\top A^{-1} u}$. Good choices of A make this geometry align with the curvature of f; heuristically, $A \approx \nabla^2 f(x)^{-1}$ reduces the effective condition number. For the ridge logistic objective (5), this picture is quite concrete. The weighted Gram matrix $X^\top W(\theta) X$ records how sharply the loss curves in different directions, while the ridge term mI adds the same amount of curvature everywhere. A useful

preconditioner therefore tries to match this combined geometry rather than treating every coordinate direction identically.

In practice, explicit matrix preconditioners are often approximated or restricted to be cheap. For example, many "adaptive" optimizers in machine learning can be interpreted as choosing a diagonal (or block-diagonal) preconditioner based on running second-moment statistics of the stochastic gradients; see the EMA discussion in Section 1.5.3.

Preconditioning still asks us to choose a metric from outside the objective. Second-order methods push the same idea one step further by building the metric from the local Hessian itself. That shift turns every update into a structured linear-algebra problem, which is the next layer of the optimization story.

1.4 Second-order methods and structured linear algebra

Preconditioning rescales first-order information with a chosen metric. Newton-type methods instead extract the metric from the Hessian itself, so each step is defined by a local quadratic model and implemented through a linear solve. This is the point where curvature, factorization, and iterative linear algebra become inseparable.

1.4.1 Newton's method

When f is twice differentiable, the second-order Taylor approximation around x is

$$f(y) \approx f(x) + \nabla f(x)^\top (y - x) + \frac{1}{2}(y - x)^\top \nabla^2 f(x)(y - x). \tag{27}$$

If $\nabla^2 f(x) \succ 0$, this quadratic model is strictly convex in y, so its unique minimizer gives the *Newton step*

$$\Delta x_{\mathrm{nt}} = -\nabla^2 f(x)^{-1} \nabla f(x), \tag{28}$$

and Newton's method updates $x \leftarrow x + \Delta x_{\mathrm{nt}}$. If $\nabla^2 f(x)$ is singular or indefinite, the quadratic model need not have a minimizer, so the basic Newton step is not defined without modification. If f is exactly quadratic with positive definite Hessian, Newton reaches the minimizer in one step. For the ridge logistic objective $s(\theta)$ from Example 1.1, the Newton direction is especially transparent: it solves

$$\left(X^\top W(\theta) X + mI\right) \Delta\theta = -\nabla s(\theta).$$

This is the same Hessian structure that appeared earlier, now used directly to reshape the local geometry before taking a step.

Lemma 1.19 (Newton direction is a descent direction)**.** *If $\nabla^2 f(x) \succ 0$ and $\nabla f(x) \neq 0$, then the Newton direction $\Delta x_{\mathrm{nt}} = -\nabla^2 f(x)^{-1} \nabla f(x)$ satisfies $\nabla f(x)^\top \Delta x_{\mathrm{nt}} < 0$.*

Proof. Let $g = \nabla f(x)$ and $H = \nabla^2 f(x) \succ 0$. Then

$$\nabla f(x)^\top \Delta x_{\mathrm{nt}} = -g^\top H^{-1} g.$$

Since $H^{-1} \succ 0$, we have $g^\top H^{-1} g > 0$ whenever $g \neq 0$, so the expression is strictly negative. ■

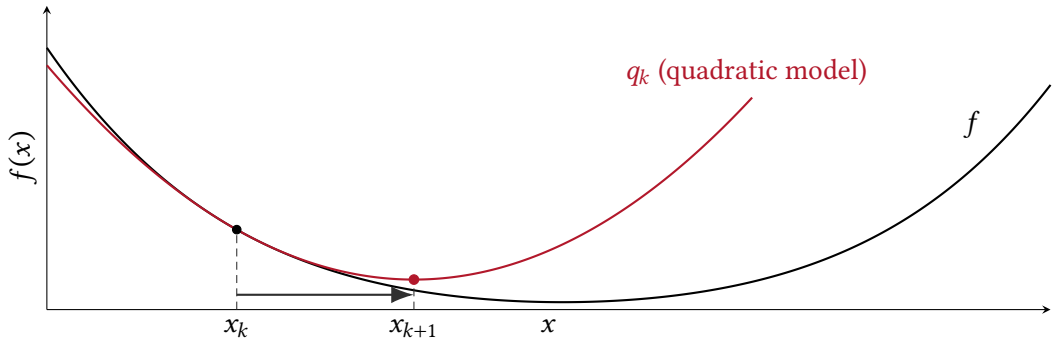

Figure 5: Newton's method: minimize the local quadratic (second-order Taylor) model of f around x_k. The minimizer of the quadratic model gives the Newton step $x_{k+1} = x_k - \nabla^2 f(x_k)^{-1} \nabla f(x_k)$.

Definition 1.20 (Newton decrement)**.** When $\nabla^2 f(x) \succ 0$, the Newton decrement is

$$\lambda(x)^2 = \nabla f(x)^\top \nabla^2 f(x)^{-1} \nabla f(x). \tag{29}$$

Equivalently, $\lambda(x)^2 = -\nabla f(x)^\top \Delta x_{\mathrm{nt}}$. A common stopping criterion is $\lambda(x)^2/2 \le \varepsilon$.

The basic algorithm is therefore stated for the regime where the Hessian stays positive definite along the iterates.

Algorithm 4 Newton's method with backtracking line search

Input: Twice differentiable f, initial point $x^{(0)}$ such that $\nabla^2 f(x^{(k)}) \succ 0$ for every iterate, tolerance $\varepsilon > 0$, parameters $\alpha \in (0, 1/2)$, $\beta \in (0, 1)$

Output: Approximate minimizer x

```
k ← 0
while true do
    g^(k) ← ∇f(x^(k))
    H^(k) ← ∇²f(x^(k))
    Solve H^(k) Δx^(k) = −g^(k) for Δx^(k)                ▷ Newton direction
    λ_k² ← −g^(k)ᵀ Δx^(k)                                 ▷ Newton decrement squared
    if λ_k²/2 ≤ ε then
        break
    end if
    t^(k) ← BacktrackingLineSearch(f, x^(k), Δx^(k), α, β)   ▷ Algorithm 2
    x^(k+1) ← x^(k) + t^(k) Δx^(k)
    k ← k + 1
end while
return x^(k)
```

The Newton decrement $\lambda(x)$ measures how much quadratic curvature would help you improve the objective at x. When $\lambda(x)$ is small, the Newton step is small and the quadratic approximation is typically accurate, so Newton's method often takes full steps ($t = 1$) and converges very quickly. When $\lambda(x)$ is large, damping/backtracking is needed to stay in the region where the local quadratic model is trustworthy (see Boyd and Vandenberghe (2004, §9.5.3)).

Boyd and Vandenberghe (2004, §9.5.3) analyzes Newton's method in two phases: far from the optimum, backtracking may be needed and the decrease is more modest; once close enough that the quadratic model is accurate, the method typically accepts $t = 1$ and exhibits quadratic convergence. Per-iteration cost can be high (solving linear systems in dimension d), and numerical issues can arise in high dimension or with ill-conditioned Hessians.

One also sometimes uses a *regularized* (or damped) Newton step, solving $(\nabla^2 f(x) + \lambda I)\Delta x = -\nabla f(x)$ with $\lambda > 0$, to improve numerical stability or to ensure the system is well-conditioned. This is already a geometric decision, not merely a linear-algebra safeguard: adding λI floors the local curvature and prevents the update from overreacting to directions that are poorly resolved. When exact Hessians are unavailable or too expensive to form, the next compromise is to approximate the same local geometry rather than abandon it.

1.4.2 Quasi-Newton updates

Quasi-Newton methods keep the local-model viewpoint but replace the exact inverse Hessian by a positive-definite approximation updated from gradients and steps. Methods such as BFGS and L-BFGS are designed to preserve the secant information revealed by successive iterates while avoiding the cost of forming or factoring a full Hessian at every step; see Nocedal and Wright (2006, §6.1–§6.2). In practice this often gives a useful middle ground between first-order robustness and full Newton steps: the method still follows a curvature-adapted direction, but the per-iteration linear algebra is much cheaper.

Whether the curvature comes from a true Hessian, a quasi-Newton update, or a covariance surrogate, the implementation still depends on stable positive-definite representations and efficient linear solves. The next destinations make that computational geometry explicit: first the reusable SPD toolkit, then the complementary large-scale solver viewpoint provided by conjugate gradient. The next three destinations answer a concrete numerical question: once an algorithm produces a Hessian, covariance, Gram matrix, or local metric, how should we actually compute with it? The recurring answer is to factor the matrix, solve systems instead of inverting them, and regularize noisy geometry before trusting it. These tools appear here because Newton systems force the issue first, but the same Cholesky factorizations, linear solves, and damping rules will reappear in Laplace proposals, Kalman and EM covariance updates, HMC mass-matrix adaptation, and Gaussian diffusion calculations.

1.4.3 Structure-preserving positive-definite geometry

Many optimization methods are built from local quadratic objects: Hessians, Gram matrices, covariance surrogates, and preconditioners. To use them reliably, we need stable numerical primitives. The recurring ones are simple: represent positive definite matrices through Cholesky factors, solve linear systems instead of forming inverses, and damp noisy geometry before trusting it. These choices keep the computation aligned with the local shape of the problem.

The first primitive is the Cholesky factorization and the structure-preserving parameterizations it enables. For symmetric positive definite matrices, the Cholesky factorization is the basic structure-preserving representation; see Golub and Van Loan (2013, Thm. 4.2.7) and Nocedal and Wright (2006, Alg. A.2). If $K \succ 0$, write $K = LL^\top$ with L lower triangular and positive diagonal. This representation keeps positive definiteness explicit and turns matrix calculations into triangular ones. It also matches the parameterization ideas from Section 1.6.4: instead of optimizing or adapting directly over the cone of SPD matrices, one works with unconstrained or lightly constrained entries of a factor. In floating-point arithmetic, a matrix that is theoretically positive definite can acquire a tiny negative eigenvalue after noisy estimation, rounding, or an unfortunate update if one stores and manipulates its entries directly. By contrast, if one stores a factor and reconstructs the matrix as $LL^\top$, positive definiteness is built into the representation itself.

When a method is built around a covariance, precision, or local metric, writing that object through a factor prevents the computation from drifting outside the positive definite cone. Cholesky factors therefore appear naturally in Gaussian objectives, covariance parameterizations, and later in mass-matrix adaptation for HMC. The factor is the stable computational object; the SPD matrix is then recovered from it.

1.4.4 Solves, log-determinants, and Gaussian quadratics

Once a positive definite matrix has been factored, the next principle is to solve linear systems rather than form explicit inverses. If $K = LL^\top$ and $r \in \mathbb{R}^d$, then

$$r^\top K^{-1} r = \|L^{-1} r\|_2^2, \qquad \log\det K = 2\sum_{i=1}^{d} \log L_{ii}. \tag{30}$$

The quadratic form is evaluated by two triangular solves, and the log-determinant is read directly from the diagonal of the factor. This is exactly the computational pattern that follows from the Cholesky factorization and its associated forward and back substitutions (Nocedal and Wright, 2006, Alg. A.2). This is the stable pattern behind Gaussian log densities, Laplace approximations, and Newton systems with quadratic models.

In practice, "invert, then multiply" is almost never the right implementation. Explicit inverses amplify conditioning problems and discard the triangular structure we just paid to compute. They also answer a more fragile question than the algorithm actually asks. The method typically needs $K^{-1}r$, not every entry of K^{-1}. A solve keeps the same mathematical answer while preserving sparsity, symmetry, and numerical accuracy. The same algebra will return whenever an algorithm is driven by local Gaussian geometry.

1.4.5 Local Gaussian geometry and damping

Once a quadratic approximation is available, the same matrix can often be reused in several ways. If a smooth objective or negative log density is approximated near a stationary point μ by

$$q(x) \approx q(\mu) + \frac{1}{2}(x-\mu)^\top H (x-\mu), \qquad H \succ 0,$$

then H is simultaneously a local curvature matrix, a local precision matrix, and a candidate preconditioner. The associated Gaussian approximation has log density

$$\log g(x) = \text{const} + \frac{1}{2}\log\det H - \frac{1}{2}(x-\mu)^\top H (x-\mu),$$

so the same Cholesky factor used to solve Newton systems also evaluates the quadratic form and log-determinant stably. This is the computational core of a Laplace approximation, but the point is broader than approximation alone.

If $H = LL^\top$, then the rescaled coordinate

$$z := L^\top (x-\mu)$$

turns the local quadratic model into $\frac{1}{2}\|z\|_2^2$. In that coordinate system, one unit of displacement corresponds to one local standard deviation, or equivalently to one unit of quadratic energy. In those whitened coordinates, a spherical proposal or unit step is locally matched to the geometry, whereas the same move in the original coordinates would be stretched along shallow directions and compressed along steep ones. A local Hessian or covariance estimate can therefore serve as a proposal scale, an initialization scale, or a preconditioner for gradient-based methods as well as an approximation to the target itself. The Laplace proposal from importance sampling is one example, and mass-matrix adaptation in HMC is another.

The local Gaussian picture is only a local approximation. If the target is strongly skewed, heavy tailed, or multimodal, then the quadratic geometry may still be useful for scaling the method even when it is not globally accurate as a distributional approximation. Its adequacy is then checked with the diagnostics natural to the algorithm at hand: log-weight histograms and ESS in importance sampling, or trace plots and mixing diagnostics in MCMC.

In practice the local geometry used by an algorithm is usually estimated rather than known exactly. Even with a good representation, local geometry estimates are often noisy or badly scaled. Newton's method may produce a Hessian that is too ill conditioned to trust directly, and a covariance estimated from a short pilot run can be nearly singular even when the underlying target is well behaved. The common response is to alter the geometry in a controlled way rather than pretending the raw estimate is exact. Adding a multiple of the identity,

$$K_\lambda := K + \lambda I, \qquad \lambda > 0,$$

is the simplest example. In optimization language, this is the basic modified-Cholesky or Hessian-damping move; see Nocedal and Wright (2006, Alg. 3.3). In optimization this appears as ridge regularization or a damped Newton system. In covariance estimation it appears as diagonal loading or a covariance floor. In shrinkage form one may instead replace $\hat{\Sigma}$ by a convex combination of $\hat{\Sigma}$ and a simpler reference metric, often diagonal or isotropic.

These modifications are not ad hoc numerical patches. They encode the judgment that very small estimated curvature or variance should not be trusted literally until the data supporting it are stronger. The effect is to floor the geometry: steep directions remain steep, but directions supported only by noise are prevented from becoming infinitely sharp or completely flat. This is the same idea behind preconditioning in Section 1.3.2. One chooses a metric that makes the important directions comparably scaled, so that a single step size can resolve them all. Later, the same perspective explains why mild regularization is useful when EM updates weighted covariances and why HMC warm-up treats mass-matrix adaptation as preconditioning from noisy pilot draws rather than as exact covariance estimation.

1.4.6 Conjugate gradient for SPD systems

Newton's method and many preconditioned updates reduce each iteration to solving a symmetric positive-definite linear system. Conjugate gradient is the canonical solver in that setting when matrix–vector products are cheap but dense factorizations are too expensive. The Cholesky-based discussion above is the direct-factorization route; CG is the complementary large-scale route when we can apply the geometry but do not want to factor it.

Let $A \in \mathbb{R}^{d\times d}$ be symmetric positive definite (SPD) and let $b \in \mathbb{R}^d$. We want to solve the linear system

$$Ax = b. \tag{31}$$

This is equivalent to minimizing the strictly convex quadratic

$$\phi(x) := \frac{1}{2}x^\top Ax - b^\top x. \tag{32}$$

Indeed, $\nabla\phi(x) = Ax - b$, so $\nabla\phi(x^\star) = 0$ is exactly (31).

In many applications, A is large and sparse, so direct methods (e.g. dense elimination) are expensive and can destroy sparsity. In such settings we typically do not want to form A^{-1} explicitly; instead we want an algorithm that uses A only through fast matrix–vector products $v \mapsto Av$. Also, Newton-type methods for nonlinear optimization repeatedly solve linear systems of the form

$$\nabla^2 f(x)\,\Delta x = -\nabla f(x),$$

so understanding fast linear solvers is directly relevant to optimization. Since $A \succ 0$, it defines an inner product and norm on $\mathbb{R}^d$:

$$\langle u, v\rangle_A := u^\top A v, \qquad \|u\|_A := \sqrt{u^\top A u}.$$

We will measure errors in the A-norm because it is the natural geometry of ϕ. Indeed, if $x^\star = A^{-1}b$, then completing the square shows

$$\phi(x) - \phi(x^\star) = \frac{1}{2}\|x - x^\star\|_A^2,$$

so decreasing the quadratic objective is equivalent to shrinking the error in A-norm.

Define the residual $r_k := b - Ax_k = -\nabla\phi(x_k)$. Conjugate gradient generates a sequence of search directions p_k that are mutually A-orthogonal ("A-conjugate") and chooses step sizes by exact line search. If we were to use steepest descent on ϕ, we would step in direction r_k. CG modifies this basic idea by combining gradient information across iterations in a way that avoids repeatedly "undoing" progress along previously explored directions.

Algorithm 5 Conjugate gradient for $Ax = b$ with $A \succ 0$

Input: SPD matrix A, vector b, initial point x_0
Output: Approximate solution x_k to $Ax = b$

```
r_0 ← b − A x_0
p_0 ← r_0
for k = 0, 1, 2, ... until converged do
    α_k ← (r_k^⊤ r_k) / (p_k^⊤ A p_k)
    x_{k+1} ← x_k + α_k p_k
    r_{k+1} ← r_k − α_k A p_k
    β_{k+1} ← (r_{k+1}^⊤ r_{k+1}) / (r_k^⊤ r_k)
    p_{k+1} ← r_{k+1} + β_{k+1} p_k
end for
return x_k
```

CG can be viewed as a descent method for ϕ with direction p_k and exact line search. In particular,

$$\alpha_k \in \arg\min_{\alpha\in\mathbb{R}} \phi(x_k + \alpha p_k).$$

Lemma 1.21 (Line search formula). *For any $x \in \mathbb{R}^d$ and nonzero direction p, the minimizer of $\alpha \mapsto \phi(x + \alpha p)$ is*

$$\alpha^\star = \frac{p^\top(b - Ax)}{p^\top A p}.$$

In CG (where $r_k = b - Ax_k$ and $p = p_k$), this yields $\alpha_k = \frac{p_k^\top r_k}{p_k^\top A p_k}$.

Proof. Expand the univariate quadratic:

$$\phi(x + \alpha p) = \frac{1}{2}(x + \alpha p)^\top A(x + \alpha p) - b^\top(x + \alpha p) = \phi(x) + \alpha\, p^\top(Ax - b) + \frac{\alpha^2}{2} p^\top A p.$$

Differentiate and set to zero:

$$\frac{d}{d\alpha}\phi(x+\alpha p) = p^\top(Ax-b) + \alpha\, p^\top Ap = -p^\top(b-Ax) + \alpha\, p^\top Ap = 0,$$

so $\alpha^\star = \frac{p^\top(b-Ax)}{p^\top Ap}$. Since $A \succ 0$ and $p \neq 0$, we have $p^\top Ap > 0$, so the minimizer is unique. In CG, one can show $p_k^\top r_k = r_k^\top r_k$ (see Lemma 1.22), giving the stated update in Algorithm 5. ■

1.4.7 Orthogonality and conjugacy in conjugate gradient

Lemma 1.22 (Residual orthogonality and A-conjugacy)**.** *For the CG iterates (x_k, r_k, p_k) in Algorithm 5, the following hold for all $i < j$:*

$$p_i^\top r_j = 0, \qquad r_i^\top r_j = 0, \qquad p_i^\top A p_j = 0.$$

In particular, $p_k^\top r_k = r_k^\top r_k$ for all k.

Proof. We prove the three identities by induction on j. For $j = 0$, there is nothing to prove.

Assume the identities hold for all pairs (i, j') with $i < j' \leq k$. We prove them for index $j = k+1$.

First we show that $p_i^\top r_{k+1} = 0$ for all $i \leq k$. We start by proving that $p_k^\top r_k = r_k^\top r_k$. For $k = 0$ this is immediate since $p_0 = r_0$. For $k \geq 1$, use $p_k = r_k + \beta_k p_{k-1}$ and the induction hypothesis $p_{k-1}^\top r_k = 0$ to get

$$p_k^\top r_k = r_k^\top r_k + \beta_k p_{k-1}^\top r_k = r_k^\top r_k.$$

Therefore the step size in Algorithm 5 can be written as

$$\alpha_k = \frac{p_k^\top r_k}{p_k^\top A p_k}.$$

Now use $r_{k+1} = r_k - \alpha_k A p_k$ to obtain

$$p_k^\top r_{k+1} = p_k^\top r_k - \alpha_k p_k^\top A p_k = 0.$$

If $i < k$, then

$$p_i^\top r_{k+1} = p_i^\top r_k - \alpha_k p_i^\top A p_k = 0 - 0 = 0,$$

by the induction hypotheses $p_i^\top r_k = 0$ and $p_i^\top A p_k = 0$.

Next we show that $r_i^\top r_{k+1} = 0$ for all $i \leq k$. From the direction recurrence $p_j = r_j + \beta_j p_{j-1}$ we can solve for r_j:

$$r_j = p_j - \beta_j p_{j-1},$$

so $r_j \in \operatorname{span}\{p_0, \dots, p_j\}$ for every j. From the first part, r_{k+1} is orthogonal to every $p_0, \dots, p_k$, hence to every vector in $\operatorname{span}\{p_0, \dots, p_k\}$. In particular, for each $i \leq k$ (so $r_i \in \operatorname{span}\{p_0, \dots, p_i\} \subseteq \operatorname{span}\{p_0, \dots, p_k\}$) we have $r_i^\top r_{k+1} = 0$.

Finally we show that $p_i^\top A p_{k+1} = 0$ for all $i \leq k$. Rearrange the residual update $r_{j+1} = r_j - \alpha_j A p_j$ to express

$$A p_j = \frac{r_j - r_{j+1}}{\alpha_j}. \tag{33}$$

Fix $i < k$. From the second part, r_{k+1} is orthogonal to both r_i and r_{i+1}, hence (33) gives

$$r_{k+1}^\top A p_i = \frac{r_{k+1}^\top r_i - r_{k+1}^\top r_{i+1}}{\alpha_i} = 0.$$

Using $p_{k+1} = r_{k+1} + \beta_{k+1} p_k$ and the induction hypothesis $p_k^\top A p_i = 0$,

$$p_{k+1}^\top A p_i = r_{k+1}^\top A p_i + \beta_{k+1} p_k^\top A p_i = 0.$$

For $i = k$, use (33) with $j = k$ and the second part ($r_{k+1}^\top r_k = 0$):

$$r_{k+1}^\top A p_k = \frac{r_{k+1}^\top r_k - r_{k+1}^\top r_{k+1}}{\alpha_k} = -\frac{r_{k+1}^\top r_{k+1}}{\alpha_k}.$$

Therefore,

$$p_{k+1}^\top A p_k = r_{k+1}^\top A p_k + \beta_{k+1} p_k^\top A p_k = -\frac{r_{k+1}^\top r_{k+1}}{\alpha_k} + \frac{r_{k+1}^\top r_{k+1}}{r_k^\top r_k} p_k^\top A p_k.$$

Since $\alpha_k = \frac{r_k^\top r_k}{p_k^\top A p_k}$, we have $\frac{1}{\alpha_k} = \frac{p_k^\top A p_k}{r_k^\top r_k}$, and the two terms cancel, giving $p_{k+1}^\top A p_k = 0$. ■

The identities in Lemma 1.22 help explain why CG is effective: each iteration moves in a genuinely new direction in the A-geometry, avoiding the "zig-zag" behavior of steepest descent on ill-conditioned quadratics.

1.4.8 Convergence guarantees for conjugate gradient

Let $x^\star = A^{-1}b$ be the unique solution and write the error $e_k := x_k - x^\star$. Since $A \succ 0$, the A-norm is equivalent to Euclidean norm, and it is the natural norm for measuring the quadratic suboptimality:

$$\phi(x_k) - \phi(x^\star) = \frac{1}{2}\|e_k\|_A^2.$$

Corollary 1.23 (Monotonic decrease of the quadratic objective)**.** *For the CG iterates in Algorithm 5, the quadratic objective is nonincreasing:*

$$\phi(x_{k+1}) \le \phi(x_k) \qquad \text{for all } k \ge 0.$$

Equivalently, the error decreases in A-norm: $\|e_{k+1}\|_A \le \|e_k\|_A$.

Proof. By exact line search (see Lemma 1.21), $x_{k+1} = x_k + \alpha_k p_k$ minimizes $\alpha \mapsto \phi(x_k + \alpha p_k)$, hence $\phi(x_{k+1}) \le \phi(x_k)$. The A-norm statement follows from $\phi(x) - \phi(x^\star) = \frac{1}{2}\|x - x^\star\|_A^2$. ■

Theorem 1.24 (Finite termination in exact arithmetic)**.** *In exact arithmetic, CG produces the exact solution in at most d iterations. More generally, if A has p distinct eigenvalues, CG terminates in at most p iterations.*

Proof. If CG terminates early, then $r_k = 0$ for some k and therefore $Ax_k = b$. Assume it does not terminate before iteration d.

By Lemma 1.22, the directions $p_0, \dots, p_{d-1}$ are pairwise A-conjugate and nonzero. In particular they are linearly independent: if $\sum_{i=0}^{d-1} c_i p_i = 0$, then multiplying by $p_j^\top A$ and using $p_j^\top A p_i = 0$ for $i \neq j$ gives $c_j\, p_j^\top A p_j = 0$, hence $c_j = 0$ for every j. Thus $p_0, \dots, p_{d-1}$ form a basis of $\mathbb{R}^d$.

Again by Lemma 1.22, we have $p_i^\top r_d = 0$ for every $i < d$. Since the p_i form a basis, this implies $r_d = 0$. Therefore $b - Ax_d = r_d = 0$, so x_d is the exact solution.

The refinement to p distinct eigenvalues follows from the polynomial characterization of CG (the error is $e_k = q_k(A)e_0$ for a degree-k polynomial q_k with $q_k(0) = 1$) and the fact that a degree-$(p-1)$ polynomial can interpolate $1/\lambda$ at p distinct eigenvalues; see Golub and Van Loan (2013, Thm. 11.3.1). ■

Theorem 1.25 (Condition-number convergence bound)**.** *Let $\kappa(A) := \lambda_{\max}(A)/\lambda_{\min}(A)$ be the spectral condition number. Then the CG error satisfies*

$$\frac{\|e_k\|_A}{\|e_0\|_A} \leq 2\left(\frac{\sqrt{\kappa(A)}-1}{\sqrt{\kappa(A)}+1}\right)^k. \tag{34}$$

Proof. The proof uses the polynomial form $e_k = q_k(A)e_0$ and chooses q_k to be a scaled and shifted Chebyshev polynomial that is small on the spectrum of A; see Golub and Van Loan (2013, §11.3). ■

For large $\kappa(A)$, the factor in (34) is well-approximated by an exponential:

$$\left(\frac{\sqrt{\kappa(A)}-1}{\sqrt{\kappa(A)}+1}\right)^k = \exp\left(k \log \frac{\sqrt{\kappa(A)}-1}{\sqrt{\kappa(A)}+1}\right) \approx \exp\left(-\frac{2k}{\sqrt{\kappa(A)}}\right).$$

This makes the role of conditioning more transparent: CG typically needs on the order of $\sqrt{\kappa(A)}$ iterations to reduce the error by a constant factor.

CG is particularly attractive when A is sparse and we can compute matrix-vector products $v \mapsto Av$ quickly. Each iteration costs one multiplication by A plus a few inner products, so the overall cost can be far below $O(d^3)$ when we only need a moderately accurate solution. However, Theorem 1.25 shows that poor conditioning (large $\kappa(A)$) slows convergence. This motivates *preconditioning*, which replaces $Ax = b$ with an equivalent system with a smaller effective condition number; see Nocedal and Wright (2006, §5.1). This is the same basic idea as the preconditioning viewpoint in Section 1.3.2, now applied to linear systems rather than gradient steps.

Finally, note that CG is tailored to SPD systems. If A is not symmetric (or not positive definite), one can sometimes reduce to an SPD system (e.g. the normal equations $A^\top Ax = A^\top b$), but this typically squares the condition number and can significantly worsen numerical stability. In such cases one usually prefers Krylov methods designed for the structure of the problem (e.g. least-squares methods like LSQR, or nonsymmetric solvers like GMRES). After the second-order and linear-solver material, we return to the first-order update template and remove two different simplifying assumptions from the basic deterministic story. Stochastic optimization keeps the same descent geometry while replacing exact gradients with noisy ones. Constrained optimization keeps exact derivatives but restricts which directions are feasible. Both themes will reappear later once momentum, augmentation, and sampling enter the picture.

1.5 Stochastic optimization

In many statistical learning problems we minimize

$$f(\theta) = \frac{1}{n}\sum_{i=1}^{n} f_i(\theta), \tag{35}$$

e.g. each f_i is the loss contributed by data point i. When n is large, evaluating $\nabla f(\theta)$ can be expensive. The basic idea of SGD is to replace the full gradient $\nabla f(\theta)$ with the gradient of a single randomly sampled term $f_{i_t}(\theta)$ (or a small mini-batch), which makes each iteration cheap. The running logistic regression example from Example 1.1 fits this template after normalizing by n. If we define

$$f_i(\theta) := \log\left(1 + e^{x_i^\top \theta}\right) - y_i x_i^\top \theta + \frac{m}{2n}\|\theta\|_2^2,$$

then $\frac{1}{n}s(\theta) = \frac{1}{n}\sum_{i=1}^{n} f_i(\theta)$. So one stochastic gradient step uses the loss contribution from a single observation together with the deterministic ridge gradient $(m/n)\theta$.

1.5.1 Stochastic gradients and mini-batches

Algorithm 6 Stochastic gradient descent (SGD)

Input: Functions $f_1, \dots, f_n$, step sizes $(\gamma_t)_{t\geq 0}$, initial point $\theta^{(0)}$, number of steps T
Output: Iterate $\theta^{(T)}$

```
for t = 0, 1, ..., T − 1 do
    Sample i_t ~ Unif{1, ..., n}
    g_t ← ∇f_{i_t}(θ^(t))
    θ^(t+1) ← θ^(t) − γ_t g_t
end for
return θ^(T)
```

Let $i_t \sim \mathrm{Unif}\{1, \dots, n\}$ and write

$$g_t = \nabla f_{i_t}(\theta^{(t)}).$$

Then g_t is an unbiased estimator of the full gradient:

$$\mathbb{E}[g_t \mid \theta^{(t)}] = \nabla f(\theta^{(t)}).$$

It is convenient to separate g_t into "signal" and "noise" by defining the gradient noise

$$\xi_t = g_t - \nabla f(\theta^{(t)}), \qquad \mathbb{E}[\xi_t \mid \theta^{(t)}] = 0.$$

The conditional covariance of this noise can be written explicitly as

$$\Sigma(\theta) := \mathbb{E}\left[\xi_t \xi_t^\top \mid \theta^{(t)} = \theta\right] = \frac{1}{n}\sum_{i=1}^{n}\left(\nabla f_i(\theta) - \nabla f(\theta)\right)\left(\nabla f_i(\theta) - \nabla f(\theta)\right)^\top. \tag{36}$$

This matrix summarizes how heterogeneous the per-sample gradients are at θ. Even at an optimum $\theta^\star$ (where $\nabla f(\theta^\star) = 0$), the per-sample gradients $\nabla f_i(\theta^\star)$ need not vanish, so the stochastic gradient typically has nonzero variance.

In practice one usually averages a batch of B independent indices $i_{t,1}, \dots, i_{t,B}$ and uses the mini-batch gradient

$$g_t^{(B)} = \frac{1}{B}\sum_{j=1}^{B} \nabla f_{i_{t,j}}(\theta^{(t)}).$$

This is still unbiased, $\mathbb{E}[g_t^{(B)} \mid \theta^{(t)}] = \nabla f(\theta^{(t)})$. Moreover, if the indices are sampled independently with replacement, then the noise covariance shrinks by a factor $1/B$:

$$\mathbb{E}\left[(g_t^{(B)} - \nabla f(\theta))(g_t^{(B)} - \nabla f(\theta))^\top \mid \theta^{(t)} = \theta\right] = \frac{1}{B}\Sigma(\theta). \tag{37}$$

Thus larger batches reduce variance but cost more compute per iteration; mini-batching trades off these two effects. These considerations already suggest the qualitative behavior of SGD: with a constant step size the iterates do not settle exactly at $\theta^\star$ (there is persistent noise), while decreasing step sizes and/or averaging can stabilize and improve convergence.

1.5.2 Step sizes and convergence

With a fixed step size $\gamma_t \equiv \gamma$, SGD typically does *not* converge exactly to $\theta^\star = \arg\min f(\theta)$; it fluctuates around it. Decreasing step sizes allow convergence to the optimum. Another standard device is to average the iterates to reduce noise; the bound below is for a weighted average (with weights proportional to the step sizes), which is a convenient form of iterate averaging in theory and practice.

The convergence bound below is standard; see Garrigos and Gower (2024, Thm. 5.3) for a reference. We include a proof for completeness.

Theorem 1.26 (Averaged SGD bound). *Let $\theta^\star = \arg\min_{\theta\in\mathbb{R}^d} f(\theta)$ and define*

$$\sigma_\star^2 = \mathbb{E}_i\left[\|\nabla f_i(\theta^\star) - \nabla f(\theta^\star)\|_2^2\right] = \mathbb{E}_i\left[\|\nabla f_i(\theta^\star)\|_2^2\right],$$

where i is uniform on $\{1, \dots, n\}$ and $\nabla f(\theta^\star) = 0$. Assume:

- *$f_1, \dots, f_n$ are convex,*
- *$\nabla^2 f_i(\theta) \preceq M_{\max} I$ for all θ and all i.*

Let $(\theta^{(t)})$ be generated by SGD with step sizes $0 < \gamma_t \le 1/(4M_{\max})$. Define the weighted average iterate

$$\bar\theta^T = \frac{\sum_{t=0}^{T-1} \gamma_t \theta^{(t)}}{\sum_{t=0}^{T-1} \gamma_t}. \tag{38}$$

Then

$$\mathbb{E}\left[f(\bar\theta^T) - f(\theta^\star)\right] \le \frac{\|\theta^{(0)} - \theta^\star\|_2^2}{\sum_{t=0}^{T-1}\gamma_t} + \frac{2\sigma_\star^2 \sum_{t=0}^{T-1}\gamma_t^2}{\sum_{t=0}^{T-1}\gamma_t}. \tag{39}$$

Proof. Fix an iteration t and write $g_t = \nabla f_{i_t}(\theta^{(t)})$. Expand the squared distance to $\theta^\star$:

$$\|\theta^{(t+1)} - \theta^\star\|_2^2 = \|\theta^{(t)} - \theta^\star\|_2^2 - 2\gamma_t g_t^\top(\theta^{(t)} - \theta^\star) + \gamma_t^2\|g_t\|_2^2.$$

Taking conditional expectation given $\theta^{(t)}$ and using unbiasedness $\mathbb{E}[g_t \mid \theta^{(t)}] = \nabla f(\theta^{(t)})$ yields

$$\mathbb{E}\left[\|\theta^{(t+1)} - \theta^\star\|_2^2 \mid \theta^{(t)}\right] = \|\theta^{(t)} - \theta^\star\|_2^2 - 2\gamma_t \nabla f(\theta^{(t)})^\top(\theta^{(t)} - \theta^\star) + \gamma_t^2 \mathbb{E}\left[\|g_t\|_2^2 \mid \theta^{(t)}\right]. \tag{40}$$

By convexity of f, we have $\nabla f(\theta^{(t)})^\top(\theta^{(t)} - \theta^\star) \ge f(\theta^{(t)}) - f(\theta^\star)$.

It remains to bound the second moment of g_t. Define

$$\tilde f_i(\theta) = f_i(\theta) - f_i(\theta^\star) - \nabla f_i(\theta^\star)^\top(\theta - \theta^\star).$$

Each $\tilde{f}_i$ is convex and $M_{\max}$-smooth (subtracting an affine function does not change convexity or smoothness), and satisfies $\nabla \tilde{f}_i(\theta^\star) = 0$, so $\theta^\star$ minimizes $\tilde{f}_i$. For any convex $M_{\max}$-smooth function h with minimizer $x^\star$ one has

$$\|\nabla h(x)\|_2^2 \le 2M_{\max}\big(h(x) - h(x^\star)\big), \tag{41}$$

which follows by applying the smoothness upper bound in Theorem 1.6 to h at x with the test point $x - \frac{1}{M_{\max}}\nabla h(x)$ and using optimality of $x^\star$. Applying (41) to $h = \tilde{f}_i$ gives

$$\|\nabla f_i(\theta) - \nabla f_i(\theta^\star)\|_2^2 = \|\nabla \tilde{f}_i(\theta)\|_2^2 \le 2M_{\max}\tilde{f}_i(\theta).$$

Averaging over i and using $\mathbb{E}_i[\nabla f_i(\theta^\star)] = \nabla f(\theta^\star) = 0$ yields

$$\mathbb{E}_i\|\nabla f_i(\theta) - \nabla f_i(\theta^\star)\|_2^2 \le 2M_{\max}\big(f(\theta) - f(\theta^\star)\big).$$

Finally,

$$\mathbb{E}_i\|\nabla f_i(\theta)\|_2^2 \le 2\mathbb{E}_i\|\nabla f_i(\theta) - \nabla f_i(\theta^\star)\|_2^2 + 2\mathbb{E}_i\|\nabla f_i(\theta^\star)\|_2^2 \le 4M_{\max}\big(f(\theta) - f(\theta^\star)\big) + 2\sigma_\star^2.$$

Plug this into (40) and use $\gamma_t \le 1/(4M_{\max})$ to get

$$\mathbb{E}\big[\|\theta^{(t+1)} - \theta^\star\|_2^2 \mid \theta^{(t)}\big] \le \|\theta^{(t)} - \theta^\star\|_2^2 - \gamma_t\big(f(\theta^{(t)}) - f(\theta^\star)\big) + 2\sigma_\star^2\gamma_t^2.$$

Rearrange, take full expectation, and sum from $t = 0$ to $T - 1$ to obtain

$$\sum_{t=0}^{T-1}\gamma_t\,\mathbb{E}\big[f(\theta^{(t)}) - f(\theta^\star)\big] \le \|\theta^{(0)} - \theta^\star\|_2^2 + 2\sigma_\star^2\sum_{t=0}^{T-1}\gamma_t^2.$$

Since f is convex and $\bar{\theta}^T$ is a weighted average, $f(\bar{\theta}^T) \le \frac{\sum_t \gamma_t f(\theta^{(t)})}{\sum_t \gamma_t}$. Dividing by $\sum_{t=0}^{T-1}\gamma_t$ gives (39). ■

The right-hand side of (39) has a simple structure: the first term is an "optimization" term, which decreases when $\sum_t \gamma_t$ is large; the second is a "noise" term, which decreases when $\sum_t \gamma_t^2$ is small. Choosing a step size schedule balances these two effects.

The bound (39) is a clean bias–variance trade-off. Large step sizes make rapid early progress (large $\sum_t \gamma_t$) but amplify gradient noise (large $\sum_t \gamma_t^2$), leading to a noise floor. Shrinking step sizes reduces fluctuations and can drive the expected suboptimality to 0, but typically slows optimization. At a heuristic level, one can think of SGD as gradient descent perturbed by state-dependent noise whose strength is set by the step size and batch size. Writing the recursion as

$$\theta^{(t+1)} = \theta^{(t)} - \gamma_t\nabla f(\theta^{(t)}) - \gamma_t\xi_t$$

makes the analogy more concrete. If $\gamma_t \equiv \gamma$ is small, then on the rescaled time axis $s = t\gamma$ the drift term is precisely the explicit Euler discretization of gradient flow. Over $O(1/\gamma)$ steps, the accumulated noise is of order $\sqrt{\gamma}$ by a central-limit heuristic, and for a mini-batch of size B this suggests the diffusion approximation

$$d\Theta_s = -\nabla f(\Theta_s)\,ds + \sqrt{\frac{\gamma}{B}}\,\Sigma(\Theta_s)^{1/2}\,dW_s, \tag{42}$$

interpreted informally as a weak-limit statement under additional regularity assumptions; see Mandt et al. (2015, §2) for one clean continuous-time derivation in the constant-step regime. Thus gradient descent is the zero-noise limit, while constant-step SGD is the corresponding Euler–Maruyama discretization with a state-dependent diffusion term. Near $\theta^\star$, this diffusion does not disappear unless γ is reduced or B is

increased, which is the dynamical reason constant-step SGD hovers in a neighborhood of the minimizer rather than settling exactly there. That intuition is consistent with (39): the same schedule that drives the optimization term down also sets the size of the noise floor.

Some consequences for step size schedules are:

(1) *Constant step size.* If $\gamma_t \equiv \gamma$ then
$$\frac{\sum_{t=0}^{T-1} \gamma_t^2}{\sum_{t=0}^{T-1} \gamma_t} = \gamma,$$
so (39) becomes
$$\mathbb{E}[f(\bar{\theta}^T) - f(\theta^\star)] \le \frac{\|\theta^{(0)} - \theta^\star\|_2^2}{T\gamma} + 2\sigma_\star^2 \gamma.$$
Choosing $\gamma \propto 1/\sqrt{T}$ yields the familiar $O(1/\sqrt{T})$ rate.

(2) *Decaying step sizes.* A common choice is $\gamma_t = \gamma_0/(t+1)^\alpha$ with $1/2 < \alpha \le 1$ (often $\alpha = 1$), with γ_0 chosen so that $\gamma_t \le 1/(4M_{\max})$ (or by clipping γ_t to satisfy the bound). Then $\sum_t \gamma_t = \infty$ but $\sum_t \gamma_t^2 < \infty$, so (39) forces $\mathbb{E}[f(\bar{\theta}^T) - f(\theta^\star)] \to 0$ as $T \to \infty$.

(3) *Under strong convexity.* If additionally f is strongly convex, then constant-step SGD has a linear transient followed by a noise floor whose size is proportional to γ; see Garrigos and Gower (2024, Thm. 5.8). Decreasing the step size from there, for example with $\gamma_t = \gamma_0/(t+1)$, trades that residual noise for exact convergence, and sharper $O(1/T)$-type bounds are available under stronger assumptions; see Garrigos and Gower (2024, §5.2).

Representative theorems for these schedule-dependent consequences appear in Garrigos and Gower (2024, Thm. 5.3 and §5.2).

1.5.3 Averaging and practical guidance

SGD is useful when full gradient computations are bottlenecked by very large data sets or streaming data. The table below is only a workload heuristic. It mixes the linear-rate strongly convex regime for GD with the $O(1/\rho)$-type noisy convex scaling suggested by the SGD bounds above, so it should be read as an order-of-magnitude comparison rather than a theorem.

A rough trade-off is:

	GD	SGD	Mini-batch SGD
Time / iteration	n	1	B
Representative iterations to error scale ρ	$\log(1/\rho)$	$1/\rho$	$1/\rho$
Representative time scale	$n\log(1/\rho)$	$1/\rho$	B/ρ

A few practical implementation notes:

(1) Resampling without replacement / shuffling data between epochs is common and can improve performance in practice (also for mini-batches).

(2) Tune learning rates on a small subset first, then scale up. A typical decreasing schedule is $\gamma_t = \gamma_0/(t+1)$; another common heuristic is $\gamma_t = \gamma_0/(1+\gamma_0 m t)$ for an estimate of a strong convexity constant m.

(3) If possible, monitor both a training-side quantity and a held-out generalization quantity: for example, a full or large-batch estimate of the objective together with validation/test error checked every few

epochs. Per-mini-batch losses fluctuate too much to support a reliable stopping rule on their own, and watching only training loss can hide the onset of overfitting.

(4) With a constant or piecewise-constant step size, expect a noise-dominated plateau rather than exact convergence. Once the monitored metric stops improving for several checks, the right response is usually to reduce γ_t, increase the batch size, or stop; continuing with the same settings mainly reshuffles the noise floor predicted by (39) and (42).

(5) If the objective is strongly convex (often true once we add regularization), the last iterate $\theta^{(T)}$ is a reasonable estimator; otherwise use an averaged iterate such as (38), or a *tail average*

$$\bar{\theta}_{(i)}^{T} = \frac{\sum_{t=i}^{T-1} \gamma_t \theta^{(t)}}{\sum_{t=i}^{T-1} \gamma_t}$$

for a sufficiently large burn-in index i, chosen after the iterates have entered their final fluctuation regime.

(6) When gradient norms are available only through noise, estimate $\|\nabla f(\theta^{(t)})\|_2$ on the full data or on a large representative subset before using it as a stopping test. This is much more stable than thresholding a single stochastic gradient.

(7) Mini-batching stabilizes noisy gradients; common batch sizes in ML practice are on the order of tens to a few hundreds (e.g. 32–256), depending on compute and data.

The stopping logic mirrors the theory. With a fixed step size γ and batch size B, the optimization term in (39) decays like $1/(T\gamma)$, while the noise floor stays on the scale of γ/B. Once the former is much smaller than the latter, more iterations at the same settings mostly average noise rather than meaningfully decrease the objective. At that point, either change the schedule or stop and keep an averaged iterate.

Averaging appears constantly in modern ML practice. One common variant is an exponential moving average of parameters,

$$\hat{\theta}^{(t)} = \beta \hat{\theta}^{(t-1)} + (1-\beta)\theta^{(t)}, \qquad \beta \in (0,1),$$

which is a geometrically weighted average of past iterates and acts like a low-pass filter that reduces high-frequency noise. One reason averaging helps is that it reduces the effective gradient noise; Theorem 1.26 is one simple guarantee for (weighted) iterate averaging in the convex smooth setting. EMA is a practically convenient variant that emphasizes recent iterates. Relatedly, many optimizers used in deep learning maintain exponential moving averages of gradients (and sometimes squared gradients), which can be interpreted as adding smoothing and (diagonal) preconditioning heuristics.

1.6 Constrained optimization and reparameterization

Many statistical models impose constraints on parameters: probability vectors live on a simplex, variances must be positive, and covariance matrices must be positive semidefinite. Abstractly, we consider the constrained optimization problem

$$\min_{x \in C} f(x), \tag{43}$$

where $C \subseteq \mathbb{R}^d$ is a constraint set. Unlike in (1), a naive gradient step $x - t\nabla f(x)$ may leave C. Projected methods address this by combining unconstrained updates with a projection back onto the feasible set. This gives the geometric view of constrained optimization. We then turn to the algebraic view, where Lagrange multipliers and dual variables encode how the active constraints push back on the objective.

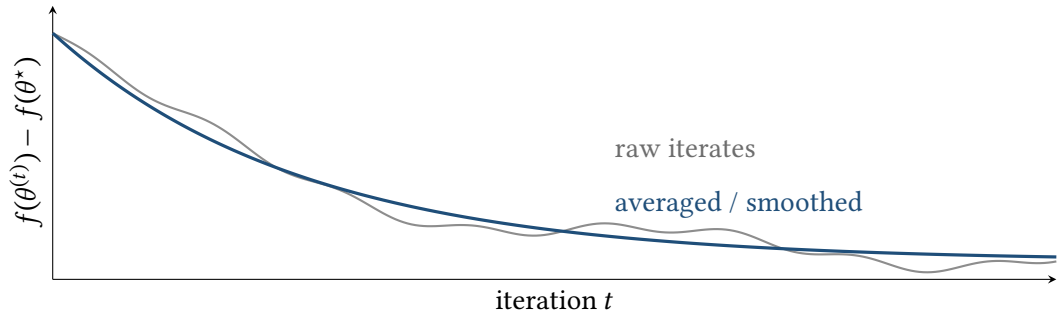

Figure 6: Typical behavior of SGD on the objective gap: noisy descent with a long tail. Averaging (or tail-averaging) stabilizes iterates and makes the underlying decreasing trend easier to see, matching the motivation behind Theorem 1.26.

1.6.1 Projections and projected gradient descent

Definition 1.27 (Euclidean projection)**.** Let $C \subseteq \mathbb{R}^d$ be nonempty, closed, and convex. The (Euclidean) projection onto C is the map $\Pi_C : \mathbb{R}^d \to C$ defined by

$$\Pi_C(z) := \arg\min_{y \in C} \|y - z\|_2.$$

Closedness ensures the minimum is attained; convexity plus the strict convexity of $\|\cdot\|_2^2$ ensures the minimizer is unique. Equivalently, $\Pi_C(z)$ is the closest feasible point to z in Euclidean distance.

Algorithm 7 Projected gradient descent

Input: Differentiable f, closed convex set C, initial point $x_0 \in C$, step sizes $(t_k)_{k \geq 0}$
Output: Iterate $x_k \in C$

```
for k = 0, 1, 2, ... until converged do
    x_{k+1} ← Π_C(x_k − t_k ∇f(x_k))
end for
return x_k
```

Projected gradient descent is conceptually simple: take an unconstrained step, then project back to the feasible set. In practice, the main difficulty is often computational: computing Π_C can be easy for some sets (like boxes) and nontrivial for others. When projections are expensive, other approaches can be more appropriate. For example, one can add penalties for constraint violations, use barrier terms (which blow up near the boundary) to keep iterates strictly feasible (see Section 1.6.6), or introduce Lagrange multipliers and work with optimality conditions (see Section 1.6.5). Here we focus on projections because they make the feasibility mechanism explicit.

1.6.2 Affine constraints

A common and especially simple constraint set is an affine set

$$C := \{x \in \mathbb{R}^d : Ax = b\},$$

where $A \in \mathbb{R}^{p \times d}$ has full row rank. In this case the Euclidean projection admits a closed form.

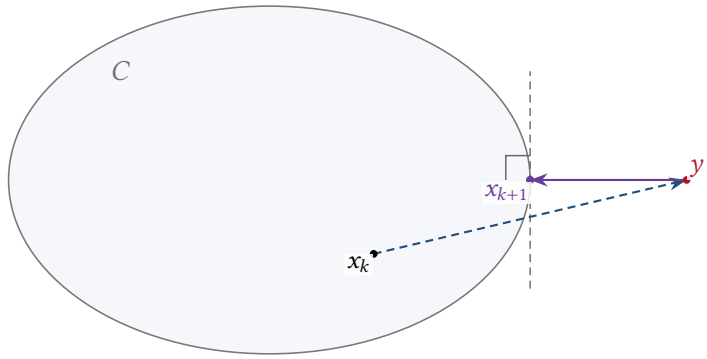

Figure 7: Geometry of a projected gradient step. Starting from a feasible iterate $x_k \in C$, we first take an unconstrained step to $y = x_k - t_k \nabla f(x_k)$. If $y \notin C$, projected gradient descent maps it back to the feasible set via $x_{k+1} = \Pi_C(y)$, the closest point in C to y in Euclidean distance. At the projection point (for smooth boundaries) the correction direction $y - x_{k+1}$ is orthogonal to the boundary.

Lemma 1.28 (Projection onto an affine set). *Assume A has full row rank and let $C = \{x \in \mathbb{R}^d : Ax = b\}$. Then for any $z \in \mathbb{R}^d$,*

$$\Pi_C(z) = z - A^\top (AA^\top)^{-1}(Az - b).$$

Proof. The projection solves $\min_x \frac{1}{2}\|x - z\|_2^2$ subject to $Ax = b$. Form the Lagrangian $\mathcal{L}(x, \nu) = \frac{1}{2}\|x - z\|_2^2 + \nu^\top(Ax - b)$. Stationarity gives $0 = \nabla_x \mathcal{L} = x - z + A^\top \nu$, so $x = z - A^\top \nu$. Enforcing feasibility yields $Az - AA^\top \nu = b$, hence $\nu = (AA^\top)^{-1}(Az - b)$ since A has full row rank. ■

Applying Algorithm 7 with this projection keeps all iterates feasible. Equivalently, the projected gradient step moves in the nullspace $\mathcal{N}(A)$:

$$x_{k+1} = x_k - t_k \, \Delta x_k, \qquad \Delta x_k = \mathrm{Proj}_{\mathcal{N}(A)}\big(\nabla f(x_k)\big),$$

so feasibility is preserved for any step size t_k. Here $\mathcal{N}(A) = \{v \in \mathbb{R}^d : Av = 0\}$ is the nullspace of A, and $\mathrm{Proj}_{\mathcal{N}(A)}$ denotes the Euclidean (orthogonal) projection onto this subspace. See Boyd and Vandenberghe (2004, §10.2, Ex. 10.2). When A has full row rank, one can write $\mathrm{Proj}_{\mathcal{N}(A)} = I - A^\top (AA^\top)^{-1} A$.

1.6.3 Simplex and positive-semidefinite projections

Each of the projections below is itself a convex optimization problem. Often, writing a Lagrangian and working out the optimality conditions reveals a simple thresholding structure (an "active set" of tight inequalities). We formalize this viewpoint later via the KKT conditions in Section 1.6.5; here we use it as a computational tool.

A first example is the probability simplex in $\mathbb{R}^d$,

$$\Delta := \{x \in \mathbb{R}^d : x_i \geq 0, \ \textstyle\sum_{i=1}^d x_i = 1\}.$$

The naive operation "threshold then renormalize" is *not* the Euclidean projection in general. The true projection has a clean structure.

Proposition 1.29 (Projection onto the simplex has a threshold form)**.** *Let $z \in \mathbb{R}^d$. There exists a scalar $\tau \in \mathbb{R}$ such that the Euclidean projection $x^\star = \Pi_\Delta(z)$ satisfies*

$$x_i^\star = \max\{z_i - \tau, 0\}, \qquad i = 1, \dots, d, \tag{44}$$

with τ chosen so that $\sum_i x_i^\star = 1$.

Proof. Consider the convex problem $\min_{x\in\Delta} \frac{1}{2}\|x-z\|_2^2$. Form the Lagrangian with multiplier τ for the equality constraint and multipliers $\nu_i \geq 0$ for the inequalities:

$$\mathcal{L}(x, \tau, \nu) = \frac{1}{2}\|x - z\|_2^2 + \tau\Big(\sum_{i=1}^d x_i - 1\Big) - \sum_{i=1}^d \nu_i x_i.$$

Stationarity gives $0 = \nabla_x \mathcal{L} = x - z + \tau\mathbf{1} - \nu$, i.e. $x_i = z_i - \tau + \nu_i$. Complementary slackness $\nu_i x_i = 0$ implies: if $x_i > 0$ then $\nu_i = 0$ and $x_i = z_i - \tau$; if $x_i = 0$ then $\nu_i = \tau - z_i \geq 0$ so $z_i - \tau \leq 0$. Thus $x_i = \max\{z_i - \tau, 0\}$. Finally, feasibility requires $\sum_i x_i = 1$, which determines τ. ■

Algorithm 8 Projection onto the simplex Δ (sorting + threshold)

Input: Vector $z \in \mathbb{R}^d$
Output: Projection $x = \Pi_\Delta(z)$

1: Let u be z sorted in descending order: $u_1 \geq \cdots \geq u_d$
2: $\rho \leftarrow \max\Big\{ j \in \{1, \dots, d\} \;:\; u_j - \frac{1}{j}\Big(\sum_{i=1}^j u_i - 1\Big) > 0 \Big\}$
3: $\tau \leftarrow \frac{1}{\rho}\Big(\sum_{i=1}^\rho u_i - 1\Big)$
4: $x_i \leftarrow \max\{z_i - \tau, 0\}$ for $i = 1, \dots, d$
5: **return** x

Algorithm 8 implements the threshold form in Proposition 1.29 by finding the active set size ρ and corresponding threshold τ via sorting. Complementary slackness identifies an "active set" of coordinates that remain positive, and sorting is a convenient way to find its size. Because Δ contains interior points (for example $(1/d)\mathbf{1}$), Slater's condition holds for this convex problem, so the KKT conditions in Section 1.6.5 indeed characterize the projection.

Another common constraint set is the cone of symmetric positive semidefinite (PSD) matrices $\mathbb{S}_+^d \subseteq \mathbb{R}^{d\times d}$. The Euclidean projection (for the Frobenius norm) of a symmetric matrix Z onto $\mathbb{S}_+^d$ is obtained by thresholding eigenvalues. Here the Frobenius norm is $\|M\|_F = \big(\sum_{i,j} M_{ij}^2\big)^{1/2}$. If Z is not symmetric, one typically replaces it by its symmetric part $(Z + Z^\top)/2$ before projecting.

Proposition 1.30 (Projection onto the PSD cone)**.** *Let $Z \in \mathbb{R}^{d\times d}$ be symmetric with eigendecomposition $Z = Q\Lambda Q^\top$, where $\Lambda = \mathrm{diag}(\lambda_1, \dots, \lambda_d)$. Define $\Lambda_+ = \mathrm{diag}(\max\{\lambda_1, 0\}, \dots, \max\{\lambda_d, 0\})$. Then*

$$\Pi_{\mathbb{S}_+^d}(Z) = Q\Lambda_+ Q^\top.$$

Proof. We minimize $\|X - Z\|_F^2$ over $X \succeq 0$. By orthogonal invariance of the Frobenius norm,

$$\|X - Z\|_F^2 = \|Q^\top X Q - \Lambda\|_F^2.$$

Since $X \succeq 0$ iff $Q^\top XQ \succeq 0$, we can equivalently minimize over $Y \succeq 0$ the quantity $\|Y - \Lambda\|_F^2$. For diagonal Λ, the optimal Y is diagonal as well (off-diagonal entries only increase the Frobenius norm), reducing the problem to projecting $(\lambda_1, \dots, \lambda_d)$ onto $\mathbb{R}_+^d$, i.e. replacing negative coordinates by 0. ■

This is the same basic thresholding phenomenon as in Proposition 1.29, but expressed in an eigenbasis rather than in coordinates. In the eigen-directions where the PSD constraint is inactive, the projection keeps the eigenvalue unchanged; in the active directions, the eigenvalue is clipped to 0.

1.6.4 Reparameterization for strict constraints

The projection Π_C is only guaranteed to be well-defined when C is closed. Strict constraints (e.g. $x_i > 0$ or $\Sigma \succ 0$) define open sets, which can cause projection-based methods to misbehave near the boundary. In such cases, a minimizer can fail to exist: the infimum may be approached only at boundary points that are not feasible.

Example 1.31 (Open constraints can destroy projections)**.** If $C = (0, 1) \subset \mathbb{R}$ and $z = 0$, then $\inf_{x \in C} \|x - z\|_2 = 0$ but the infimum is not attained in C.

A common alternative is to *reparameterize* the constrained object in terms of unconstrained variables.

A common example is enforcing that a covariance matrix Σ is symmetric positive definite. If $\Sigma \succ 0$ is symmetric, it has a Cholesky factorization $\Sigma = LL^\top$ with L lower triangular and $L_{ii} > 0$. We can enforce strict positivity by writing $L_{ii} = \exp(b_{ii})$ for unconstrained $b_{ii} \in \mathbb{R}$. Then any choice of parameters produces $\Sigma \succ 0$ automatically.

Similarly, the strict simplex

$$\Delta^\circ := \{x \in \mathbb{R}^d : \ x_i > 0, \ \textstyle\sum_{i=1}^d x_i = 1\}$$

can be reparameterized by the softmax map: for $y \in \mathbb{R}^d$, define

$$x_i = \frac{e^{y_i}}{\sum_{j=1}^d e^{y_j}}, \qquad i = 1, \dots, d.$$

This enforces $x \in \Delta^\circ$ automatically and turns optimization over Δ° into unconstrained optimization in y (up to the invariance $y \mapsto y + c\mathbf{1}$). The next example shows the same idea in a matrix-valued statistical model, where reparameterization removes a fragile positivity constraint and isolates the remaining structural restriction.

Example 1.32 (Gaussian precision estimation with a sparsity constraint)**.** Let $x_1, \dots, x_n \in \mathbb{R}^d$ be i.i.d. with $x_i \sim \mathcal{N}(\mu, \Sigma)$, where $\Sigma \succ 0$. Up to an additive constant, the negative log-likelihood is

$$\mathcal{L}(\mu, \Sigma) = \frac{n}{2} \log\det \Sigma + \frac{1}{2} \sum_{i=1}^n (x_i - \mu)^\top \Sigma^{-1} (x_i - \mu). \tag{45}$$

Suppose we impose the (structural) constraint that the $(1, 2)$ and $(2, 1)$ entries of the precision matrix $\Omega := \Sigma^{-1}$ are zero:

$$\Omega_{12} = \Omega_{21} = 0.$$

(In Gaussian models, such sparsity constraints correspond to conditional independences.)
Optimizing over μ first gives $\hat{\mu} = \bar{x}$, where $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$. Rewriting the objective in terms of $\Omega = \Sigma^{-1}$

then yields

$$\mathcal{L}(\bar{x}, \Omega) = \frac{n}{2}\Big(-\log\det\Omega + \operatorname{tr}(\Omega S)\Big), \qquad S := \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})(x_i - \bar{x})^\top. \tag{46}$$

The matrix S is the sample covariance. The objective (46) is convex over $\Omega \succ 0$ (since $-\log\det$ is convex and the trace term is linear); see Boyd and Vandenberghe (2004, §3.1.5). Adding the linear constraint $\Omega_{12} = 0$ preserves convexity.

To enforce $\Omega \succ 0$, write $\Omega = LL^\top$ with L lower triangular and $L_{ii} = \exp(b_{ii})$. This is exactly the structure-preserving parameterization from Section 1.4.3, and the corresponding log-determinant and quadratic identities are developed in Section 1.4.4. Here the point is that the strict positivity constraint is absorbed into the factor, while the sparsity constraint becomes simpler. Indeed, because L is lower triangular,

$$\Omega_{21} = (LL^\top)_{21} = L_{21}L_{11} = 0 \quad \Rightarrow \quad L_{21} = 0,$$

because $L_{11} = \exp(b_{11}) > 0$. So the constrained precision estimation problem reduces to an optimization over the factor entries, with one simple linear constraint in place of the original positivity-plus-sparsity constraints:

$$\min_{L} \; -n\sum_{i=1}^{d} b_{ii} + \frac{n}{2}\operatorname{tr}(L^\top S L) \quad \text{s.t.} \quad L \text{ lower triangular},\; L_{ii} = \exp(b_{ii}),\; L_{21} = 0. \tag{47}$$

Concretely, the free variables are the real numbers $b_{11}, \dots, b_{dd}$ and the strictly lower-triangular entries of L (with L_{21} fixed to 0), and every such choice produces a valid precision matrix $\Omega \succ 0$. This is the main lesson of the example. Reparameterization removes the fragile positivity constraint, and the remaining structural constraint can be handled either by eliminating the variable or through the KKT framework developed next.

1.6.5 KKT conditions and equality-constrained Newton

The projection examples above already showed the basic multiplier idea. If we want to minimize $f(x)$ subject to one equality constraint $h(x) = 0$, we form the Lagrangian

$$\mathcal{L}(x, \nu) = f(x) + \nu h(x).$$

At a constrained optimum, the gradient of f need not vanish. Instead it is balanced by the gradient of the constraint:

$$\nabla f(x^\star) + \nu^\star \nabla h(x^\star) = 0.$$

The multiplier records how strongly the constraint pushes back against further decrease of the objective. The KKT conditions are the general version of this first-order balance.

Consider a problem with inequality and equality constraints,

$$\min_{x\in\mathbb{R}^d} f(x) \quad \text{s.t.} \quad g_i(x) \le 0,\; i = 1, \dots, m, \qquad h_j(x) = 0,\; j = 1, \dots, p, \tag{48}$$

where f, g_i, h_j are differentiable. The constrained problem (48) is called the *primal problem*, and x is the *primal variable*. For multipliers $\lambda \in \mathbb{R}^m_+$ (i.e. $\lambda_i \ge 0$) and $\nu \in \mathbb{R}^p$, define the Lagrangian

$$\mathcal{L}(x, \lambda, \nu) := f(x) + \sum_{i=1}^{m} \lambda_i g_i(x) + \sum_{j=1}^{p} \nu_j h_j(x).$$

For fixed multipliers, $\mathcal{L}(x, \lambda, \nu)$ is just an unconstrained objective in x: the extra terms encode how much we penalize or reward constraint violation. The same construction also defines the *dual function*

$$q(\lambda, \nu) := \inf_{x \in \mathbb{R}^d} \mathcal{L}(x, \lambda, \nu),$$

and the associated *dual problem*

$$\max_{\lambda \in \mathbb{R}^m_+,\ \nu \in \mathbb{R}^p} q(\lambda, \nu).$$

The dual function is the best lower bound on the primal objective that the multipliers can certify, and the dual problem searches for the strongest such bound. For each fixed (λ, ν), the dual function is obtained by minimizing the Lagrangian over the primal variable x. This automatically makes q a concave function of (λ, ν), even if the primal problem itself is not convex. Let $p^\star$ denote the optimal value of (48), and let $d^\star$ denote the optimal value of the dual. Weak duality gives $d^\star \le p^\star$. Indeed, if x is primal feasible and $\lambda \ge 0$, then

$$\mathcal{L}(x, \lambda, \nu) = f(x) + \sum_{i=1}^{m} \lambda_i g_i(x) + \sum_{j=1}^{p} \nu_j h_j(x) \le f(x),$$

because the inequality-constraint terms are nonpositive and the equality-constraint terms vanish. Taking the infimum over x gives $q(\lambda, \nu) \le f(x)$. More generally, for any primal feasible x and dual feasible (λ, ν) (meaning $\lambda \ge 0$ and $q(\lambda, \nu) > -\infty$) we have $q(\lambda, \nu) \le f(x)$, so the nonnegative quantity $f(x) - q(\lambda, \nu)$ is a computable *duality gap* certificate; see Boyd and Vandenberghe (2004, §5.1.2–§5.1.3).

Example 1.33 (Dual problem for affine projection)**.** Consider the affine projection problem from Lemma 1.28,

$$\min_{x \in \mathbb{R}^d} \frac{1}{2}\|x - z\|_2^2 \qquad \text{s.t.} \qquad Ax = b,$$

where A has full row rank. Its Lagrangian is $\mathcal{L}(x, \nu) = \frac{1}{2}\|x - z\|_2^2 + \nu^\top(Ax - b)$. Minimizing over x gives $x = z - A^\top \nu$, and substituting this back yields the dual function

$$q(\nu) = \nu^\top(Az - b) - \frac{1}{2}\nu^\top AA^\top \nu.$$

The dual problem is therefore the unconstrained concave quadratic

$$\max_{\nu \in \mathbb{R}^p} \ \nu^\top(Az - b) - \frac{1}{2}\nu^\top AA^\top \nu.$$

Its maximizer is $\nu^\star = (AA^\top)^{-1}(Az - b)$, and substituting back into $x = z - A^\top \nu$ recovers the projection formula from Lemma 1.28. This example makes the roles clear: the primal variable is the candidate point x, while the dual variable ν enforces the feasibility condition $Ax = b$.

The Karush–Kuhn–Tucker (KKT) conditions are the standard first-order optimality conditions associated with (48). For convex problems they can give an exact, checkable characterization of optimality; for non-convex problems they should be viewed as necessary conditions (under mild regularity of the constraints),

not as a guarantee of global optimality. We say that $(x^\star, \lambda^\star, \nu^\star)$ satisfies the KKT conditions if

$$\begin{aligned} g_i(x^\star) &\le 0, \quad i = 1, \dots, m, \\ h_j(x^\star) &= 0, \quad j = 1, \dots, p, \\ \lambda_i^\star &\ge 0, \quad i = 1, \dots, m, \\ \lambda_i^\star g_i(x^\star) &= 0, \quad i = 1, \dots, m, \\ \nabla f(x^\star) + \sum_{i=1}^m \lambda_i^\star \nabla g_i(x^\star) + \sum_{j=1}^p \nu_j^\star \nabla h_j(x^\star) &= 0. \end{aligned} \tag{49}$$

The last line in (49) is called *stationarity*: at optimality, the objective gradient is balanced by a linear combination of constraint gradients. Complementary slackness, $\lambda_i^\star g_i(x^\star) = 0$, captures the "active set" idea: if constraint $g_i(x) \le 0$ is *inactive* at $x^\star$ (meaning $g_i(x^\star) < 0$), then necessarily $\lambda_i^\star = 0$, so it plays no role in stationarity. Only constraints that are tight ($g_i(x^\star) = 0$) can have nonzero multipliers; we call such constraints *active*. These multipliers appear in the stationarity equation as a linear combination of the gradients of the active constraints. For a differentiable constraint g_i, the vector $\nabla g_i(x^\star)$ is orthogonal to the boundary $\{x : g_i(x) = 0\}$ at $x^\star$, so it acts as a (local) normal vector to the constraint surface. For an inequality constraint $g_i(x) \le 0$, $\nabla g_i(x^\star)$ points locally in the direction where g_i increases, i.e. toward constraint violation.

This matches the basic geometry of constrained minima. If $x^\star$ lies in the interior of the feasible region, then all inequalities are inactive, complementary slackness forces $\lambda^\star = 0$, and stationarity reduces to $\nabla f(x^\star) = 0$. If $x^\star$ lies on the boundary, then stationarity says that $-\nabla f(x^\star)$ can be written as a linear combination of constraint gradients, with nonnegative coefficients on the active inequalities. In other words, any direction that decreases f must point (at least a little) out of the feasible set, and the multipliers encode how the active constraints "push back" against the objective gradient. In the unconstrained case ($m = p = 0$), (49) reduces to $\nabla f(x^\star) = 0$. See Figure 8 for a two-dimensional picture of this balance.

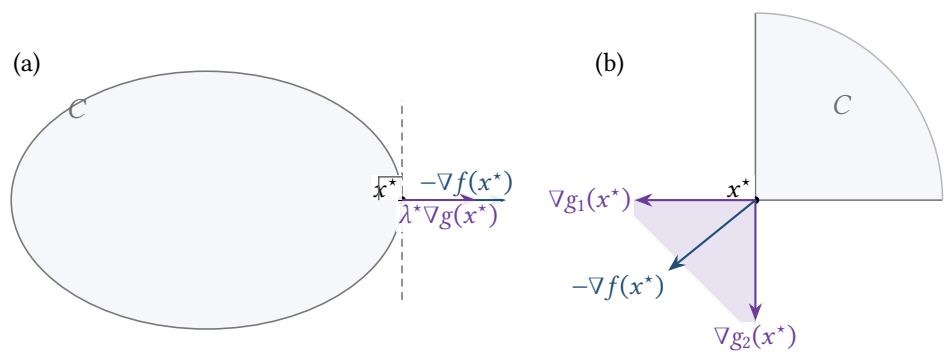

Figure 8: Geometry of KKT stationarity and complementary slackness. Left: with a single active inequality at a smooth boundary point $x^\star$, stationarity forces $-\nabla f(x^\star)$ to align with the constraint normal $\nabla g(x^\star)$, with some scale $\lambda^\star \ge 0$. Right: at a corner with two active inequalities, stationarity forces $-\nabla f(x^\star)$ to lie in the set of nonnegative combinations of the active normals $\nabla g_1(x^\star)$ and $\nabla g_2(x^\star)$. This same "normal direction" geometry underlies Euclidean projection steps, such as the projected update in Figure 7.

This same KKT geometry also explains the geometry of Euclidean projection. The projection $\Pi_C(z)$ is defined as the minimizer of $\min_{x \in C} \frac{1}{2}\|x - z\|_2^2$, which is a convex problem when C is convex. When C is described by differentiable constraints, applying the KKT stationarity equation to this projection problem shows that $z - \Pi_C(z)$ must be a linear combination of gradients of the active constraints at the projection

point. For example, if $C = \{x : g(x) \le 0\}$ with differentiable g and $x^\star = \Pi_C(z)$ lies on the boundary $g(x^\star) = 0$, then there exists $\lambda^\star \ge 0$ such that

$$x^\star - z + \lambda^\star \nabla g(x^\star) = 0, \qquad \text{so} \qquad z - x^\star = \lambda^\star \nabla g(x^\star).$$

This is exactly the normal-direction geometry illustrated in Figure 7: for a smooth boundary, the projection correction is orthogonal to the tangent.

Theorem 1.34 (KKT optimality conditions for convex problems)**.** *Consider the constrained problem* (48)*. Assume that f and g_i are convex, that each equality constraint h_j is affine (linear plus a constant), and that there exists a* strictly feasible *point $\tilde{x}$ with $g_i(\tilde{x}) < 0$ for all i and $h_j(\tilde{x}) = 0$ for all j (Slater's condition). Then $x^\star$ solves* (48) *if and only if there exist multipliers $\lambda^\star \in \mathbb{R}^m_+$ and $\nu^\star \in \mathbb{R}^p$ such that $(x^\star, \lambda^\star, \nu^\star)$ satisfies the KKT conditions* (49)*. See Boyd and Vandenberghe (2004, §5.5.3).*

Under Slater's condition, strong duality holds ($p^\star = d^\star$) and the KKT multipliers can be viewed as a dual certificate of optimality (zero duality gap); see Boyd and Vandenberghe (2004, §5.5.3). This is the basic *primal–dual* viewpoint: one seeks a primal variable x and dual multipliers (λ, ν) that simultaneously satisfy primal feasibility, dual feasibility, complementary slackness, and stationarity. The KKT system is precisely the set of equations and inequalities that couples these two sides of the problem.

The KKT conditions are not only a certificate of optimality; they can also be viewed as a system of equations to solve. A particularly transparent case is the equality constrained problem

$$\min_{x \in \mathbb{R}^d} f(x) \quad \text{s.t.} \quad Ax = b, \tag{50}$$

where $A \in \mathbb{R}^{p \times d}$ has full row rank $p < d$ and f is twice differentiable. Projected gradient descent can enforce feasibility via the closed-form projection in Lemma 1.28.

Newton's method applies the same feasibility idea to the local quadratic model of f. Assume x is feasible ($Ax = b$) and let $H = \nabla^2 f(x)$. The equality constrained Newton step Δx_{nt} is the minimizer of the quadratic model subject to preserving feasibility to first order, i.e. staying in the tangent space:

$$\Delta x_{\text{nt}} \in \arg\min_{\Delta x} \ \nabla f(x)^\top \Delta x + \frac{1}{2} \Delta x^\top H \Delta x \quad \text{s.t.} \quad A\Delta x = 0.$$

Introducing a Lagrange multiplier $\nu \in \mathbb{R}^p$ for the constraint $A\Delta x = 0$, the KKT conditions (see Section 1.6.5) yield the linear system

$$\begin{bmatrix} H & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} \Delta x_{\text{nt}} \\ \nu \end{bmatrix} = \begin{bmatrix} -\nabla f(x) \\ 0 \end{bmatrix}. \tag{51}$$

The top block row is the stationarity condition for the quadratic model (with the equality constraint incorporated via $A^\top \nu$), while the bottom block row enforces feasibility $A\Delta x_{\text{nt}} = 0$. The block matrix in (51) is called the (equality-constrained) KKT matrix. If A has full row rank and H is positive definite on $\mathcal{N}(A)$ (i.e. $\Delta x \ne 0$ and $A\Delta x = 0$ implies $\Delta x^\top H \Delta x > 0$), then this matrix is nonsingular and (51) has a unique solution; see Boyd and Vandenberghe (2004, §10.2). One typically combines Δx_{nt} with a backtracking line search on f (feasibility is preserved since $A\Delta x_{\text{nt}} = 0$).

1.6.6 Barrier methods

For inequality constraints, another common approach is to keep the iterates strictly inside the feasible region by adding a term that blows up at the boundary. One exact but nonsmooth way to express this is to

rewrite each inequality $g_i(x) \le 0$ with the indicator of the negative half-line,

$$I_{(-\infty,0]}(u) := \begin{cases} 0, & u \le 0, \\ +\infty, & u > 0. \end{cases}$$

Then the constrained problem can be written equivalently as minimizing

$$f(x) + \sum_{i=1}^{m} I_{(-\infty,0]}\big(g_i(x)\big) \qquad \text{subject to} \qquad Ax = b.$$

This makes the role of the inequalities completely explicit, but it is not suitable for Newton-like updates because the indicator is flat on the interior and singular at the boundary. The log barrier replaces this hard wall by the smooth surrogate $-(1/t)\log(-u)$, which is finite on the interior and diverges as $u \uparrow 0$. Multiplying the resulting surrogate objective by t leads to the standard barrier form $tf(x) + \phi(x)$. This is the standard starting point for barrier methods; see Boyd and Vandenberghe (2004, §11.3). The model example is the positive half-line $x > 0$, where the function $-\log x$ is mild in the interior and diverges as $x \downarrow 0$. Barrier methods use the same idea in higher-dimensional constrained problems. To make the discussion concrete, consider the convex problem

$$\min_{x \in \mathbb{R}^d} f(x) \quad \text{s.t.} \quad g_i(x) \le 0,\ i = 1, \dots, m, \qquad Ax = b, \tag{52}$$

where f and the g_i are convex and twice differentiable, and A has full row rank. We assume there exists a strictly feasible point $\tilde{x}$ with $g_i(\tilde{x}) < 0$ for all i and $A\tilde{x} = b$.

For the inequalities $g_i(x) \le 0$, the logarithmic barrier is

$$\phi(x) := -\sum_{i=1}^{m} \log\big(-g_i(x)\big), \qquad \operatorname{dom}\phi = \{x \ :\ g_i(x) < 0 \text{ for all } i\}.$$

For a parameter $t > 0$, define the barrier objective

$$F_t(x) := tf(x) + \phi(x).$$

The associated centering problem is

$$x^\star(t) \in \arg\min_x \ F_t(x) \quad \text{s.t.} \quad Ax = b. \tag{53}$$

The barrier term ϕ keeps iterates strictly feasible, while increasing t pushes the minimizer of F_t toward a solution of the original constrained problem.

For later use, it is helpful to record the derivatives of ϕ:

$$\nabla\phi(x) = -\sum_{i=1}^{m} \frac{1}{g_i(x)} \nabla g_i(x),$$
$$\nabla^2\phi(x) = \sum_{i=1}^{m} \left(\frac{1}{g_i(x)^2} \nabla g_i(x) \nabla g_i(x)^\top - \frac{1}{g_i(x)} \nabla^2 g_i(x) \right).$$

Because $g_i(x) < 0$, the coefficients $-1/g_i(x)$ are positive, so (under convexity of g_i) the barrier Hessian is positive semidefinite.

The minimizer $x^\star(t)$ of the centering problem traces out the *central path* as t varies. It also yields a dual feasible point and a simple duality-gap certificate. This central-path viewpoint and the resulting gap bound

are the core conclusions of Boyd and Vandenberghe (2004, §11.3). Let $p^\star$ denote the optimal value of (52). The KKT conditions for (53) state that there exists a multiplier $\hat{\nu}(t) \in \mathbb{R}^p$ such that

$$t\nabla f\big(x^\star(t)\big) + \nabla\phi\big(x^\star(t)\big) + A^\top \hat{\nu}(t) = 0, \qquad Ax^\star(t) = b. \tag{54}$$

Using the explicit form of $\nabla\phi$, we can rewrite stationarity as

$$\nabla f\big(x^\star(t)\big) + \sum_{i=1}^{m} \lambda_i(t)\,\nabla g_i\big(x^\star(t)\big) + A^\top \nu(t) = 0,$$

where we define

$$\lambda_i(t) := -\frac{1}{t\,g_i(x^\star(t))} > 0, \qquad \nu(t) := \frac{1}{t}\hat{\nu}(t).$$

Consider the Lagrangian for (52),

$$\mathcal{L}(x,\lambda,\nu) := f(x) + \sum_{i=1}^{m} \lambda_i g_i(x) + \nu^\top(Ax - b).$$

In the convex setting, $\lambda(t) \geq 0$ implies that $\mathcal{L}(x,\lambda(t),\nu(t))$ is convex in x. The stationarity equation above is exactly $\nabla_x \mathcal{L}(x^\star(t),\lambda(t),\nu(t)) = 0$. Since $\mathcal{L}(\cdot,\lambda(t),\nu(t))$ is convex and differentiable, this implies that $x^\star(t)$ minimizes $\mathcal{L}(\cdot,\lambda(t),\nu(t))$, so the dual function value $q(\lambda(t),\nu(t))$ from Section 1.6.5 is attained at $x^\star(t)$. Since $Ax^\star(t) = b$, this gives

$$q\big(\lambda(t),\nu(t)\big) = f\big(x^\star(t)\big) + \sum_{i=1}^{m} \lambda_i(t)\,g_i\big(x^\star(t)\big).$$

Moreover,

$$\sum_{i=1}^{m} \lambda_i(t)\,g_i\big(x^\star(t)\big) = \sum_{i=1}^{m} \left(-\frac{1}{t\,g_i(x^\star(t))}\right) g_i\big(x^\star(t)\big) = -\frac{m}{t},$$

so the duality gap for the primal–dual pair $\big(x^\star(t),\lambda(t),\nu(t)\big)$ equals m/t. In particular, $f(x^\star(t)) - p^\star \leq m/t$. Thus, in principle, computing a central point with $m/t \leq \varepsilon$ yields an ε-suboptimal solution of (52). See Boyd and Vandenberghe (2004, §11.2).

The barrier method computes approximate central points for an increasing sequence of t values, terminating once the duality-gap bound m/t is below a tolerance.

Algorithm 9 Barrier method (logarithmic barrier)

Input: Convex f and g_i, affine constraint $Ax = b$, strictly feasible x_0 with $g_i(x_0) < 0$
Input: Parameters $t_0 > 0$, $\mu > 1$, tolerance $\varepsilon > 0$

```
x ← x_0, t ← t_0
while m/t > ε do
    Approximately solve (53) starting at x (centering step), and update x to the computed solution
    t ← μt
end while
return x
```

Within each centering step, one can apply Newton's method to minimize F_t subject to $Ax = b$. At a current strictly feasible point x with $Ax = b$, let $r := \nabla F_t(x)$ and $H := \nabla^2 F_t(x)$. The equality-constrained Newton step solves the KKT linear system

$$\begin{bmatrix} H & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ w \end{bmatrix} = \begin{bmatrix} -r \\ 0 \end{bmatrix}. \tag{55}$$

One then updates $x \leftarrow x + \alpha\Delta x$ with a backtracking line search that (i) preserves strict feasibility $g_i(x + \alpha\Delta x) < 0$ and (ii) decreases F_t (e.g. using an Armijo condition). See Boyd and Vandenberghe (2004, §11.3).

2 Sampling

Sampling asks for expectations under a target law π, not just one good point. Exact independent sampling is often unavailable, so the problem is to design estimators and transition mechanisms that still approximate those expectations reliably. Stable probability calculations, proposal geometry, and gradient information remain central throughout, whether the method is i.i.d. Monte Carlo or MCMC. The recurring objects are the target law π, proposal objects q, and transition kernels K.

2.1 Monte Carlo methods

Monte Carlo methods use random samples to approximate expectations. The basic object in these notes is an expectation under some law p, because once we can draw i.i.d. samples from p, estimation becomes averaging. Ordinary numerical integration is one example of this same idea: after choosing a sampling density, an integral can be rewritten as an expectation. The rest of the subsection asks two practical questions: how to generate useful draws, and how to spend a fixed simulation budget well.

2.1.1 Monte Carlo estimation

The starting point is the law-of-large-numbers estimator for an expectation under a tractable sampling law. A basic task is to evaluate an integral

$$I = \int_D g(x)\,dx,$$

which can be rewritten as an expectation under any density p on $\mathcal{D}$ that is positive wherever g is nonzero:

$$I = \mathbb{E}_{X\sim p}\left[\frac{g(X)}{p(X)}\right].$$

In many applications we instead want

$$\mu = \mathbb{E}_{X\sim p}[f(X)],$$

where the target density is only known up to a normalizing constant, $p(x) = \tilde{p}(x)/Z$. Thus our two recurring goals are: (i) generate i.i.d. samples from p, and (ii) estimate μ from those samples. The second task is the one we ultimately care about, but the first task is often the main computational bottleneck. Much of Monte Carlo is about trading algorithmic effort for variance reduction so that m samples are as informative as possible.

If $X_1, \dots, X_m \sim p$ i.i.d. and $\mathbb{E}_p[f(X)^2] < \infty$, then the Monte Carlo estimator

$$\hat{\mu}_m = \frac{1}{m}\sum_{i=1}^{m} f(X_i)$$

converges almost surely to μ, and $\sqrt{m}(\hat{\mu}_m - \mu)$ is asymptotically normal with variance $\mathrm{Var}_p(f(X))$ (LLN/CLT). This implies a standard error that scales like $1/\sqrt{m}$, so doubling accuracy typically requires roughly four times as many samples unless we use variance reduction. Throughout this unit we assume we can numerically generate i.i.d. samples from $\mathrm{Unif}(0,1)$ and $\mathcal{N}(0,1)$.

2.1.2 Basic sampling constructions

Many Monte Carlo procedures start from the ability to sample from a target distribution. We record two classical one-dimensional constructions that we will reuse as building blocks later. Both methods illustrate a common theme: we trade analytic structure (closed-form CDFs or a tractable envelope) for efficient sampling.

Algorithm 10 Rejection sampling

Input: Unnormalized target $\tilde{p}(x)$, proposal density $g(x)$, constant M with $\tilde{p}(x) \le Mg(x)$

```
repeat
    Draw Y ~ g and U ~ Unif(0, 1).
until U ≤ p̃(Y)/(Mg(Y))
return Y
```

The first construction pushes uniform randomness through a cumulative distribution function.

Proposition 2.1 (Inverse CDF method)**.** *Suppose* $U \sim \mathrm{Unif}(0,1)$ *and* F *is a one-dimensional cumulative distribution function. Define*

$$X = F^{-1}(U) := \inf\{x : F(x) \ge U\}.$$

Then X *has cdf* F*.*

Proof. For any x, $\{X \le x\} = \{U \le F(x)\}$, so $\mathbb{P}(X \le x) = F(x)$. ■

The inverse CDF method is exact and simple, but it requires either a closed form for F^{-1} or a fast numerical inverse. In higher dimensions we often rely on conditional CDFs, transformations, or acceptance-rejection instead. See Liu (2002, §2.1) for additional discussion.

Example 2.2 (Exponential distribution)**.** For $\mathrm{Exp}(\lambda)$, the cdf is $F(x) = \left(1 - e^{-\lambda x}\right)\mathbf{1}\{x \ge 0\}$, so $F^{-1}(u) = -\lambda^{-1}\log(1-u)$. If $u_i \sim \mathrm{Unif}(0,1)$, then $x^{(i)} = -\lambda^{-1}\log(1-u_i)$ are i.i.d. $\mathrm{Exp}(\lambda)$.

The second construction samples from an easier envelope and keeps only the proposals that fall under the target. Suppose the target density is $p(x) = \tilde{p}(x)/Z$ where $\tilde{p}$ is an unnormalized density and $Z = \int \tilde{p}(x)\,dx$. Assume we can sample from a proposal density g and that there is a constant $M > 0$ with $\tilde{p}(x) \le Mg(x)$ for all x. Algorithm 10 draws a proposal and accepts it with probability $\tilde{p}(x)/(Mg(x))$; the accepted sample has density p. Indeed, the joint density of *accepted* draws is proportional to $\tilde{p}$, and the acceptance probability is Z/M. Equivalently, the expected number of proposals per accepted sample is M/Z, so choosing g (and thus M) controls efficiency. See Liu (2002, §2.2) for details.

Example 2.3 (Truncated Gamma tail)**.** We want $X \sim \Gamma(4,1)$ conditioned on $X > 10$. An unnormalized density is

$$\tilde{p}(x) = x^3 e^{-x}\mathbf{1}\{x > 10\}.$$

Take a shifted exponential proposal $Y = 10 + E$ with $E \sim \mathrm{Exp}(\lambda)$, i.e.

$$g(y) = \lambda e^{-\lambda(y-10)}\mathbf{1}\{y > 10\}.$$

Then

$$\frac{\tilde{p}(x)}{g(x)} = \frac{x^3}{\lambda}\exp\big(-(1-\lambda)x - 10\lambda\big).$$

Choosing $\lambda = \frac{1}{2}$ gives $\tilde{p}(x)/g(x) = 2x^3 e^{-x/2-5}$, which is decreasing for $x \ge 10$; thus we can take $M = 2 \cdot 10^3 e^{-10} = 2000e^{-10}$. The acceptance ratio for a proposed x is

$$\frac{\tilde{p}(x)}{Mg(x)} = 10^{-3}x^3 e^{-x/2+5} \le 1,$$

Algorithm 11 Self-normalized importance sampling

Input: Unnormalized target $\tilde{p}(x)$, proposal density $g(x)$, test function $h(x)$

1: Draw $X_1, \dots, X_m \sim g$ i.i.d.
2: Compute weights $\tilde{w}_i = \tilde{p}(X_i)/g(X_i)$ and normalize $w_i = \tilde{w}_i / \sum_{j=1}^m \tilde{w}_j$.
3: **return** $\hat{\mu} = \sum_{i=1}^m w_i h(X_i)$.

with equality at $x = 10$.

Rejection sampling works best when the proposal closely tracks the target and M is small. In high dimensions it can become impractical because acceptance rates decay quickly, especially if g underestimates the tails of p.

Example 2.4 (A multidimensional rejection construction)**.** Suppose the target is the uniform distribution on the unit disk

$$\mathcal{D} := \{x \in \mathbb{R}^2 : \|x\|_2 \le 1\}.$$

Draw $Y \sim \mathrm{Unif}([-1,1]^2)$ and accept when $Y \in \mathcal{D}$. Conditioned on acceptance, the draw is uniform on $\mathcal{D}$, and the acceptance probability is

$$\frac{\mathrm{area}(\mathcal{D})}{\mathrm{area}([-1,1]^2)} = \frac{\pi}{4}.$$

This simple example is the multivariate version of the one-dimensional picture: rejection sampling is still exact, but its efficiency is the ratio between the target-set volume and the envelope volume. For balls inside boxes that ratio deteriorates quickly with dimension, which is one concrete reason the method becomes fragile in high-dimensional problems.

2.1.3 Importance sampling

Importance sampling estimates expectations under p using samples from a more convenient proposal g. Assume $\mathrm{supp}(p) \subseteq \mathrm{supp}(g)$ and define the importance weight $w(x) = p(x)/g(x)$. Then

$$\mu = \mathbb{E}_{X\sim p}[h(X)] = \mathbb{E}_{X\sim g}[w(X)h(X)]\,.$$

Intuitively, $w(x)$ corrects for the fact that we are sampling from g instead of p. A simple estimator is

$$\tilde{\mu} = \frac{1}{m}\sum_{i=1}^m w(X_i)h(X_i), \qquad X_i \sim g.$$

When $p(x) = \tilde{p}(x)/Z$ and Z is unknown, we use the self-normalized estimator

$$\hat{\mu} = \frac{\sum_{i=1}^m \tilde{w}_i h(X_i)}{\sum_{i=1}^m \tilde{w}_i}, \qquad \tilde{w}_i = \frac{\tilde{p}(X_i)}{g(X_i)}.$$

The estimator $\hat{\mu}$ is generally biased for finite m, but it is consistent under standard moment conditions and is the natural choice when Z is unknown. Its finite-sample accuracy is still governed by weight variability: highly uneven weights make both the bias and the variance worse. See Liu (2002, §2.5).

A particularly useful case is when we already know how to sample from a base law π, and then new side information y arrives. If that information enters through a nonnegative likelihood factor $L_y(x)$, the new

target is the reweighted law

$$\pi^y(dx) := \frac{L_y(x)\,\pi(dx)}{Z_y}, \qquad Z_y := \int L_y(x)\,\pi(dx).$$

This is already interesting on its own: it answers a concrete Monte Carlo question. If we already have samples from π and then data arrive that change the target, can we recycle those samples rather than start the whole computation from scratch?

Example 2.5 (Reweighting existing samples after data arrive)**.** Assume $0 < Z_y < \infty$, and let h be π^y-integrable. Then

$$\int h(x)\,\pi^y(dx) = \frac{\int h(x)L_y(x)\,\pi(dx)}{\int L_y(x)\,\pi(dx)}. \tag{56}$$

So if $X_1, \dots, X_m \sim \pi$ are i.i.d., then

$$\int h(x)\,\pi^y(dx) \approx \frac{\sum_{i=1}^m L_y(X_i)\,h(X_i)}{\sum_{i=1}^m L_y(X_i)}.$$

In other words, conditioning is a special case of self-normalized importance sampling with proposal π, target π^y, and unnormalized weights

$$\tilde{w}_i = L_y(X_i).$$

The unknown constant Z_y cancels after normalization. If π is a prior law and $L_y(x) = p(y \mid x)$, then π^y is exactly the posterior law given y, and posterior expectations can be estimated by reweighting prior samples.

This conditioning-by-reweighting viewpoint will reappear whenever side information changes an existing sampling problem.

Importance sampling is only as useful as the weight computations behind it. Because those weights are ratios of potentially tiny densities, the next block collects the log-scale and transformed-density bookkeeping that will recur here, in Metropolis–Hastings acceptance ratios, in tempering, in SMC, and later in diffusion objectives.

2.1.4 Working on the log scale

The Monte Carlo calculations above already expose the main numerical issue for sampling. In floating-point arithmetic there are only finitely many representable numbers, so the computer does not literally manipulate the real numbers that appear in the mathematics. Probabilities are often tiny, likelihood ratios can differ by many orders of magnitude, and a naive translation of the algebra into floating-point arithmetic can therefore overflow, underflow, or round away the smaller of two competing contributions entirely. The log-weight checks around Algorithm 11 are the first example, and the same issue will return immediately in Markov-chain acceptance ratios and transformed target densities. Stable evaluation means rewriting the same mathematics so that the scale that matters for the algorithm remains visible to the machine. One repeatedly tries to cancel irrelevant common factors, keep intermediate quantities in a representable range, and avoid subtractions in which all significant digits are lost. In the sampling methods introduced so far, the algorithm usually needs a ratio, a normalization, a tail comparison, or a log normalizing constant rather than an absolute probability value.

The first recurring move is to stay on the log scale whenever the algorithm only cares about relative probabilities. Whenever an objective or density is built from products of positive terms, the logarithm is

usually the natural computational scale. Products become sums, ratios become differences, and a common multiplicative scale becomes an additive shift that can be removed without changing any normalized quantity. Suppose a calculation produces log weights $\ell_1, \dots, \ell_n$. The normalized weights are then

$$w_i = \frac{e^{\ell_i}}{\sum_{j=1}^n e^{\ell_j}},$$

and the same normalizing sum also determines the log normalizing constant $\log \sum_{j=1}^n e^{\ell_j}$. Directly exponentiating the ℓ_i can be numerically disastrous: large positive values can overflow, while very negative values can underflow to zero before the ratios among the smaller terms are ever compared. Writing

$$a := \max_i \ell_i$$

and subtracting this common shift before exponentiating gives

$$\log \sum_{i=1}^n e^{\ell_i} = a + \log \sum_{i=1}^n e^{\ell_i - a}. \tag{57}$$

This is the log-sum-exp identity. It simply factors out the largest scale e^a. After that shift, every exponent in (57) is at most zero, so each exponential lies in $(0, 1]$ and the sum is computed in a much safer floating-point range while preserving the relative sizes of the terms.

The same shift gives normalized weights directly:

$$w_i = \frac{e^{\ell_i - a}}{\sum_{j=1}^n e^{\ell_j - a}}.$$

If we want an average rather than a sum, then

$$\log \frac{1}{n} \sum_{i=1}^n e^{\ell_i} = -\log n + \log \sum_{i=1}^n e^{\ell_i},$$

which is often called a log-mean-exp calculation. This pattern is what keeps self-normalized importance sampling stable when only a few draws dominate, and it is equally useful in latent-variable models where responsibilities must be renormalized at every iteration. If only relative weights matter, one should remove the largest common scale before asking floating-point arithmetic to exponentiate anything. In practice one usually stays on the log scale until the last step that genuinely requires ordinary probabilities.

The same issue appears when a calculation ends with a probability close to 0 or 1. If a tail probability is so small that floating-point arithmetic rounds it to zero, then taking a logarithm afterwards loses all information. For that reason one often works directly with $\log F(x)$ or $\log(1 - F(x))$, depending on whether the lower or upper tail is the quantity of interest. In statistics software these are usually exposed as log-CDF or log-survival evaluations. Here the stable object is the logarithm of the probability, not the probability itself. Similarly, if $F(x)$ is extremely close to 1, then the subtraction $1 - F(x)$ may round to zero before the logarithm is taken. Stable log-survival calculations avoid that dangerous subtraction rather than trying to repair it afterwards. More generally, subtracting two nearly equal floating-point numbers is a common source of lost precision, so algebraically equivalent forms that avoid such subtractions are usually preferable.

This matters whenever the algorithm compares tail events rather than raw densities. Gaussian tail approximations, rejection envelopes, and likelihood-based tests can all fail in the far tails if the computation leaves the log scale too early. The same discipline also clarifies diagnostics: a histogram of log weights is often informative precisely because the logarithm turns multiplicative instability into additive spread that we can see and reason about. The same habit of carrying the dominant scale explicitly will reappear below when Metropolis–Hastings and Langevin-type samplers evaluate log targets and acceptance ratios.

2.1.5 Transformed densities and Jacobians

Often the numerically convenient coordinates are not the scientifically meaningful ones. We introduce an unconstrained variable z, optimize or sample there, and map back to the original parameter by $x = T(z)$. This is useful for positive parameters, simplex coordinates, and Cholesky factors, but it changes the density: once we change coordinates, a Jacobian term must be included to keep the target correct. The basic identity is the ordinary change-of-variables formula. If $x = T(z)$ is a smooth bijection, then

$$p_Z(z) = p_X\big(T(z)\big)\,|\det J_T(z)|.$$

Taking logarithms gives

$$\log p_Z(z) = \log p_X\big(T(z)\big) + \log|\det J_T(z)|. \tag{58}$$

Equation (58) is therefore just the change-of-variables formula written on the log scale. The transformed log density is the original log density evaluated at the mapped point, plus an additive volume correction. This additive form is also the stable one computationally: multiplicative Jacobian factors can vary over many orders of magnitude, while on the log scale they enter as one more term in a sum. The computational and probabilistic issues coincide here. We change coordinates because the transformed variable z may be easier to optimize over or sample in, but the Jacobian term is what makes that computational convenience represent the original model rather than a different one.

For a positive scalar parameter, the basic example is $x = e^z$. Then (58) becomes

$$\log p_Z(z) = \log p_X(e^z) + z.$$

The exponential map keeps x positive automatically, while the Jacobian term records how intervals on the z scale stretch when mapped back to the original variable. The same pattern extends to vector transforms, simplex maps, and Cholesky parameterizations. Whenever we optimize or sample in transformed coordinates, carrying the Jacobian term on the log scale is what keeps the target mathematically correct and numerically stable. This becomes especially important in latent-variable models and in gradient-based samplers, where transformed coordinates are often the difference between a feasible computation and an impossible one.

The same distinction matters after the computation is finished. If an optimizer or sampler is run on an unconstrained variable z but the scientific parameter of interest is $x = T(z)$, then summaries should usually be formed on the x scale by transforming the retained iterates first. For nonlinear maps, the order matters:

$$\mathbb{E}[T(Z)] \neq T(\mathbb{E}[Z])$$

in general, and the same is true for credible intervals and other nonlinear summaries. One therefore stores the draws or iterates in the computational coordinates that make the algorithm stable, but reports means, intervals, and other interpretable quantities on the model scale that the transformation defines.

Transformed objectives are also one of the most common places for implementation errors. If gradients are coded by hand, then a useful consistency check is to compare the analytic gradient of the log density or potential at a few representative points with a finite-difference derivative of that same scalar function. The finite-difference calculation is not meant to replace the final algorithm, and in high dimension it is far too expensive to use iteratively. Its role is narrower and more important: it catches missing Jacobian terms, sign mistakes, and other local errors before those errors are amplified by Newton updates, Langevin proposals, or leapfrog trajectories. These same ideas reappear whenever we sample in transformed coordinates and need the algorithmic convenience of the new parameterization to remain faithful to the original target. With that numerical bookkeeping in place, we can ask whether the proposal is statistically well matched to the target.

Figure 9: Proposal quality in importance sampling. The target $p(x)$ is shown in black and the proposal $g(x)$ in red. Good proposals match both modes and tails, whereas poorly aligned proposals place too much mass in regions where $p(x)$ is small. Overly diffuse proposals are workable but less efficient.

2.1.6 Proposal quality and effective sample size

When $X_i \sim p$ i.i.d., a CLT yields approximate confidence intervals $\bar{h} \pm 1.96\, s_h/\sqrt{m}$ for μ, where $\bar{h} = m^{-1}\sum_{i=1}^m h(X_i)$ and $s_h^2 = (m-1)^{-1}\sum_{i=1}^m (h(X_i) - \bar{h})^2$. For importance sampling, proposal quality is often summarized by how concentrated the normalized weights are. If all normalized weights are close to $1/m$, then every draw contributes comparably to the estimate. If one or two normalized weights dominate, then only a few draws are doing most of the work. This motivates the weight-based effective sample size proxy

$$\mathrm{ESS} \approx \frac{1}{\sum_{i=1}^m w_i^2},$$

where w_i now denotes the normalized importance weight. When p and g are normalized so that $\mathbb{E}_g[w(X)] = 1$, a large-sample heuristic gives

$$m \sum_{i=1}^m w_i^2 \approx \mathbb{E}_g[w(X)^2] = 1 + \mathrm{Var}_g(w(X)),$$

and therefore

$$\mathrm{ESS}(m) \approx \frac{m}{1 + \mathrm{Var}_g(w(X))}.$$

This is the weight-based notion of ESS: it measures how uneven the normalized importance weights are, not the autocorrelation ESS for correlated MCMC chains that appears later in Section 2.2.4. (See Liu (2002, §2.5.3).) If p and g are normalized densities, then $\mathrm{Var}_g(w(X)) = \chi^2(p\|g)$, where $\chi^2(p\|g) = \int (p(x) - g(x))^2/g(x)\,dx$ is the chi-squared divergence. Thus we want $w(X)$ to be as close to constant as possible, i.e., $g(x)$ should closely match $p(x)$ in both modes and tails. Figure 9 illustrates typical outcomes.

A useful heuristic is to choose g to approximate $|h(x)|p(x)$, which controls the variance of $w(X)h(X)$. In practice this often means matching the dominant modes and tail decay. If g is lighter-tailed than p, then a few extreme samples can dominate the estimator, leading to weight degeneracy. When $p(x) \propto e^{-q(x)}$ with smooth q, a Laplace approximation suggests

$$\nabla q(\mu) = 0, \qquad \Sigma = \left[\nabla^2 q(\mu)\right]^{-1},$$

and $g = \mathcal{N}(\mu, \Sigma)$. For multimodal targets, a mixture proposal can capture multiple modes; for heavy tails, a t_ν proposal is often safer.

A common workflow is:

(1) Start with a pilot proposal g and a moderate sample size m. Compute a rough ESS estimate (e.g., via the sample weights) and inspect a histogram of log-weights.

(2) If weights are highly skewed, increase the variance of g and/or add mixture components where large weights occur, then repeat step (1).

(3) Once the weights stabilize, run a larger batch for the final estimate.

The histogram is often more informative than raw weights, since it makes tail behavior and outliers visible.

2.1.7 Variance reduction

Monte Carlo error decreases as m grows, but we can often reduce variance further by exploiting structure. The idea is to transform an unbiased estimator into another unbiased estimator with smaller variance by subtracting (or conditioning on) a related quantity whose expectation is known. We summarize one widely used technique here; see Liu (2002, §2.3) for additional methods. Suppose we want $\mu = \mathbb{E}[f(X)]$ and we have another random variable Y with known expectation $\mathbb{E}[Y]$. For any $b \in \mathbb{R}$, the control-variate estimator

$$Z = f(X) + b\big(Y - \mathbb{E}[Y]\big)$$

has the same mean as $f(X)$. Its variance is

$$\mathrm{Var}(Z) = \mathrm{Var}(f(X)) + b^2\,\mathrm{Var}(Y) + 2b\ \mathrm{Cov}(f(X), Y).$$

The minimizer is

$$b^* = -\frac{\mathrm{Cov}(f(X), Y)}{\mathrm{Var}(Y)} = -\rho_{f(X),Y}\frac{\mathrm{sd}(f(X))}{\mathrm{sd}(Y)},$$

which yields

$$\mathrm{Var}(Z) = \big(1 - \rho^2_{f(X),Y}\big)\,\mathrm{Var}(f(X)) \le \mathrm{Var}(f(X)).$$

Thus strong (negative) correlation between $f(X)$ and Y produces the largest variance reduction. This guarantee is for the population-optimal coefficient b^*. In practice one often replaces b^* by an empirical estimate $\hat{b}$ computed from the same Monte Carlo sample. That plug-in estimator is often still useful, but finite-sample variance reduction is no longer automatic: if the correlation is weak, estimating $\hat{b}$ can slightly increase variance rather than decrease it.

Example 2.6 (Rare-event estimation with IS + control variates)**.** Consider $\theta = \mathbb{E}\big[e^X \mathbf{1}\{X > 3\}\big]$ with $X \sim \mathcal{N}(0, 1)$. Naive Monte Carlo is inefficient because $\mathbb{P}(X > 3)$ is tiny. An importance-sampling alternative draws $Y \sim \mathcal{N}(\mu, 1)$ for some $\mu > 3$, with weight

$$w(y) = \frac{\phi(y)}{\phi(y - \mu)} = \exp\big(-\mu y + \tfrac{1}{2}\mu^2\big),$$

and estimator

$$\tilde{\theta}_{\mathrm{IS}} = \frac{1}{m}\sum_{i=1}^{m} e^{Y_i}\,\mathbf{1}\{Y_i > 3\}\,w(Y_i), \qquad Y_i \sim \mathcal{N}(\mu, 1).$$

Here ϕ denotes the standard normal density. To further reduce variance, treat Y as a control variate with $\mathbb{E}[Y] = \mu$. Let $H(y) = e^y\,\mathbf{1}\{y > 3\}\,w(y)$ and define

$$\hat{\theta}_{\mathrm{IS+CV}} = \frac{1}{m}\sum_{i=1}^{m}\Big(H(Y_i) + b\,(Y_i - \mu)\Big).$$

A practical estimate of b is

$$\hat{b} = -\frac{\sum_{i=1}^{m}(H(Y_i) - \bar{H})(Y_i - \bar{Y})}{\sum_{i=1}^{m}(Y_i - \bar{Y})^2}, \qquad \bar{H} = \frac{1}{m}\sum_{i=1}^{m} H(Y_i),\ \bar{Y} = \frac{1}{m}\sum_{i=1}^{m} Y_i.$$

In this example, choosing $\mu \approx 4$ yields a noticeable variance reduction in practice. Empirical estimation of b can slightly increase variance, and the benefit is strongest when $H(Y)$ and Y are strongly (typically negatively) correlated.

2.2 Markov chains, invariant distributions, and ergodicity

In many problems our goal is to compute expectations under a distribution π on $\mathbb{R}^d$,

$$\mu := \mathbb{E}_{X\sim\pi}[g(X)],$$

where π may be known only up to a normalizing constant. If we can draw i.i.d. samples from π, then the basic Monte Carlo estimator is immediate. However, in moderate or high dimensions, designing good i.i.d. proposals can be difficult; for instance, rejection sampling can have vanishing acceptance probabilities and importance sampling can suffer from weight degeneracy (see Section 2.1.2).

Markov chain Monte Carlo (MCMC) takes a different approach. Instead of sampling i.i.d., we simulate a Markov chain whose *marginal* distribution converges to π. The resulting samples are correlated, but if the chain mixes well then empirical averages of $g(X_n)$ can still be accurate. Thus an MCMC method is a choice of a one-step transition rule K that is (i) easy to simulate, (ii) has invariant distribution π, and (iii) moves through the state space efficiently.

Definition 2.7 (Transition kernel)**.** Let $\mathcal{X} = \mathbb{R}^d$ and let $\mathcal{B}(\mathcal{X})$ denote its Borel σ-algebra. A *transition kernel* is a function

$$K : \mathcal{X} \times \mathcal{B}(\mathcal{X}) \to [0,1], \qquad (x, A) \mapsto K(x, A),$$

such that: (i) for each fixed $x \in \mathcal{X}$, the map $A \mapsto K(x, A)$ is a probability measure on $\mathcal{B}(\mathcal{X})$, and (ii) for each fixed $A \in \mathcal{B}(\mathcal{X})$, the map $x \mapsto K(x, A)$ is measurable.

The interpretation is that $K(x, A)$ is the probability that one step of the chain moves into the set A when the current state is x. We often write $K(x, dy)$ as a notational reminder that, for each fixed x, $K(x, \cdot)$ is a measure on $\mathcal{X}$.

If $K(x, \cdot)$ is absolutely continuous with respect to Lebesgue measure, we write $k(y \mid x)$ for a conditional density and

$$K(x, A) = \int_A k(y \mid x)\, dy.$$

More generally, for any measurable $f : \mathcal{X} \to \mathbb{R}$ with $\int |f(y)|\, K(x, dy) < \infty$, define the associated Markov operator

$$(Kf)(x) := \int_{\mathcal{X}} f(y)\, K(x, dy).$$

Then $(Kf)(x)$ is the conditional expectation $\mathbb{E}[f(X_{n+1}) \mid X_n = x]$ when (X_n) has transition kernel K.

Definition 2.8 (Markov chain with kernel K)**.** A sequence $(X_n)_{n\geq 0}$ is a (time-homogeneous) Markov chain with transition kernel K if for every $n \geq 0$ and every $A \in \mathcal{B}(\mathcal{X})$,

$$\mathbb{P}(X_{n+1} \in A \mid X_0, \dots, X_n) = \mathbb{P}(X_{n+1} \in A \mid X_n) = K(X_n, A)$$

almost surely.

Equivalently, conditional on $X_n = x$, we draw the next state as

$$X_{n+1} \mid X_n = x \sim K(x, \cdot).$$

An initial distribution for X_0, together with K, determines the joint law of $(X_0, X_1, \dots)$. We focus on discrete-time, time-homogeneous chains because they are directly implementable, though many MCMC methods are motivated by continuous-time Markov processes and their discretizations.

2.2.1 Transition kernels and iterated kernels

Let K be a transition kernel. Define the m-step kernel K^m recursively by $K^1 = K$ and

$$K^{m+1}(x, A) := \int_{\mathcal{X}} K^m(y, A)\, K(x, dy).$$

Then $K^m(x, A) = \mathbb{P}(X_{n+m} \in A \mid X_n = x)$, and the recursion is the discrete-time Chapman–Kolmogorov equation. If μ is a distribution for X_0, the marginal distribution after m steps is the measure

$$(\mu K^m)(A) := \int_{\mathcal{X}} K^m(x, A)\, \mu(dx).$$

Thinking in these terms makes the MCMC objective precise: we want μK^m to approach π as m grows, even when μ is far from π. Continuous-time Markov processes use the same notation with a time index: for a time-homogeneous process $(X_t)_{t\geq 0}$, one writes

$$K_t(x, A) := \mathbb{P}(X_{s+t} \in A \mid X_s = x), \qquad t \geq 0,$$

which does not depend on s. Then $K_0(x, A) = \mathbf{1}\{x \in A\}$, and the Chapman–Kolmogorov equation becomes

$$K_{s+t}(x, A) = \int_{\mathcal{X}} K_t(y, A)\, K_s(x, dy), \qquad s, t \geq 0.$$

This is the continuous-time analogue of the discrete iterated-kernel recursion above.

Example 2.9 (Gaussian AR(1) chain)**.** Fix $\rho \in (-1, 1)$ and let $(\varepsilon_n)_{n\geq 1}$ be i.i.d. $\mathcal{N}(0, 1)$. Define

$$X_{n+1} = \rho X_n + \sqrt{1 - \rho^2}\, \varepsilon_{n+1}.$$

Then $K(x, \cdot) = \mathcal{N}(\rho x,\ 1 - \rho^2)$. Iterating the recursion shows that for every deterministic starting point $X_0 = x_0$,

$$X_n \sim \mathcal{N}\left(\rho^n x_0,\ 1 - \rho^{2n}\right).$$

In particular, if $X_0 = 0$, then $X_n \sim \mathcal{N}(0, 1 - \rho^{2n})$, and if $X_0 \sim \mathcal{N}(0, 1)$, then $X_n \sim \mathcal{N}(0, 1)$ for all n, so $\pi = \mathcal{N}(0, 1)$ is an invariant distribution.

Example 2.10 (Ornstein–Uhlenbeck process)**.** The Ornstein–Uhlenbeck (OU) process is a continuous-time Markov process with a Gaussian transition kernel. In a standard parameterization, if $X_s = x$ then

$$X_{s+t} \mid X_s = x \sim \mathcal{N}\left(e^{-t}x,\ 1 - e^{-2t}\right), \qquad t \geq 0.$$

Equivalently, for a deterministic initial value $X_0 = x_0$,

$$X_t \sim \mathcal{N}\left(e^{-t}x_0,\ 1 - e^{-2t}\right).$$

Thus from $X_0 = 0$ the transient marginal is $\mathcal{N}(0, 1 - e^{-2t})$, which relaxes to stationarity as $t \to \infty$. The corresponding stationary distribution is $\mathcal{N}(0, 1)$.

2.2.2 Invariant distributions and Monte Carlo averages

The central design goal in MCMC is to build a kernel K whose invariant distribution is the target distribution π. In terms of the operator $\mu \mapsto \mu K$ defined by

$$(\mu K)(A) := \int_{\mathcal{X}} K(x, A)\, \mu(dx),$$

invariance simply means that π is a fixed point: $\pi K = \pi$.

Definition 2.11 (Invariant distribution)**.** A probability measure π on $\mathcal{X}$ is *invariant* (or *stationary*) for K if for every $A \in \mathcal{B}(\mathcal{X})$,

$$\pi(A) = \int_{\mathcal{X}} K(x, A)\, \pi(dx).$$

Equivalently, if $X \sim \pi$ and $Y \mid X = x \sim K(x, \cdot)$, then $Y \sim \pi$.

Proposition 2.12 (Starting from π gives stationarity)**.** *If $(X_n)_{n\geq 0}$ is a Markov chain with kernel K and invariant distribution π, and $X_0 \sim \pi$, then $X_n \sim \pi$ for all $n \geq 0$.*

Proof. The claim holds for $n = 0$ by assumption. Assume $X_n \sim \pi$. Then for any measurable A,

$$\mathbb{P}(X_{n+1} \in A) = \mathbb{E}\big[\mathbb{P}(X_{n+1} \in A \mid X_n)\big] = \mathbb{E}\big[K(X_n, A)\big] = \int_{\mathcal{X}} K(x, A)\, \pi(dx) = \pi(A),$$

where the last equality is invariance. ■

Stationarity means that all one-dimensional marginals are π, but the samples are generally *not* independent. This dependence is central to how we analyze the efficiency of MCMC estimators.

2.2.3 Reversibility and detailed balance

Many MCMC kernels are constructed by enforcing a stronger symmetry property than invariance.

Definition 2.13 (Detailed balance)**.** A kernel K is *reversible* with respect to π if for all measurable sets $A, B \in \mathcal{B}(\mathcal{X})$,

$$\int_A K(x, B)\, \pi(dx) = \int_B K(x, A)\, \pi(dx).$$

Equivalently, the joint distribution of (X_0, X_1) is symmetric when (X_n) is stationary: if $X_0 \sim \pi$ and $X_1 \mid X_0 \sim K(X_0, \cdot)$, then (X_0, X_1) has the same law as (X_1, X_0).

Detailed balance implies invariance by taking $A = \mathcal{X}$. Reversibility is not necessary for MCMC, but it is a convenient sufficient condition and will be the main tool behind Metropolis–Hastings kernels.

2.2.4 Autocorrelation and effective sample size

Let $(X_n)_{n\geq 0}$ be stationary with $X_n \sim \pi$ and let $g : \mathcal{X} \to \mathbb{R}$ satisfy $\mathbb{E}_\pi[g(X)^2] < \infty$. We estimate $\mu = \mathbb{E}_\pi[g(X)]$ by the empirical average

$$\hat{\mu}_n := \frac{1}{n} \sum_{i=1}^{n} g(X_i).$$

Then $\mathbb{E}[\hat{\mu}_n] = \mu$, but $\mathrm{Var}(\hat{\mu}_n)$ is typically larger than the i.i.d. variance $\mathrm{Var}_\pi(g(X))/n$ because of serial correlation.

Define the autocovariance $\gamma_k = \mathrm{Cov}_\pi(g(X_0), g(X_k))$. By expanding the variance of a sum and using stationarity,

$$\mathrm{Var}(\hat{\mu}_n) = \frac{1}{n^2} \sum_{i=1}^{n} \sum_{j=1}^{n} \mathrm{Cov}\big(g(X_i), g(X_j)\big) = \frac{1}{n^2} \left(n\gamma_0 + 2 \sum_{k=1}^{n-1} (n-k)\gamma_k \right). \tag{59}$$

In the i.i.d. case, the off-diagonal covariances vanish and $\mathrm{Var}(\hat{\mu}_n) = \gamma_0/n$. This suggests defining the *effective sample size* by matching variances:

$$N_{\mathrm{eff}}(n, g) := \frac{\mathrm{Var}_\pi(g(X_0))}{\mathrm{Var}(\hat{\mu}_n)} = \frac{\gamma_0}{\mathrm{Var}(\hat{\mu}_n)}.$$

When the samples are i.i.d., $N_{\mathrm{eff}}(n, g) = n$. When there is positive serial correlation, $N_{\mathrm{eff}}(n, g)$ can be much smaller than n.

Writing $\rho_k = \gamma_k/\gamma_0$ (when $\gamma_0 > 0$) gives

$$\mathrm{Var}(\hat{\mu}_n) = \frac{\gamma_0}{n}\left(1 + 2\sum_{k=1}^{n-1}\left(1 - \frac{k}{n}\right)\rho_k\right).$$

If the series $\sum_{k\geq 1} \rho_k$ is summable, the coefficient in parentheses stabilizes as n grows. This motivates the *integrated autocorrelation time*

$$\tau_{\mathrm{int}} := 1 + 2\sum_{k=1}^{\infty} \rho_k,$$

and the corresponding *effective sample size*

$$N_{\mathrm{eff}} \approx \frac{n}{\tau_{\mathrm{int}}}.$$

The interpretation is that n correlated MCMC draws provide roughly the same estimation accuracy as N_{eff} i.i.d. samples from π. Both τ_{int} and N_{eff} depend on the function g; a chain can mix quickly for some summaries of the state while remaining highly correlated for others. This is a different notion from the weight-based ESS in importance sampling and SMC. Here ESS measures the loss of information caused by serial correlation in one chain, whereas there it measures how uneven a set of normalized weights has become. This autocorrelation-based ESS formula is standard in MCMC diagnostics; see Betancourt (2017, §2.2, p. 11).

Example 2.14 (Integrated autocorrelation time for the AR(1) chain)**.** Consider the stationary AR(1) chain from Example 2.9 and take $g(x) = x$. Then $\rho_k = \rho^k$, so

$$\tau_{\mathrm{int}} = 1 + 2\sum_{k=1}^{\infty} \rho^k = 1 + \frac{2\rho}{1-\rho} = \frac{1+\rho}{1-\rho}.$$

Thus $N_{\mathrm{eff}} \approx n(1-\rho)/(1+\rho)$. When ρ is close to 1, successive states are extremely similar and the effective sample size can be much smaller than n.

2.2.5 Ergodicity, mixing time, and burn-in

In practice we rarely start the chain exactly from π. Early iterates still remember the initialization, so averages formed too soon can be biased by where the chain started rather than by the target law itself. Burn-in is the attempt to discard enough of those early steps that this memory has mostly washed out. To make that idea precise, we ask that $K^m(x, \cdot)$ approach π as $m \to \infty$ for a wide range of starting states x. Invariance alone does not guarantee this: a chain can have π invariant but fail to reach it from some initial states, or oscillate without settling down.

Algorithm 12 Generic MCMC estimation with burn-in

Input: Transition kernel K with invariant distribution π, initial state $x^{(0)}$, burn-in B, total steps T, test function g

```
for t = 0, 1, ..., T − 1 do
    Draw x^(t+1) ~ K(x^(t), ·)
end for
Discard x^(0), ..., x^(B) and keep x^(B+1), ..., x^(T)
return μ̂ = (T − B)^−1 Σ_{t=B+1}^{T} g(x^(t))
```

Definition 2.15 (Total variation distance)**.** For probability measures μ, ν on $(\mathcal{X}, \mathcal{B}(\mathcal{X}))$, their total variation distance is

$$\|\mu - \nu\|_{\mathrm{TV}} := \sup_{A \in \mathcal{B}(\mathcal{X})} \left|\mu(A) - \nu(A)\right|.$$

Total variation is a worst-case discrepancy over events: it controls how much any probability statement can change when we replace μ by ν. If μ and ν have densities p and q, then

$$\|\mu - \nu\|_{\mathrm{TV}} = \frac{1}{2} \int_{\mathcal{X}} |p(x) - q(x)| \, dx.$$

A standard way to summarize convergence speed from a particular starting state x is the ε-mixing time

$$t_{\mathrm{mix}}(\varepsilon; x) := \min\{m \geq 0 \,:\, \|K^m(x, \cdot) - \pi\|_{\mathrm{TV}} \leq \varepsilon\}.$$

On unbounded state spaces, the more stringent "worst-case over x" notion can be infinite; what matters in practice is convergence from the initializations we actually use. Under geometric ergodicity (Definition 2.16), one has $\|K^m(x, \cdot) - \pi\|_{\mathrm{TV}} \leq M(x)\rho^m$, so it is enough to take m on the order of $\log(M(x)/\varepsilon)/\log(1/\rho)$. The burn-in length B in Algorithm 12 is an attempt to choose $B \gtrsim t_{\mathrm{mix}}(\varepsilon; x^{(0)})$ for some small ε, but in most realistic problems we cannot compute t_{mix} and instead rely on diagnostic checks. This separation between early bias reduction and later variance reduction is the same qualitative warm-up picture emphasized in Betancourt (2017, §2.2, pp. 11–12).

Definition 2.16 (Geometric ergodicity)**.** A Markov chain with transition kernel K and invariant distribution π is *geometrically ergodic* if there exist $\rho \in (0, 1)$ and a function $M : \mathcal{X} \to [0, \infty)$ such that for all $m \geq 0$,

$$\|K^m(x, \cdot) - \pi\|_{\mathrm{TV}} \leq M(x)\rho^m.$$

Geometric ergodicity is a strong form of convergence: it says the distribution contracts at a linear (geometric) rate in total variation. The Gaussian AR(1) chain in Example 2.9 and the OU process are clean examples of this behavior: their linear drift contracts toward the origin while the noise prevents collapse.

Discarding burn-in reduces bias from initialization, but it does not make the remaining samples independent. The residual serial dependence is what drives the variance inflation in Equation (59) and is summarized by τ_{int} (or N_{eff}).

When comparing MCMC algorithms, two recurring questions are: (i) how long it takes for the chain to approach stationarity (mixing time / burn-in), and (ii) how much correlation remains after convergence (which controls N_{eff} via Equation (59)). We return to practical diagnostics for both questions in Section 2.3.3.

Algorithm 13 Metropolis–Hastings

Input: Unnormalized target $\tilde{\pi}(x)$, proposal density $q(y \mid x)$, initial state $x^{(0)}$, steps T

```
for t = 0, 1, ..., T − 1 do
    Draw proposal y ~ q(· | x^(t)) and u ~ Unif(0, 1).
    Compute α(x^(t), y) from (60).
    if u ≤ α(x^(t), y) then
        x^(t+1) ← y
    else
        x^(t+1) ← x^(t)
    end if
end for
return x^(0), x^(1), ..., x^(T)
```

2.3 Markov chain Monte Carlo

The next three internal topics instantiate the abstract kernel framework: first accept–reject and conditional-update constructions, then practical diagnostics, and finally gradient-based chains.

2.3.1 Metropolis–Hastings

Metropolis–Hastings turns an easy-to-simulate proposal kernel into an MCMC kernel that leaves a target distribution π invariant. The key correction is an accept/reject step that enforces detailed balance, so the method still works when π is known only up to a normalizing constant. See Liu (2002, §5.1–§5.2).

Let the target density on $\mathcal{X} = \mathbb{R}^d$ be

$$\pi(x) = \frac{\tilde{\pi}(x)}{Z}, \qquad Z = \int_{\mathbb{R}^d} \tilde{\pi}(x)\, dx.$$

Let $Q(x, \cdot)$ be a proposal kernel with density $q(y \mid x)$. Given the current state x, we draw $Y \sim Q(x, \cdot)$ and accept the proposal with probability

$$\alpha(x, y) := \min\left\{1,\ \frac{\tilde{\pi}(y)\, q(x \mid y)}{\tilde{\pi}(x)\, q(y \mid x)}\right\}. \tag{60}$$

Because (60) only involves a ratio of unnormalized target values, the unknown constant Z cancels. The resulting Markov kernel is

$$K_{\mathrm{MH}}(x, A) = \int_A q(y \mid x)\alpha(x, y)\, dy + \mathbf{1}\{x \in A\}\left(1 - \int_{\mathbb{R}^d} q(z \mid x)\alpha(x, z)\, dz\right). \tag{61}$$

The first term moves to an accepted proposal in A, while the second term is the probability of rejecting and staying at x. Figure 10 sketches this move-or-stay structure.

Proposition 2.17 (Metropolis–Hastings satisfies detailed balance)**.** *The kernel K_{MH} in (61) is reversible with respect to π. In particular, π is invariant for K_{MH}.*

Proof. Define

$$r(x, y) := \frac{\tilde{\pi}(y)\, q(x \mid y)}{\tilde{\pi}(x)\, q(y \mid x)}, \qquad \alpha(x, y) = \min\{1, r(x, y)\}.$$

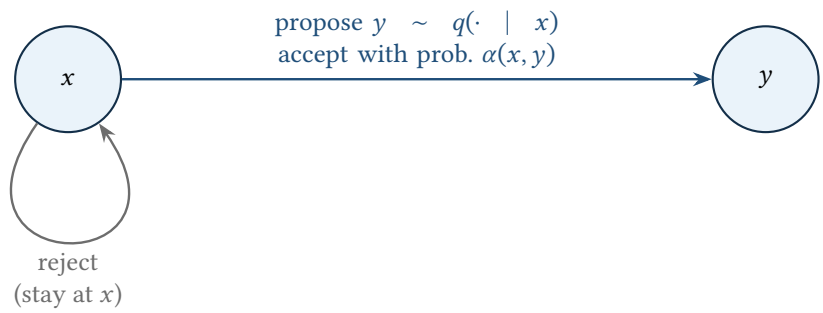

Figure 10: Schematic form of a Metropolis–Hastings step. A proposal $y \sim q(\cdot \mid x)$ is accepted with probability $\alpha(x, y)$; otherwise the chain stays at x.

Then

$$\tilde{\pi}(x)\, q(y \mid x)\alpha(x, y) = \min\{\tilde{\pi}(x)\, q(y \mid x), \tilde{\pi}(y)\, q(x \mid y)\} = \tilde{\pi}(y)\, q(x \mid y)\alpha(y, x).$$

Dividing by Z gives

$$\pi(x)\, q(y \mid x)\alpha(x, y) = \pi(y)\, q(x \mid y)\alpha(y, x).$$

Integrating this identity over $x \in A$ and $y \in B$ yields detailed balance for the move part of (61), and the reject-and-stay mass preserves the same symmetry. ■

For implementation, the stable quantity is the log ratio rather than the ratio itself. As in Section 2.1.4, it is usually safest to compute

$$\log r(x, y) = \log \tilde{\pi}(y) - \log \tilde{\pi}(x) + \log q(x \mid y) - \log q(y \mid x)$$

and accept whenever $\log u \leq \min\{0, \log r(x, y)\}$. The same change-of-variables bookkeeping from Section 2.1.5 applies when the chain is run in unconstrained coordinates. If $x = T(z)$ is a smooth bijection and we sample on the z scale, then the unnormalized target becomes

$$\tilde{\pi}_Z(z) = \tilde{\pi}_X\big(T(z)\big)\, |\det J_T(z)|, \qquad \log \tilde{\pi}_Z(z) = \log \tilde{\pi}_X\big(T(z)\big) + \log|\det J_T(z)|. \tag{62}$$

Equation (60) must then be applied with $\tilde{\pi}_Z$, not with the original density in the constrained coordinates. Otherwise the sampler targets the wrong distribution even if every proposal is accepted or rejected correctly on its own computational scale.

Proposal design is the real modeling choice inside Metropolis–Hastings. The accept/reject step guarantees the correct stationary law, but the proposal kernel determines whether the chain moves coherently through the typical set or wastes most iterations on rejected or highly correlated proposals. Recording the acceptance rate is therefore useful, since it gives a quick indication of whether the proposal is too conservative or too aggressive; see Liu (2002, §5.8).

A standard choice is random-walk Metropolis:

$$Y = x + \varepsilon, \qquad \varepsilon \sim g,$$

with proposal density $q(y \mid x) = g(y - x)$. If g is symmetric, then the proposal terms cancel and (60) simplifies to

$$\alpha(x, y) = \min\left\{1, \frac{\tilde{\pi}(y)}{\tilde{\pi}(x)}\right\}.$$

See Liu (2002, §5.4.1). The method is simple, but the same local scale must serve every direction. If the steps are too small, acceptance is high but autocorrelation remains large; if the steps are too large, most proposals land in low-density regions and are rejected. In practice one tunes the proposal scale by monitoring both acceptance and the resulting effective sample size (Section 2.2.4).

Random-walk Metropolis theory is driven by a simple geometric theme. The proposal must have genuine local support, and the target must not force the chain through increasingly thin bottlenecks in the tails. Those hypotheses appear in many theorem-level conditions with different technical wrappers, but the practical message is the same: random-walk proposals work best when the level sets are reasonably regular and when no direction becomes arbitrarily harder to traverse than the others. This is also why detailed balance, by itself, is not enough. It guarantees the correct stationary law, but it does not guarantee a usable convergence rate. Targets with heavy tails, strong anisotropy, or narrow tail funnels can all make a perfectly correct random walk mix very poorly.

A practical workflow is to tune the proposal in stages rather than hoping one scale works everywhere. One usually starts with an isotropic Gaussian proposal,

$$Y \sim \mathcal{N}(x, \sigma^2 I_d),$$

with $\sigma^2 = 2.4^2/d$ as a common default scale, runs a short pilot chain, discards warm-up, and uses the retained draws to estimate both an acceptance rate and a pilot covariance. As a rough diagnostic, acceptance rates around 50% in one dimension and around 25% in higher-dimensional problems are often reasonable starting points. If the observed acceptance rate is too small, the proposal scale should be shrunk; if it is too large, the scale can be increased. When the pilot draws reveal strong anisotropy, a better next proposal is

$$Y \sim \mathcal{N}\left(x, \frac{1}{d}\hat{\Sigma}\right),$$

where $\hat{\Sigma}$ is the empirical covariance from the pilot run. The right pattern is then *pilot run → estimate covariance → restart the chain with the new proposal*, repeating until the mixing behavior is acceptable.

This workflow also explains why reparameterization is often more valuable than endless scalar tuning. If the negative log density is ill-conditioned, isotropic random walks face the same steep-versus-flat tradeoff as gradient descent: a small step size is needed to avoid rejection in stiff directions, while a large step size is needed to move in flat directions. Rescaling or whitening the parameters, choosing a proposal that respects constraints, or moving to unconstrained coordinates via Section 2.1.5, can turn a badly tuned random walk into a workable one. One should not adapt $\hat{\Sigma}$ indefinitely inside the main run without care, since that makes the transition rule depend on the past and therefore breaks the simple Markov-chain analysis unless one uses a genuine adaptive-MCMC scheme.

Another useful family is the independence proposal, where $q(y \mid x) = g(y)$ does not depend on the current state. Then

$$\alpha(x, y) = \min\left\{1, \frac{\tilde{\pi}(y)\, g(x)}{\tilde{\pi}(x)\, g(y)}\right\}.$$

See Liu (2002, §5.4.2). Independence proposals can mix quickly when g already approximates π well, but they fail for exactly the same reason that importance sampling fails: if g misses important modes or has lighter tails than π, the correction weights become extreme and accepted moves are rare.

2.3.2 Gibbs sampling

Gibbs sampling updates one block of variables at a time by drawing from the corresponding full conditional distribution. It is most attractive when those conditionals are available in closed form, often because of conjugacy or simple Gaussian structure. See Liu (2002, §6.1).

Algorithm 14 Systematic-scan Gibbs sampler

Input: Target density $\pi(x_1, \dots, x_d)$ up to normalization, initial state $x^{(0)}$, sweeps T

```
for t = 0, 1, ..., T − 1 do
    for i = 1, 2, ..., d do
        Sample x_i^(t+1) ~ π(· | x_1^(t+1), ..., x_{i−1}^(t+1), x_{i+1}^(t), ..., x_d^(t)).
    end for
end for
return {x^(t)}_{t=0}^T
```

Let $X = (X_1, \dots, X_d)$ have target density π on $\mathbb{R}^d$. For each coordinate i, write x_{-i} for the vector of all coordinates except x_i. The full conditional density of X_i given $X_{-i} = x_{-i}$ is

$$\pi(x_i \mid x_{-i}) \propto \pi(x_1, \dots, x_d),$$

viewed as a function of x_i with x_{-i} fixed. If $\pi(x) \propto \tilde{\pi}(x)$ is known only up to a global normalizing constant, that constant cancels inside the conditional as well. Gibbs sampling is therefore a natural choice when conditional sampling is easy even though direct sampling from π is not.

The systematic-scan Gibbs sampler cycles through the coordinates and redraws each one from its full conditional: Within one sweep, the already updated coordinates use their time-$(t+1)$ values while the remaining coordinates stay at time t. In two dimensions this means alternating horizontal and vertical moves through the level sets of the target; Figure 11 shows the basic geometry for a correlated Gaussian. The random-scan version uses different bookkeeping: at step t, draw an index $I_t \in \{1, \dots, d\}$ from probabilities $(w_1, \dots, w_d)$, update only coordinate I_t from its full conditional, and leave all other coordinates unchanged. If K_i denotes the kernel that refreshes coordinate i, then one random-scan step has kernel

$$K_{\mathrm{RS}} = \sum_{i=1}^{d} w_i K_i,$$

while one systematic sweep has kernel

$$K_{\mathrm{SS}} = K_d \cdots K_2 K_1.$$

Both target the same invariant law π, but they produce different short-lag dependence and different notions of what counts as "one iteration" in practice. Their convergence behavior can nevertheless be dramatically different: strong posterior dependence can make successive coordinate refreshes nearly deterministic, while sensible blocking can make a full sweep behave much more like an independent refresh. That is why blocking matters so much: the more nearly a full sweep behaves like a sequence of weakly correlated refreshes, the closer the chain moves toward i.i.d. behavior.

Proposition 2.18 (A Gibbs coordinate update leaves π invariant)**.** *Fix $i \in \{1, \dots, d\}$ and consider the kernel K_i that maps $x = (x_i, x_{-i})$ to (x_i', x_{-i}) with $x_i' \sim \pi(\cdot \mid x_{-i})$. Then π is invariant for K_i. Consequently, both systematic-scan Gibbs (Algorithm 14) and random-scan Gibbs have invariant distribution π.*

Proof. Let f be bounded and measurable. By definition of K_i,

$$(K_i f)(x_i, x_{-i}) = \int f(x_i', x_{-i})\, \pi(x_i' \mid x_{-i})\, dx_i'.$$

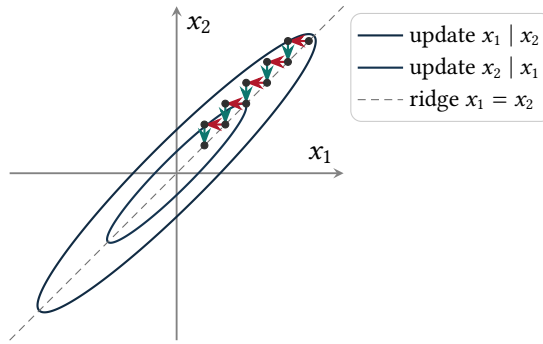

Figure 11: Geometry of a two-coordinate Gibbs sampler. The blue ellipses are level sets of a correlated Gaussian target. A Gibbs sweep alternates between sampling $x_1 \mid x_2$ (red horizontal moves) and sampling $x_2 \mid x_1$ (teal vertical moves).

Integrating against $\pi(x_i, x_{-i}) = \pi(x_i \mid x_{-i})\pi(x_{-i})$ gives

$$\begin{aligned}\int (K_i f)(x)\,\pi(x)\,dx &= \int \left[\int f(x_i', x_{-i})\,\pi(x_i' \mid x_{-i})\,dx_i'\right] \pi(x_i \mid x_{-i})\pi(x_{-i})\,dx_i\,dx_{-i} \\ &= \int f(x_i', x_{-i})\,\pi(x_i' \mid x_{-i})\pi(x_{-i})\,dx_i'\,dx_{-i} \\ &= \int f(x)\,\pi(x)\,dx.\end{aligned}$$

■

Gibbs updates can be read as a special Metropolis–Hastings proposal that is always accepted. If we update coordinate i by proposing $y = (y_i, x_{-i})$ with $y_i \sim \pi(\cdot \mid x_{-i})$, then

$$q(y \mid x) = \pi(y_i \mid x_{-i}), \qquad q(x \mid y) = \pi(x_i \mid x_{-i}),$$

and the Hastings ratio equals 1 because each conditional is proportional to the corresponding slice of $\tilde{\pi}$. This viewpoint motivates Metropolis-within-Gibbs: when one full conditional is unavailable, we can still update that block with an MH step targeting the conditional while holding the other coordinates fixed; see Liu (2002, §6.3.2).

Example 2.19 (Correlated Gaussian Gibbs mixing)**.** Let $(X_1, X_2) \sim \mathcal{N}(0, \Sigma)$ with

$$\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}, \qquad |\rho| < 1.$$

Then

$$X_1 \mid X_2 = x_2 \sim \mathcal{N}\left(\rho x_2,\ 1 - \rho^2\right), \qquad X_2 \mid X_1 = x_1 \sim \mathcal{N}\left(\rho x_1,\ 1 - \rho^2\right).$$

When $|\rho|$ is close to 1, the conditional variance $1 - \rho^2$ is small, so each update makes only a short axis-aligned move. Geometrically, the chain traces a staircase path along the narrow ridge of the target instead of moving directly along that ridge. If we record X_1 after each full sweep, the resulting marginal chain is

$$X_{1,t+1} = \rho^2 X_{1,t} + \sqrt{1 - \rho^4}\,\varepsilon_{t+1}, \qquad \varepsilon_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1),$$

so the lag-k autocorrelation is $(\rho^2)^k$. Thus, as $|\rho| \to 1$, successive Gibbs iterates become highly correlated and the effective sample size decays (cf. Section 2.2.4).

This Gaussian example is the right template for thinking about Gibbs more generally. The acceptance rate is always 1, so the main efficiency question is not acceptance but autocorrelation. When strongly coupled variables are updated one at a time, the chain can only move through the posterior by a sequence of short axis-aligned corrections.

Example 2.20 (Hierarchical Gibbs sampler and why blocking helps)**.** Suppose grouped data Y_{ij} are modeled by

$$Y_{ij} \mid \alpha_j, \sigma^2 \sim \mathcal{N}(\alpha_j, \sigma^2), \qquad \alpha_j \mid \mu, \tau^2 \sim \mathcal{N}(\mu, \tau^2),$$

for groups $j = 1, \dots, J$ and observations $i = 1, \dots, n_j$. Give the hyperparameters the conjugate priors

$$\mu \sim \mathcal{N}(m_0, s_0^2), \qquad \sigma^2 \sim \text{InvGamma}(a_\sigma, b_\sigma), \qquad \tau^2 \sim \text{InvGamma}(a_\tau, b_\tau).$$

Writing

$$\bar{Y}_j := \frac{1}{n_j} \sum_{i=1}^{n_j} Y_{ij}, \qquad S_j := \sum_{i=1}^{n_j} (Y_{ij} - \bar{Y}_j)^2,$$

the full conditionals are

$$\alpha_j \mid \mu, \tau^2, \sigma^2, Y \sim \mathcal{N}\left(\frac{n_j \bar{Y}_j / \sigma^2 + \mu/\tau^2}{n_j/\sigma^2 + 1/\tau^2}, \frac{1}{n_j/\sigma^2 + 1/\tau^2} \right),$$

$$\mu \mid \alpha, \tau^2, Y \sim \mathcal{N}\left(\frac{m_0/s_0^2 + \sum_{j=1}^J \alpha_j/\tau^2}{1/s_0^2 + J/\tau^2}, \frac{1}{1/s_0^2 + J/\tau^2} \right),$$

$$\sigma^2 \mid \alpha, \mu, \tau^2, Y \sim \text{InvGamma}\left(a_\sigma + \frac{1}{2} \sum_{j=1}^J n_j, b_\sigma + \frac{1}{2} \sum_{j=1}^J \left[S_j + n_j (\bar{Y}_j - \alpha_j)^2 \right] \right),$$

and

$$\tau^2 \mid \alpha, \mu, \sigma^2, Y \sim \text{InvGamma}\left(a_\tau + \frac{J}{2}, b_\tau + \frac{1}{2} \sum_{j=1}^J (\alpha_j - \mu)^2 \right).$$

A straightforward Gibbs sampler updates $\alpha_1, \dots, \alpha_J$, then μ, then σ^2, then τ^2 one at a time. That sampler is easy to code, but it can mix poorly because the group means α_j, the global mean μ, and the between-group scale τ^2 are strongly correlated in the posterior. Each single-site update moves only a little, and the chain spends many sweeps walking back and forth along the same posterior ridge. This is especially visible when the groups are weakly informed, for example when each n_j is small, because then the posterior coupling between μ and the α_j remains strong and autocorrelations decay slowly. If we collect the Gaussian variables into one block,

$$\theta := (\mu, \alpha_1, \dots, \alpha_J),$$

then $\theta \mid \tau^2, \sigma^2, Y$ is jointly Gaussian with analytically available mean and covariance. In matrix form, this block conditional can be written as

$$\theta \mid \tau^2, \sigma^2, Y \sim \mathcal{N}(Q^{-1} b, Q^{-1}),$$

where

$$Q = \begin{bmatrix} s_0^{-2} + J\tau^{-2} & -\tau^{-2}\mathbf{1}_J^\top \\ -\tau^{-2}\mathbf{1}_J & \tau^{-2}I_J + \sigma^{-2}\operatorname{diag}(n_1, \dots, n_J) \end{bmatrix}, \qquad b = \begin{bmatrix} m_0/s_0^2 \\ \sigma^{-2}n_1\bar{Y}_1 \\ \vdots \\ \sigma^{-2}n_J\bar{Y}_J \end{bmatrix}.$$

We can therefore replace many correlated one-dimensional Gaussian updates by a single multivariate draw for θ, followed by the inverse-gamma draws for σ^2 and τ^2. This blocked Gibbs step moves in the correlated directions all at once, so the lag-one autocorrelation drops sharply. In practice the blocked chain is often much closer to an i.i.d. sequence in its autocorrelation plot, and the resulting Monte Carlo variance for estimating quantities such as $\mathbb{E}[\mu \mid Y]$ is correspondingly smaller than for the one-at-a-time Gibbs chain.

The lesson is not merely "use bigger blocks." The real principle is to block together variables that are strongly coupled *when the joint conditional is still easy to sample.* Blocking is worthwhile precisely because it attacks autocorrelation at its source. State augmentation is another way to create such tractable blocks: introducing latent variables can turn an awkward marginal problem into a sequence of easy conditional updates. If a clean joint conditional is still unavailable, then the next best option is often Metropolis-within-Gibbs: use exact Gibbs updates on the tractable blocks and an MH step on the rest.

2.3.3 Multi-chain diagnostic workflow

In realistic problems we usually assess convergence and Monte Carlo accuracy from finite runs rather than from a closed-form mixing calculation. The standard workflow is concrete: run several chains from dispersed initial conditions, discard warm-up, choose a few scalar summaries $h(X)$, and inspect trace plots, split-$\hat{R}$, effective sample size, and interval estimates for those summaries. These diagnostics are tools for detecting pathologies rather than certificates that the chain has explored every region where π places mass. In particular, diagnostics only see the regions the chain has already visited, so a deceptive run can still miss an isolated mode or a narrow neck. This is why empirical checks complement, rather than replace, the structural arguments from the Markov-chain theory subsection; see Betancourt (2017, §2.2–§2.3) for the conceptual role of diagnostics and Vehtari et al. (2021, §1.1) for the modern multi-chain workflow.

The standard workflow is to run C independent chains from dispersed initial conditions. For chain $c \in \{1, \dots, C\}$, run $b + m$ iterations, discard the first b warm-up steps, and retain

$$X_{1,c}, \dots, X_{m,c} \in \mathbb{R}^d.$$

Diagnostics are then applied to scalar summaries of the state, such as a coordinate projection $h(x) = x_j$ or the log target $h(x) = \log \tilde{\pi}(x)$. Write

$$h_{i,c} := h(X_{i,c}), \qquad i = 1, \dots, m.$$

For each summary h, it is useful to report the empirical mean and central intervals from the pooled post-warm-up draws, and then place beside them the Monte Carlo diagnostics for that same summary: split-$\hat{R}$, effective sample size, and often an autocorrelation plot. The interval summarizes uncertainty under the target distribution, while the diagnostics summarize how reliably the finite correlated simulation has estimated that uncertainty.

In practice a simple routine works well:

(1) Run multiple chains, often $C = 4$, from dispersed initial conditions and use a conservative warm-up.

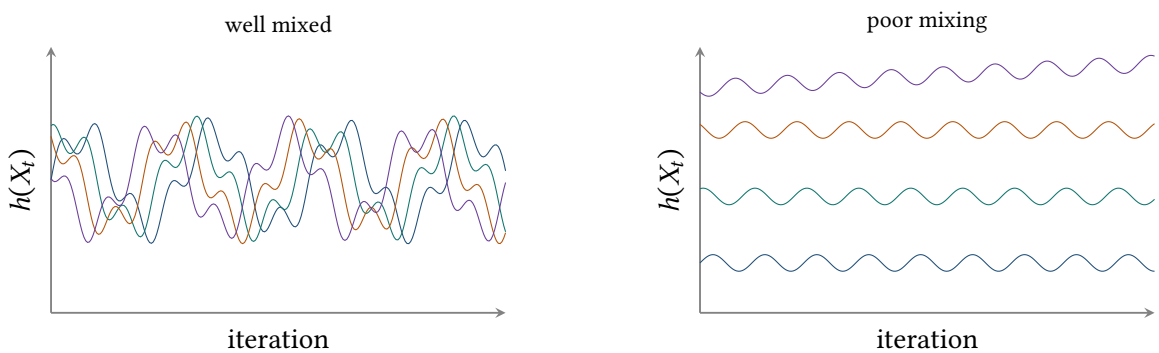

Figure 12: Schematic trace plots for four chains. The left panel shows good mixing, with chains overlapping and no visible drift. The right panel shows poor mixing, with chains stuck in different regions and slow trends that suggest non-stationarity.

(2) Inspect trace plots and pair plots for a few coordinates, transformed parameters, and the log target.

(3) Compute split-$\hat{R}$ and effective sample size for each coordinate and for scientifically meaningful derived summaries.

(4) If the diagnostics look good, extend the chains only to reduce Monte Carlo error.

(5) If the diagnostics look bad, change the algorithm, the parameterization, or the initialization strategy rather than merely increasing m.

Trace plots overlay $h_{i,c}$ against iteration i for each chain. Well-mixed chains look like overlapping noisy bands with no visible trend, while poor mixing appears as drift, stickiness, long periods trapped in different regions, or chains that never overlap at all. Pair plots complement this one-dimensional view by revealing narrow ridges, strong correlations, and multimodality that make a sampler move slowly. Figure 12 sketches the most basic visual distinction.

When diagnostics suggest poor mixing, the first response should not be to simply run longer. If the chain has trouble moving between regions of high probability, additional samples often repeat the same information. Common remedies include tuning proposal scales, improving warm-up and initializations, or reparameterizing the model so the sampler sees less extreme correlations and fewer funnel-like geometries. The next example is a canonical warning sign.

Example 2.21 (Neal's funnel)**.** Let $y \sim \mathcal{N}(0, 3^2)$ and, conditional on y, let $x \in \mathbb{R}^K$ have independent coordinates

$$x_j \mid y \sim \mathcal{N}(0, e^y), \qquad j = 1, \dots, K.$$

When y is negative, the conditional variance e^y is tiny and the mass in x concentrates in a narrow neck near 0. When y is positive, the conditional variance is enormous and the mass spreads out. This creates the funnel-shaped geometry in Figure 13: generic local proposals must trade off moving through the neck against moving efficiently in the wide part of the state space. In practice the pathology shows up as poor trace plots, small effective sample sizes, and split-$\hat{R}$ values noticeably above 1.
A standard remedy is a non-centered parameterization. If we write $x = e^{y/2}z$ with $z \sim \mathcal{N}(0, I_K)$ independent of y, then the joint law of (z, y) is a product of Gaussians and the narrow neck disappears. For hierarchical models, this often changes the sampler's geometry more than any tuning adjustment.

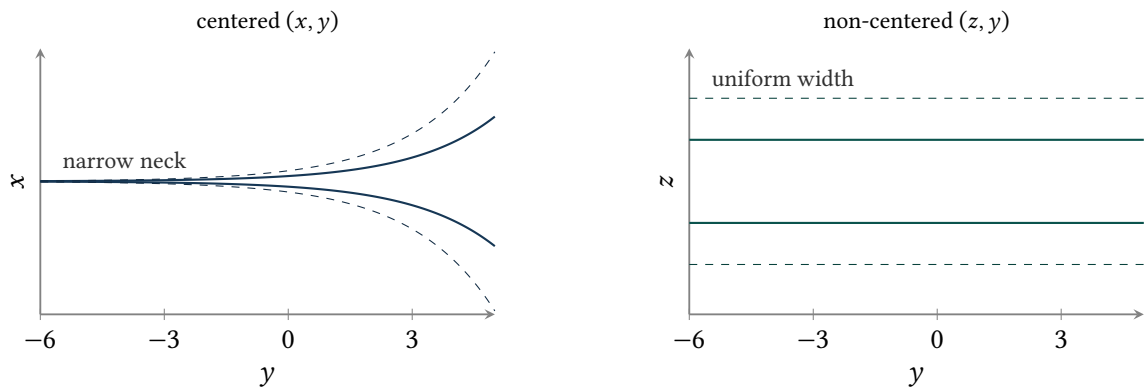

Figure 13: Neal's funnel in the centered parameterization (left) and the corresponding non-centered reparameterization (right) for $K = 1$. In (x, y) coordinates the density concentrates in a narrow neck when y is negative, while spreading out rapidly when y is positive. Writing $x = e^{y/2}z$ yields independent (z, y) and removes the funnel-shaped geometry.

2.3.4 Split-$\hat{R}$ and effective sample size

Once the multi-chain workflow is in place, two scalar diagnostics summarize the two questions from Section 2.2.5: have the chains reached the same stationary regime, and how much Monte Carlo information remains after accounting for autocorrelation? The first question is addressed by split-$\hat{R}$ and the second by effective sample size; see Betancourt (2017, §2.2–§2.3) for the conceptual interpretation and Vehtari et al. (2021, §§3.2–4.5) for the refinements used in modern implementations.

For each retained chain, define the within-chain mean and variance

$$\bar{h}_c := \frac{1}{m}\sum_{i=1}^{m} h_{i,c}, \qquad s_c^2 := \frac{1}{m-1}\sum_{i=1}^{m}(h_{i,c} - \bar{h}_c)^2,$$

and let $\bar{h} := C^{-1}\sum_{c=1}^{C}\bar{h}_c$. The within-chain and between-chain variance estimates are

$$W := \frac{1}{C}\sum_{c=1}^{C} s_c^2, \qquad B := \frac{m}{C-1}\sum_{c=1}^{C}(\bar{h}_c - \bar{h})^2.$$

The classical potential scale reduction factor is

$$\hat{R} := \sqrt{\frac{\hat{V}}{W}}, \qquad \hat{V} := \frac{m-1}{m}W + \frac{1}{m}B. \tag{63}$$

If all chains are exploring the same stationary law, then B and W should be similar and $\hat{R}$ should be close to 1. If different chains are still trapped in different regions, B is inflated and $\hat{R}$ grows above 1.

In practice one usually computes the split version. Each chain is cut in half, the $2C$ half-chains are treated as separate chains of length $m/2$, and (63) is applied to those half-chains. This makes the diagnostic sensitive to slow drifts that would be hidden if each chain were summarized only by its full-run mean. Values such as split-$\hat{R} \lesssim 1.01$ are common targets, but the threshold is not magic and should be read together with the trace plots rather than in isolation. Equation (63) is the classical variance-ratio formula because it makes the idea transparent. In practice, implementations usually report the rank-normalized split-$\hat{R}$ together

with bulk and tail ESS summaries, since those are more reliable for heavy-tailed or skewed posteriors; see Vehtari et al. (2021, §§4.1–4.5).

Effective sample size measures the second question: once the chain has reached stationarity, how much Monte Carlo precision is lost to serial dependence? For each chain c, define the lag-k sample autocovariance

$$\tilde{\gamma}_c(k) := \frac{1}{m-k} \sum_{i=1}^{m-k} (h_{i,c} - \bar{h}_c)(h_{i+k,c} - \bar{h}_c), \qquad k = 0, 1, \dots, m-1,$$

and the corresponding sample autocorrelation $\tilde{\rho}_c(k) := \tilde{\gamma}_c(k)/\tilde{\gamma}_c(0)$. A simple multi-chain aggregate is

$$\tilde{\rho}(k) := \frac{1}{C} \sum_{c=1}^{C} \tilde{\rho}_c(k).$$

Two mundane implementation details are worth recording. Some software centers the lag products with the pooled mean over all chains rather than the within-chain mean $\bar{h}_c$, and many implementations normalize the lag-k autocovariance by m rather than $m-k$. Those choices introduce a small bias but can reduce variance in the very noisy large-lag regime. Large-lag estimates are noisy, so the usual practice is to truncate the sum once the estimated autocorrelations stop behaving like a decaying signal. One conservative rule is to choose the first lag T with $\tilde{\rho}(T) < 0$ and then define the variance-inflation estimate using only the preceding nonnegative lags:

$$\hat{\tau}_{\text{int}} := 1 + 2 \sum_{k=1}^{T-1} \tilde{\rho}(k), \qquad \text{ESS} := \frac{mC}{\hat{\tau}_{\text{int}}}. \tag{64}$$

This is the empirical version of the integrated autocorrelation-time calculation from Section 2.2.4. More stable multi-chain ESS estimators combine this truncation idea with between-chain variance information; see Hoffman and Gelman (2014, App. A) and Vehtari et al. (2021, §3.2). ESS is always summary-dependent: a sampler may mix rapidly for one coordinate and extremely slowly for another or for a nonlinear function of the state. That is why diagnostics should be reported for transformed parameters and scientifically relevant summaries, not only for the raw coordinates stored internally by the algorithm.

2.3.5 Langevin diffusion and the unadjusted Langevin algorithm

Many modern targets can be written in energy form,

$$\pi(x) \propto e^{-U(x)}, \qquad x \in \mathbb{R}^d,$$

where $U : \mathbb{R}^d \to \mathbb{R}$ is differentiable. The basic random-walk Metropolis construction from Section 2.3.1 treats π as a black box: proposals are centered at the current state and do not use any derivative information about where the density is increasing. When gradients are available, we can do better. The vector $-\nabla U(x)$ points toward higher-probability regions, so it gives a local directional hint that random-walk proposals ignore. Langevin methods use that hint while still adding noise, so the algorithm samples from the distribution instead of simply descending toward a mode.

The conceptual starting point is the overdamped Langevin diffusion

$$dX_t = -\nabla U(X_t)\, dt + \sqrt{2}\, dB_t, \tag{65}$$

where $(B_t)_{t \geq 0}$ is standard Brownian motion in $\mathbb{R}^d$. For the broader Langevin-to-SGLD viewpoint, see Welling and Teh (2011, §§2–3). The drift $-\nabla U(X_t)$ points downhill in the potential and therefore toward regions where π is larger, while the Brownian term prevents the process from collapsing onto a mode. For small dt,

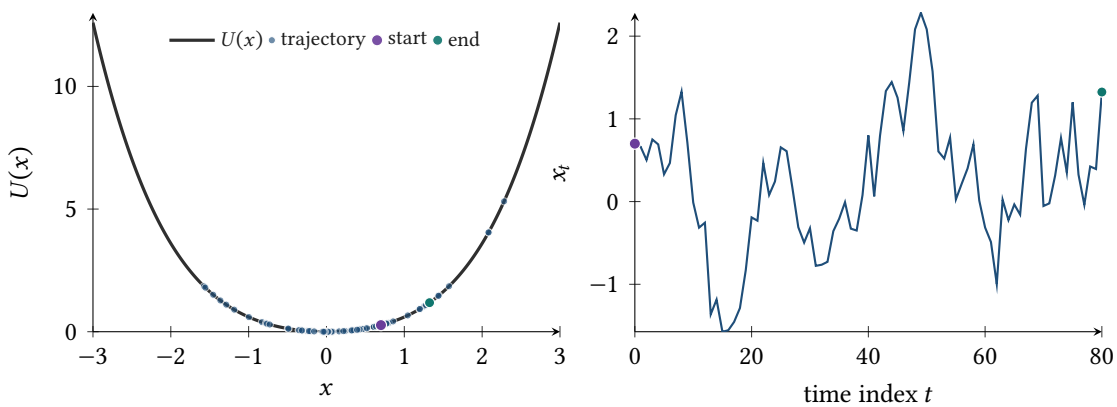

Figure 14: A one-dimensional potential U and a ULA trajectory for one fixed noise realization. The drift term $-\nabla U(x)$ pulls the iterate toward low-energy regions, while the Gaussian noise provides exploration.

the drift contributes an $O(dt)$ deterministic move and the noise contributes an $O(\sqrt{dt})$ random perturbation. This combination is exactly what distinguishes sampling from optimization: the drift alone would behave like gradient descent, but the added noise keeps the process exploring the typical set rather than converging to a single maximizer. To even formulate (65) cleanly, one usually assumes that the drift is globally Lipschitz, for example

$$\|\nabla U(x) - \nabla U(y)\| \le L\|x - y\| \qquad \text{for all } x, y \in \mathbb{R}^d.$$

Under this condition the SDE is well posed: there is a unique global solution, there is no finite-time blow-up, and the law of the process depends continuously on the initial condition. If $U \in C^2$, this is implied by a uniform Hessian bound $\|\nabla^2 U(x)\| \le L$.

Applying Euler–Maruyama to (65) with step size $\epsilon > 0$ gives the unadjusted Langevin algorithm (ULA),

$$x_{t+1} = x_t - \epsilon \nabla U(x_t) + \sqrt{2\epsilon}\, \xi_{t+1}, \qquad \xi_{t+1} \sim \mathcal{N}(0, I_d), \tag{66}$$

with Gaussian one-step kernel

$$K_\epsilon^{\mathrm{ULA}}(x, dy) = \mathcal{N}(y;\ x - \epsilon \nabla U(x),\ 2\epsilon I_d)\, dy.$$

ULA is immediately implementable and often mixes much better than a pure random walk because the drift discourages proposals deep in low-density regions. At the same time, (66) is only a discretization of (65), so for fixed ϵ it usually targets an ϵ-dependent approximation to π rather than π itself. Reducing ϵ decreases this bias but increases autocorrelation because successive states become more similar. The practical takeaway is therefore simple: under the standard regularity assumptions used to define the diffusion and its Fokker–Planck equation, the continuous-time Langevin diffusion preserves π exactly, while ULA is its cheap Euler discretization and inherits a step-size bias. Figure 14 shows the drift-plus-noise geometry in one dimension.

For the continuous-time process it is useful to introduce the associated transition kernel

$$K_t(x, A) := \mathbb{P}(X_t \in A \mid X_0 = x),$$

and the Markov semigroup acting on test functions,

$$(P_t g)(x) := \mathbb{E}[g(X_t) \mid X_0 = x] = \int g(y)\, K_t(x, dy).$$

Then invariance means $\pi P_t = \pi$ for every $t \geq 0$, and the generator is the infinitesimal derivative

$$\mathcal{L}g = \lim_{t \downarrow 0} \frac{P_t g - g}{t}$$

on smooth functions. The kernels satisfy the continuous-time Chapman–Kolmogorov relation

$$K_{s+t}(x, A) = \int K_t(y, A)\, K_s(x, dy), \qquad s, t \geq 0,$$

or equivalently $P_{s+t} = P_s P_t$. This semigroup viewpoint is the clean way to organize the next calculation: the generator describes the local drift-diffusion balance, while the semigroup describes how those infinitesimal moves accumulate over time.

The next calculation records the infinitesimal balance behind that exactness. At stationarity, the inward drift and the outward diffusion cancel one another so that the target law is preserved rather than collapsed onto the mode. For a smooth test function g, the generator of (65) is

$$(\mathcal{L}g)(x) = -\langle \nabla U(x) \rangle \nabla g(x) + \Delta g(x), \tag{67}$$

and the corresponding Fokker–Planck equation for the density f_t of X_t is

$$\partial_t f_t(x) = \nabla \cdot \big(f_t(x) \nabla U(x) \big) + \Delta f_t(x) = \nabla \cdot \big(f_t(x) \nabla U(x) + \nabla f_t(x) \big). \tag{68}$$

The probability flux in (68) vanishes when $f_t(x) \propto e^{-U(x)}$, because then $\nabla f_t = -f_t \nabla U$. This is the infinitesimal reason the diffusion preserves the Gibbs density.

Proposition 2.22 (Generator identity for the Langevin Gibbs law)**.** *Assume U is differentiable and e^{-U} is integrable on $\mathbb{R}^d$. Let $\pi(x) = Z^{-1} e^{-U(x)}$. If boundary terms vanish for the test functions of interest, then*

$$\int (\mathcal{L}g)(x)\, \pi(x)\, dx = 0$$

for every smooth test function g with sufficient decay.

Proof. Let g be smooth with sufficient decay. Using (67),

$$\int (\mathcal{L}g)(x)\, \pi(x)\, dx = \frac{1}{Z} \int \left[-\langle \nabla U(x) \rangle \nabla g(x) + \Delta g(x) \right] e^{-U(x)}\, dx.$$

Integration by parts gives

$$\int (\Delta g) e^{-U}\, dx = -\int \langle \nabla g \rangle \nabla (e^{-U})\, dx = \int \langle \nabla g \rangle \nabla U\, e^{-U}\, dx, \qquad \nabla(e^{-U}) = -e^{-U} \nabla U,$$

so the two terms cancel and $\int (\mathcal{L}g)\, \pi = 0$. ■

This generator identity is the infinitesimal stationarity condition. Combined with the standard well-posedness assumptions that justify the Fokker–Planck equation and its stationary solutions, it yields invariance of π for the Langevin semigroup because the flux in (68) vanishes when $f_t = \pi$.

To quantify convergence, it is convenient to work with couplings. A *coupling* of probability measures μ and ν on $\mathbb{R}^d$ is a joint law γ on $\mathbb{R}^d \times \mathbb{R}^d$ whose marginals are μ and ν. The 2-Wasserstein distance is then

$$W_2(\mu,\nu) := \inf_{\gamma \in \mathcal{C}(\mu,\nu)} \left(\int \|x-y\|^2 \, \gamma(dx,dy) \right)^{1/2},$$

where $\mathcal{C}(\mu,\nu)$ denotes the set of all couplings. This metric is natural for Langevin diffusions because synchronous couplings give direct pathwise control of $\|X_t - Y_t\|$. Earlier, in Section 2.2.5, we used total variation because it controls discrepancies of arbitrary events for general chains. Here the dynamics have an explicit Euclidean geometry, so W_2 is often the more informative metric: it is built from couplings of whole trajectories and interacts directly with convexity of the drift.

Under strong convexity the diffusion also forgets its initialization at an exponential rate. The cleanest proof uses synchronous coupling: if two copies of the diffusion share the same Brownian motion, then the noise cancels in their difference and only the contracting drift remains. For a probability measure μ, we write μK_t for the law obtained by evolving the initial distribution μ under the Langevin diffusion for time t.

Proposition 2.23 (Synchronous coupling gives exponential Wasserstein contraction)**.** *Assume ∇U is globally Lipschitz and that U is m-strongly convex, meaning*

$$\langle \nabla U(x) - \nabla U(y) \rangle x - y \geq m\|x-y\|^2 \qquad \text{for all } x, y \in \mathbb{R}^d.$$

Let (X_t) and (Y_t) be the corresponding unique strong solutions to (65) driven by the same Brownian motion and started from $X_0 = x, Y_0 = y$. Then for all $t \geq 0$,

$$\|X_t - Y_t\| \leq e^{-mt}\|x-y\|.$$

Consequently, for any probability measures μ, ν on $\mathbb{R}^d$,

$$W_2(\mu K_t, \nu K_t) \leq e^{-mt} W_2(\mu,\nu),$$

where K_t denotes the Langevin transition kernel at time t.

Proof. With the same Brownian motion, the difference $Z_t := X_t - Y_t$ satisfies the integral equation

$$Z_t = Z_0 - \int_0^t \big(\nabla U(X_s) - \nabla U(Y_s)\big)\, ds.$$

Hence Z_t is absolutely continuous, and for almost every t,

$$\frac{d}{dt}\|Z_t\|^2 = -2\langle Z_t \rangle \nabla U(X_t) - \nabla U(Y_t) \leq -2m\|Z_t\|^2.$$

Gronwall's inequality yields $\|Z_t\| \leq e^{-mt}\|Z_0\|$. Now let γ_0 be any coupling of μ and ν, draw $(X_0, Y_0) \sim \gamma_0$, and evolve both coordinates with the same Brownian motion. Then (X_t, Y_t) is a coupling of μK_t and νK_t, so

$$W_2^2(\mu K_t, \nu K_t) \leq \mathbb{E}\|X_t - Y_t\|^2 \leq e^{-2mt}\mathbb{E}\|X_0 - Y_0\|^2.$$

Taking the infimum over all initial couplings γ_0 gives

$$W_2^2(\mu K_t, \nu K_t) \leq e^{-2mt} W_2^2(\mu,\nu),$$

and hence the claimed contraction bound. ∎

This result explains why gradient-based samplers are especially clean for strongly log-concave targets: the continuous-time dynamics contract naturally toward equilibrium. For the Euler discretization, however, one must separate two distinct questions. If K_ϵ denotes the ULA kernel and π_ϵ its invariant law when it exists, then

$$W_2(\mu K_\epsilon^n, \pi) \le W_2(\mu K_\epsilon^n, \pi_\epsilon) + W_2(\pi_\epsilon, \pi).$$

The first term is convergence of the discrete chain toward its own equilibrium, while the second is the step-size bias introduced by discretizing the diffusion. Under strong log-concavity and sufficiently small ϵ, ULA is itself geometrically ergodic toward π_ϵ; the central limitation is that at fixed ϵ one typically has $\pi_\epsilon \neq \pi$. For strongly log-concave targets with L-Lipschitz gradient, this creates a genuine two-scale requirement: when $0 < \epsilon < 2/L$, one can drive $W_2(\mu K_\epsilon^n, \pi_\epsilon)$ down geometrically in n, but one still has to choose ϵ small enough that $W_2(\pi_\epsilon, \pi)$ is itself below the target tolerance; see Vempala and Wibisono (2019, Thm. 1 and §7.1) for one precise local reference.

This decomposition clarifies the practical tradeoff. Larger ϵ makes each step more aggressive and can reduce the number of iterations needed to approach π_ϵ, but it also enlarges the asymptotic bias. Smaller ϵ reduces that bias while making neighboring states more correlated. For ULA there is therefore no single notion of "mixing fast": to hit a target accuracy δ, one typically needs both

$$W_2(\mu K_\epsilon^n, \pi_\epsilon) \lesssim \delta/2 \qquad \text{and} \qquad W_2(\pi_\epsilon, \pi) \lesssim \delta/2.$$

The first condition is a mixing requirement for the discrete chain, while the second is an approximation requirement for the time step.

The same split appears in Monte Carlo estimation. If we estimate $\mathbb{E}_\pi[g(X)]$ by the empirical average

$$\hat{\mu}_n^{\mathrm{ULA}} := \frac{1}{n}\sum_{k=1}^{n} g(X_k),$$

then after burn-in the chain average is centered at $\mathbb{E}_{\pi_\epsilon}[g(X)]$, not at $\mathbb{E}_\pi[g(X)]$. The total error therefore combines initialization bias, Monte Carlo variance from the correlated chain, and discretization bias $\mathbb{E}_{\pi_\epsilon}[g] - \mathbb{E}_\pi[g]$. Equivalently,

$$\hat{\mu}_n^{\mathrm{ULA}} - \mathbb{E}_\pi[g] = \underbrace{\hat{\mu}_n^{\mathrm{ULA}} - \mathbb{E}_{\pi_\epsilon}[g]}_{\text{initialization + correlated Monte Carlo}} + \underbrace{\mathbb{E}_{\pi_\epsilon}[g] - \mathbb{E}_\pi[g]}_{\text{discretization bias}}.$$

For an ergodic ULA chain the first term vanishes as $n \to \infty$, but the second persists whenever ϵ is held fixed. For first-order Euler discretizations that bias is typically $O(\epsilon)$ for smooth observables, which is why it is useful to check whether estimates stabilize as the step size is reduced. In practice one either treats ULA as an approximate sampler and checks that estimates stabilize as ϵ is reduced, or removes the bias with a Metropolis correction.

2.3.6 MALA, stochastic gradients, and the road to HMC

The Metropolis-adjusted Langevin algorithm (MALA) removes the discretization bias of ULA by using the ULA transition as a Metropolis–Hastings proposal. With

$$q_\epsilon(y \mid x) = \mathcal{N}(y;\ x - \epsilon\nabla U(x),\ 2\epsilon I_d), \tag{69}$$

the acceptance probability is

$$\alpha(x, y) = \min\left\{1, \frac{\tilde{\pi}(y)\, q_\epsilon(x \mid y)}{\tilde{\pi}(x)\, q_\epsilon(y \mid x)}\right\} = \min\left\{1, \exp\big(\log r(x, y)\big)\right\}, \tag{70}$$

Algorithm 15 Metropolis-adjusted Langevin algorithm (MALA)

Input: Potential $U(x)$ and gradient $\nabla U(x)$, step size $\epsilon > 0$, initial state $x^{(0)}$, steps T

```
for t = 0, 1, ..., T − 1 do
    Draw ξ ~ N(0, I_d) and set y ← x^(t) − ε∇U(x^(t)) + √(2ε) ξ.
    Compute log r(x^(t), y) from (71) and draw u ~ Unif(0, 1).
    if log u ≤ min{0, log r(x^(t), y)} then
        x^(t+1) ← y
    else
        x^(t+1) ← x^(t)
    end if
end for
return x^(0), x^(1), ..., x^(T)
```

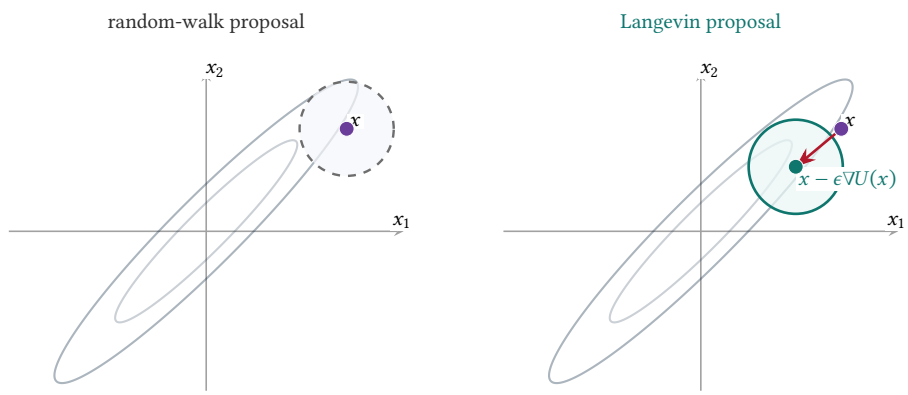

Figure 15: Schematic comparison of a random-walk Gaussian proposal (dashed) and a Langevin proposal (solid). The Langevin proposal shifts the mean by $-\epsilon\nabla U(x)$, biasing proposals toward higher-density regions.

where

$$\log r(x, y) = -U(y) + U(x) - \frac{1}{4\epsilon}\left(\|x - y + \epsilon\nabla U(y)\|^2 - \|y - x + \epsilon\nabla U(x)\|^2\right). \tag{71}$$

Computing (71) on the log scale is the stable implementation of the Metropolis correction. By Proposition 2.17, the resulting kernel leaves π invariant exactly.

The correction term in (71) has a simple interpretation. The first part, $-U(y) + U(x)$, is the usual log target ratio from Metropolis–Hastings. The quadratic part corrects for proposal asymmetry: it checks whether the forward drift from x makes y plausible and whether the reverse drift from y makes x plausible. Small step sizes therefore make MALA behave like ULA with an acceptance rate near 1, while larger step sizes produce more rejections because the Euler proposal no longer tracks the diffusion accurately.

Neal's discussion of the Langevin method makes the connection to HMC explicit: one leapfrog step of Hamiltonian Monte Carlo, followed by a Metropolis correction, is exactly a Langevin proposal of this form; see Neal (2011, §5.2). From that augmented-state viewpoint, MALA is the one-step overdamped relative of HMC. HMC keeps the gradient information but carries momentum over many leapfrog steps, which suppresses the random-walk behavior that remains when every move is only a local Langevin correction. NUTS then replaces the hand-tuned trajectory length by the no-U-turn stopping rule; see Hoffman and Gelman (2014, §3.1) and Sections 4.1 and 4.2. This is the main conceptual bridge: ULA and MALA already

show how gradients tilt local proposals, while HMC and NUTS show how augmentation converts that same local information into long coherent trajectories.

In large-data problems, evaluating the full gradient can dominate the cost of each step. If

$$U(x) = \sum_{i=1}^{N} u_i(x),$$

then stochastic gradient Langevin dynamics (SGLD) replaces the exact gradient by a mini-batch estimate; this is the large-data variant introduced by Welling and Teh (2011, §3). With batch size B, draw indices $i_1, \dots, i_B$ uniformly and define

$$\nabla\hat{U}(x) := \frac{N}{B} \sum_{b=1}^{B} \nabla u_{i_b}(x).$$

The update becomes

$$x_{t+1} = x_t - \epsilon_t \nabla\hat{U}(x_t) + \sqrt{2\epsilon_t}\, \xi_{t+1}, \qquad \xi_{t+1} \sim \mathcal{N}(0, I_d), \tag{72}$$

with a user-chosen step-size schedule (ϵ_t). If we write

$$\nabla\hat{U}(x_t) = \nabla U(x_t) + \eta_t, \qquad \mathbb{E}[\eta_t \mid x_t] = 0,$$

then

$$x_{t+1} = x_t - \epsilon_t \nabla U(x_t) + \sqrt{2\epsilon_t}\, \xi_{t+1} - \epsilon_t \eta_t. \tag{73}$$

Equation (73) makes the approximation explicit: besides the calibrated Langevin noise, SGLD contains extra noise from subsampling. For very small step sizes that additional term becomes negligible, but at practical step sizes it can substantially change the stationary behavior. SGLD is therefore best understood as an approximate large-scale sampler whose per-iteration cost is cheap precisely because it gives up the exact invariance enjoyed by MALA.

Two operating regimes are worth separating. If asymptotic exactness matters, then one uses a decreasing schedule with $\sum_t \epsilon_t = \infty$ and $\sum_t \epsilon_t^2 < \infty$, so the Euler bias and the mini-batch perturbation are both annealed away over time. This is the regime analyzed in the original SGLD paper (Welling and Teh, 2011, §3). In large-scale practice, however, fixed or slowly decaying step sizes are more common because they mix faster per unit gradient cost; the price is that the chain then settles near a biased limiting distribution, just as ULA does; see Welling and Teh (2011, §4.1) for the corresponding practical discussion of the transition into the Langevin phase. This is why practitioners often use larger step sizes during early exploration or warm-up and smaller ones when collecting approximate posterior draws.

The batch size must be chosen jointly with the step size. The stochastic-gradient perturbation in (73) enters at scale $\epsilon_t \eta_t$, whereas the calibrated Langevin perturbation enters at scale $\sqrt{\epsilon_t}\, \xi_{t+1}$. Under sampling with replacement, the covariance of the gradient-estimation error typically scales like $N^2\Sigma(x_t)/B$ for a suitable per-datum covariance matrix $\Sigma(x_t)$. Consequently the extra SGLD perturbation has covariance on the order of $\epsilon_t^2 N^2 \Sigma(x_t)/B$, whereas the injected Langevin noise has covariance $2\epsilon_t I_d$. Thus very noisy mini-batches can dominate the intended diffusion unless ϵ_t is small enough or B is large enough. A practical rule is to choose B so that the mini-batch term is a correction to the Gaussian noise rather than the main source of randomness once the chain is near stationarity; if one increases ϵ_t, one usually has to increase B as well. This is why SGLD often uses rougher settings only for early exploration, then reduces the step size and often enlarges the batch size before collecting approximate posterior draws. In practice it is useful to calibrate this near a high-density point rather than in the tails. A short pilot run can estimate the scale of the mini-batch gradient error, and the diagnostic question is simple: is the extra perturbation $\epsilon_t^2 \operatorname{Var}(\eta_t \mid x_t)$ comfortably smaller than the injected Langevin variance $2\epsilon_t I_d$? If not, one should shrink ϵ_t, increase B, or

both. Batch sizes in the same rough range used in stochastic optimization are a common starting point, but stability checks across several (B, ϵ) pairs are more informative than trusting one nominal default; this is exactly the kind of schedule-versus-noise calibration emphasized by Welling and Teh (2011, §4). Comparing posterior histograms or summary estimates across those settings is often the quickest way to tell whether the stochastic-gradient noise is dominating the intended diffusion.

At a high level, the tradeoffs are now visible. ULA is the cheapest gradient-based method but carries step-size bias, MALA removes that bias with an accept/reject step, SGLD trades exactness for mini-batch scalability, and HMC or NUTS spend full gradients to convert the same local information into much longer coherent trajectories. Equivalently, ULA and fixed-step SGLD buy cheaper iterations by tolerating an ϵ-dependent limiting law, whereas MALA, HMC, and NUTS spend extra work per iteration to keep the exact target invariant.

All of these gradient-based samplers inherit the transformed-density bookkeeping from Section 2.1.5. If the computation runs on unconstrained coordinates z, then both the potential $U(z)$ and its gradient must include the Jacobian contribution from the change of variables. Otherwise the sampler follows the right local geometry for the wrong target distribution.

Part II

Augmented-state methods

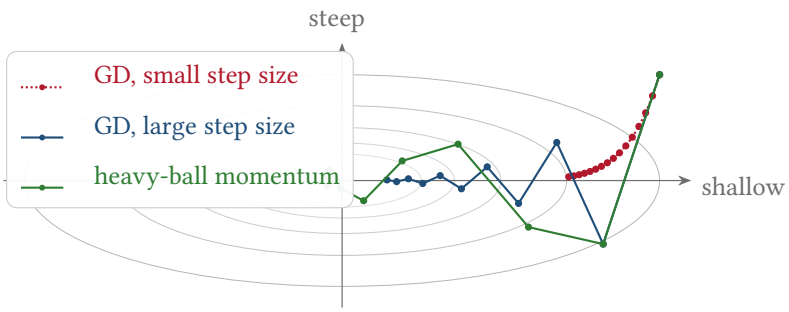

Figure 16: Ill-conditioned geometry on $f(x_1, x_2) = x_1^2/9 + x_2^2$. In the steep direction, large steps can induce oscillations; a momentum term damps these oscillations by combining information across iterations.

3 Augmented-state optimization

Augmented-state optimization enlarges the variable we update rather than working only in the original decision variable. The added state is chosen so that the enlarged dynamics are easier to stabilize, explore, or optimize while remaining tied to the original objective. Momentum adds velocity, simulated annealing adds temperature, and EM introduces latent variables; in each case the auxiliary quantity makes a difficult optimization problem more tractable.

3.1 Momentum-based methods

Gradient descent reacts only to the current gradient. It updates

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k),$$

so the next iterate depends only on the current point. On an ill-conditioned objective that can produce the familiar zig-zag pattern: the method keeps correcting the last step instead of building sustained motion along the useful direction. Momentum methods change this by carrying an auxiliary state from one iteration to the next. That state is a velocity for heavy-ball and NAG, a cumulative dual statistic for dual averaging, and diagonal moment accumulators for the AdaGrad/RMSProp/Adam family. The unifying idea is simple: a local gradient should not merely determine the next correction; it should update a short-term memory that makes the future trajectory more coherent.

This is the first optimization chapter where augmentation is the main design move rather than a secondary implementation detail. It also marks a clean split between method families. Heavy-ball and NAG belong to the smooth deterministic story, dual averaging organizes the constrained and prox-geometry viewpoint from the previous chapter, and Adam is a mini-batch method with deliberately heuristic defaults. The shared state-space viewpoint matters throughout, but the tuning rules and guarantees are not interchangeable.

3.1.1 Ill-conditioning and zig-zagging

On an ill-conditioned objective, the curvature can be very different in different directions. The simplest picture is a quadratic with elongated level sets, e.g. $f(x_1, x_2) = x_1^2/9 + x_2^2$. Gradient descent with a conservative step size is stable but may make painfully slow progress along the shallow direction. Taking a larger step size can speed up progress, but typically introduces oscillations in the steep direction: successive gradients point in alternating directions, and the iterates repeatedly "undo" part of the previous step. Figure 16 illustrates this behavior.

The basic heuristic is that averaging reinforces directions that persist over several iterations and cancels

directions that alternate rapidly. This is why momentum helps on elongated valleys: it preserves progress along the shallow direction while damping the back-and-forth motion in the steep one.

3.1.2 Heavy-ball momentum

A common momentum scheme is Polyak's *heavy-ball* method,

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k) + \beta_k (x_k - x_{k-1}), \tag{74}$$

where $\alpha_k > 0$ is a step size and $\beta_k \in [0, 1)$ controls how much of the previous step is carried forward. Defining the velocity $v_k := x_k - x_{k-1}$ turns (74) into the augmented-state recursion

$$v_{k+1} = \beta_k v_k - \alpha_k \nabla f(x_k), \qquad x_{k+1} = x_k + v_{k+1}. \tag{75}$$

In this form, the algorithm is "first-order" in the state (x_k, v_k): the next state depends only on the current state and the current gradient.

3.1.3 Acceleration by look-ahead gradients

Nesterov's accelerated gradient (NAG) modifies Equation (74) by taking a *look-ahead* point and evaluating the gradient there. Given step sizes (α_k) and momentum parameters (β_k), define

$$y_k = x_k + \beta_k (x_k - x_{k-1}), \qquad x_{k+1} = y_k - \alpha_k \nabla f(y_k). \tag{76}$$

The first line extrapolates in the direction of the current velocity $x_k - x_{k-1}$, and the second line uses a gradient step to "steer" from the look-ahead point.

Algorithm 16 Nesterov accelerated gradient (NAG)

Input: Differentiable f, step sizes $(\alpha_k)_{k \geq 0}$, momentum parameters $(\beta_k)_{k \geq 0}$, initial point x_0, number of steps T

Output: Iterate x_T

```
x_{-1} ← x_0
for k = 0, 1, ..., T − 1 do
    y_k ← x_k + β_k(x_k − x_{k−1})          ▷ look-ahead point
    x_{k+1} ← y_k − α_k ∇f(y_k)
end for
return x_T
```

It is sometimes helpful to write NAG in velocity form. Let $v_k := x_k - x_{k-1}$. Then $y_k = x_k + \beta_k v_k$, and the update becomes

$$v_{k+1} = \beta_k v_k - \alpha_k \nabla f(x_k + \beta_k v_k), \qquad x_{k+1} = x_k + v_{k+1}. \tag{77}$$

The parameter β_k controls how much previous velocity is retained, while α_k controls how strongly the new gradient information overrides this extrapolation. The difference from heavy-ball is conceptual as well as algebraic: heavy-ball uses the stored velocity to smooth the step itself, whereas NAG uses it to decide *where* the next gradient should be evaluated. That distinction is why acceleration arguments for NAG are tied to smooth convex structure rather than to the generic oscillation-damping heuristic from the previous subsection.

In the regimes where acceleration theory applies, the improvement over plain gradient descent is explicit rather than merely heuristic. For an L-smooth convex objective, gradient descent gives a suboptimality bound of order $O(1/k)$, whereas NAG improves this to $O(1/k^2)$. If f is moreover m-strongly convex with

condition number $\kappa = L/m$, then gradient descent requires $O(\kappa \log(1/\varepsilon))$ iterations to reach accuracy ε, while accelerated schemes reduce this to $O(\sqrt{\kappa} \log(1/\varepsilon))$. We record these rates only as the classical benchmark that motivates the method; the notes use them for orientation rather than as a theorem proved from our local source base. This is the main meaning of the word *acceleration*: NAG changes the dependence on the condition number, not just the constant in front of the rate.

3.1.4 Averaging with dual states

Nesterov also introduced *dual averaging*, which carries a different auxiliary state. Here the stored quantity is the cumulative gradient summary s_k; instead of extrapolating the current iterate, the algorithm updates this summary and maps it back to the decision variable through a regularized minimization step. Let $C \subseteq \mathbb{R}^d$ be a closed convex set, let $\psi : C \to \mathbb{R}$ be a strongly convex prox function, and let g_k be a gradient or subgradient observed at iteration k. Define the cumulative gradient summary

$$s_k := \sum_{i=1}^{k} g_i,$$

and then choose the next iterate by

$$x_{k+1} = \arg\min_{x \in C} \left\{ \langle s_k \rangle x + \frac{1}{\eta_k} \psi(x) \right\}, \tag{78}$$

where $\eta_k > 0$ is a scale parameter.

The vector s_k is the auxiliary dual accumulator because it acts on the primal variable x through the linear form $\langle s_k \rangle x$. Using the Euclidean identification introduced at the start of the optimization chapter, we often write dual vectors as ordinary vectors, so the distinction is conceptual rather than notational, but the roles are different: the dual variable stores aggregated first-order information, while the primal variable is the point at which the objective is evaluated. The prox term ψ keeps the minimization in (78) well behaved and converts the accumulated dual information into a new primal iterate.

In the simplest Euclidean case, where $C = \mathbb{R}^d$ and $\psi(x) = \frac{1}{2}\|x\|_2^2$, the optimality condition for (78) gives

$$x_{k+1} = -\eta_k s_k = -\eta_k \sum_{i=1}^{k} g_i.$$

Up to the scale factor η_k, the iterate therefore depends on an average or cumulative sum of past gradients rather than on a look-ahead gradient at an extrapolated point. This is still an augmented-state method, but the state no longer represents literal velocity. Instead it stores a running first-order summary that is interpreted through the prox geometry. That viewpoint will reappear in Section 4.2.4, where dual averaging is used to tune a sampler rather than to update model parameters.

3.1.5 Adaptive moment methods

In large-scale problems, f often has a finite-sum form (as in Equation (35)), and gradients are estimated using mini-batches. Adam keeps the augmentation idea but changes the carried quantities. It maintains two auxiliary states: a first-moment estimate m_k and a second-moment estimate v_k. The first carries directional information forward, while the second rescales coordinates according to recent gradient magnitudes. It is best viewed together with AdaGrad and RMSProp as part of a small family of diagonal adaptive-scaling methods: AdaGrad keeps a cumulative sum of squared gradients, RMSProp replaces that cumulative memory by an exponential moving average, and Adam adds a first-moment exponential average on top of

the RMSProp-style denominator. This combination is one reason Adam-type methods became standard in deep-network training, where coordinates can have very different scales and mini-batch gradients are both noisy and nonstationary (Kingma and Ba, 2015, §1).

Algorithm 17 Adam (adaptive moment estimation)

Input: Functions $f_1, \dots, f_n$, step size $\alpha > 0$, decay rates $\beta_1, \beta_2 \in (0, 1)$, $\epsilon > 0$, batch size B, initial point x_0, number of steps T

Output: Iterate x_T

$m_0 \leftarrow 0, v_0 \leftarrow 0$
for $k = 1, 2, \dots, T$ **do**
  Sample a mini-batch $S_k \subseteq \{1, \dots, n\}$ of size B (with replacement)
  $g_k \leftarrow \frac{1}{B} \sum_{i \in S_k} \nabla f_i(x_{k-1})$ ▷ stochastic gradient
  $m_k \leftarrow \beta_1 m_{k-1} + (1 - \beta_1) g_k$ ▷ first moment
  $v_k \leftarrow \beta_2 v_{k-1} + (1 - \beta_2)(g_k \odot g_k)$ ▷ second moment (elementwise)
  $\hat{m}_k \leftarrow \dfrac{m_k}{1 - \beta_1^k}, \hat{v}_k \leftarrow \dfrac{v_k}{1 - \beta_2^k}$ ▷ bias correction
  $x_k \leftarrow x_{k-1} - \alpha \hat{m}_k \odot \frac{1}{\sqrt{\hat{v}_k} + \epsilon}$ ▷ all operations elementwise
end for
return x_T

Common default choices are $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$ (Kingma and Ba, 2015, §2 and Alg. 1). In many applications these are kept near their defaults and one primarily tunes the global step size α. The parameter β_1 controls how quickly the momentum estimate forgets past directions, while β_2 controls the memory length of the squared-gradient estimate. A useful rule of thumb is that β_2 averages over about $1/(1 - \beta_2)$ iterations, so $\beta_2 = 0.999$ corresponds to a window on the order of 10^3 updates. Increasing β_2 makes the denominator steadier, which is often helpful when mini-batch gradients are very noisy; decreasing it makes the scaling react faster after a curvature change or a long plateau, but can make the effective coordinatewise step sizes much more erratic. Values of β_2 close to 1 are especially common when gradients are sparse or highly variable across iterations, because then a longer memory stabilizes the second-moment estimate (Kingma and Ba, 2015, §6.4).

The moving average m_k is Adam's momentum-like state, and the second-moment estimate v_k is its adaptive scaling state. If a particular coordinate has very large or very noisy gradients, then the corresponding entries of v_k grow, and dividing by $\sqrt{\hat{v}_k} + \epsilon$ reduces the effective step size in that coordinate. This behavior is reminiscent of diagonal preconditioning, and it can be particularly effective when the objective has strong anisotropy (e.g. very different curvature or gradient scales across coordinates).

Unrolling the recursions shows that m_k and v_k are exponential moving averages:

$$m_k = (1 - \beta_1) \sum_{i=1}^{k} \beta_1^{k-i} g_i, \qquad v_k = (1 - \beta_2) \sum_{i=1}^{k} \beta_2^{k-i} (g_i \odot g_i).$$

The bias-corrected quantities $\hat{m}_k$ and $\hat{v}_k$ renormalize these weights so that, roughly speaking, they behave like averages rather than sums during the early iterations (when m_k and v_k are still close to 0). Compared with dual averaging, the key change is that the memory is now exponential rather than cumulative. That makes the method responsive to nonstationary gradient scales, which is useful in mini-batch training, but it also means the method is best understood as a practical adaptive heuristic rather than as a direct extension of the deterministic acceleration formulas above. This flexibility comes with an important caveat. Because v_k is an exponential moving average rather than a monotone accumulator, the coordinatewise

denominator can decrease after a transient, so the effective learning rates can increase again. There are convex examples where the original Adam recursion fails to converge to the minimizer for exactly this reason (Reddi et al., 2018, §3–§4). The same analysis motivates "long-memory" fixes such as AMSGrad, which replace the raw second-moment estimate by a coordinatewise running maximum to prevent this denominator from decreasing (Reddi et al., 2018, §4). That is why Adam belongs in the "practical adaptive method" bucket rather than in the same guarantee class as gradient descent or NAG.

3.1.6 Scope-limited tuning guidance

Tuning guidance for momentum methods is family-specific. For heavy-ball and NAG on deterministic smooth problems, the step sizes α_k play the same role as in gradient descent: one can use a hand-tuned constant, a decreasing schedule, or, when f is cheap enough to evaluate, a line search. The momentum parameters $\beta_k \in [0, 1)$ control how quickly the method forgets past velocities. In practice a constant β in the range 0.8–0.95 is a common heuristic starting point, and one lowers it if the iterates exhibit persistent oscillations in steep directions.

When global curvature bounds are known, more structured formulas are available, but they are tied to specific recursions rather than to "momentum methods" as a whole. For example, if f is twice differentiable and satisfies

$$mI \preceq \nabla^2 f(x) \preceq LI \qquad \text{for all } x \text{ in the region explored,}$$

then $\alpha_k \equiv 1/L$ is a natural step-size scale for the NAG update (76), and a common strongly-convex choice is

$$\beta_k \equiv \frac{\sqrt{L} - \sqrt{m}}{\sqrt{L} + \sqrt{m}} = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}, \qquad \kappa := \frac{L}{m}.$$

Heavy-ball has a different parameter analysis even on quadratic objectives, so this formula should not be read as a generic prescription for every two-step method. For merely convex problems, a popular increasing-momentum schedule is

$$\beta_k = \frac{t_k - 1}{t_{k+1}}, \qquad t_1 = 1, \qquad t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}, \tag{79}$$

which is the momentum rule used in FISTA-type accelerated methods. Neither formula should be transplanted wholesale to noisy mini-batch settings: they are derived for structured smooth convex regimes, not for general adaptive training.

For dual averaging, the main design choice is the prox geometry together with the scale sequence η_k. Larger η_k lets current gradients dominate more quickly, while smaller η_k keeps the regularization effect of ψ active for longer. This is the right method family when constraints or non-Euclidean geometry matter, not when one merely wants a faster version of heavy-ball.

For Adam, one usually keeps β_1 and ϵ near their standard defaults and tunes the global step size α. The main extra family-specific choice is β_2: larger values smooth the denominator when stochastic gradients are very noisy, while smaller values make the scaling react faster after a plateau or regime change. Those defaults are conventions rather than universal constants, so the safest interpretation is that Adam trades some theoretical clarity for a robust, easy-to-tune implementation pattern. In large-scale learning problems, a fixed number of parameter updates is often used as a stopping rule. Another common approach is early stopping: periodically evaluate a held-out validation metric and stop if it does not improve for a chosen number of checks. Finally, when full-gradient evaluations are feasible, or on a sufficiently large subsample, one can stop when $\|\nabla f(x_k)\|_2$ is below a chosen tolerance.

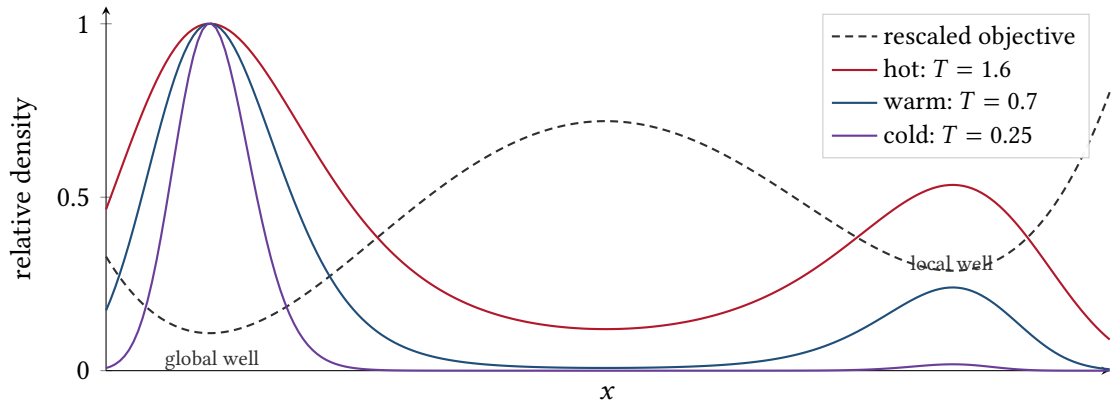

Figure 17: For a fixed two-well objective (shown faintly in gray after rescaling), the quantities proportional to $\exp(-f(x)/T)$ are much flatter when T is large and much more concentrated near low-energy states when T is small. Simulated annealing exploits this by exploring broadly while the system is hot and becoming increasingly selective as it cools.

3.2 Simulated annealing

Gradient descent and its stochastic variants are fundamentally local search methods. They can move quickly once they are inside a good basin, but on objectives with several separated local minima, saddles, or energy barriers they do not provide a reliable global-optimization strategy. One response is to use careful initialization; another is brute force, i.e. many dispersed restarts followed by local refinement. Simulated annealing takes a different approach: it combines the accept–reject mechanism of Metropolis–Hastings with a temperature schedule that is high at the beginning and small later on. Here the auxiliary mechanism is the scalar temperature itself. At iteration k the method works on an enlarged description (x, T_k) rather than on the state x alone: the temperature temporarily changes the landscape, then is cooled back toward the original optimization problem. The high-temperature phase makes it easier to leave poor local minima, while the low-temperature phase gradually concentrates the search near low-energy states.

3.2.1 Boltzmann targets and temperature flattening

Let $f : \mathcal{X} \to \mathbb{R}$ be the objective we want to minimize. For each temperature $T > 0$, define the Gibbs or Boltzmann distribution

$$\pi_T(x) := \frac{\exp(-f(x)/T)}{Z_T}, \qquad Z_T := \int_{\mathcal{X}} \exp(-f(x)/T)\, dx, \tag{80}$$

with the integral replaced by a sum when $\mathcal{X}$ is discrete. The theoretical probabilistic interpretation requires $Z_T < \infty$, but the algorithm itself only uses ratios of the form $\pi_T(y)/\pi_T(x)$, so the normalizing constant never needs to be computed explicitly.

Two simple facts explain why (80) is useful. First, for every $T > 0$, maximizing $\pi_T(x)$ is exactly the same as minimizing $f(x)$ because the map $u \mapsto e^{-u/T}$ is strictly decreasing. Second, temperature controls how strongly energy differences are emphasized. When T is large, the factor $e^{-f(x)/T}$ changes slowly with $f(x)$, so the target is relatively flat. When T is small, differences in objective value are amplified and the mass concentrates around the lowest-energy states.

Thinking of T as a physical temperature gives the right intuition: when the system is hot, the wells of the landscape are relatively shallow, so movement between them is easier; when the system is cold, those same wells become deep, and the dynamics become much more reluctant to move uphill. Figure 17 makes this

concrete: the hot distribution still assigns substantial mass to the higher-energy local well and even to the barrier region, whereas the cold distribution is sharply concentrated near the global well.

3.2.2 Metropolis updates and simulated annealing

Suppose that at a fixed temperature T we use a proposal kernel $q(y \mid x)$. If we were running an ordinary Metropolis–Hastings chain targeting π_T, then the acceptance probability would be

$$\alpha_T(x, y) = \min\left\{1,\ \exp\left(-\frac{f(y) - f(x)}{T}\right)\frac{q(x \mid y)}{q(y \mid x)}\right\}. \tag{81}$$

For a symmetric random-walk proposal, $q(y \mid x) = q(x \mid y)$, so (81) simplifies to

$$\alpha_T(x, y) = \min\left\{1,\ \exp\left(-\frac{f(y) - f(x)}{T}\right)\right\}. \tag{82}$$

This form is especially transparent: downhill moves ($f(y) \leq f(x)$) are always accepted, while uphill moves are accepted with a probability that decays exponentially in the energy increase $f(y) - f(x)$. Large temperature makes uphill moves fairly likely; small temperature suppresses them. Temperature turns uphill acceptance into a controllable exploration mechanism: hot chains are willing to climb barriers, while cold chains behave much more like descent.

Simulated annealing uses the same accept–reject idea, but with a nonincreasing sequence of temperatures

$$T_0 \geq T_1 \geq T_2 \geq \cdots \downarrow 0.$$

At iteration k, the algorithm updates the pair $(x^{(k)}, T_k)$ by holding the temperature fixed at its current value and performing a Metropolis–Hastings step for the target π_{T_k}. Because the temperature changes over time, the chain is no longer trying to sample from one fixed distribution. Instead, it gradually transitions from exploratory dynamics at high temperature to almost-greedy local search at low temperature.

Algorithm 18 Random-walk simulated annealing

Input: Objective f, symmetric proposal kernel $q(\cdot \mid x)$, temperatures $T_0, \ldots, T_{K-1}$ with $T_{k+1} \leq T_k$, initial state $x^{(0)}$

Output: Best state encountered during the run

```
x_best ← x^(0), f_best ← f(x^(0))
for k = 0, 1, ..., K − 1 do
    Sample a proposal y ~ q(· | x^(k))
    α_k ← min{ 1, exp(−(f(y) − f(x^(k)))/T_k) }
    Draw U_k ~ Unif[0, 1]
    if U_k ≤ α_k then
        x^(k+1) ← y
    else
        x^(k+1) ← x^(k)
    end if
    if f(x^(k+1)) < f_best then
        x_best ← x^(k+1), f_best ← f(x^(k+1))
    end if
end for
return x_best
```

The best-so-far bookkeeping is important. Even when the current state wanders away from a good point, the algorithm retains the lowest objective value it has seen so far. For asymmetric proposals one simply replaces the acceptance formula in Algorithm 18 by the full Metropolis–Hastings ratio (81). As T_k becomes small, the chain behaves more and more like randomized local descent on f, but the early hot phase is what gives the method a chance to reach a good basin before that local behavior takes over.

3.2.3 Cooling schedules and global guarantees

If one wants theorem-level global guarantees rather than a practical heuristic, the cooling schedule must be extremely slow. A classical theorem-level schedule is logarithmic cooling of the form

$$T_k = \frac{c}{\log(k + k_0)}, \qquad c > 0, \tag{83}$$

in the finite-state Gibbs setting analyzed by Geman and Geman (1984, eq. (12.7), Thm. C, and p. 732). In that theorem, the constant c is problem-dependent and reflects the barrier structure of the energy landscape. These schedules are slow precisely because they are designed to give the algorithm enough time to escape increasingly deep local wells. We are using (83) only as a theorem-level schematic, not as a drop-in prescription: outside that finite-state setting, the exact hypotheses and constants depend delicately on the proposal kernel and the geometry of the state space. Its message is structural: provable global exploration is much slower than the schedules people usually want to run.

3.2.4 Practical schedules and proposal design

The proposal distribution matters just as much here as it does in ordinary Metropolis–Hastings. For continuous state spaces, Gaussian random-walk proposals are a common default. On structured discrete spaces, one instead designs local combinatorial moves that preserve validity of the state, such as swaps, reversals, or Gibbs-style coordinate updates.

In practical computation one often uses faster heuristics such as a linear schedule

$$T_k = T_{\max} - \frac{k}{K}\big(T_{\max} - T_{\min}\big), \tag{84}$$

or an exponential schedule

$$T_k = T_{\max}\alpha^k, \qquad \alpha \in (0, 1), \tag{85}$$

because they cool more aggressively. These schedules are heuristics: they often work far better computationally than logarithmic cooling, but they do not come with the same global guarantee. Cool too quickly and the method behaves like a greedy local search that may freeze in a poor basin; cool more slowly and the method explores better, but at higher computational cost. The temperature schedule is therefore the main dial that trades global exploration against local refinement. Because the method is stochastic and still sensitive to initialization, one often runs several independent seeds and compares the best objective values they find rather than treating a single annealing trajectory as definitive. That comparison separates two questions that are easy to conflate in a single run: whether one schedule explores a basin effectively once it reaches it, and whether the overall procedure reliably reaches good basins at all.

This tension between hot exploration and cold exploitation is the bridge to tempering in the next unit. Simulated annealing commits to a single cooling schedule and eventually gives up the hot dynamics. Tempering keeps a finite ladder of hot auxiliary distributions alive throughout the run and couples them to the cold target, so barrier-crossing remains available even after the chain has returned to the original distribution; see Section 4.3.

Example 3.1 (Traveling salesman and a 2-opt proposal)**.** Given locations $z_1, \dots, z_n \in \mathbb{R}^2$, a tour is a permutation $\sigma = (i_1, \dots, i_n)$ of $\{1, \dots, n\}$. The traveling-salesman objective is

$$f(\sigma) = \sum_{j=1}^{n} \|z_{i_j} - z_{i_{j+1}}\|_2, \qquad i_{n+1} := i_1,$$

and the goal is to minimize $f(\sigma)$ over all tours. This problem is NP-hard, so exhaustive search is generally impossible.

A standard symmetric proposal is the *2-opt* move: choose two indices $a < b$ and reverse the segment $(i_a, i_{a+1}, \dots, i_b)$. This keeps the tour valid while making a large enough local change to repair bad crossings or reorganize an awkward portion of the route. Because the move is symmetric, the acceptance probability is exactly (82). Early in the run the algorithm is willing to accept a longer tour if that helps reorganize the route; later, once the temperature is small, it behaves much more like a greedy local improvement method.

The handwritten comparison on one fixed TSP instance made the schedule tradeoff concrete. A very short aggressive schedule froze quickly and often kept visibly poor crossings, because the chain stopped accepting uphill reorganizations before the 2-opt proposals had time to rearrange the route. A longer linear schedule improved the tour more reliably, and a slower exponential schedule typically produced the cleanest final route on that example. The point was not that one named schedule always dominates. It was that schedule length has to be matched to the proposal's ability to make useful large-scale repairs before the search becomes effectively greedy.

3.3 Expectation–maximization (EM)

Many statistical models are easiest to describe by introducing a latent or missing variable. This is the probabilistic version of augmentation: the auxiliary variable z makes the complete-data model simple even though the observed-data likelihood is harder. Suppose the observed data are x, the latent data are z, and the model is specified by a joint density or mass function $p_\theta(x, z)$. If z were observed, then we could often optimize the complete-data model directly. The difficulty is that maximum likelihood depends on the marginalized quantity

$$p_\theta(x) = \int p_\theta(x, z)\, dz, \qquad \ell(\theta; x) := \log p_\theta(x),$$

with the integral replaced by a sum when z is discrete. The awkward step is that the logarithm sits outside the integral, so even a simple complete-data model can lead to a difficult observed-data optimization problem. EM replaces this direct maximization by alternating two simpler steps. With the current iterate $\theta^{(t)}$, the E-step computes the current conditional law

$$q_t(z) := p_{\theta^{(t)}}(z \mid x),$$

and the M-step maximizes the averaged complete-data log-likelihood

$$Q(\theta \mid \theta^{(t)}) := \mathbb{E}_{Z \sim q_t}[\log p_\theta(x, Z)]\,.$$

Later in the section we will rewrite this same alternation as coordinate ascent on a free-energy functional, which is the bridge to variational inference. Classical examples include Gaussian mixture models, hidden Markov models, and Gaussian models with missing data.

3.3.1 Free energy, the Q-function, and coordinate ascent

Let $\theta^{(t)}$ be the current iterate and define the current posterior law of the latent variable by

$$q_t(z) := p_{\theta^{(t)}}(z \mid x).$$

The associated EM surrogate is

$$Q(\theta \mid \theta^{(t)}) := \mathbb{E}\big[\log p_\theta(x, Z) \mid X = x, \theta^{(t)}\big] = \int \log p_\theta(x, z)\, q_t(z)\, dz. \tag{86}$$

The E-step computes the conditional law q_t, and the M-step maximizes the surrogate (86) with respect to θ. $Q(\theta \mid \theta^{(t)})$ averages the complete-data log-likelihood under the current belief about the missing data, so the M-step often reduces to a familiar weighted maximum-likelihood problem. The E-step updates the auxiliary latent law, and the M-step optimizes with that law held fixed.

Algorithm 19 Expectation–maximization (EM)

Input: Observed data x, joint model $p_\theta(x, z)$, initial guess $\theta^{(0)}$, stopping tolerance ε
Output: Approximate maximizer of the observed-data log-likelihood

```
for t = 0, 1, 2, ... do
    E-step: compute the conditional law q_t(z) = p_{θ^(t)}(z | x)
    Form Q(θ | θ^(t)) = E_{Z~q_t}[log p_θ(x, Z)]
    M-step: choose θ^(t+1) ∈ arg max_θ Q(θ | θ^(t))
    if ‖θ^(t+1) − θ^(t)‖_2 ≤ ε then
        return θ^(t+1)
    end if
end for
```

The M-step does not have to be solved exactly to preserve the usual ascent property. Any update that increases $Q(\theta \mid \theta^{(t)})$ is called a *generalized EM* step. This generalized form is standard in the modern EM literature; see, for example, Balakrishnan et al. (2014, §2.1). Equation (86) is the operational form of the same free-energy problem introduced above, because

$$\mathcal{F}(q, \theta) := \mathbb{E}_q[\log p_\theta(x, Z)] + H(q) = \int q(z) \log \frac{p_\theta(x, z)}{q(z)}\, dz. \tag{87}$$

EM is exact coordinate ascent on this joint objective.

Proposition 3.2 (EM as coordinate ascent on free energy)**.** *For any density q supported where $p_\theta(x, z) > 0$,*

$$\ell(\theta; x) = \mathcal{F}(q, \theta) + \mathrm{KL}\big(q(z) \,\|\, p_\theta(z \mid x)\big). \tag{88}$$

Consequently:

1. *for fixed θ, the unique maximizer of $\mathcal{F}(q, \theta)$ over all densities q is the exact posterior $p_\theta(z \mid x)$;*
2. *exact EM alternates the exact q-update*

$$q^{(t+1)}(z) = p_{\theta^{(t)}}(z \mid x)$$

with the exact θ-update

$$\theta^{(t+1)} \in \arg\max_\theta \mathcal{F}(q^{(t+1)}, \theta);$$

3. *if q is restricted to a tractable family $\mathcal{Q}$, then maximizing $\mathcal{F}(q,\theta)$ over $q \in \mathcal{Q}$ and θ is precisely the variational-inference relaxation of EM.*

Proof. Starting from Bayes' rule,

$$p_\theta(z \mid x) = \frac{p_\theta(x,z)}{p_\theta(x)}.$$

Substituting this identity into the definition of KL divergence gives

$$\mathrm{KL}\big(q(z) \,\|\, p_\theta(z \mid x)\big) = \int q(z) \log \frac{q(z)\, p_\theta(x)}{p_\theta(x,z)}\, dz.$$

Rearranging yields (88). For fixed θ, the term $\ell(\theta; x)$ is constant in q, so maximizing $\mathcal{F}(q,\theta)$ is equivalent to minimizing the KL divergence to the exact posterior. That proves the first claim.

For the second claim, the exact q-update is just the maximizer from the first part. Once $q^{(t+1)}$ is fixed, the entropy term $H(q^{(t+1)})$ is constant in θ, so maximizing $\mathcal{F}(q^{(t+1)},\theta)$ over θ is equivalent to maximizing

$$\mathbb{E}_{q^{(t+1)}}[\log p_\theta(x,Z)] = Q(\theta \mid \theta^{(t)}),$$

which is exactly the M-step. The third claim is the same statement with the q-optimization restricted from all densities to the smaller family $\mathcal{Q}$. ■

Proposition 3.3 (EM monotonicity)**.** *Let $\ell(\theta; x) = \log p_\theta(x)$ and let $q_t(z) = p_{\theta^{(t)}}(z \mid x)$. If $\theta^{(t+1)}$ satisfies*

$$Q(\theta^{(t+1)} \mid \theta^{(t)}) \geq Q(\theta^{(t)} \mid \theta^{(t)}),$$

then

$$\ell(\theta^{(t+1)}; x) \geq \ell(\theta^{(t)}; x).$$

In particular, exact EM does not decrease the observed-data log-likelihood.

Proof. For any θ,

$$\ell(\theta; x) = \log \int q_t(z)\, \frac{p_\theta(x,z)}{q_t(z)}\, dz.$$

Applying Jensen's inequality to the concave logarithm gives

$$\ell(\theta; x) \geq \int q_t(z) \log \frac{p_\theta(x,z)}{q_t(z)}\, dz = Q(\theta \mid \theta^{(t)}) - \int q_t(z) \log q_t(z)\, dz.$$

The second term does not depend on θ. Moreover, when $\theta = \theta^{(t)}$, Bayes' rule gives

$$\frac{p_{\theta^{(t)}}(x,z)}{q_t(z)} = \frac{p_{\theta^{(t)}}(x,z)}{p_{\theta^{(t)}}(z \mid x)} = p_{\theta^{(t)}}(x),$$

which is constant in z, so Jensen's inequality is tight at $\theta^{(t)}$. Therefore

$$\ell(\theta^{(t+1)}; x) - \ell(\theta^{(t)}; x) \geq Q(\theta^{(t+1)} \mid \theta^{(t)}) - Q(\theta^{(t)} \mid \theta^{(t)}) \geq 0.$$

■

This proposition explains why EM is often described as a monotone ascent method. It is important, however, not to over-interpret the result: monotonicity guarantees improvement in likelihood, but it does *not* guarantee convergence to the global maximizer. Observed-data likelihoods for latent-variable models are usually nonconcave, so EM should be viewed as a local refinement method.

3.3.2 Worked example: a two-coin mixture

Suppose coin A lands heads with probability θ_A and coin B lands heads with probability θ_B. For each experiment $i = 1, \dots, N$:

1. choose coin A with probability π and coin B with probability $1 - \pi$,
2. toss the chosen coin m times,
3. record only the number of heads $X_i \in \{0, 1, \dots, m\}$.

The latent variable is

$$Z_i \in \{A, B\},$$

which records which coin was used in experiment i. Conditional on $Z_i = A$, we have $X_i \sim \mathrm{Bin}(m, \theta_A)$, and conditional on $Z_i = B$, we have $X_i \sim \mathrm{Bin}(m, \theta_B)$.

If the labels $Z_1, \dots, Z_N$ were observed, estimation would be straightforward. Indeed, the complete-data likelihood is

$$\begin{aligned} p(x, z \mid \pi, \theta_A, \theta_B) = \prod_{i=1}^{N} &\left[\pi \binom{m}{x_i} \theta_A^{x_i} (1-\theta_A)^{m-x_i}\right]^{\mathbf{1}\{z_i = A\}} \\ &\times \left[(1-\pi) \binom{m}{x_i} \theta_B^{x_i} (1-\theta_B)^{m-x_i}\right]^{\mathbf{1}\{z_i = B\}}. \end{aligned} \tag{89}$$

The corresponding complete-data log-likelihood is

$$\begin{aligned} \log p(x, z \mid \pi, \theta_A, \theta_B) = \sum_{i=1}^{N} \Big[&\mathbf{1}\{z_i = A\}\big(\log \pi + x_i \log \theta_A + (m - x_i) \log(1 - \theta_A)\big) \\ &+ \mathbf{1}\{z_i = B\}\big(\log(1-\pi) + x_i \log \theta_B + (m - x_i) \log(1 - \theta_B)\big) \\ &+ \log \binom{m}{x_i} \Big]. \end{aligned} \tag{90}$$

If the indicators were known, maximizing (90) would just mean taking the empirical fraction of experiments assigned to each coin and the empirical fraction of heads within each coin.

The observed-data likelihood is harder because we must sum over the unknown labels:

$$p(x \mid \pi, \theta_A, \theta_B) = \prod_{i=1}^{N} \left[\pi \binom{m}{x_i} \theta_A^{x_i} (1-\theta_A)^{m-x_i} + (1-\pi) \binom{m}{x_i} \theta_B^{x_i} (1-\theta_B)^{m-x_i}\right]. \tag{91}$$

Taking a logarithm turns the product into a sum, but each summand still contains a logarithm of a *sum*, so direct maximization is awkward.

Given current estimates $(\pi^{(t)}, \theta_A^{(t)}, \theta_B^{(t)})$, the E-step computes the posterior probability that experiment i used coin A:

$$\gamma_{i,A}^{(t)} := \Pr(Z_i = A \mid X_i = x_i, \pi^{(t)}, \theta_A^{(t)}, \theta_B^{(t)}) = \frac{\pi^{(t)} (\theta_A^{(t)})^{x_i} (1-\theta_A^{(t)})^{m-x_i}}{\pi^{(t)} (\theta_A^{(t)})^{x_i} (1-\theta_A^{(t)})^{m-x_i} + (1-\pi^{(t)}) (\theta_B^{(t)})^{x_i} (1-\theta_B^{(t)})^{m-x_i}}. \tag{92}$$

The binomial coefficient cancels in the ratio, so only the relative plausibility of the two coins matters. We also write

$$\gamma_{i,B}^{(t)} := 1 - \gamma_{i,A}^{(t)}.$$

Using these posterior weights in (90) yields the surrogate

$$Q(\pi, \theta_A, \theta_B \mid \pi^{(t)}, \theta_A^{(t)}, \theta_B^{(t)}) = \sum_{i=1}^{N} \Big[\gamma_{i,A}^{(t)} \big(\log \pi + x_i \log \theta_A + (m - x_i) \log(1 - \theta_A) \big) + \gamma_{i,B}^{(t)} \big(\log(1 - \pi) + x_i \log \theta_B + (m - x_i) \log(1 - \theta_B) \big) \Big] + \text{const}, \tag{93}$$

where the constant term does not depend on $(\pi, \theta_A, \theta_B)$.

The M-step maximizes (93). Because the parameters separate, the maximizers are available in closed form:

$$\pi^{(t+1)} = \frac{1}{N} \sum_{i=1}^{N} \gamma_{i,A}^{(t)}, \tag{94}$$

$$\theta_A^{(t+1)} = \frac{\sum_{i=1}^{N} \gamma_{i,A}^{(t)} x_i}{m \sum_{i=1}^{N} \gamma_{i,A}^{(t)}}, \qquad \theta_B^{(t+1)} = \frac{\sum_{i=1}^{N} \gamma_{i,B}^{(t)} x_i}{m \sum_{i=1}^{N} \gamma_{i,B}^{(t)}}. \tag{95}$$

These formulas have a simple interpretation: EM replaces the unknown label of each experiment by a *soft assignment*. The updated coin-bias estimates are therefore just weighted fractions of heads, and the updated mixing weight is the average posterior probability of using coin A. In the language of clustering, the E-step softly separates the experiments into "coin A-like" and "coin B-like" groups, and the M-step re-fits the two coins to those soft groups.

3.3.3 Stable responsibilities and regularized updates

The algebra of the E-step is simple, but the computation can still become unstable when some latent states are much less plausible than others. The general log-scale normalization rule from Section 2.1.4 is therefore part of the standard EM toolkit. If a model has latent states $k = 1, \dots, K$, one first forms log responsibilities

$$\ell_{ik} := \log \pi_k + \log p(x_i \mid z_i = k, \theta_k),$$

then normalizes them by subtracting a common scale before exponentiating:

$$r_{ik} = \exp(\ell_{ik} - \mathrm{LSE}(\ell_{i1}, \dots, \ell_{iK})).$$

Equivalently, one may subtract $\max_k \ell_{ik}$ first and normalize the resulting exponentials. Both calculations preserve the same mathematical answer, but the log-scale version avoids underflow when one component is exponentially less plausible than another.

In models with covariance or precision updates, the M-step raises a second issue. Weighted empirical covariance matrices can be unstable when a latent state receives very little effective mass, and the resulting estimate may be nearly singular even though the population problem is well posed. A small covariance floor, diagonal loading term, or other mild regularization then plays the same role as damping in Newton's method: it preserves positive definiteness and prevents the next E-step from being driven by spurious tiny eigenvalues; see Section 1.4.3. This does not change the basic EM logic; it keeps the geometry of the update trustworthy while the effective sample size assigned to a component is still small.

Initialization remains the third computational safeguard. Because EM is a local method, multiple dispersed starts are often as important as any single numerical trick. They separate two different failure modes:

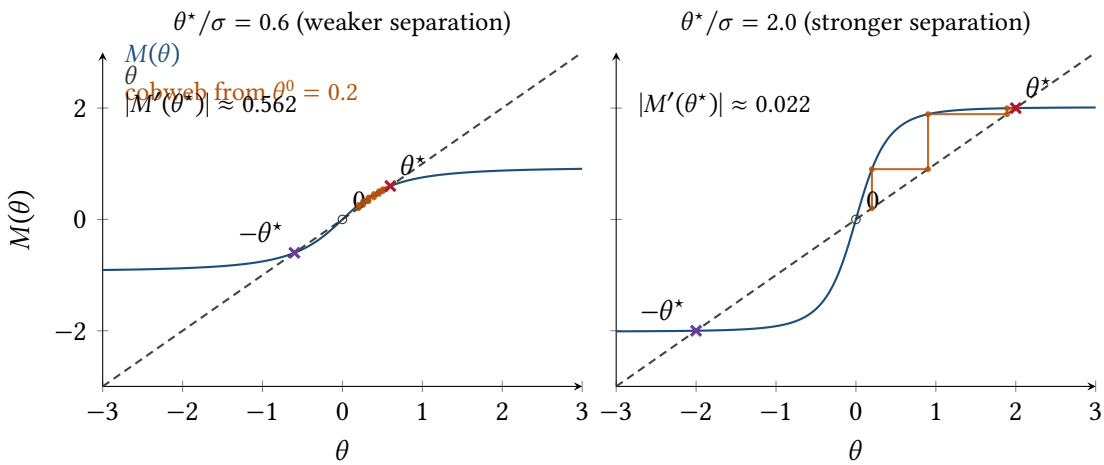

Figure 18: A one-dimensional population EM map for a symmetric Gaussian mixture. The fixed points are the intersections with the diagonal; the filled points are stable and the open point at the origin is unstable. The cobweb paths, shown here from the common initialization $\theta^{(0)} = 0.2$, show how the limiting point depends on the initialization. Weaker separation (left) leaves the local slope near the stable fixed point around 0.56, so refinement is slower; stronger separation (right) drives that slope down to about 0.02, producing much sharper local contraction. This basin picture is why EM is best understood as a local refinement method rather than a globally convergent optimizer.

unstable local arithmetic inside a run, and convergence to different basins across runs. These same log-scale and geometry-control ideas reappear in more complicated latent-variable procedures, including hidden-state models and mixture models, where repeated normalization and covariance estimation are unavoidable.

3.3.4 Initialization, local maxima, and the bridge to VI

The two-coin example highlights why EM is different from a hard-imputation procedure. One could try to replace each unobserved Z_i by its most likely value under the current parameter and then maximize the complete-data likelihood. That approach throws away uncertainty, especially for observations that are compatible with either latent state. EM keeps this uncertainty in the E-step by averaging over the full conditional law $p_{\theta^{(t)}}(z \mid x)$.

At the same time, EM remains a local method. The likelihood ascent property from Proposition 3.3 says that EM moves uphill, but it does not say *which* hill the algorithm will climb. For latent-variable models with multiple basins of attraction, different initial conditions can lead to different fixed points or different local maxima. Practitioners therefore frequently run EM from several dispersed initializations and retain the run with the largest final likelihood.

The two panels of Figure 18 sharpen two central intuitions. First, EM is basin-based: the initialization determines which stable fixed point the iterates approach. Second, once EM is inside the right basin, the local slope of the map controls the speed of refinement. When the latent classes are weakly separated, the map lies closer to the diagonal near the target and convergence is slower; when the classes are better separated, the map bends more sharply and the iterates lock in much faster. For Gaussian mixtures, these basin and contraction phenomena are formalized in both local and global analyses: Balakrishnan et al. (2014, Cor. 1–2) prove local contractivity inside a basin around the truth, while Daskalakis et al. (2016, Thm. 1) and Xu et al. (2016, Thms. 3–4) analyze the fixed-point geometry and broader convergence picture.

There is also a useful comparison with Gibbs sampling and with the coordinate-ascent variational updates

that appear later in the course. All three methods exploit latent-variable structure by alternating conditional updates, but they use those conditionals differently:

$$\text{Gibbs: sample } z \sim p_\theta(z \mid x), \qquad \text{EM: set } q(z) = p_\theta(z \mid x) \text{ and maximize in } \theta,$$

$$\text{CAVI: update } q_j(z_j) \propto \exp\big(\mathbb{E}_{q_{-j}}[\log p_\theta(x, z)]\big) \text{ inside a restricted family.}$$

Gibbs alternates conditional sampling, EM alternates exact conditional expectation and maximization, and CAVI keeps the same free-energy logic while restricting the latent law to a tractable family. This is another instance of the augmented-state viewpoint used throughout this unit: introducing extra probabilistic structure can make it possible to explore or optimize a landscape in ways that purely local deterministic updates cannot.

The important conceptual handoff is that EM and VI optimize the same free-energy identity. Exact EM uses the unrestricted update

$$q^{(t+1)}(z) = p_{\theta^{(t)}}(z \mid x),$$

whereas variational inference keeps the model $p_\theta(x, z)$ fixed and replaces that exact posterior by the best approximation inside a tractable family $\mathcal{Q}$:

$$q^{(t+1)} \in \arg\max_{q \in \mathcal{Q}} \mathcal{F}(q, \theta^{(t)}).$$

The next unit adopts this (x, z, θ) notation throughout and rewrites the same free-energy objective as the evidence lower bound in Section 5.1.

4 Augmented-state sampling

Augmented-state sampling also enlarges the state, but now the goal is to improve how we move through a target family rather than to change the objective being optimized. For fixed-target MCMC, auxiliary variables improve exploration while preserving the target law π; for sequential methods, a weighted or interacting particle population and its ancestry represent a target that changes as new data arrive. Momentum, adaptive trajectory building, temperature ladders, and particle systems all follow the same idea: add state so local computations can make more global progress.

4.1 Hamiltonian Monte Carlo

In Section 2.3.5 we used gradients to bias proposals toward regions where the target density is large. That helps a chain find the high-density part of the distribution, but it does not by itself solve the harder geometric problem: once the chain reaches that region, how should it travel through it without falling back into short random-walk-style moves? Overdamped Langevin dynamics still pushes directly toward lower potential energy. That inward drift is useful for finding the right region, but once the chain is there it becomes a limitation: in high dimensions the mode is not where most of the probability mass lies, and moving straight downhill does not efficiently travel through the typical set. Even for a simple Gaussian target, pointwise density decreases with radius while the available volume increases, so the bulk of the probability mass concentrates in an annulus away from the mode; Figure 19 sketches this volume-versus-density tradeoff. HMC addresses this by augmenting the state with momentum and following nearly energy-preserving dynamics that turn local gradient information into long, coherent motion through that shell; see Betancourt (2017, §3) and Neal (2011, §§2–3) for complementary expositions. It is therefore the sampling analogue of the augmented-state viewpoint from Section 3.1. The parallel with momentum methods is explicit:

$$\text{optimization:} \qquad v_{k+1} \approx \beta v_k - \eta \nabla f(x_k), \qquad x_{k+1} = x_k + v_{k+1},$$

$$\text{HMC:} \qquad \dot{x}_t = M^{-1} p_t, \qquad \dot{p}_t = -\nabla U(x_t).$$

In both cases the extra variable stores motion so that local gradient information shapes a longer trajectory rather than a single local correction.

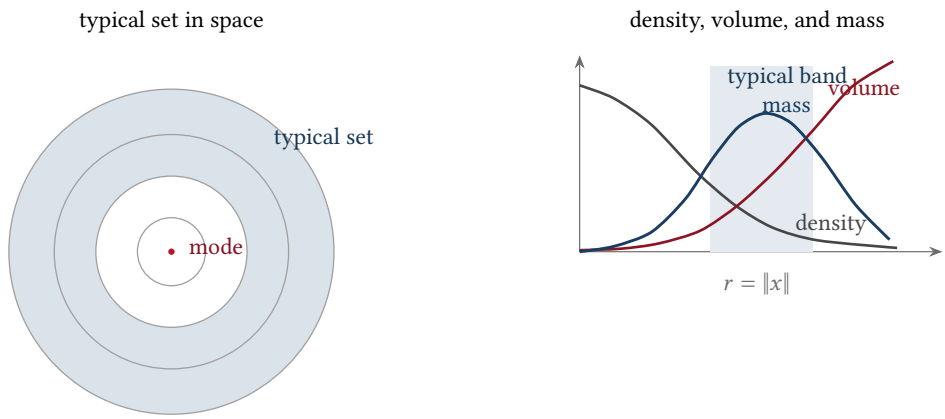

Figure 19: The typical-set viewpoint. Left: for an isotropic target, the mode sits at the center but the relevant mass lies in a shell away from that center. Right: pointwise density decreases with radius while the available volume increases; their product concentrates in an intermediate band that carries most of the probability mass.

The construction therefore starts from one concrete move: add momentum, use it to convert local gradients into a long deterministic trajectory, and then discretize that trajectory carefully enough that a Metropolis correction can repair the remaining numerical error.

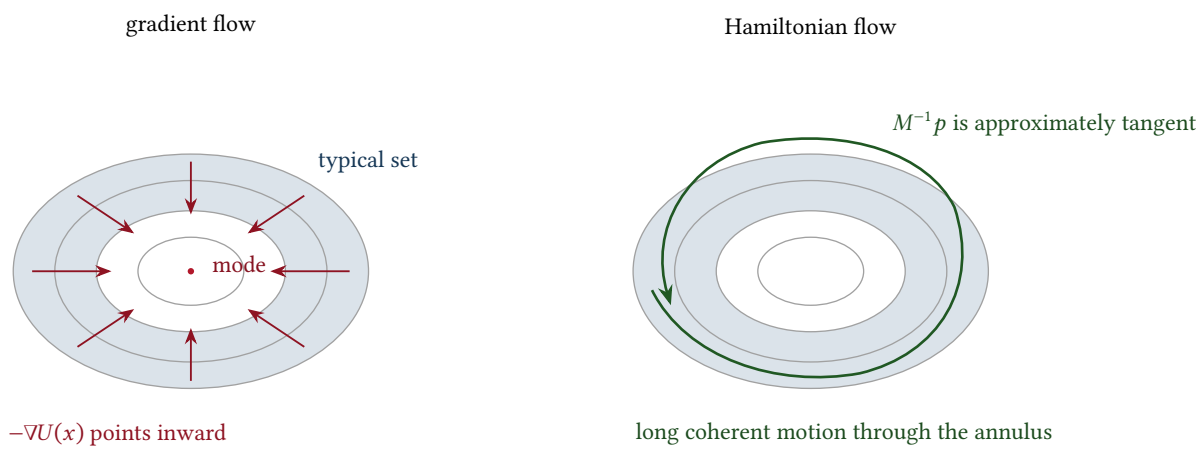

Figure 20: Left: the gradient field points inward toward the mode, so following $-\nabla U(x)$ alone does not move efficiently through the typical set. Right: after augmenting with momentum, Hamiltonian dynamics produces a nearly tangential flow that can move coherently through the typical set while still respecting the target geometry.

4.1.1 Momentum augmentation and Hamiltonian dynamics

Let the target density on $\mathbb{R}^d$ be

$$\pi(x) \propto e^{-U(x)},$$

where $U : \mathbb{R}^d \to \mathbb{R}$ is differentiable. The construction mirrors the momentum-based methods from Section 3.1: we keep the position variable x, add an auxiliary momentum variable p, and choose a simple Gaussian law for that momentum. Specifically, let $M \in \mathbb{R}^{d\times d}$ be positive definite and draw

$$p \sim \mathcal{N}(0, M).$$

The resulting joint density is

$$\Pi(x, p) := \pi(x)\,\varphi_M(p) \propto \exp\big(-H(x, p)\big), \tag{96}$$

where the Hamiltonian is

$$H(x, p) := U(x) + \frac{1}{2}p^\top M^{-1}p. \tag{97}$$

The second term is the *kinetic energy*; the first is the *potential energy*. Because the momentum can be marginalized out explicitly, the x-marginal of Π is exactly π, so it is enough to sample from the joint law and then discard p. The momentum variable does not change the target distribution; it gives the sampler a direction and scale of travel once the deterministic dynamics begin. More generally, one can think of choosing a kinetic energy $K(p)$ and writing $H(x, p) = U(x) + K(p)$. That choice is not cosmetic. Each momentum refresh draws a new value of $K(p)$, which in turn selects the next total energy level $H(x, p)$ from which the deterministic trajectory starts. Good performance therefore requires an *energy match*: the kinetic term should inject momentum on scales that are commensurate with the variation in U, so refreshed trajectories land on energy shells that explore the typical set rather than repeatedly over-energizing stiff directions and under-energizing flat ones. The standard quadratic choice $K(p) = \frac{1}{2}p^\top M^{-1}p$ achieves that matching through the mass matrix M, so choosing M well is both a velocity-scaling decision and an energy-matching decision.

The Hamiltonian in (97) generates the ODE

$$\dot{x} = \nabla_p H(x, p) = M^{-1}p, \qquad \dot{p} = -\nabla_x H(x, p) = -\nabla U(x). \tag{98}$$

The first equation turns momentum into a velocity, while the second says that momentum changes according to the negative gradient of the potential. Compared with overdamped Langevin dynamics, the gradient now acts through a second-order system: instead of determining the whole move directly, it bends an existing trajectory.

Two geometric facts now explain why this deterministic motion is useful for sampling. First, exact trajectories preserve total energy, so they do not simply slide downhill into the mode. Instead, potential and kinetic energy are exchanged along the path while the total remains fixed.

Proposition 4.1 (Hamiltonian flow preserves the Hamiltonian)**.** *Let (x_t, p_t) solve (98). Then $H(x_t, p_t)$ is constant in t.*

Proof. Differentiate (97) along the trajectory:

$$\frac{d}{dt}H(x_t, p_t) = \langle \nabla_x H(x_t, p_t)\rangle \dot{x}_t + \langle \nabla_p H(x_t, p_t)\rangle \dot{p}_t.$$

Using (98),

$$\frac{d}{dt}H(x_t, p_t) = \langle \nabla_x H \rangle \nabla_p H + \langle \nabla_p H \rangle - \nabla_x H = 0.$$

Hence $H(x_t, p_t)$ is constant. ■

The conservation law means that exact Hamiltonian trajectories remain on a fixed energy surface. Informally, one can decompose the joint distribution into a distribution over energy levels $E = H(x, p)$ and a conditional distribution on each energy surface $\{(x, p) : H(x, p) = E\}$. This conditional law is often called the *microcanonical distribution*. From this perspective, an ideal HMC transition alternates between two mechanisms: momentum refreshment changes the energy level, while Hamiltonian dynamics explores the corresponding energy surface. Ideally the conditional distribution of total energy after momentum refreshment, given the current position x, should resemble the marginal energy distribution induced by Π. Otherwise the chain faithfully explores the wrong energy shells: exact dynamics still preserves those shells, but refreshment keeps selecting shells that are badly matched to the typical set. Figure 21 shows this geometry in phase space.

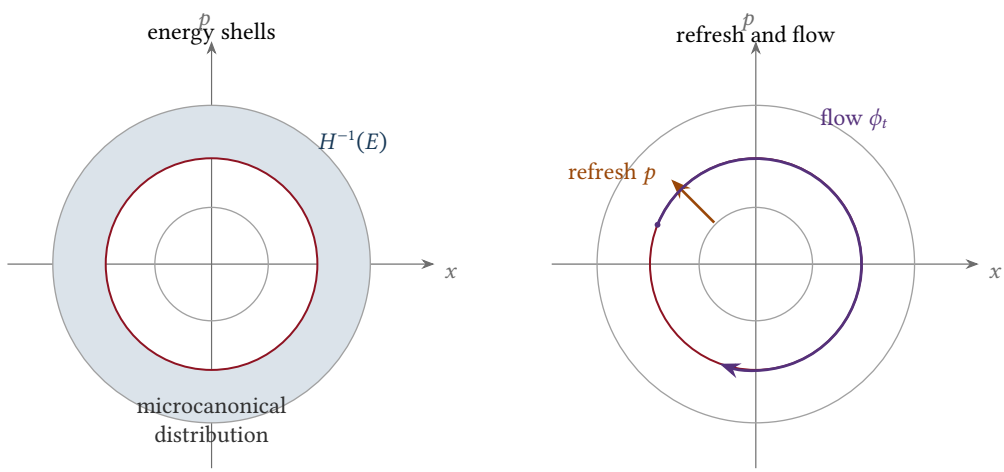

Figure 21: Phase-space geometry for HMC. Left: the canonical distribution on (x, p) can be organized by Hamiltonian level sets. Right: momentum refreshment moves the chain between energy levels, while Hamiltonian flow explores one selected level set without changing its energy.

The second key fact is that exact Hamiltonian flow does not squeeze or stretch phase space. If it did, following the flow would require a Jacobian correction and would not preserve the canonical density in the simple way we want.

Proposition 4.2 (Hamiltonian vector fields are divergence free)**.** *Assume H is twice continuously differentiable. Then the vector field*

$$F(x, p) := \big(\nabla_p H(x, p), -\nabla_x H(x, p)\big)$$

has zero divergence:

$$\nabla \cdot F = 0.$$

Consequently, the exact Hamiltonian flow preserves Lebesgue measure on phase space.

Proof. Write $x = (x_1, \dots, x_d)$ and $p = (p_1, \dots, p_d)$. Then

$$\nabla \cdot F = \sum_{i=1}^{d} \frac{\partial^2 H}{\partial x_i \, \partial p_i} - \sum_{i=1}^{d} \frac{\partial^2 H}{\partial p_i \, \partial x_i} = 0$$

by equality of mixed partial derivatives. The volume-preservation statement is Liouville's theorem for this divergence-free flow. ■

The two propositions capture the essential geometry. Because H is constant along exact trajectories, the density $e^{-H(x,p)}$ is also constant along those trajectories. Because the flow preserves phase-space volume, there is no Jacobian correction. Hence exact Hamiltonian dynamics leaves the canonical density $\Pi(x, p) \propto e^{-H(x,p)}$ invariant. An exact HMC step therefore has a clean interpretation: refresh the momentum to choose an energy level and a direction of travel, then move deterministically along the corresponding level set.

4.1.2 Exact Hamiltonian transitions

If exact solutions of (98) were available, one HMC iteration would be conceptually simple. Starting from the current position $x^{(t)}$, draw fresh momentum $p^{(t)} \sim \mathcal{N}(0, M)$, follow Hamiltonian dynamics for some integration time τ, and keep the resulting position. Because the momentum refreshment step samples from the correct conditional law under (96), and the Hamiltonian flow preserves Π, the resulting Markov chain leaves π invariant.

This description already explains why HMC suppresses random-walk behavior. After the momentum is refreshed, the trajectory can travel a macroscopic distance through the typical set before the next stochastic step. In the idealized exact-dynamics picture there is also no accept/reject cost: every simulated trajectory is already distributed correctly. The only reason a Metropolis correction appears in practice is that we must approximate the Hamiltonian flow numerically.

4.1.3 Leapfrog integration and Metropolis correction

Exact Hamiltonian trajectories are rarely available in closed form, so we must discretize (98). A naive forward Euler step is unstable for long trajectories because it systematically injects or dissipates energy, causing the numerical path to drift away from the intended energy surface. For the separable Hamiltonian (97), the standard discretization is the leapfrog method:

$$p_{n+\frac{1}{2}} = p_n - \frac{\epsilon}{2} \nabla U(x_n), \tag{99}$$

$$x_{n+1} = x_n + \epsilon M^{-1} p_{n+\frac{1}{2}}, \tag{100}$$

$$p_{n+1} = p_{n+\frac{1}{2}} - \frac{\epsilon}{2} \nabla U(x_{n+1}), \tag{101}$$

where $\epsilon > 0$ is the step size. If we apply (99)–(101) for L steps, we obtain a numerical proposal map

$$(x, p) \mapsto (\tilde{x}, \tilde{p}).$$

The three lines have a simple interpretation: half a momentum update, then a full position update, then the second half of the momentum update. This symmetric "kick–drift–kick" structure is what gives leapfrog its good long-time behavior.

A subtle point is that the raw leapfrog endpoint $(\tilde{x}, \tilde{p})$ is not yet the right reversible proposal. To retrace the same discrete path one must run the integrator backward in time, which for this symmetric scheme

is equivalent to flipping momentum at the endpoint. That is why HMC packages the proposal as $(\tilde{x}, -\tilde{p})$ rather than $(\tilde{x}, \tilde{p})$ before applying the Metropolis correction. If $\Psi_{\epsilon,L}$ denotes the L-step leapfrog map and $R(x, p) := (x, -p)$ the momentum-flip map, then the reversibility identity is

$$\Psi_{\epsilon,L}^{-1} = R \circ \Psi_{\epsilon,L} \circ R.$$

So the leapfrog endpoint itself is not the self-reversible proposal used in the Metropolis ratio; the reversible proposal is $R \circ \Psi_{\epsilon,L}$.

The leapfrog map is volume preserving and reversible after a momentum flip, so a Metropolis correction yields an exact sampler. Specifically, if we view $(\tilde{x}, -\tilde{p})$ as the proposal, then the acceptance probability is

$$\alpha\big((x, p), (\tilde{x}, -\tilde{p})\big) = \min\Big\{1, \exp\big(-H(\tilde{x}, \tilde{p}) + H(x, p)\big)\Big\}. \tag{102}$$

Because the kinetic energy is even in p, there is no difference between $H(\tilde{x}, \tilde{p})$ and $H(\tilde{x}, -\tilde{p})$. If we could integrate the ODE exactly, then the energy difference in (102) would vanish and every proposal would be accepted. The qualitative reason leapfrog works so well is that its numerical error remains organized around the true energy shell rather than drifting coherently away from it. By contrast, a naive Euler scheme will typically spiral off the intended level set. This is the geometric content of Figure 22.

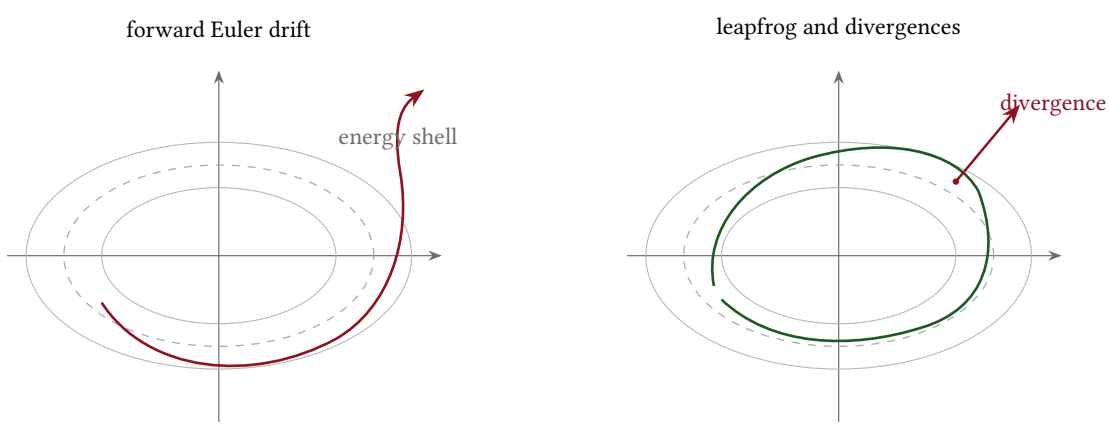

Figure 22: Numerical integration for HMC. Left: a naive Euler discretization drifts across energy levels and is unsuitable for long trajectories. Right: leapfrog remains close to the intended level set for long times, although sufficiently severe local curvature can still produce a sharp departure that implementations detect as a divergence.

Algorithm 20 Hamiltonian Monte Carlo

Input: Potential U, gradient ∇U, mass matrix $M \succ 0$, step size ϵ, leapfrog steps L, initial state $x^{(0)}$, iterations T

```
for t = 0, 1, ..., T − 1 do
    Draw p^(0) ~ N(0, M).
    (x, p) ← (x^(t), p^(0)).
    for ℓ = 1, 2, ..., L do
        p ← p − (ε/2) ∇U(x).
        x ← x + ε M^−1 p.
        p ← p − (ε/2) ∇U(x).
    end for
    ΔH ← H(x, p) − H(x^(t), p^(0)).
    Draw u ~ Unif(0, 1).
    if log u ≤ −ΔH then
        x^(t+1) ← x
    else
        x^(t+1) ← x^(t)
    end if
end for
return x^(0), x^(1), ..., x^(T)
```

The rejection step in Algorithm 20 corrects the discretization error but does not remove the need for careful tuning. If the numerical trajectory is too inaccurate, then many proposals will be rejected and the chain will again behave like a local random walk. The gain from HMC comes from keeping the leapfrog trajectory accurate *enough* that long moves are accepted with substantial probability. HMC is therefore best understood as a geometric method: its efficiency comes from aligning the numerical trajectory with the target geometry over many leapfrog steps. For the same reason, small implementation errors in ∇U can be much more damaging here than in a one-step local proposal. Each leapfrog path reuses the gradient repeatedly, so a missing Jacobian term or a sign error can show up as systematic energy drift long before any tuning parameter is adjusted. When gradients are coded by hand, it is therefore worth checking them at a few representative points against finite-difference derivatives of the same scalar potential U, including any transformation terms built into U itself as in Section 2.1.5. The remaining tuning burden is the trajectory length. Choosing too few leapfrog steps brings back local random-walk behavior, while choosing too many wastes computation by retracing the same orbit. The next subsection addresses exactly that burden. The finite-difference calculation is not the final sampler. It is a local correctness check that helps separate tuning problems from coding errors before warm-up begins.

4.1.4 Mass-matrix adaptation and trajectory design

The matrix M plays the role of a preconditioner. If the target is approximately Gaussian with covariance Σ, then locally

$$U(x) \approx \frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu), \qquad \ddot{x} \approx -M^{-1}\Sigma^{-1}(x-\mu).$$

As in Section 1.4.3, one wants the metric to balance scales across directions rather than forcing one numerical step size to resolve every curvature at once. Here that general principle has a concrete sampler consequence: if M^{-1} is comparable to Σ, equivalently M to the local precision Σ^{-1}, then the velocity $M^{-1}p$ is smaller in steep directions and larger in shallow ones, so a single step size ϵ is usable across more of the state space. Figure 23 illustrates how an adapted metric redistributes motion toward the shallow directions instead of

wasting most of the numerical effort in the steep ones. This is the same energy-matching principle from Section 4.1.1 in a more concrete form: a well-chosen mass matrix makes a refreshed momentum draw inject a plausible amount of kinetic energy in each direction, so the chain starts each trajectory on an energy shell that actually intersects the typical set in a useful way. In practice one often starts with $M = I_d$ or a diagonal estimate and adapts the inverse mass matrix M^{-1} during warm-up using pilot samples. Those estimates are noisy, so the same safeguards from Section 1.4.3 apply here as well: diagonal approximations, shrinkage, or ridge regularization are used to freeze a positive definite metric that captures the dominant anisotropy without overreacting to unstable covariance estimates.

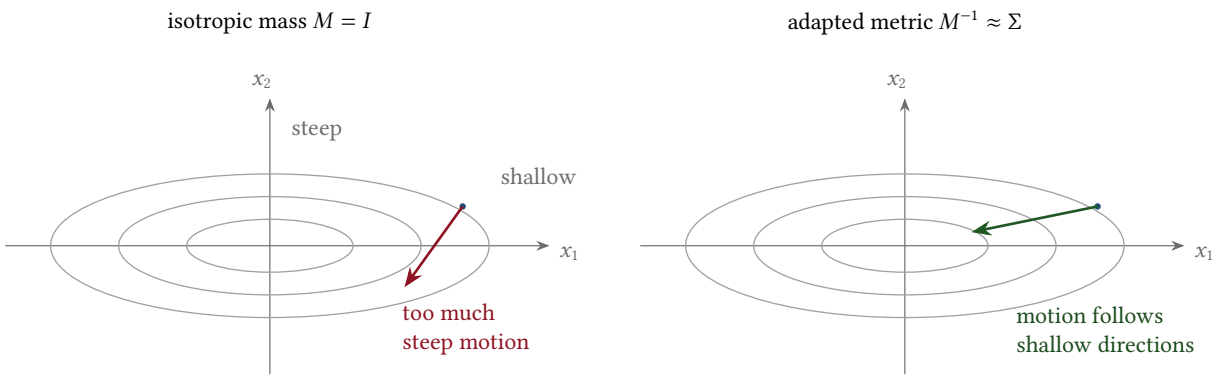

Figure 23: Mass-matrix adaptation in an anisotropic target. The target contours are elongated, so a single isotropic momentum scale is poorly matched to the geometry. Choosing M^{-1} closer to the target covariance, equivalently M closer to the target precision, damps motion in steep directions and allows longer, more balanced travel through shallow directions.

The step size ϵ controls numerical accuracy. If ϵ is too large, the leapfrog error in ΔH grows, the acceptance rate falls, and the integrator may become unstable. If ϵ is too small, each individual leapfrog step is accurate but the algorithm wastes work on tiny moves. A useful practical target is an average acceptance rate somewhere around 0.6–0.8, with many modern implementations defaulting near the upper end of that range; see Neal (2011, §4.2) and Hoffman and Gelman (2014, §3.2).

The path length ϵL controls how far each deterministic excursion travels through phase space. If L is too small, then even accurate trajectories move only a short distance and the chain reverts toward random-walk behavior. If L is too large, the trajectory can loop back toward where it started, spending gradient evaluations undoing earlier movement. Moreover, an unfortunate fixed value of L can resonate with the natural frequencies of the Hamiltonian system. The practical warning is specific: a perfectly deterministic choice of path length can accidentally line up with a near-periodic orbit, so successive trajectories keep returning close to where they started even when the integrator itself is accurate. One common hedge is therefore to randomize L over a modest range across iterations; Neal (2011, §4, p. 25) recommends this precisely to avoid such near-periodicity. The more systematic remedy is NUTS, which chooses the path length adaptively. The practical lesson is that M, ϵ, and L should be tuned together: the mass matrix sets the geometry, the step size resolves that geometry numerically, and the path length determines how fully each trajectory exploits it. The Metropolis correction guarantees invariance of π, but quantitative convergence is a separate theorem-level question. Durmus et al. (2019, Thms. 1–2 and 9) make that distinction precise: they prove irreducibility and Harris recurrence under an $A1$-type global regularity/growth condition on U, and geometric ergodicity under stronger directly verifiable assumptions on the potential. Betancourt (2017, §6) complements that theorem-level view by emphasizing the practical obstructions, especially heavy tails

and neighborhoods of extreme curvature. We will not prove those results here, but it is worth keeping the distinction in view: exactness comes from reversibility and volume preservation, whereas exponential forgetting of the initial state requires additional drift control that is not automatic from the Metropolis correction alone.

4.1.5 Divergences and reparameterization

The most important failure mode to recognize in practice is a *divergence*. Very high curvature can make the leapfrog discretization numerically unstable before the intended trajectory length is completed. When this happens, the simulated path departs sharply from the approximate energy surface and the implementation flags the transition as divergent. A divergence is therefore more informative than an ordinary rejection: it is a warning that the numerical trajectory is no longer faithfully tracking the target geometry.

Mild divergences can sometimes be removed by decreasing ϵ. Persistent divergences, however, usually indicate a geometric problem rather than a mere tuning problem. Strong anisotropy, tight funnels, or highly correlated parameterizations can force HMC to choose between moving accurately in the narrowest direction and moving efficiently everywhere else. In such cases reparameterization is often more effective than continued step-size reduction. Neal's funnel in Figure 13 is a canonical illustration: a non-centered parameterization can dramatically improve the geometry seen by HMC and NUTS.

4.2 The No-U-Turn Sampler

One of the hardest parts of tuning HMC is choosing the trajectory length. The No-U-Turn Sampler (NUTS) of Hoffman and Gelman (2014, §3.1) automates this choice. Its basic idea is simple: keep extending the HMC trajectory while it is still making productive forward progress, and stop once further integration would mostly retrace the same region. The step size ϵ can still be tuned from acceptance behavior, but NUTS replaces the hand-chosen leapfrog count L with a geometric stopping rule. In a nearly elliptical geometry that rule tries to run long enough to cross the orbit without circling back and undoing earlier movement.

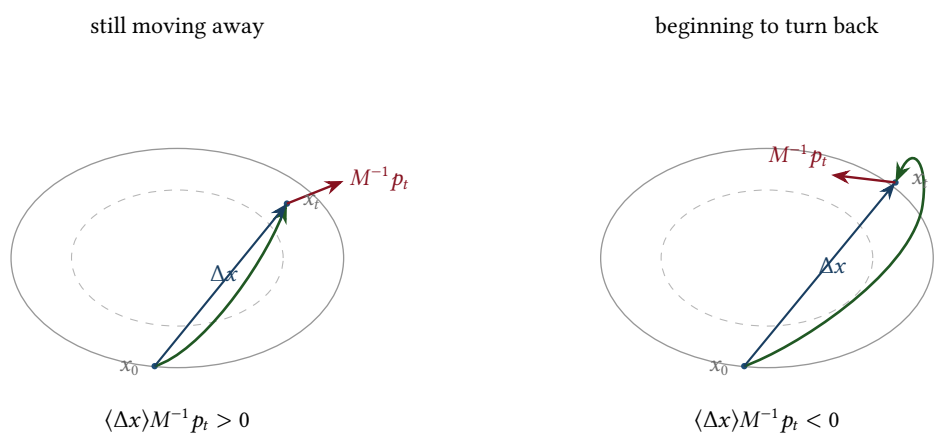

Figure 24: A schematic of the NUTS stopping idea. While the secant vector Δx and the velocity $M^{-1}p$ point in roughly the same direction, extending the trajectory is still productive. Once their inner product becomes negative, the trajectory is turning back toward where it came from and further leapfrog steps are likely to waste computation.

4.2.1 The U-turn stopping criterion

Suppose an exact Hamiltonian trajectory starts at x_0 and reaches x_t with momentum p_t. If we write $\Delta x_t := x_t - x_0$, then the squared distance from the starting point changes according to

$$\frac{d}{dt}\frac{1}{2}\|x_t - x_0\|^2 = \langle x_t - x_0\rangle \dot{x}_t = \langle x_t - x_0\rangle M^{-1} p_t. \tag{103}$$

Equation (103) is the key geometric diagnostic. As long as it is positive, the trajectory is still moving away from its starting point. When the derivative becomes negative, the trajectory has begun to turn back. On an elliptical orbit this is exactly the moment at which continuing the simulation starts to undo previous movement rather than discovering new regions of the typical set.

This observation gives the core stopping idea: once the trajectory begins to turn back, additional leapfrog steps are no longer buying new exploration. The probabilistic difficulty is that we cannot simply stop at the first realized U-turn and keep that endpoint, because the stopping rule itself depends on the realized path. The remaining construction packages this geometric stopping idea into a symmetric HMC transition.

4.2.2 Symmetric tree expansion and candidate selection

NUTS implements that stopping idea with two pieces of machinery. First, a slice variable excludes states with excessively large energy. Second, a doubling construction grows a symmetric set of candidate states around the current point instead of privileging a single realized endpoint.

The slice variable

$$u \sim \mathrm{Unif}\left(0, e^{-H(x_0,p_0)}\right), \tag{104}$$

where (x_0, p_0) is the current phase-space state after momentum refreshment. Only states satisfying

$$e^{-H(x,p)} \geq u$$

are eligible candidates for the next draw. This slice restriction prevents obviously poor numerical states from entering the candidate set and makes the correctness argument symmetric.

Starting from the current state, NUTS builds a balanced binary tree by repeatedly choosing a direction $v_j \in \{-1, +1\}$ and integrating 2^j leapfrog steps of size $v_j\epsilon$ at depth j. At every stage it maintains left and right endpoints (x^-, p^-) and (x^+, p^+), along with the candidate states encountered so far that satisfy the slice condition. Concretely, depth 0 adds one leapfrog step to one side of the current state. If no stop is triggered, depth 1 appends a fresh subtree of length 2, then depth 2 appends one of length 4, and so on, so the explored path length doubles at each successful stage. The tree is therefore just a bookkeeping device for this recursive concatenation. One can think of each subtree as carrying a small summary: its left and right endpoints, the number of slice-admissible states inside it, and a flag saying whether a U-turn or divergence has already appeared. Those summaries are enough to keep the construction symmetric between forward and backward time without privileging the order in which individual leapfrog states were generated. This bookkeeping also explains candidate selection. After a valid subtree has been grown, NUTS does not simply keep the newest endpoint. Instead it merges the old and new subtrees and updates the candidate by sampling from the union with probability proportional to the number of slice-admissible states contributed by each side. That recursive size-weighted merge is how implementations reproduce the uniform law on the admissible candidate set without explicitly storing every visited point in a long trajectory.

Tree expansion stops as soon as one of two things happens. The first is a divergence, meaning the energy error has become too large for the numerical trajectory to be trusted. The second is an endpoint version of

the U-turn condition:

$$\langle x^+ - x^- \rangle M^{-1} p^- < 0 \qquad \text{or} \qquad \langle x^+ - x^- \rangle M^{-1} p^+ < 0. \tag{105}$$

The secant $x^+ - x^-$ summarizes the currently explored trajectory segment, and the two endpoint momenta check whether either end is now pointing back across that segment. When (105) holds, further doubling would mostly revisit the same region rather than extend exploration. Figure 25 illustrates the doubling logic schematically. There is also no separate Metropolis accept/reject step at the end of this construction. Energy fluctuations are handled indirectly by the slice restriction and by terminating the expansion when a divergence is detected, after which the next state is drawn from the admissible set already built inside the slice.

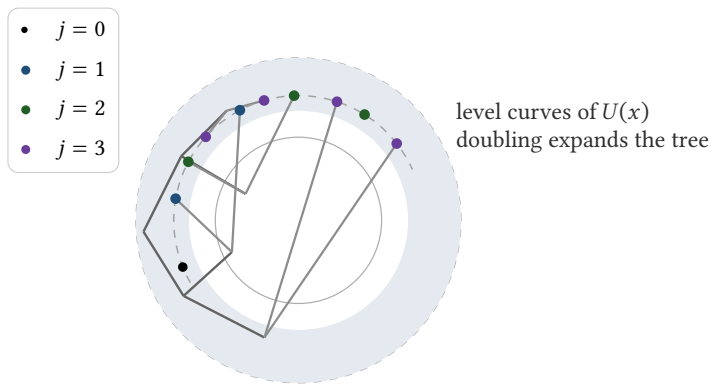

Figure 25: A schematic NUTS tree grown along an HMC trajectory. Each doubling adds a new half-tree of leapfrog states, increasing the explored path length exponentially until a U-turn or a divergence stops the expansion.

Algorithm 21 High-level NUTS iteration

Input: Current state $x^{(t)}$, mass matrix M, step size ϵ, maximum tree depth $J_{\max}$

Draw $p_0 \sim \mathcal{N}(0, M)$ and $u \sim \text{Unif}\left(0, e^{-H(x^{(t)}, p_0)}\right)$.
Initialize left and right endpoints at $(x^{(t)}, p_0)$ and candidate set $\mathcal{C} \leftarrow \{(x^{(t)}, p_0)\}$.
for $j = 0, 1, \ldots, J_{\max}$ **do**
    Choose a direction $v_j \in \{-1, +1\}$.
    Extend the corresponding endpoint by 2^j leapfrog steps of size $v_j \epsilon$.
    Add any new states satisfying the slice condition $e^{-H(x,p)} \geq u$ to $\mathcal{C}$.
    if a divergence occurs or the endpoint criterion (105) holds **then**
        break
    end if
end for
Sample the next state $x^{(t+1)}$ using the candidate-selection rule that preserves the uniform law on admissible candidates in $\mathcal{C}$.

The original algorithm samples uniformly from the admissible candidate set inside the slice. Modern implementations often use a closely related multinomial sampling rule that weights states by $e^{-H(x,p)}$ instead of enforcing the slice variable explicitly. The implementation details differ, but their role is the

same: turn the geometric rule "stop when the trajectory starts to retrace" into a valid symmetric transition kernel. The maximum tree depth $J_{\max}$ is only a computational cap on this doubling procedure. Hitting that cap means the sampler would have kept exploring if allowed; it does not indicate a correctness failure, only that the implementation has limited the longest admissible trajectory for cost reasons.

4.2.3 Why the transition is still valid

The key point is that NUTS is *not* "run HMC until the first U-turn and keep the endpoint." That naive rule would privilege one realized stopping point and would generally break the symmetry needed for detailed balance. Instead, NUTS uses the slice variable and the doubling tree to construct a candidate set that contains the current state, excludes obviously bad high-energy states, and is built in a way that is symmetric between forward and backward integration; this is the core construction in Hoffman and Gelman (2014, §§3.1.1–3.1.2).

Once that symmetric candidate set has been built, the next state is drawn by a transition rule that leaves the uniform distribution on the admissible candidates invariant. In the simplified presentation that means sampling uniformly from the slice-restricted set. In the efficient implementation it means using subtree sizes to reproduce the same uniform law without storing every visited state explicitly. The correctness mechanism is therefore easy to summarize even though the implementation details are intricate: NUTS preserves HMC's reversible geometry by replacing a single path-dependent endpoint with a symmetric set of admissible states and then sampling from that set with a reversible rule.

4.2.4 Warm-up and practical tuning

In practice NUTS is almost always paired with automatic adaptation of the step size and mass matrix during warm-up. Warm-up has two jobs: choose a step size that gives stable, high-acceptance trajectories, and estimate a mass matrix that matches the scale of the target. The original NUTS paper uses dual averaging on $\log \epsilon$ to target a chosen average acceptance statistic δ; see Hoffman and Gelman (2014, §3.2, p. 1608). In the optimization language of Section 3.1.4, the quantity being updated is now the scalar tuning parameter $\log \epsilon$ rather than the model parameter itself, and the role of the noisy gradient is played by the acceptance error. If α_j denotes the acceptance statistic from warm-up iteration j, one defines $H_j := \delta - \alpha_j$ and updates

$$\bar{H}_j = \left(1 - \frac{1}{j + t_0}\right) \bar{H}_{j-1} + \frac{1}{j + t_0} H_j, \qquad \log \epsilon_j = \mu - \frac{\sqrt{j}}{\gamma} \bar{H}_j,$$

for a baseline value μ, then forms the smoothed average

$$\log \bar{\epsilon}_j = j^{-\kappa} \log \epsilon_j + \left(1 - j^{-\kappa}\right) \log \bar{\epsilon}_{j-1}.$$

The first recursion aggregates acceptance errors, while the second maps that accumulated dual signal back to the primal tuning variable $\log \epsilon$. The step-size recursion is therefore also called *Nesterov's primal-dual averaging*: it maintains an aggregated dual error signal and maps it back to a primal tuning variable. It is therefore distinct from Nesterov's accelerated gradient, which updates the model parameter by a look-ahead gradient step. The commonly used defaults from that construction are $\gamma = 0.05$, $t_0 = 10$, and $\kappa = 0.75$.

Warm-up is staged because the two tuning problems live on different time scales. The step size reacts quickly: after a short run it is already clear whether trajectories are numerically stable and whether the acceptance statistic is near the target. The mass matrix is slower because it requires a covariance estimate from a window of warm-up draws that is long enough to say something reliable about scale.

A typical workflow therefore has three phases. One starts with $M = I$ (or a simple diagonal guess) and tunes only ϵ until trajectories are numerically stable and the average acceptance statistic is in a sensible

range. Then come the slower covariance windows: warm-up draws are accumulated, a new estimate of M^{-1} is formed, and the step-size adaptation is restarted because the geometry has just changed. This middle stage is repeated with longer windows until the metric estimate stops moving enough to matter. Only then does one freeze M and do a final short retuning pass for ϵ with the geometry held fixed. This is the practical meaning of the staged warm-up used in modern NUTS implementations: fast adaptation for stability, slower windowed adaptation for geometry, then fast retuning once the geometry has been fixed. The common summary is therefore "fast-slow-fast": short early adaptation for ϵ, long middle windows for M, then a short final cleanup pass for ϵ with M fixed.

As in HMC, the geometric rationale for the metric comes from Section 1.4.3; the sampler-specific point is that a better metric makes the no-U-turn rule act on a trajectory whose numerical scale is already better balanced across directions. Because the covariance estimates are noisy, practical implementations use windowed updates, diagonal approximations, or mild shrinkage and ridge regularization before freezing the metric. Once warm-up is finished, the tuning parameters are frozen and standard post-warm-up diagnostics such as $\hat{R}$, effective sample size, trace plots, and divergence counts become the relevant checks; see Section 2.3.3.

From a practical point of view, NUTS removes the most fragile hand-tuned HMC parameter. After warm-up, the user no longer chooses L directly: the realized trajectory length is whatever the no-U-turn logic allows before retracing begins. Modern probabilistic programming systems therefore often expose NUTS, rather than plain fixed-L HMC, as the default general-purpose gradient-based sampler.

4.3 Tempering methods

Hamiltonian Monte Carlo, NUTS, MALA, and random-walk Metropolis all make *local* moves. That locality is not a serious problem when the target has one connected typical set, but it becomes a major obstacle when $\pi(x) \propto e^{-U(x)}$ has several well-separated modes. If two basins are separated by a region where $U(x)$ is much larger, then any local chain must spend a long time proposing moves through exponentially unlikely states before it can switch basins. The practical symptom is familiar: the chain mixes rapidly *within* one mode but spends an intractably long time discovering the others.

Tempering addresses this by introducing auxiliary distributions whose energy barriers are lower than those of the target. For an inverse temperature $\beta \in (0, 1]$, define the tempered density

$$\pi_\beta(x) \propto e^{-\beta U(x)}. \tag{106}$$

When $\beta = 1$ we recover the target distribution, and when $\beta < 1$ the landscape is flatter because every energy difference is scaled down by β. This is the same flattening mechanism that appeared in simulated annealing in Section 3.2, but the goal is different here: we are not cooling toward an optimizer, but using hot auxiliary levels to help an MCMC chain travel between cold modes while still preserving the original target at $\beta = 1$. This temperature-ladder viewpoint is a standard extension of local MCMC for multimodal targets; see Neal (2011, §5.7) for the high-level sampling motivation and Earl and Deem (2005, §§2.1–2.3) for the standard replica-exchange formulation. Figure 26 sketches this geometry.

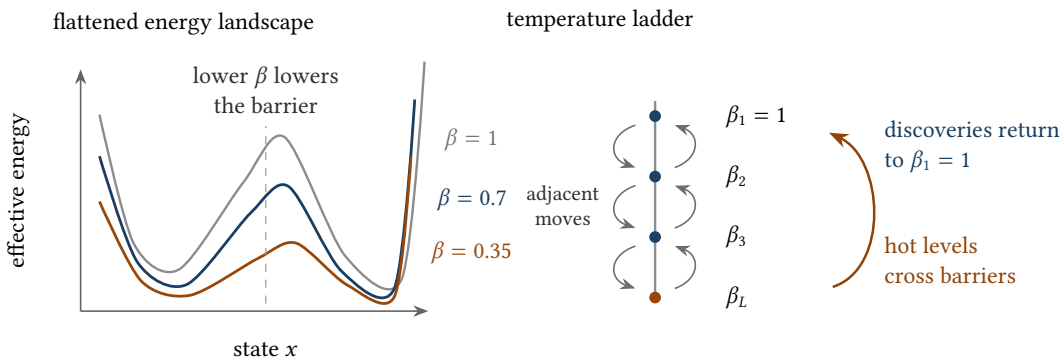

Figure 26: Lower inverse temperatures flatten the effective energy landscape and make barrier crossing easier. Tempering methods introduce a finite ladder of such auxiliary levels and then move information between them so that discoveries made by hot chains can propagate back to the cold target.

The augmented-state logic is the same in both tempering methods. One introduces auxiliary hot levels on which local kernels can cross barriers more easily, then couples those levels to the cold target so that improved basin exploration at high temperature can be transferred back to $\beta = 1$. Simulated tempering and parallel tempering are the two standard realizations of that idea: the first augments the state with a temperature index, while the second augments it with a full collection of replicas across the ladder.

In practice one fixes a finite inverse-temperature ladder

$$1 = \beta_1 > \beta_2 > \cdots > \beta_L > 0 \tag{107}$$

and, for each level ℓ, chooses a Markov kernel K_ℓ that leaves π_{β_ℓ} invariant. The kernels can all be of the same type, such as random-walk Metropolis or HMC with temperature-dependent tuning, or they can differ across levels. What matters is that each level mixes reasonably well *locally*, since the purpose of the ladder is to handle global movement across modes.

4.3.1 From annealing to temperature ladders

Simulated annealing uses one chain and a temperature schedule that is eventually driven toward 0, so the hot exploratory dynamics disappear as the run proceeds. Tempering keeps the same flattening idea but organizes it differently. Instead of a single cooling schedule, it augments the state with a finite ladder of temperatures and preserves a joint stationary law whose cold marginal is still the target. Hot exploration is therefore not a transient prelude to the real computation; it remains available throughout the run and can continue to feed information back to the cold chain.

The practical role of the ladder is to balance two competing demands. The hottest levels should be flat enough to cross barriers that trap the cold target, while neighboring temperatures must still be close enough that information can move back down the ladder. If the gaps are too large, the hot and cold levels effectively decouple. If they are too small, one spends computation maintaining many nearly identical replicas or temperature states. The ladder therefore plays the same role as a geometric discretization: it decides how finely the auxiliary state must resolve the hard part of the original problem.

4.3.2 Simulated tempering

Simulated tempering lets a single chain move between temperature levels. At the current level it updates x with the local kernel for that temperature, then it proposes changing the temperature index and accepts

or rejects that move by a Metropolis rule. If those temperature moves mix well, one chain can alternate between hot exploration and cold sampling instead of maintaining a full population of replicas.

To write the invariant law, augment the state with the temperature index itself. Let $I \in \{1, \dots, L\}$ denote the current inverse-temperature level, and choose log-weights $g_1, \dots, g_L$. The joint target on (x, I) is

$$\Pi_{\mathrm{ST}}(x, \ell) \propto \exp\bigl(-\beta_\ell U(x) + g_\ell\bigr). \tag{108}$$

The x-conditional given $I = \ell$ is exactly π_{β_ℓ}. The temperature marginal is

$$\Pi_{\mathrm{ST}}(I = \ell) \propto e^{g_\ell} Z_\ell, \qquad Z_\ell := \int e^{-\beta_\ell U(x)}\, dx. \tag{109}$$

Hence choosing $g_\ell \approx -\log Z_\ell$ makes the ladder approximately flat under stationarity. This is useful because a simulated-tempering chain that almost never visits the hot levels cannot use them to move between modes. In practice one does not need highly accurate free-energy estimates here. What matters is that the empirical distribution of the temperature index is not grossly imbalanced. Because only differences of the g_ℓ matter in (110), it is natural to tune them directly on the log scale: one can start with all g_ℓ equal, or with rough pilot estimates of $\log Z_\ell$ obtained from short runs or neighboring-temperature importance estimates, and then adjust the g_ℓ after a pilot run so that rarely visited temperatures get larger weights and overvisited temperatures get smaller ones. For neighboring levels this estimation problem is mildest because

$$\frac{Z_{\ell+1}}{Z_\ell} = \mathbb{E}_{\pi_{\beta_\ell}}\bigl[\exp\bigl(-(\beta_{\ell+1} - \beta_\ell)U(X)\bigr)\bigr],$$

so short pilot runs can estimate successive log-ratios and then accumulate them along the ladder. As usual with importance sampling, this is reliable only when neighboring temperatures overlap well enough. It is therefore common to work with auxiliary pilot parameters $v_\ell := g_\ell$ and update those directly: if temperature ℓ is visited too rarely, increase v_ℓ; if it is visited too often, decrease v_ℓ. Those pilot values are only a device for keeping the index I moving; they need not be precise estimates of the normalizing constants themselves.

There are two natural move types. At fixed temperature we update x using the kernel K_ℓ. At fixed x we propose a new temperature index and accept or reject it by a Metropolis step on (108).

Proposition 4.3 (Temperature-move acceptance ratio)**.** *Suppose the current state is (x, ℓ) and propose a new level m from a proposal mass function $r(m \mid \ell)$. Then the Metropolis acceptance probability for the target (108) is*

$$\alpha_{\mathrm{ST}}\bigl((x, \ell), (x, m)\bigr) = \min\left\{1,\ \exp\bigl(-(\beta_m - \beta_\ell)U(x) + g_m - g_\ell\bigr)\frac{r(\ell \mid m)}{r(m \mid \ell)}\right\}. \tag{110}$$

Proof. The Metropolis ratio is

$$\frac{\Pi_{\mathrm{ST}}(x, m)\, r(\ell \mid m)}{\Pi_{\mathrm{ST}}(x, \ell)\, r(m \mid \ell)}.$$

Substituting (108) gives

$$\frac{\exp\bigl(-\beta_m U(x) + g_m\bigr)\, r(\ell \mid m)}{\exp\bigl(-\beta_\ell U(x) + g_\ell\bigr)\, r(m \mid \ell)} = \exp\bigl(-(\beta_m - \beta_\ell)U(x) + g_m - g_\ell\bigr)\frac{r(\ell \mid m)}{r(m \mid \ell)}.$$

Applying the usual $\min\{1, \cdot\}$ rule gives (110). ∎

The key practical point in (110) is that the unknown normalizing constants Z_ℓ cancel. We need only the energy $U(x)$ and the chosen weights g_ℓ. Simulated tempering is therefore feasible even though the ideal choice $g_\ell = -\log Z_\ell$ is itself unavailable exactly. In implementation those ratios are evaluated on the log scale, exactly as in Section 2.1.4, because the energies and ladder weights can vary by many orders of magnitude across temperatures. This coupling between the ladder (β_ℓ) and the balancing weights (g_ℓ) is also what makes simulated tempering more delicate to tune than parallel tempering.

Algorithm 22 Simulated tempering

Input: Inverse temperatures $\beta_1 > \cdots > \beta_L$, kernels $K_1, \dots, K_L$, log-weights $g_1, \dots, g_L$, temperature proposal $r(\cdot \mid \cdot)$, state updates per temperature move M, initial state $(x^{(0)}, I^{(0)})$, iterations T

```
for t = 0, 1, ..., T − 1 do
    x ← x^(t), i ← I^(t)
    for m = 1, 2, ..., M do
        Draw x ~ K_i(x, ·)
    end for
    Propose j ~ r(· | i)
    Draw u ~ Unif(0, 1)
    if u ≤ α_ST((x, i), (x, j)) then
        I^(t+1) ← j
    else
        I^(t+1) ← i
    end if
    x^(t+1) ← x
end for
return (x^(t), I^(t))_{t=0}^T
```

Because each x-update preserves the conditional law π_{β_i} at fixed i, and each temperature move is a valid Metropolis step for (108), the combined chain leaves Π_{ST} invariant. The target samples are the states with $I = 1$; the higher-temperature visits are auxiliary moves that help those cold states change basins.

4.3.3 Parallel tempering

Parallel tempering, also called *replica exchange MCMC*, keeps one replica at each temperature instead of letting a single chain climb up and down the ladder. The joint target on L replicas is the product measure

$$\Pi_{\mathrm{PT}}(x_1, \dots, x_L) := \prod_{\ell=1}^{L} \pi_{\beta_\ell}(x_\ell). \tag{111}$$

At fixed replica labels, each chain evolves with its own kernel K_ℓ. Communication between temperatures happens through occasional swap proposals, usually between adjacent levels.

Suppose we propose to swap the states at levels ℓ and $\ell + 1$. Because the proposal is symmetric, the acceptance probability depends only on how the product target changes under that swap.

Proposition 4.4 (Swap acceptance ratio). *Let the current replica states be $(x_1, \dots, x_L)$ and propose to exchange x_ℓ and $x_{\ell+1}$ for some $\ell \in \{1, \dots, L-1\}$. Then the Metropolis acceptance probability is*

$$\alpha_{\mathrm{swap}} = \min\Big\{1,\ \exp\big((\beta_\ell - \beta_{\ell+1})(U(x_\ell) - U(x_{\ell+1}))\big)\Big\}. \tag{112}$$

Proof. Under (111), all factors other than levels ℓ and $\ell+1$ cancel in the Metropolis ratio. Hence

$$\frac{\pi_{\beta_\ell}(x_{\ell+1})\,\pi_{\beta_{\ell+1}}(x_\ell)}{\pi_{\beta_\ell}(x_\ell)\,\pi_{\beta_{\ell+1}}(x_{\ell+1})} = \exp\big(-\beta_\ell U(x_{\ell+1}) - \beta_{\ell+1}U(x_\ell) + \beta_\ell U(x_\ell) + \beta_{\ell+1}U(x_{\ell+1})\big),$$

which simplifies to (112). Applying $\min\{1,\cdot\}$ yields the stated acceptance probability. ■

Algorithm 23 Parallel tempering

Input: Inverse temperatures $\beta_1 > \cdots > \beta_L$, kernels $K_1, \dots, K_L$, initial replicas $x_1^{(0)}, \dots, x_L^{(0)}$, iterations T

```
for t = 0, 1, ..., T − 1 do
    for ℓ = 1, 2, ..., L do
        Draw x_ℓ ~ K_ℓ(x_ℓ, ·)
    end for
    Choose an adjacent pair (ℓ, ℓ + 1)
    Draw u ~ Unif(0, 1)
    if u ≤ α_swap from (112) then
        Exchange x_ℓ and x_{ℓ+1}
    end if
    Record the cold replica x_1
end for
return (x_1^(t), ..., x_L^(t))_{t=0}^T
```

Parallel tempering has a complementary trade-off to simulated tempering. It avoids the unknown weight problem because the joint target is just a product of tempered marginals, but it requires maintaining L replicas at once. The conceptual benefit is clean: the hot replicas can cross barriers that trap the cold chain, and successful swap moves then pull that new basin membership back down the ladder to $\beta_1 = 1$. In modern implementations the replica updates can often be run in parallel, which is where the method gets its name, but from the probabilistic point of view the essential ingredient is the swap mechanism rather than the hardware.

4.3.4 Choosing the ladder in practice

The main tuning question is how to choose the ladder (107). It should be fine enough that neighboring temperatures communicate, but not so fine that computation is wasted on too many nearly identical levels. A common heuristic is to tune adjacent temperature moves or swap proposals to be accepted at a moderate rate, often around 20% to 50%. The standard acceptance-rate discussion is reviewed by Earl and Deem (2005, §2.3), who summarize both the empirical 20% rule and the related 23% calculation from diffusion-style approximations. Very high acceptance usually means the ladder is denser than it needs to be, while very low acceptance means the gaps between temperatures are too large for useful communication. It is important to inspect these acceptance rates *pair by pair*, not only through an average across the whole ladder. A ladder with adjacent swap rates $(0.8, 0.3, 0.04, 0.1)$ still has an average near 0.31, but the single 4% gap effectively splits the ladder into two weakly communicating pieces. In that situation the right repair is usually local—insert one extra temperature inside the bad gap or shrink that particular spacing—rather than uniformly densifying the whole ladder.

For parallel tempering, a more informative view looks at the *energy* random variable

$$E_\beta := U(X_\beta), \qquad X_\beta \sim \pi_\beta.$$

Neighboring swap moves work well only when the histograms of E_{β_ℓ} and $E_{\beta_{\ell+1}}$ overlap appreciably. If the hotter level typically has much larger energy than the colder one, then a proposed swap compares two states whose energies are already well separated, and the exponent in (112) is usually very negative. This is why energy histograms are often a better ladder diagnostic than visual inspection of the x-marginals; see Earl and Deem (2005, §2.3). They also help diagnose whether the ladder is hot enough at all: if the hottest levels still look trapped in one basin, then the problem is not merely the spacing between adjacent temperatures, but that β_L itself is still too large.

That overlap heuristic also leads to a useful small-gap calculation. Write

$$Z(\beta) := \int e^{-\beta U(x)}\,dx, \qquad \mu(\beta) := \mathbb{E}_{\pi_\beta}[U(X)].$$

Then

$$\mu(\beta) = -\frac{d}{d\beta}\log Z(\beta),$$

and differentiating once more gives the thermodynamic identity

$$\mu'(\beta) = -\operatorname{Var}_{\pi_\beta}(U(X)). \tag{113}$$

To see this directly, differentiate under the integral sign:

$$\mu'(\beta) = \frac{d}{d\beta}\left(\frac{1}{Z(\beta)}\int U(x)e^{-\beta U(x)}\,dx\right) = -\mathbb{E}_{\pi_\beta}[U(X)^2] + \mathbb{E}_{\pi_\beta}[U(X)]^2.$$

Now compare neighboring levels β and $\beta - \Delta\beta$ with $\Delta\beta > 0$ small, and let

$$X \sim \pi_\beta, \qquad X' \sim \pi_{\beta-\Delta\beta}, \qquad \Delta U := U(X') - U(X).$$

The swap acceptance probability becomes

$$\alpha = \min\left\{1, e^{-\Delta\beta\,\Delta U}\right\},$$

so if $A(\Delta\beta) := \mathbb{E}[\alpha]$, then

$$1 - A(\Delta\beta) = \mathbb{E}\left[\left(1 - e^{-\Delta\beta\,\Delta U}\right)\mathbf{1}_{\{\Delta U>0\}}\right].$$

The exact small-gap behavior depends on the distribution of the positive part $(\Delta U)_+$, not only on its variance. Still, the natural scale is easy to read off from the first two moments of ΔU. Using (113),

$$\mathbb{E}[\Delta U] = \mu(\beta - \Delta\beta) - \mu(\beta) = -\mu'(\beta)\,\Delta\beta + O\big((\Delta\beta)^2\big) = \operatorname{Var}_{\pi_\beta}(U(X))\,\Delta\beta + O\big((\Delta\beta)^2\big),$$

while independence of X and X' gives

$$\mathbb{E}[\Delta U^2] = \operatorname{Var}_{\pi_{\beta-\Delta\beta}}(U(X)) + \operatorname{Var}_{\pi_\beta}(U(X)) + \big(\mathbb{E}[\Delta U]\big)^2 = 2\,\operatorname{Var}_{\pi_\beta}(U(X)) + O(\Delta\beta).$$

Moreover, $1 - e^{-u} \le u$ for every $u \ge 0$, so

$$1 - A(\Delta\beta) \le \Delta\beta\,\mathbb{E}[(\Delta U)_+] \le \Delta\beta\,\sqrt{\mathbb{E}[\Delta U^2]}.$$

Therefore the rejection probability is controlled at the scale

$$1 - A(\Delta\beta) = O\left(\Delta\beta\sqrt{\operatorname{Var}_{\pi_\beta}(U(X))}\right). \tag{114}$$

up to a constant that depends on the law of $(\Delta U)_+$. This bound does not identify the exact asymptotic constant, but it does show which dimensionless quantity governs neighboring-level swaps. That leads to the practical spacing rule

$$(\beta_\ell - \beta_{\ell+1})\sqrt{\mathrm{Var}_{\pi_{\beta_\ell}}(U(X))} \approx \text{constant} \tag{115}$$

if one wants roughly comparable swap acceptance along the ladder. In words, the inverse-temperature gaps should shrink where the energy fluctuates more and can be wider where the energy variance is small. This is the small-gap logic behind the geometric-spacing heuristic reviewed by Earl and Deem (2005, §2.3).

The second question is how many local state updates to run at each temperature before attempting a temperature move or swap. If the chain changes temperature too often relative to local exploration, it never has time to exploit the geometry of the current level. In many implementations one therefore performs several local updates between temperature proposals, or alternates deterministic blocks of local moves with occasional swap attempts. A useful starting cadence is about 10–100 within-temperature state updates per temperature move or swap attempt, consistent with the standard replica-exchange workflow described by Earl and Deem (2005, §2.1). Equivalently, if move types are interleaved at random, one wants temperature changes to be much rarer than local state updates. When the local kernel is itself geometric, such as HMC or NUTS, these within-temperature updates are what turn a hot level into a useful exploration device rather than a mere bookkeeping state. If that cadence is too aggressive, the chain spends its effort bookkeeping temperature changes before each level has mixed locally; if it is too sparse, new basin information discovered at hot levels diffuses back to $\beta_1 = 1$ too slowly.

The variance rule (115) is most useful in pilot adaptation. One runs a short preliminary tempering simulation, estimates the adjacent swap rates and the energy variances at each level, and then redistributes the β_ℓ so that the bad gaps are split and the neighboring acceptance rates are closer to uniform. This is another place where the earlier one-bad-gap diagnostic matters: the goal is not a cosmetically good average acceptance, but steady communication all the way down the ladder.

The requirement that the ladder be finite also matters theoretically. To prove geometric ergodicity one needs drift and minorization control that is uniform across the temperature levels, and that is substantially harder to obtain on an infinite ladder. Practical tempering methods therefore work with a finite collection of temperatures and treat the ladder itself as a tuning object.

4.4 Sequential Monte Carlo and particle filters

The earlier methods in this unit target one fixed distribution. State-space models change the problem: after observing $y_1, \dots, y_t$ we want the current posterior for the latent state, and at time $t + 1$ that target changes because a new observation arrives. Sequential Monte Carlo (SMC) is built for exactly this update cycle. At each time step it represents the current filtering law by particles, reweights those particles using the new observation, and resamples when the weights have become too uneven.

The augmentation is therefore different from the momentum and temperature variables in the previous chapters. Here the auxiliary state is a whole interacting population, together with weights and eventually ancestry variables, that tracks an evolving sequence of posterior distributions. The particles propagate through the latent dynamics, the weights incorporate the newest observation, and resampling prevents the approximation from collapsing onto a single lineage.

4.4.1 State-space models and filtering recursions

Let $x_t \in \mathbb{R}^d$ denote a latent state and $y_t \in \mathbb{R}^m$ the corresponding observation. A state-space model is specified by an initial law p_0, a transition density f_θ, and an observation density g_θ:

$$x_0 \sim p_0, \qquad x_t \mid x_{t-1} \sim f_\theta(\cdot \mid x_{t-1}), \qquad y_t \mid x_t \sim g_\theta(\cdot \mid x_t). \tag{116}$$

The latent process $(x_t)_{t\geq 0}$ is Markov, and the observations are conditionally independent given the states. For data $y_{1:T}$, the four recurring inferential tasks are

$$\text{filtering: } p_\theta(x_t \mid y_{1:t}), \qquad \text{prediction: } p_\theta(x_{t+k} \mid y_{1:t}),$$

$$\text{smoothing: } p_\theta(x_{0:t} \mid y_{1:T}), \qquad \text{parameter learning: infer } \theta \text{ from } y_{1:T}.$$

Filtering is the central online problem, because once we can update $p_\theta(x_t \mid y_{1:t})$ sequentially, prediction and likelihood evaluation follow from the same recursion, while smoothing and parameter learning can be built on top of those filtered quantities.

Proposition 4.5 (Prediction and update)**.** *For the state-space model* (116)*, the predictive and filtering laws satisfy*

$$p_\theta(x_t \mid y_{1:t-1}) = \int f_\theta(x_t \mid x_{t-1})\, p_\theta(x_{t-1} \mid y_{1:t-1})\, dx_{t-1}, \tag{117}$$

and

$$p_\theta(x_t \mid y_{1:t}) = \frac{g_\theta(y_t \mid x_t)\, p_\theta(x_t \mid y_{1:t-1})}{z_t}, \qquad z_t := p_\theta(y_t \mid y_{1:t-1}) = \int g_\theta(y_t \mid x_t)\, p_\theta(x_t \mid y_{1:t-1})\, dx_t. \tag{118}$$

Proof. Using the Markov property of the latent chain,

$$p_\theta(x_t \mid x_{t-1}, y_{1:t-1}) = p_\theta(x_t \mid x_{t-1}) = f_\theta(x_t \mid x_{t-1}).$$

Therefore

$$p_\theta(x_t \mid y_{1:t-1}) = \int p_\theta(x_t \mid x_{t-1}, y_{1:t-1})\, p_\theta(x_{t-1} \mid y_{1:t-1})\, dx_{t-1},$$

which gives (117). For the update, Bayes's rule and conditional independence of y_t from $y_{1:t-1}$ given x_t yield

$$p_\theta(x_t \mid y_{1:t}) = \frac{p_\theta(y_t \mid x_t, y_{1:t-1})\, p_\theta(x_t \mid y_{1:t-1})}{p_\theta(y_t \mid y_{1:t-1})} = \frac{g_\theta(y_t \mid x_t)\, p_\theta(x_t \mid y_{1:t-1})}{p_\theta(y_t \mid y_{1:t-1})}.$$

Integrating the numerator over x_t gives the normalizing constant z_t and proves (118). ■

4.4.2 Kalman filtering as a tractable baseline

The prediction-update recursion becomes especially simple in the linear-Gaussian model

$$x_0 \sim \mathcal{N}(m_0, P_0), \qquad x_t = Ax_{t-1} + q_t, \quad q_t \sim \mathcal{N}(0, Q), \qquad y_t = Cx_t + r_t, \quad r_t \sim \mathcal{N}(0, R), \tag{119}$$

with independent Gaussian noises. If

$$x_{t-1} \mid y_{1:t-1} \sim \mathcal{N}(m_{t-1}, P_{t-1}),$$

then (117) implies

$$x_t \mid y_{1:t-1} \sim \mathcal{N}(\hat{m}_t, \hat{P}_t), \qquad \hat{m}_t = Am_{t-1}, \qquad \hat{P}_t = AP_{t-1}A^\top + Q. \tag{120}$$

The update is equally explicit. Conditional on $y_{1:t-1}$, the observation equation in (119) gives

$$y_t = Cx_t + r_t, \qquad r_t \sim \mathcal{N}(0, R),$$

with r_t independent of x_t. Therefore the pair (x_t, y_t) is jointly Gaussian with

$$\begin{bmatrix} x_t \\ y_t \end{bmatrix} \mid y_{1:t-1} \sim \mathcal{N}\left(\begin{bmatrix} \hat{m}_t \\ C\hat{m}_t \end{bmatrix}, \begin{bmatrix} \hat{P}_t & \hat{P}_t C^\top \\ C\hat{P}_t & C\hat{P}_t C^\top + R \end{bmatrix} \right). \tag{121}$$

For a jointly Gaussian pair (U, V) with block mean (μ_U, μ_V) and block covariance

$$\begin{bmatrix} \Sigma_{UU} & \Sigma_{UV} \\ \Sigma_{VU} & \Sigma_{VV} \end{bmatrix},$$

completing the square in the joint density shows that

$$U \mid V = v \sim \mathcal{N}\left(\mu_U + \Sigma_{UV}\Sigma_{VV}^{-1}(v - \mu_V), \Sigma_{UU} - \Sigma_{UV}\Sigma_{VV}^{-1}\Sigma_{VU}\right).$$

Applying this formula to (121) yields the innovation covariance

$$S_t := C\hat{P}_t C^\top + R$$

When $R \succ 0$, the matrix S_t is invertible, and the Kalman gain is

$$K_t := \hat{P}_t C^\top S_t^{-1} = \hat{P}_t C^\top (C\hat{P}_t C^\top + R)^{-1}, \tag{122}$$

$$m_t = \hat{m}_t + K_t(y_t - C\hat{m}_t), \qquad P_t = \hat{P}_t - K_t C\hat{P}_t = (I - K_t C)\hat{P}_t. \tag{123}$$

The correction term $y_t - C\hat{m}_t$ is the innovation: it measures how far the new observation lies from its prediction under the current state estimate, and the matrix K_t determines how strongly that discrepancy should pull the posterior mean. The important structural point is not the formula itself, but the Gaussian closure behind it: linear dynamics and Gaussian noise keep the filtering distribution inside a tractable parametric family. If the latent states $x_{0:T}$ were observed, parameter learning in (119) would reduce to ordinary multivariate regression and residual covariance estimation. For example, one would estimate the dynamics by regressing x_t on x_{t-1},

$$\hat{A} = \left(\sum_{t=1}^{T} x_t x_{t-1}^\top\right)\left(\sum_{t=1}^{T} x_{t-1} x_{t-1}^\top\right)^{-1}, \qquad \hat{Q} = \frac{1}{T}\sum_{t=1}^{T}(x_t - \hat{A}x_{t-1})(x_t - \hat{A}x_{t-1})^\top,$$

while the observation model would give

$$\hat{C} = \left(\sum_{t=1}^{T} y_t x_t^\top\right)\left(\sum_{t=1}^{T} x_t x_t^\top\right)^{-1}, \qquad \hat{R} = \frac{1}{T}\sum_{t=1}^{T}(y_t - \hat{C}x_t)(y_t - \hat{C}x_t)^\top.$$

The hidden-state difficulty is exactly that these regressors are unobserved.

That is why Kalman filtering solves the online problem exactly, and why a backward Gaussian-smoothing pass is the natural bridge to the EM updates from Section 3.3. Running the Kalman filter forward gives the

filtered Gaussian laws, and a backward Rauch–Tung–Striebel-type pass then returns smoothed means and covariances such as

$$m_t^{\mathrm{s}} := \mathbb{E}_{\theta^{(k)}}[x_t \mid y_{1:T}], \qquad P_t^{\mathrm{s}} := \mathrm{Var}_{\theta^{(k)}}(x_t \mid y_{1:T}), \qquad P_{t,t-1}^{\mathrm{s}} := \mathrm{Cov}_{\theta^{(k)}}(x_t, x_{t-1} \mid y_{1:T}).$$

The E-step for a current parameter value $\theta^{(k)}$ therefore reduces to computing the smoothed one-step and lag-one moments

$$\mathbb{E}_{\theta^{(k)}}[x_t \mid y_{1:T}], \qquad \mathbb{E}_{\theta^{(k)}}[x_t x_t^\top \mid y_{1:T}], \qquad \mathbb{E}_{\theta^{(k)}}[x_t x_{t-1}^\top \mid y_{1:T}], \tag{124}$$

which are the sufficient statistics of the complete-data Gaussian model. Equivalently,

$$Q(\theta \mid \theta^{(k)}) := \mathbb{E}_{\theta^{(k)}}[\log p_\theta(x_{0:T}, y_{1:T}) \,|\, y_{1:T}]$$

depends on θ only through these expectations, so the M-step simply plugs them into the same regression and covariance formulas that would be used if the states were observed. In that sense, EM for linear-Gaussian state-space models is just "regression with missing states," where the backward smoother supplies the conditional replacements for the unobserved regressors and cross-products. Once the model becomes nonlinear or non-Gaussian, the recursion from Proposition 4.5 still holds, but the filtered law no longer closes inside a finite-dimensional family.

4.4.3 Particle approximations and weight degeneracy

For a general state-space model, SMC is an online version of the self-normalized importance-sampling construction from Section 2.1.3. Instead of approximating one fixed target once, it updates the approximation each time a new observation arrives. This progression from sequential importance sampling to a general SMC framework is developed systematically in Liu (2002, §2.6.3 and Ch. 3). Concretely, it replaces the filtering distribution by a weighted empirical measure

$$\pi_t^N(dx) := \sum_{i=1}^N w_t^{(i)} \delta_{x_t^{(i)}}(dx) \approx p_\theta(x_t \mid y_{1:t})(dx), \tag{125}$$

where the particle locations $x_t^{(i)}$ summarize likely states and the weights satisfy $w_t^{(i)} \geq 0$ and $\sum_i w_t^{(i)} = 1$. As in ordinary importance sampling, expectations under the filtering law are then approximated by weighted averages:

$$\mathbb{E}_\theta[h(x_t) \mid y_{1:t}] \approx \sum_{i=1}^N w_t^{(i)} h(x_t^{(i)}). \tag{126}$$

At this point it is worth separating the empirical approximation problem from the specific sequential mechanics. In SMC the particles interact through weighting and resampling, so after one update step the cloud is typically no longer i.i.d. That is not itself a defect. What matters is how well the empirical measure approximates the target law. Suppose our task is to estimate expectations under π from a particle cloud of fixed size N. Then the right question is not "are the particles independent?" but "what does dependence do to the variance of empirical averages?" The next proposition records the basic variance calculation behind that point.

Proposition 4.6 (Empirical averages for an exchangeable particle cloud)**.** *Let $X^{(1)}, \dots, X^{(N)}$ be ex-*

changeable random variables with common marginal law π, and define the empirical measure

$$\hat{\pi}_N := \frac{1}{N}\sum_{i=1}^{N} \delta_{X^{(i)}}.$$

Then for every integrable test function h,

$$\mathbb{E}\left[\int h(x)\,\hat{\pi}_N(dx)\right] = \int h(x)\,\pi(dx). \tag{127}$$

If $h \in L^2(\pi)$, then

$$\mathrm{Var}\left(\int h(x)\,\hat{\pi}_N(dx)\right) = \frac{1}{N}\,\mathrm{Var}_\pi\big(h(X^{(1)})\big) + \frac{N-1}{N}\,\mathrm{Cov}\big(h(X^{(1)}), h(X^{(2)})\big). \tag{128}$$

Proof. Because each $X^{(i)}$ has marginal law π,

$$\mathbb{E}\left[\int h(x)\,\hat{\pi}_N(dx)\right] = \frac{1}{N}\sum_{i=1}^{N}\mathbb{E}[h(X^{(i)})] = \int h(x)\,\pi(dx),$$

which proves (127). For the variance, write

$$\int h(x)\,\hat{\pi}_N(dx) = \frac{1}{N}\sum_{i=1}^{N} h(X^{(i)}).$$

Expanding the variance of that average gives

$$\mathrm{Var}\left(\frac{1}{N}\sum_{i=1}^{N} h(X^{(i)})\right) = \frac{1}{N^2}\sum_{i=1}^{N}\mathrm{Var}(h(X^{(i)})) + \frac{1}{N^2}\sum_{i\neq j}\mathrm{Cov}(h(X^{(i)}), h(X^{(j)})).$$

Exchangeability makes all variance terms equal and all off-diagonal covariance terms equal, which yields (128). ■

For i.i.d. particles, the covariance term in (128) vanishes. If the dependence is negative for a class of important test functions h, then the coupled cloud can approximate π more accurately than an i.i.d. cloud of the same size. So from the viewpoint of empirical approximation, independence is sufficient but not sacred. This matters already in SMC, where reweighting and resampling create interaction automatically and the real question is whether the resulting empirical measure is good. Later, in Section 6.3.3, we will see a diffusion-model example where one specifies a joint law on the particle cloud directly in order to improve finite-particle approximation (Corso et al., 2023, §3).

If the weighted particles at time $t-1$ approximate $p_\theta(x_{t-1} \mid y_{1:t-1})$, the most direct sequential importance step uses the transition density itself as the proposal. We first propagate

$$x_t^{*,(i)} \sim f_\theta(\cdot \mid x_{t-1}^{(i)}), \qquad i = 1, \dots, N, \tag{129}$$

which approximates the predictive law (117), and then reweight with the new likelihood factor:

$$\tilde{w}_t^{(i)} = w_{t-1}^{(i)}\, g_\theta(y_t \mid x_t^{*,(i)}), \qquad w_t^{(i)} = \frac{\tilde{w}_t^{(i)}}{\sum_{j=1}^{N}\tilde{w}_t^{(j)}}. \tag{130}$$

Equation (130) is Bayes's rule from (118) written in sequential importance-sampling form, with exactly the same normalize-the-weights step as in Section 2.1.3. Liu presents the same recursive-weight viewpoint in Liu (2002, §2.6.3). In implementation one usually evaluates the unnormalized weights on the log scale, as in Section 2.1.4, since likelihood values can vary by many orders of magnitude across particles. The operational cycle is therefore simple: start from the weighted empirical law at time $t-1$, propagate each particle through the transition model, multiply by the new likelihood factor, renormalize, and resample only when the resulting weights become too uneven.

A standard diagnostic is the same effective-sample-size proxy from Section 2.1.6, now applied at each time step. Its role is to quantify the weight degeneracy of pure sequential importance sampling. After many observations, most particles receive tiny weights, so the approximation is carried by only a few surviving lineages.

$$\mathrm{ESS}_t := \frac{1}{\sum_{i=1}^{N}(w_t^{(i)})^2}. \tag{131}$$

This is the same weight-based ESS proxy used in importance sampling, not the autocorrelation ESS from Section 2.2.4 for a single correlated Markov chain. This equals N when the weights are uniform and decreases toward 1 when a single particle dominates. Here the interpretation is even more concrete than in one-shot importance sampling: a small ESS means the particle system has effectively forgotten most of its diversity and needs help before the next observation arrives.

4.4.4 Resampling and the bootstrap filter

Resampling combats degeneracy by replacing a highly uneven weighted sample with an equally weighted sample drawn from the current empirical measure. Given weighted particles $\{(x_t^{*,(i)}, w_t^{(i)})\}_{i=1}^N$, draw ancestor indices

$$A_t^{(1)}, \dots, A_t^{(N)} \in \{1, \dots, N\}$$

with probabilities $\mathbb{P}(A_t^{(j)} = i) = w_t^{(i)}$, then set

$$x_t^{(j)} = x_t^{*,(A_t^{(j)})}, \qquad w_t^{(j)} = \frac{1}{N}.$$

The new system duplicates high-weight particles and discards low-weight ones, so future propagation starts from a healthier empirical approximation. If the ancestor indices are retained, then the algorithm is genuinely working on an augmented state: the current particle locations approximate the filtering law, while the recorded lineages provide the bookkeeping needed for later smoothing over past states. Equivalently, one can formulate the method on path space, with particles carrying weighted partial trajectories $x_{0:t}^{(i)}$ rather than only their current endpoints; this is the partial-sample viewpoint emphasized by Liu (2002, §§3.4.3–3.4.4).

Proposition 4.7 (Resampling preserves weighted averages in expectation)**.** *Conditional on the weighted sample $\{(x_t^{*,(i)}, w_t^{(i)})\}_{i=1}^N$, let $x_t^{(1)}, \dots, x_t^{(N)}$ be resampled independently from that empirical measure. Then, for any integrable test function h,*

$$\mathbb{E}\left[\frac{1}{N}\sum_{j=1}^{N} h(x_t^{(j)}) \;\middle|\; x_t^{*,(1:N)}, w_t^{(1:N)}\right] = \sum_{i=1}^{N} w_t^{(i)} h(x_t^{*,(i)}). \tag{132}$$

Proof. Conditional on the current weighted sample, each $x_t^{(j)}$ has distribution $\sum_i w_t^{(i)} \delta_{x_t^{*,(i)}}$. Therefore

$$\mathbb{E}[h(x_t^{(j)}) \mid x_t^{*,(1:N)}, w_t^{(1:N)}] = \sum_{i=1}^{N} w_t^{(i)} h(x_t^{*,(i)}).$$

Summing over $j = 1, \dots, N$ and dividing by N gives (132). ■

Resampling improves the distribution of future weights, but it is not free. Duplicating the current high-weight particles reduces weight variance now, yet it also reduces genealogical diversity and can lead to sample impoverishment if resampling is too frequent. That is why practical filters usually resample only when the ESS in (131) falls below a threshold such as $N/2$. For discussion of this tradeoff and of simple-random versus residual resampling schemes, see Liu (2002, §§3.4.3–3.4.4).

Algorithm 24 Adaptive bootstrap particle filter

Input: Initial law p_0, transition density f_θ, likelihood g_θ, observations $y_{1:T}$, number of particles N, ESS threshold factor $c \in (0, 1]$

for $i = 1, 2, \dots, N$ **do**
  Draw $x_0^{(i)} \sim p_0$ and set $w_0^{(i)} \leftarrow 1/N$.
end for
for $t = 1, 2, \dots, T$ **do**
  for $i = 1, 2, \dots, N$ **do**
    Draw $x_t^{*,(i)} \sim f_\theta(\cdot \mid x_{t-1}^{(i)})$.
    $\tilde{w}_t^{(i)} \leftarrow w_{t-1}^{(i)} g_\theta(y_t \mid x_t^{*,(i)})$.
  end for
  $\hat{z}_t \leftarrow \sum_{i=1}^{N} \tilde{w}_t^{(i)}$.
  for $i = 1, 2, \dots, N$ **do**
    $w_t^{(i)} \leftarrow \tilde{w}_t^{(i)} / \hat{z}_t$.
  end for
  $\mathrm{ESS}_t \leftarrow 1 / \sum_{i=1}^{N} (w_t^{(i)})^2$.
  if $\mathrm{ESS}_t < cN$ **then**
    Sample ancestor indices $A_t^{(1)}, \dots, A_t^{(N)}$ with probabilities $w_t^{(1)}, \dots, w_t^{(N)}$.
    for $i = 1, 2, \dots, N$ **do**
      $x_t^{(i)} \leftarrow x_t^{*,(A_t^{(i)})}$ and $w_t^{(i)} \leftarrow 1/N$.
    end for
  else
    for $i = 1, 2, \dots, N$ **do**
      $x_t^{(i)} \leftarrow x_t^{*,(i)}$.
    end for
  end if
end for
return $\{(x_t^{(i)}, w_t^{(i)})\}_{i=1}^{N}$ for $t = 0, \dots, T$ and $\hat{\ell}_T(\theta) = \sum_{t=1}^{T} \log \hat{z}_t$

If the resampling step is carried out at every observation time, Algorithm 24 reduces to the classical sample-importance-resampling bootstrap filter. More sophisticated resampling rules such as systematic, stratified, or residual resampling reduce the extra Monte Carlo variation introduced by multinomial resampling, but they play the same probabilistic role: replace one highly skewed weighted cloud by another representation of essentially the same empirical law whose future weights will be less degenerate.

4.4.5 Likelihood increments and parameter learning

The filtering recursion also factorizes the marginal likelihood. If

$$z_t := p_\theta(y_t \mid y_{1:t-1}), \tag{133}$$

then

$$p_\theta(y_{1:T}) = \prod_{t=1}^{T} z_t, \qquad \log p_\theta(y_{1:T}) = \sum_{t=1}^{T} \log z_t. \tag{134}$$

By (118), each increment is a predictive expectation,

$$z_t = \int g_\theta(y_t \mid x_t)\, p_\theta(x_t \mid y_{1:t-1})\, dx_t. \tag{135}$$

The particle approximation therefore gives the estimator

$$\hat{z}_t := \sum_{i=1}^{N} \tilde{w}_t^{(i)} = \sum_{i=1}^{N} w_{t-1}^{(i)} g_\theta(y_t \mid x_t^{*,(i)}). \tag{136}$$

When the particles entering time t have already been reset to equal weights, this simplifies to

$$\hat{z}_t = \frac{1}{N} \sum_{i=1}^{N} g_\theta(y_t \mid x_t^{*,(i)}). \tag{137}$$

Combining (134) and (136) yields the natural particle approximation

$$\hat{p}_\theta(y_{1:T}) := \prod_{t=1}^{T} \hat{z}_t, \qquad \hat{\ell}_T(\theta) := \sum_{t=1}^{T} \log \hat{z}_t. \tag{138}$$

Even though the logarithm introduces bias by Jensen's inequality, the decomposition is still extremely useful because each term depends only on the current particle system and the newest observation. This is exactly the structure one wants for online learning. For this likelihood-increment factorization and its particle approximation, see Liu (2002, §2.6.3).

The connection back to the latent-variable viewpoint from Section 3.3 is direct. In a linear-Gaussian model, Kalman smoothing supplies exact conditional moments and EM updates the parameters analytically. In a general nonlinear or non-Gaussian model, the same latent quantities are no longer available in closed form, but SMC supplies sequential approximations to the filtered laws and to the likelihood increments (133). Those approximations can be used for approximate EM, for likelihood-based tuning, or for stochastic-gradient parameter updates as new data arrive. These parameter-learning uses of SMC are reviewed in Liu (2002, §§3.4.3–3.4.4). The same complete-data logic from (124) still applies: if the M-step would be easy given latent trajectories, then particle smoothing or weighted ancestral trajectories provide approximations to the missing sufficient statistics, and one maximizes the resulting approximate Q-function. For Bayesian parameter inference, one typically uses the likelihood estimate $\hat{p}_\theta(y_{1:T})$ rather than the log estimate (138). If the particle filter is constructed so that $\hat{p}_\theta(y_{1:T})$ is a nonnegative unbiased estimator of the true likelihood, then it can be used inside a pseudo-marginal Metropolis–Hastings update for θ. In that case the particle filter numerically integrates out the latent states while a separate sampler moves through parameter space. Without that nonnegative unbiasedness property, the same substitution should be viewed only as an approximate plug-in.

Two practical cautions matter throughout. First, state-space models can have identifiability symmetries, so parameter learning often needs constraints, regularization, or strong prior information to distinguish

dynamically equivalent parametrizations. Second, better approximation costs more computation: more particles reduce Monte Carlo error, and more aggressive resampling stabilizes the ESS, but both also increase runtime and can reduce genealogical diversity. In practice the first quantities to monitor are the ESS trajectory and the incremental log-likelihood estimates $\log \hat{z}_t$. If the ESS repeatedly collapses or the likelihood estimate becomes erratic, the filter usually needs a better proposal, more particles, or a better constrained model.

Part III

Learned maps

5 Variational inference

Variational inference starts from a latent-variable model $p_\theta(x, z)$ and the posterior $p_\theta(z \mid x)$. The difficulty is the evidence

$$p_\theta(x) = \int p_\theta(x, z)\, dz,$$

which makes exact posterior calculations intractable in many models. Variational inference chooses a tractable family q and turns posterior approximation into an optimization problem. This is the EM free-energy story with one crucial change: the exact posterior update is replaced by optimization over a restricted family. We first write the identity that makes this optimization precise, namely the ELBO. We then study factorized coordinate updates and finally move to stochastic and amortized settings, where inference itself is represented by a learned map $q_\phi(z \mid x)$.

5.1 Variational inference and the ELBO

5.1.1 From EM free energy to the ELBO

The transition from EM is literal rather than metaphorical. Proposition 3.2 introduced the free-energy functional for a latent-variable model $p_\theta(x, z)$:

$$\mathcal{F}(q, \theta; x) = \mathbb{E}_q[\log p_\theta(x, Z)] + H(q),$$

and showed that exact EM alternates between the unrestricted update $q(z) = p_\theta(z \mid x)$ and a maximization in θ. Variational inference keeps the same objective but restricts the latent law q to a tractable family. This is the only conceptual change: the model $p_\theta(x, z)$ stays in place, while the exact E-step is relaxed into an optimization problem.

For this subsection we regard θ as fixed and write

$$p_\theta(x, z) = p_\theta(x \mid z)\, p_\theta(z),$$

so the posterior is

$$p_\theta(z \mid x) = \frac{p_\theta(x, z)}{p_\theta(x)}, \qquad p_\theta(x) = \int p_\theta(x, z)\, dz.$$

The evidence $p_\theta(x)$ is exactly the quantity that makes direct Bayesian inference difficult: it is the normalizing constant of the posterior and typically requires a high-dimensional integral. Modern latent-variable methods usually introduce the ELBO at exactly this point; see Kingma and Welling (2014, §2.2) for the VAE version of the same identity.

With θ fixed, the free-energy functional becomes

$$\mathcal{F}(q, \theta; x) := \mathbb{E}_q\big[\log p_\theta(x, z)\big] + H(q),$$

where $H(q) = -\mathbb{E}_q[\log q(z)]$ is the entropy. Variational inference keeps this same objective but restricts the q-update from all densities to a smaller family

$$\mathcal{Q} = \{q_\lambda(z) \,:\, \lambda \in \Lambda\}$$

The ELBO is simply the computational form of this restricted free-energy problem.

5.1.2 ELBO–KL identity

This is the same decomposition that underlies EM, now viewed with θ fixed and q ranging over a restricted family. Writing it explicitly fixes the KL direction and makes clear why the ELBO is the computational surrogate for posterior approximation.

Proposition 5.1 (Free-energy and ELBO identity). *For any density $q(z)$ supported where $p_\theta(x, z) > 0$,*

$$\log p_\theta(x) = \mathcal{F}(q, \theta; x) + \mathrm{KL}\big(q(z) \,\|\, p_\theta(z \mid x)\big), \tag{139}$$

Consequently,

$$\mathcal{F}(q, \theta; x) \le \log p_\theta(x),$$

and maximizing $\mathcal{F}(q, \theta; x)$ over the variational family is equivalent to minimizing $\mathrm{KL}(q \| p_\theta(z \mid x))$.

Proof. Starting from the definition of KL divergence and substituting the posterior density,

$$\mathrm{KL}\big(q(z) \,\|\, p_\theta(z \mid x)\big) = \int q(z) \log \frac{q(z)}{p_\theta(x, z)/p_\theta(x)} \, dz.$$

Expanding the logarithm gives

$$\mathrm{KL}\big(q(z) \,\|\, p_\theta(z \mid x)\big) = \int q(z) \log q(z) \, dz - \int q(z) \log p_\theta(x, z) \, dz + \log p_\theta(x).$$

Rearranging yields

$$\log p_\theta(x) = \mathbb{E}_q\big[\log p_\theta(x, z) - \log q(z)\big] + \mathrm{KL}\big(q(z) \,\|\, p_\theta(z \mid x)\big),$$

which is exactly (139) because

$$\mathcal{F}(q, \theta; x) = \mathbb{E}_q[\log p_\theta(x, z)] + H(q) = \mathbb{E}_q[\log p_\theta(x, z) - \log q(z)].$$

Since KL divergence is nonnegative, (139) implies $\mathcal{F}(q, \theta; x) \le \log p_\theta(x)$, and the difference between the two objectives is the constant $\log p_\theta(x)$. ■

It is conventional to rename the same quantity

$$\mathcal{L}(q; \theta, x) := \mathbb{E}_q\big[\log p_\theta(x, z) - \log q(z)\big] = \mathcal{F}(q, \theta; x) \tag{140}$$

and call it the *evidence lower bound*, or ELBO. The new notation emphasizes the computational point: after fixing θ, the objective depends only on expectations under the tractable law q, while the intractable evidence $\log p_\theta(x)$ appears only as an additive constant. Thus

$$q^\star \in \arg\max_{q \in \mathcal{Q}} \mathcal{L}(q; \theta, x) \quad \Longleftrightarrow \quad q^\star \in \arg\min_{q \in \mathcal{Q}} \mathrm{KL}\big(q(z) \,\|\, p_\theta(z \mid x)\big). \tag{141}$$

This is where VI diverges from both EM and MCMC. EM keeps the exact posterior update but alternates it with a parameter maximization, while MCMC constructs a chain with the posterior as its stationary law. VI instead replaces posterior sampling or exact posterior evaluation by deterministic optimization over a restricted family.

5.1.3 Reconstruction–regularization decomposition

The ELBO also admits a decomposition that is often more interpretable. Using $p_\theta(x, z) = p_\theta(x \mid z) p_\theta(z)$ in (140) gives

$$\mathcal{L}(q; \theta, x) = \mathbb{E}_q[\log p_\theta(x \mid z)] - \mathrm{KL}\big(q(z) \,\|\, p_\theta(z)\big). \tag{142}$$

The first term rewards latent configurations that explain the data well, while the second penalizes variational distributions that drift too far from the prior. In latent-variable models this is the usual "fit minus regularization" interpretation. The decoder/prior language from variational autoencoders is exactly this decomposition written for a model with a fixed prior and a learned conditional distribution $p_\theta(x \mid z)$ (Kingma and Welling, 2014, §2.2).

5.1.4 What forward KL tends to prefer

There is, however, an important geometric asymmetry in (141). The direction $\mathrm{KL}(q\|p)$ heavily penalizes placing mass where the posterior is small, but it is more tolerant of failing to cover every region where the posterior is large. With a restrictive family this often leads, as a heuristic rule of thumb, to approximations that understate posterior variance or choose one mode of a multimodal posterior rather than averaging across all of them. See Blei et al. (2017, §2.4 and Fig. 1) for a concrete mean-field example of this variance-underestimation effect. This is not a flaw in the ELBO identity itself. It is the geometric tradeoff created by combining an asymmetric divergence with a tractable but limited family. The next subsection makes that tradeoff concrete by studying the most classical restricted family, namely a fully factorized mean-field approximation.

5.2 Mean-field variational inference

5.2.1 Factorized families and conditional updates

The ELBO formulation is generic. The real modeling decision is the choice of variational family $\mathcal{Q}$. If the family is too expressive, evaluating or optimizing the ELBO can be almost as hard as the original posterior problem. If the family is too simple, optimization is cheap but the approximation can be poor. Mean-field variational inference makes a deliberately simple choice: it assumes the latent coordinates are independent under the approximation, even if they are strongly dependent under the true posterior. This is the standard mean-field starting point in variational inference; see Blei et al. (2017, §2.3). To see what this means in a hierarchical model, suppose the observations are $x_{1:n}$, the shared latent block is w, and the datum-specific latent variables are $z_{1:n}$. Many posterior distributions of interest take the form

$$p_\theta(w, z_{1:n} \mid x_{1:n}) \propto p_\theta(w) \prod_{i=1}^{n} p_\theta(z_i \mid w)\, p_\theta(x_i \mid z_i, w).$$

Here the global block w is shared across the whole data set, while each z_i is local to one observation. A basic mean-field approximation replaces this coupled posterior by

$$q(w, z_{1:n}) = q(w) \prod_{i=1}^{n} q_i(z_i),$$

possibly with a finer factorization inside the global block $q(w)$. This suppresses posterior dependence, but it also isolates the global quantities that pool information from all observations from the local quantities that can be updated one datum at a time. To state the coordinate update in a uniform form, we now collect all latent blocks into $z = (z_1, \dots, z_m)$ and consider the generic factorized family below.

Write the latent variable as $z = (z_1, \dots, z_m)$ and consider the factorized family

$$q(z) = \prod_{j=1}^{m} q_j(z_j). \tag{143}$$

The factorization in (143) is strong, but it makes the ELBO amenable to coordinate updates. To derive one update, we hold all factors except q_j fixed and optimize only over the remaining factor. That one-coordinate problem is the basis of coordinate-ascent variational inference (CAVI). Mean-field VI is often the first approximation one studies because it exposes this coordinatewise logic in the simplest possible setting before one worries about richer families. The standard derivation and algorithmic viewpoint appear in Blei et al. (2017, §2.4).

5.2.2 Coordinate ascent updates

Let

$$q_{-j}(z_{-j}) := \prod_{k \neq j} q_k(z_k)$$

denote the product of all factors except the jth. If we hold q_{-j} fixed and optimize only over q_j, the ELBO has a simple closed-form maximizer.

Proposition 5.2 (Mean-field coordinate update)**.** *Fix the factors q_k for $k \neq j$. Then any maximizer of the ELBO over the remaining factor q_j satisfies*

$$q_j^\star(z_j) \propto \exp\Big(\mathbb{E}_{q_{-j}}\big[\log p_\theta(x, z)\big]\Big), \tag{144}$$

where the expectation is taken with respect to the current variational law on z_{-j} and the proportionality constant normalizes the right-hand side.

Proof. Starting from (140) and separating the factor q_j from the others,

$$\mathcal{L}(q; \theta, x) = \int q_j(z_j)\, q_{-j}(z_{-j}) \Big[\log p_\theta(x, z) - \log q_j(z_j) - \log q_{-j}(z_{-j})\Big]\, dz_j\, dz_{-j}.$$

The terms involving only q_{-j} are constant when we optimize over q_j, so

$$\mathcal{L}(q; \theta, x) = \int q_j(z_j)\, m_j(z_j)\, dz_j - \int q_j(z_j) \log q_j(z_j)\, dz_j + C,$$

where

$$m_j(z_j) := \mathbb{E}_{q_{-j}}\big[\log p_\theta(x, z)\big]$$

and C does not depend on q_j. Now define

$$\not{q}_j(z_j) \propto e^{m_j(z_j)}.$$

Then

$$\mathrm{KL}\big(q_j \,\|\, \not{q}_j\big) = \int q_j(z_j) \log q_j(z_j)\, dz_j - \int q_j(z_j) m_j(z_j)\, dz_j + \log \int e^{m_j(u)}\, du.$$

Therefore

$$\mathcal{L}(q; \theta, x) = -\mathrm{KL}\big(q_j \,\|\, \not{q}_j\big) + C',$$

for another constant C' independent of q_j. The ELBO is maximized exactly when this KL divergence is minimized, which occurs at $q_j = \not{q}_j$. This gives (144). ∎

Equation (144) is the core formula behind coordinate-ascent variational inference (CAVI). One cycles through the factors, replacing each one by the normalized exponential of the expected complete-data log density. Each update maximizes the ELBO with respect to one coordinate, so the ELBO cannot decrease from one step to the next. This is the generic CAVI update from Blei et al. (2017, §2.4, eqs. (17)–(18)). In that sense, CAVI is to VI what the E-step/M-step alternation is to EM: each block update solves the current restricted subproblem exactly. It is also the deterministic analogue of a Gibbs update:

$$\text{Gibbs:} \qquad Z_j^{\text{new}} \sim p(z_j \mid z_{-j}, x), \qquad \text{CAVI:} \qquad q_j^{\text{new}}(z_j) \propto \exp\Big(\mathbb{E}_{q_{-j}}[\log p_\theta(x, z)]\Big).$$

Gibbs sampling draws one coordinate from the exact conditional, while CAVI replaces that random draw by the factor that maximizes the ELBO given the current law on the remaining coordinates.

Algorithm 25 Coordinate-ascent mean-field VI

Input: Joint density $p_\theta(x, z)$, initial factors $q_1^{(0)}, \dots, q_m^{(0)}$, tolerance ε

1: $t \leftarrow 0$
2: **repeat**
3: **for** $j = 1, 2, \dots, m$ **do**
4: Update

$$q_j^{(t+1)}(z_j) \propto \exp\Big(\mathbb{E}_{q_{-j}^{(t+1,t)}}[\log p_\theta(x, z)]\Big),$$

where $q_{-j}^{(t+1,t)}$ uses the newest factors available in the current sweep

5: **end for**
6: $t \leftarrow t + 1$
7: **until** the ELBO increase is smaller than ε
8: **return** $q^{(t)}(z) = \prod_{j=1}^m q_j^{(t)}(z_j)$

The appeal of CAVI is not only monotone improvement of the ELBO but also a closure property in conjugate models. Suppose the exact complete conditional of z_j has exponential-family form

$$p_\theta(z_j \mid x, z_{-j}) \propto h_j(z_j) \exp\big(\eta_j(x, z_{-j})^\top T_j(z_j)\big).$$

Then (144) gives

$$q_j^\star(z_j) \propto h_j(z_j) \exp\Big(\mathbb{E}_{q_{-j}}\big[\eta_j(x, z_{-j})\big]^\top T_j(z_j)\Big).$$

So the optimal factor stays in the same exponential family; only its natural parameter is replaced by an expectation under the remaining factors. This is the basic closure principle behind classical mean-field VI: Gaussian conditionals yield Gaussian factors, categorical conditionals yield categorical factors, and Dirichlet conditionals yield Dirichlet factors. In conjugate exponential-family models the right-hand side can therefore be evaluated from current sufficient-statistic moments, and each coordinate update reduces to a familiar parameter update rather than a fresh numerical optimization. This exponential-family closure is the organizing principle in Blei et al. (2017, §4.1–4.2). That is what makes mean-field VI computationally attractive in models such as mixture models, hidden Markov models, topic models, and linear-Gaussian factor models. Once conjugacy disappears, however, those closed-form coordinate updates are no longer available. That is the point where one leaves the classical CAVI setting and moves to stochastic-gradient and amortized methods.

5.2.3 Worked examples: mixture assignments and factor analysis

Two examples are especially useful here. Gaussian mixtures show how CAVI keeps categorical, Gaussian, and Dirichlet factors inside their familiar conjugate families, while Bayesian factor analysis makes the global/local Gaussian structure visible in a form that leads naturally into amortized encoders and VAEs.

Example 5.3 (Gaussian mixtures stay inside familiar families)**.** The same closure principle appears in a conjugate Gaussian mixture model. Let $z_i \in \{1, \dots, K\}$ be the component indicator for $x_i \in \mathbb{R}^d$, let π be the mixing weights, and let $\mu_{1:K}$ be the component means. Assume the observation covariance is I_d, take the priors

$$\pi \sim \text{Dirichlet}(\alpha_1, \dots, \alpha_K), \qquad \mu_k \sim \mathcal{N}(0, \Lambda^{-1}),$$

and write the mean-field family as

$$q(\pi, \mu_{1:K}, z_{1:n}) = q(\pi) \prod_{k=1}^{K} q_k(\mu_k) \prod_{i=1}^{n} q_i(z_i).$$

Then CAVI keeps every factor in its conjugate family:

$$q_i(z_i) = \text{Categorical}(r_{i1}, \dots, r_{iK}), \qquad q_k(\mu_k) = \mathcal{N}(m_k, \Sigma_k), \qquad q(\pi) = \text{Dirichlet}(\alpha_1, \dots, \alpha_K).$$

The local responsibility update is

$$r_{ik} \propto \exp\left(\mathbb{E}_q[\log \pi_k] - \frac{1}{2}\mathbb{E}_q\|x_i - \mu_k\|^2 \right),$$

while the global factors satisfy

$$\Sigma_k^{-1} = \Lambda + \left(\sum_{i=1}^{n} r_{ik} \right) I_d, \qquad m_k = \Sigma_k \sum_{i=1}^{n} r_{ik} x_i,$$

and

$$\alpha_k = \alpha_k + \sum_{i=1}^{n} r_{ik}.$$

This is the packet's basic message in a standard conjugate model: the local z_i-updates are soft assignments, and the global updates for μ_k and π are just Gaussian and Dirichlet parameter updates driven by the current responsibilities.

Example 5.4 (Global loadings and local factors)**.** A linear-Gaussian factor model makes the global/local structure explicit. Let $x_i \in \mathbb{R}^d$ be observed, let $z_i \in \mathbb{R}^k$ with $k < d$ be a latent factor, and let $W \in \mathbb{R}^{d \times k}$ be a loading matrix. Write the rth row of W as $w_r^\top \in \mathbb{R}^k$. With fixed scales $\sigma^2, \tau^2 > 0$, consider

$$w_r \sim \mathcal{N}(0, \tau^2 I_k), \qquad z_i \sim \mathcal{N}(0, I_k), \qquad x_i \mid z_i, W \sim \mathcal{N}(W z_i, \sigma^2 I_d).$$

Then

$$p_\theta(W, z_{1:n} \mid x_{1:n}) \propto \prod_{r=1}^{d} p_\theta(w_r) \prod_{i=1}^{n} p_\theta(z_i)\, p_\theta(x_i \mid z_i, W),$$

where $\theta = (\sigma^2, \tau^2)$ collects the fixed hyperparameters. A mean-field approximation that respects the global/local split is

$$q(W, z_{1:n}) = \prod_{r=1}^{d} q_r(w_r) \prod_{i=1}^{n} q_i(z_i).$$

Because both the prior and the likelihood are Gaussian, the closure principle above implies that every optimal factor is Gaussian as well. In fact one obtains

$$q_i(z_i) = \mathcal{N}(m_i, S), \qquad q_r(w_r) = \mathcal{N}(\mu_r, \Sigma),$$

with shared covariance matrices

$$S^{-1} = I_k + \sigma^{-2}\mathbb{E}_q[W^\top W], \qquad \Sigma^{-1} = \tau^{-2} I_k + \sigma^{-2} \sum_{i=1}^{n} \mathbb{E}_q[z_i z_i^\top],$$

and means

$$m_i = \sigma^{-2} S\, \mathbb{E}_q[W]^\top x_i, \qquad \mu_r = \sigma^{-2}\Sigma \sum_{i=1}^{n} x_{ir}\, \mathbb{E}_q[z_i].$$

The required second moments are

$$\mathbb{E}_q[W^\top W] = \sum_{r=1}^{d} (\Sigma + \mu_r \mu_r^\top), \qquad \mathbb{E}_q[z_i z_i^\top] = S + m_i m_i^\top.$$

This example makes both kinds of mean-field update visible. Each local factor $q_i(z_i)$ uses only the datum x_i together with the current global moments of W. Each global factor $q_r(w_r)$ pools information from all observations through the latent second moments $\mathbb{E}_q[z_i z_i^\top]$. If W were held fixed, the local update would reduce to the exact Gaussian posterior from classical factor analysis; the Bayesian version adds one more layer of uncertainty by learning a posterior distribution over the shared loadings themselves.

5.2.4 Strengths and limitations of mean-field

Mean-field VI is often remarkably effective when the posterior is roughly unimodal and the most important uncertainty is marginal rather than strongly correlated. Its main weakness is also clear from (143): it cannot represent posterior dependence. If two coordinates are coupled under the posterior, the factorized approximation must absorb that dependence indirectly by inflating or shrinking the one-dimensional factors. This is one reason mean-field approximations often understate uncertainty.

The other major limitation is computational scaling with the data set. In models with one local latent variable per observation, CAVI introduces one local variational factor per observation as well. That is manageable for moderate data sets and conjugate models, but it becomes cumbersome when n is very large or when the coordinate updates no longer have closed form. This is exactly the setting that motivates amortized and stochastic variational inference. From this point onward the ELBO and the model $p_\theta(x, z)$ stay in place, but the optimization regime changes: instead of approximating the posterior for a fixed θ, we allow the model and the inference rule to be learned together from data.

5.3 Stochastic and amortized variational inference

5.3.1 From mean-field factors to shared encoders

Mean-field VI introduces local variational objects for each observation. If $x_1, \dots, x_n$ are observations with corresponding latent variables $z_1, \dots, z_n$, a classical approximation might introduce one local parameter λ_i for each x_i. That can be effective, but it scales poorly: every new observation comes with a fresh inference subproblem. In the Bayesian factor-analysis example from Example 5.4, this means storing one Gaussian factor $q_i(z_i)$ for every datum together with a global factor for the loading matrix W.

Amortized variational inference replaces those per-datum free parameters by a shared map. Instead of optimizing λ_i separately for every x_i, we learn parameters ϕ and set

$$q_\phi(z_i \mid x_i)$$

directly as a function of the observation. In factor analysis, a natural amortized choice is a Gaussian encoder

$$q_\phi(z_i \mid x_i) = \mathcal{N}\big(\mu_\phi(x_i), \Sigma_\phi(x_i)\big),$$

which tries to predict the posterior summaries that CAVI would otherwise compute separately for each i. If the model still contains a global latent block such as W, we keep a separate factor $q_\xi(W)$ and amortize only

the local factors:

$$q_{\xi,\phi}(W, z_{1:n}) = q_\xi(W) \prod_{i=1}^{n} q_\phi(z_i \mid x_i).$$

This is the meaning of *amortization*: we pay the cost of learning an inference rule during training and then reuse that rule across data points. When there is no separate global latent block, and the generative model has parameters θ that factorize across observations as

$$p_\theta(x_{1:n}, z_{1:n}) = \prod_{i=1}^{n} p_\theta(x_i, z_i),$$

then the training objective is the summed ELBO

$$\mathcal{J}(\theta, \phi) := \sum_{i=1}^{n} \mathcal{L}_i(\theta, \phi), \qquad \mathcal{L}_i(\theta, \phi) := \mathbb{E}_{q_\phi(z_i \mid x_i)}\big[\log p_\theta(x_i, z_i) - \log q_\phi(z_i \mid x_i)\big]. \tag{145}$$

Stochastic-gradient VI is the optimization strategy that makes this practical: we estimate gradients of (145) with Monte Carlo samples and minibatches rather than with closed-form coordinate updates. Figure 27 illustrates the difference. This encoder–decoder organization is the modern VAE setup (Kingma and Welling, 2014, §2.3).

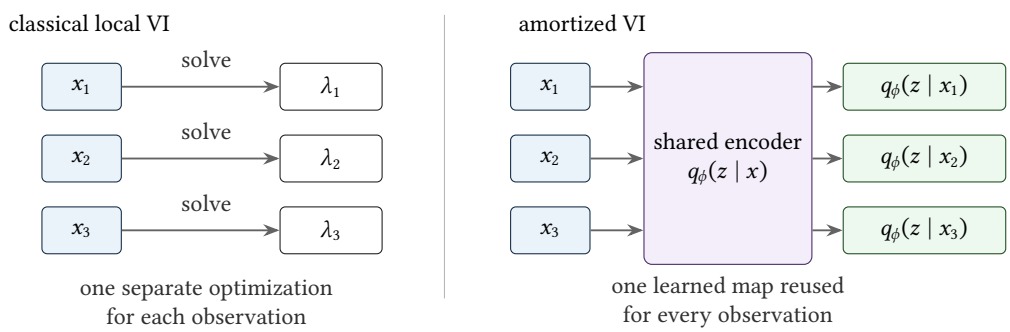

Figure 27: Classical mean-field VI uses one set of local variational parameters per observation. Amortized VI replaces those free parameters by a shared encoder $q_\phi(z \mid x)$, so inference for a new datum becomes a forward pass through the learned map.

Dividing (145) by n gives the same optimizer, and the corresponding population form replaces the empirical sum by

$$\mathbb{E}_{p_{\text{data}}(x)}\big[\mathcal{L}(q_\phi(\cdot \mid x); x)\big].$$

We keep the finite-sample form here because the main issue in this chapter is stochastic optimization over a given data set.

For the Bayesian factor-analysis model, the amortized family $q_{\xi,\phi}(W, z_{1:n})$ leads to

$$\mathcal{J}(\theta, \xi, \phi) = \mathbb{E}_{q_{\xi,\phi}}\left[\log p_\theta(W) - \log q_\xi(W) + \sum_{i=1}^{n}\big(\log p_\theta(z_i) + \log p_\theta(x_i \mid z_i, W) - \log q_\phi(z_i \mid x_i)\big)\right].$$

With Gaussian priors and Gaussian observation noise, this simplifies to

$$\mathcal{J}(\theta, \xi, \phi) = C - \mathrm{KL}\big(q_\xi(W) \,\|\, p_\theta(W)\big) - \sum_{i=1}^{n} \mathrm{KL}\big(q_\phi(z_i \mid x_i) \,\|\, p_\theta(z_i)\big) - \frac{1}{2\sigma^2} \sum_{i=1}^{n} \mathbb{E}_{q_{\xi,\phi}} \|x_i - W z_i\|^2, \tag{146}$$

where C does not depend on ξ or ϕ. The Gaussian encoder keeps the local posterior approximation in the same family as the mean-field example, but now $\mu_\phi(x)$ and $\Sigma_\phi(x)$ are shared functions rather than n separate free parameters. Up to the same additive constant, a minibatch $B \subseteq \{1, \dots, n\}$ gives the stochastic objective

$$\hat{\mathcal{J}}_B(\theta, \xi, \phi) = -\mathrm{KL}\big(q_\xi(W) \,\|\, p_\theta(W)\big) + \frac{n}{|B|} \sum_{i \in B} \ell_i(\theta, \xi, \phi), \tag{147}$$

where

$$\ell_i(\theta, \xi, \phi) := \mathbb{E}_{q_\xi(W) q_\phi(z_i \mid x_i)}\big[\log p_\theta(x_i \mid z_i, W)\big] - \mathrm{KL}\big(q_\phi(z_i \mid x_i) \,\|\, p_\theta(z_i)\big).$$

So the global term appears once, while the local reconstruction and KL terms are the ones scaled by $n/|B|$. This is the minibatch consequence that makes the factor-analysis example compatible with stochastic-gradient training.

This same example also explains the move from linear latent-variable models to variational autoencoders. If the loading matrix is treated as an ordinary model parameter rather than a latent variable, the decoder is simply the linear map $z \mapsto Wz$. A variational autoencoder keeps the Gaussian latent prior and Gaussian encoder but replaces that linear mean map by a nonlinear decoder $c_\theta : \mathbb{R}^k \to \mathbb{R}^d$, typically a neural network, and learns θ jointly with ϕ:

$$p_\theta(x \mid z) = \mathcal{N}\big(c_\theta(z), \sigma^2 I_d\big), \qquad q_\phi(z \mid x) = \mathcal{N}\big(\mu_\phi(x), \Sigma_\phi(x)\big).$$

For one datum the ELBO then becomes

$$\mathcal{L}_i(\theta, \phi) = C - \frac{1}{2\sigma^2} \mathbb{E}_{q_\phi(z_i \mid x_i)} \|x_i - c_\theta(z_i)\|^2 - \mathrm{KL}\big(q_\phi(z_i \mid x_i) \,\|\, p_\theta(z_i)\big).$$

So the conceptual move from factor analysis to a VAE is small: the latent variable z stays in place, the encoder stays Gaussian, and only the linear decoder Wz is replaced by a flexible nonlinear map (Kingma and Welling, 2014, §2.3). What changes analytically is that the linear-Gaussian closed forms disappear twice. First, the reconstruction expectation is no longer available in closed form once Wz is replaced by $c_\theta(z)$. Second, writing the ELBO gradient formally still leaves an intractable expectation under $q_\phi(z_i \mid x_i)$. That is why the pathwise reparameterization step below is not a cosmetic trick. It is the mechanism that turns this nonlinear latent-variable model back into a trainable stochastic optimization problem.

5.3.2 Reparameterization

For continuous latent variables, the Gaussian encoder from the factor-analysis example already has the right form for the reparameterization trick. The cleanest route is to rewrite the sampled latent variable as a differentiable transformation of parameter-free noise. It converts the gradient of an expectation under q_ϕ into an ordinary pathwise derivative through a fixed source of noise, which is the main computational idea behind stochastic variational autoencoders (Kingma and Welling, 2014, §2.3) and stochastic backpropagation (Rezende et al., 2014, §3). Suppose we can sample from the variational family by drawing noise ε from a fixed base distribution and applying a differentiable transformation,

$$z = T_\phi(x, \varepsilon), \qquad \varepsilon \sim s(\varepsilon), \tag{148}$$

where the law of $T_\phi(x, \varepsilon)$ is exactly $q_\phi(z \mid x)$. Then the ELBO term for one observation can be rewritten as

$$\mathcal{L}_i(\theta, \phi) = \mathbb{E}_{\varepsilon \sim s}\Big[\log p_\theta\big(x_i, T_\phi(x_i, \varepsilon)\big) - \log q_\phi\big(T_\phi(x_i, \varepsilon) \mid x_i\big)\Big]. \tag{149}$$

Now the randomness is independent of the parameters, so gradients with respect to θ and ϕ can be pushed through the sampled latent variable by ordinary backpropagation. The point is structural: once

the latent sample is written as a deterministic function of (x, ε), the ELBO becomes an expectation over a parameter-free noise source.

The standard example is a Gaussian encoder:

$$q_\phi(z \mid x) = \mathcal{N}\big(\mu_\phi(x), \Sigma_\phi(x)\big), \qquad \Sigma_\phi(x) = L_\phi(x) L_\phi(x)^\top, \qquad z = \mu_\phi(x) + L_\phi(x)\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I). \tag{150}$$

The encoder outputs the mean and a covariance factor, and one latent draw is obtained by transforming standard Gaussian noise. A diagonal covariance is the common practical specialization in VAEs, but the same pathwise idea applies more generally. Other common reparameterizable constructions fit the same template: any location-scale family, inverse-CDF transforms when the quantile map is tractable, and richer autoregressive or log-normal style transformations built from differentiable maps of base noise. This is the basic mechanism behind variational autoencoders. The encoder produces an approximate posterior, the decoder specifies $p_\theta(x \mid z)$, and the reparameterized ELBO lets us train both with ordinary stochastic gradients.

Algorithm 26 Stochastic amortized VI with reparameterization

Input: Data $x_{1:n}$, generative model $p_\theta(x, z)$, encoder $q_\phi(z \mid x)$ with reparameterization map T_ϕ, step sizes η_t

1: **for** $t = 0, 1, 2, \dots$ **do**
2: Sample a minibatch $B \subseteq \{1, \dots, n\}$
3: **for** each $i \in B$ **do**
4: Draw $\varepsilon_i \sim s(\varepsilon)$ and set $z_i \leftarrow T_\phi(x_i, \varepsilon_i)$
5: **end for**
6: Form the stochastic ELBO estimate

$$\hat{\mathcal{J}}_B(\theta, \phi) = \frac{n}{|B|} \sum_{i \in B} \Big[\log p_\theta(x_i, z_i) - \log q_\phi(z_i \mid x_i)\Big]$$

7: Update (θ, ϕ) by a stochastic-gradient step using $\nabla_{\theta,\phi} \hat{\mathcal{J}}_B$
8: **end for**

5.3.3 Minibatch ELBO optimization

The summed objective (145) is usually optimized with minibatch stochastic gradient ascent. If $B \subseteq \{1, \dots, n\}$ is a minibatch, the unbiased scaled objective

$$\hat{\mathcal{J}}_B(\theta, \phi) := \frac{n}{|B|} \sum_{i \in B} \hat{\mathcal{L}}_i(\theta, \phi) \tag{151}$$

approximates (145), where each term $\hat{\mathcal{L}}_i$ is itself estimated by Monte Carlo. This is the minibatch VAE training pattern from Kingma and Welling (2014, §2.3). With S samples $z_i^{(s)} \sim q_\phi(\cdot \mid x_i)$,

$$\hat{\mathcal{L}}_i(\theta, \phi) = \frac{1}{S} \sum_{s=1}^{S} \Big[\log p_\theta(x_i, z_i^{(s)}) - \log q_\phi(z_i^{(s)} \mid x_i)\Big]. \tag{152}$$

In the factor-analysis objective (146), the same pattern appears with one additional global term:

$$-\mathrm{KL}\big(q_\xi(W) \,\|\, p_\theta(W)\big).$$

This term stays outside the scaled minibatch sum, exactly as in (147). In practice these evaluations inherit the numerical issues from Sections 1.4.3 and 2.1.4: the ELBO is accumulated on the log scale, transformed latent

variables require the Jacobian bookkeeping from Section 2.1.5, and full-covariance Gaussian families are usually parameterized through the Cholesky constructions from Section 1.4.3. This is where the variational story reconnects to the optimization part of the course: once the ELBO is estimated on minibatches, its training dynamics are governed by the same stochastic-gradient considerations that motivated momentum and adaptive methods earlier.

5.3.4 Score-function gradients

When reparameterization is unavailable, one can still differentiate expectations under q_ϕ by a score-function, or REINFORCE, identity. The correct formula must account for any explicit ϕ-dependence in the integrand as well.

Proposition 5.5 (Score-function differentiation identity)**.** *Let q_ϕ be a differentiable density on a support that does not depend on ϕ. Let f_ϕ be an integrable function such that differentiation can be passed under the integral sign in*

$$\int f_\phi(z)\, q_\phi(z)\, dz.$$

Then

$$\nabla_\phi \mathbb{E}_{q_\phi}[f_\phi(Z)] = \mathbb{E}_{q_\phi}\big[\nabla_\phi f_\phi(Z)\big] + \mathbb{E}_{q_\phi}\big[f_\phi(Z)\, \nabla_\phi \log q_\phi(Z)\big]. \tag{153}$$

If $f_\phi = f$ does not depend on ϕ, this reduces to

$$\nabla_\phi \mathbb{E}_{q_\phi}[f(Z)] = \mathbb{E}_{q_\phi}\big[f(Z)\, \nabla_\phi \log q_\phi(Z)\big].$$

Proof. Differentiate under the integral sign:

$$\nabla_\phi \mathbb{E}_{q_\phi}[f_\phi(Z)] = \nabla_\phi \int f_\phi(z)\, q_\phi(z)\, dz = \int \nabla_\phi f_\phi(z)\, q_\phi(z)\, dz + \int f_\phi(z)\, \nabla_\phi q_\phi(z)\, dz.$$

Using $\nabla_\phi q_\phi(z) = q_\phi(z) \nabla_\phi \log q_\phi(z)$ gives

$$\nabla_\phi \mathbb{E}_{q_\phi}[f_\phi(Z)] = \mathbb{E}_{q_\phi}\big[\nabla_\phi f_\phi(Z)\big] + \int f_\phi(z)\, q_\phi(z)\, \nabla_\phi \log q_\phi(z)\, dz = \mathbb{E}_{q_\phi}\big[\nabla_\phi f_\phi(Z)\big] + \mathbb{E}_{q_\phi}\big[f_\phi(Z)\, \nabla_\phi \log q_\phi(Z)\big].$$

This is (153). ∎

Applied to one-datum ELBO term

$$\mathcal{L}_i(\theta, \phi) = \mathbb{E}_{q_\phi(z_i|x_i)}\Big[\log p_\theta(x_i, z_i) - \log q_\phi(z_i \mid x_i)\Big],$$

we take

$$f_\phi(z_i) = \log p_\theta(x_i, z_i) - \log q_\phi(z_i \mid x_i).$$

Then $\nabla_\phi f_\phi(z_i) = -\nabla_\phi \log q_\phi(z_i \mid x_i)$, and

$$\mathbb{E}_{q_\phi}\big[\nabla_\phi \log q_\phi(Z \mid x_i)\big] = \nabla_\phi \int q_\phi(z \mid x_i)\, dz = 0.$$

So (153) yields the usual score-function ELBO gradient

$$\nabla_\phi \mathcal{L}_i(\theta, \phi) = \mathbb{E}_{q_\phi(z_i|x_i)}\Big[\big(\log p_\theta(x_i, z_i) - \log q_\phi(z_i \mid x_i)\big) \nabla_\phi \log q_\phi(z_i \mid x_i)\Big].$$

This estimator is very general, but it is often noisy in practice because the scalar weight multiplying the score can vary substantially across samples. For that reason, the reparameterization path above is the default route whenever the latent variables are continuous, and score-function estimators are best viewed as the fallback that keeps the framework applicable more broadly.

Compared with classical mean-field VI, amortization replaces per-datum coordinate optimization by a single shared inference rule. That rule is approximate, but it scales to large data sets, supports minibatch training, and makes inference for a new datum a single encoder evaluation. The gain in scalability comes with two familiar risks. Even when the variational family is expressive enough for each datum separately, a shared encoder can still miss the best local approximation and leave an amortization gap. If the decoder is strong enough to explain x without using z very much, training can drift toward posterior collapse (He et al., 2019, §§3–4).

6 Diffusion models

Diffusion models are a way to sample from a target law p_0 by starting from a reference law p_1 that is easy to sample. The abstract setup has two ingredients: a joint law for the endpoint pair (X_0, X_1), with marginals p_0 and p_1, and a bridge that fills in the intermediate states. We first set up that bridge abstractly and identify the reverse kernels needed for sampling. We then specialize to Euclidean models, where the relevant reverse-time quantities can be learned by regression. We then use the course sampling toolkit to build faster, guided, and diverse diffusion samplers within the same bridge notation. The discrete-time ELBO appears as a grid-level rewriting of the same construction rather than as a separate starting point.

6.1 Bridges and infinitesimal dynamics

Let p_0 be the law we want to sample and let p_1 be a law from which sampling is easy. The problem is to start from $X_1 \sim p_1$ and construct a reverse-time dynamics that ends at p_0. The abstract setup has two ingredients. First choose a joint law for the endpoint pair (X_0, X_1) with marginals p_0 and p_1. Then choose a bridge that fills in the intermediate states. This formulation works on a general state space, not just on $\mathbb{R}^d$, so the Gaussian constructions used in continuous diffusion models are later specializations rather than the starting point.

For reverse-time sampling, the key object is the conditional bridge step from time t back to an earlier time s. In the one-sided form used throughout most of this chapter, the bridge provides the law of X_s given (X_0, X_t). The reverse kernel is obtained by averaging that bridge step against the posterior law of X_0 given X_t. This bridge-first setup matches the general presentation in Holderrieth and Erives (2026, §3.1) and Lai et al. (2025, §5.3.2).

6.1.1 Conditional and marginal probability paths

Fix a state space $\mathcal{X}$, for example $\mathbb{R}^d$ in Euclidean models or a finite set in discrete models. Let p_0 and p_1 be endpoint laws on $\mathcal{X}$, with $t = 0$ corresponding to p_0 and $t = 1$ corresponding to p_1. Choose a joint law Γ for (X_0, X_1) with marginals p_0 and p_1; this is the coupling of the endpoints. Then choose kernels $K_{t|0,1}(\cdot \mid x_0, x_1)$ describing the law of X_t given $(X_0, X_1) = (x_0, x_1)$. Averaging those kernels against Γ gives the marginal path p_t.

For reverse-time sampling, the key object is the conditional bridge step from t back to s. In the fully coupled formulation this is the law of X_s given (X_0, X_1, X_t). In many diffusion constructions it reduces to the law of X_s given (X_0, X_t), which is the notation we use below.

Definition 6.1 (Conditional and marginal probability paths)**.** Let $(X_0, X_1) \sim \Gamma$, where Γ is a coupling of (p_0, p_1). A *conditional probability path* is a family of Markov kernels $\{K_{t|0,1}(\cdot \mid x_0, x_1)\}_{t\in[0,1],\,(x_0,x_1)\in\mathcal{X}^2}$ such that

$$K_{0|0,1}(\cdot \mid x_0, x_1) = \delta_{x_0}, \qquad K_{1|0,1}(\cdot \mid x_0, x_1) = \delta_{x_1},$$

where δ_x denotes the point mass at x. The *marginal probability path* is the family of distributions

$$p_t(A) := \int_{\mathcal{X}^2} K_{t|0,1}(A \mid x_0, x_1)\,\Gamma(dx_0, dx_1), \qquad A \subseteq \mathcal{X} \text{ measurable.} \tag{154}$$

The same definition applies in continuous and discrete settings; on a finite state space the integrals are simply sums. For reverse-time sampling, let $K_{s|0,1,t}$ denote the bridge step conditioned on (X_0, X_1, X_t), and let

$$q_{0,1|t}(\cdot \mid x_t) := \mathrm{Law}\big((X_0, X_1) \mid X_t = x_t\big).$$

Then the reverse kernel from time t to time s is

$$K_{s|t}(A \mid x_t) = \int_{\mathcal{X}^2} K_{s|0,1,t}(A \mid x_0, x_1, x_t)\, q_{0,1|t}(dx_0, dx_1 \mid x_t).$$

In many constructions, including the linear Gaussian path below, this reduces to conditioning only on (X_0, X_t). From that point on we write

$$q_{0|t}(\cdot \mid x_t) := \mathrm{Law}(X_0 \mid X_t = x_t),$$

and $K_{s|0,t}$ for the corresponding bridge step conditioned on (X_0, X_t). The results below are written in this reduced notation; the fully coupled version is obtained by replacing X_0 by (X_0, X_1). For $0 \le s \le t \le u \le 1$, the reverse kernels should also compose consistently in time.

Definition 6.2 (Bridge-consistent reverse kernels)**.** Using the notation above, we say that the family $\{K_{s|t}\}_{0\le s\le t\le 1}$ is *bridge-consistent* if for every $0 \le s \le t \le u \le 1$ and every measurable set $A \subseteq \mathcal{X}$,

$$K_{s|t}(A \mid x_t) = \int_{\mathcal{X}} K_{s|0,t}(A \mid x_0, x_t)\, q_{0|t}(dx_0 \mid x_t). \tag{155}$$

and

$$K_{s|u}(A \mid x_u) = \int_{\mathcal{X}} K_{s|t}(A \mid x_t)\, K_{t|u}(dx_t \mid x_u), \tag{156}$$

Equation (156) says that reverse kernels compose correctly in time, just as forward Markov kernels do. Equation (155) says how a single reverse step is built: condition on the endpoint, take the corresponding bridge step, and then average over the posterior of that endpoint given the current state. In discrete models the same formulas hold with sums in place of integrals, and the resulting reverse dynamics is usually implemented by time-inhomogeneous jump kernels. In Euclidean models it is more convenient to pass next to infinitesimal notation.

6.1.2 Exact distributional posterior learning

Before making any tractability reduction, it is worth spelling out the exact idealization. We state it in the reduced notation just introduced; the fully coupled endpoint version is identical. Suppose that at each time t we could learn the full conditional law

$$q_{0|t}(\cdot \mid x_t) = \mathrm{Law}(X_0 \mid X_t = x_t)$$

of the endpoint given the current state. Then reverse-time sampling would proceed in two steps: first sample an endpoint from that conditional law, then take a bridge step conditioned on the sampled endpoint. This is a genuinely distributional model, because the learned object is the whole posterior law rather than a smaller statistic derived from it.

Proposition 6.3 (Exact reverse kernels transport the forward path)**.** *For every $0 \le s \le t \le 1$ and every measurable set $A \subseteq \mathcal{X}$,*

$$p_s(A) = \int_{\mathcal{X}} K_{s|t}(A \mid x_t)\, p_t(dx_t).$$

Proof. By the law of total probability,

$$\int_{\mathcal{X}} K_{s|t}(A \mid x_t)\, p_t(dx_t) = \mathbb{P}(X_s \in A) = p_s(A).$$

■

Theorem 6.4 (Distributional posterior learning is exact on any grid)**.** *Fix any decreasing time grid $1 = t_K > \cdots > t_0 = 0$. Suppose a model returns the exact conditional law*

$$q_{0|t_k}(\cdot \mid x) = \mathrm{Law}(X_0 \mid X_{t_k} = x)$$

at every grid time. Initialize $\hat{X}_{t_K} \sim p_1$, and for $k = K, \dots, 1$ generate the reverse chain by

$$\tilde{X}_0^{(k)} \mid \hat{X}_{t_k} = x \sim q_{0|t_k}(\cdot \mid x), \qquad \hat{X}_{t_{k-1}} \mid \left(\tilde{X}_0^{(k)} = x_0, \hat{X}_{t_k} = x\right) \sim K_{t_{k-1}|0,t_k}(\cdot \mid x_0, x).$$

Then $\hat{X}_{t_k} \sim p_{t_k}$ for every k. In particular $\hat{X}_{t_0} \sim p_0$.

Proof. Conditionally on $\hat{X}_{t_k} = x$, we first sample $\tilde{X}_0^{(k)} \sim q_{0|t_k}(\cdot \mid x)$ and then sample $\hat{X}_{t_{k-1}}$ from the bridge conditioned on that endpoint. Therefore, for any measurable $A \subseteq \mathcal{X}$,

$$\mathbb{P}\left(\hat{X}_{t_{k-1}} \in A \mid \hat{X}_{t_k} = x\right) = \int_{\mathcal{X}} K_{t_{k-1}|0,t_k}(A \mid x_0, x)\, q_{0|t_k}(dx_0 \mid x) = K_{t_{k-1}|t_k}(A \mid x),$$

where the second equality is the posterior-averaging identity (155). Assume inductively that $\hat{X}_{t_k} \sim p_{t_k}$. Then by Proposition 6.3,

$$\mathbb{P}(\hat{X}_{t_{k-1}} \in A) = \int_{\mathcal{X}} K_{t_{k-1}|t_k}(A \mid x)\, p_{t_k}(dx) = p_{t_{k-1}}(A).$$

Starting from $\hat{X}_{t_K} \sim p_1$, induction gives the claim for every grid point. ■

This theorem is domain-agnostic. It applies just as well on $\mathbb{R}^d$ as on a finite state space. The practical question is therefore not whether full posterior learning would be correct, but which part of that exact distributional picture can be represented tractably. On a finite state space, the learned object can sometimes literally be a simplex vector. On $\mathbb{R}^d$, the full posterior is genuinely infinite-dimensional, so the tractability move is to replace it by posterior moments that still determine the marginal dynamics. From this point on, the Euclidean development makes that reduction explicit.

6.1.3 Stochastic interpolants and Gaussian bridges

When $\mathcal{X} = \mathbb{R}^d$, one flexible way to design conditional paths is the *stochastic interpolant* framework of Albergo et al. (2025, Def. 1). The construction answers three questions: how to couple the endpoint laws, how to interpolate between sampled endpoints, and how much additional bridge noise to add along the way. It specifies a random variable X_t of the form

$$X_t := I(t, X_0, X_1) + \gamma(t)\xi, \tag{157}$$

where (X_0, X_1) is a coupling of (p_0, p_1), $\xi \sim \mathcal{N}(0, I_d)$ is independent noise, $I(0, x_0, x_1) = x_0$, $I(1, x_0, x_1) = x_1$, and $\gamma(0) = \gamma(1) = 0$. The structural results below depend only on this path specification. The main continuous-diffusion specialization is the linear Gaussian family, because in that case the posterior summaries and reverse-time dynamics become completely explicit. For the Gaussian conditional-path specialization used in flow matching, see also Lipman et al. (2023, §§3.1 and 4). In that specialization,

$$X_t = \alpha(t)X_0 + \sigma(t)Z, \qquad Z \sim \mathcal{N}(0, I_d) \text{ independent of } X_0. \tag{158}$$

Here $\alpha(0) = 1$, $\alpha(1) = 0$, $\sigma(0) = 0$, and $\sigma(1) = 1$. This still sits inside the stochastic-interpolant framework. In particular, if we take the linear interpolant $I(t, x_0, x_1) = (1-t)x_0 + tx_1$, let $X_1 \sim \mathcal{N}(0, I_d)$, and keep a nonzero bridge noise $\gamma(t)\xi$, then conditioned on X_0 the resulting path is Gaussian with

$$\alpha(t) = 1 - t, \qquad \sigma(t)^2 = t^2 + \gamma(t)^2.$$

We therefore parametrize the running example directly by α and σ, since those are the quantities that enter the conditional laws and reverse-time formulas.

Conditioned on $X_0 = x_0$, the law of X_t is Gaussian:

$$X_t \mid X_0 = x_0 \sim \mathcal{N}\big(\alpha(t)x_0,\ \sigma(t)^2 I_d\big). \tag{159}$$

The endpoint conditions imply

$$X_1 \mid X_0 = x_0 \sim \mathcal{N}(0, I_d) = p_1, \qquad X_0 \mid X_0 = x_0 = x_0,$$

so the conditional path really does end at the full reference law p_1, independently of x_0. (159) can be read as the density of the conditional kernel $K_{t|0}(\cdot \mid x_0)$, which couples the endpoint x_0 to the reference law through a linear Gaussian bridge. The corresponding marginal density is the mixture

$$p_t(x) = \int_{\mathbb{R}^d} K_{t|0}(x \mid x_0)\, p_0(x_0)\, dx_0. \tag{160}$$

The conditional/marginal distinction in (159)–(160) is also the starting point for flow matching (Lipman et al., 2023, §3.1). Figure 28 illustrates the induced conditional and marginal laws.

The probability path induces a posterior distribution at every time: conditioning on $X_t = x$ produces a conditional law for the retained endpoint variables, and, if we use a construction like (157), for the auxiliary latent variables as well. This posterior is the exact object from which the reverse kernel is obtained, by the averaging identity above. In the Euclidean setting we typically do not try to model the full posterior on all latent variables directly. Instead, the deterministic and stochastic dynamics below depend only on conditional expectations under this posterior, and those expectations are exactly the infinitesimal quantities needed to generate the marginal path.

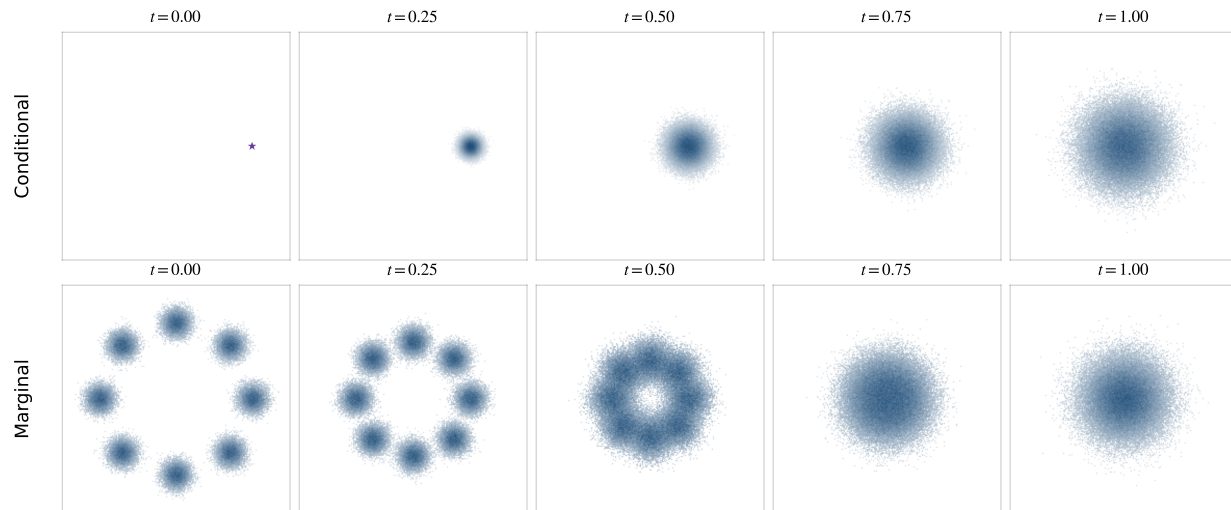

Figure 28: The Gaussian conditional path (159) for a fixed x_0 (top row) and the induced marginal law p_t when $X_0 \sim p_0$ (bottom row), illustrated here for the simple choice $\alpha(t) = 1 - t$ and $\sigma(t) = t$. Sampling starts from p_1 at $t = 1$ and runs the reverse-time dynamics back to $t = 0$.

6.1.4 Velocity, Endpoint Means, and Score Recovery

For reverse-time simulation we need a field that depends only on the current state x, not on the hidden variables used to generate the bridge. The Euclidean tractability move is therefore to replace the full posterior law $q_{0|t}(\cdot \mid x)$ by conditional expectations under it. For a differentiable interpolant, define the time derivative

$$\dot{X}_t := \partial_t I(t, X_0, X_1) + \gamma'(t)\xi,$$

which still depends on the latent variables and so cannot be used directly as a state-dependent drift. Its posterior mean is the *velocity field*

$$b(t, x) := \mathbb{E}[\dot{X}_t \mid X_t = x]. \tag{161}$$

This is the primary posterior summary in the continuous-time theory. See Holderrieth and Erives (2026, §4.1, eqs. (38)–(40)) for the same velocity/score/endpoint-mean conversion pattern in a general bridge exposition. For the linear Gaussian bridge (158), it is also useful to introduce the posterior mean of the endpoint at time 0,

$$m(t, x) := \mathbb{E}[X_0 \mid X_t = x]. \tag{162}$$

Once these two summaries are in hand, the score will follow algebraically from the same Gaussian identities. For the linear Gaussian bridge (158), we have

$$\dot{X}_t = \alpha'(t)X_0 + \sigma'(t)Z,$$

so $b(t, x) = \mathbb{E}[\alpha'(t)X_0 + \sigma'(t)Z \mid X_t = x]$. Moreover, conditioning on $(X_0, X_t) = (x_0, x)$ yields

$$\mathbb{E}[\dot{X}_t \mid X_0 = x_0, X_t = x] = \alpha'(t)x_0 + \frac{\sigma'(t)}{\sigma(t)}\big(x - \alpha(t)x_0\big),$$

which is the closed-form conditional vector field used in conditional flow matching. The score field $s(t, x) := \nabla \log p_t(x)$ is then determined by b as well.

Proposition 6.5 (Derived endpoint mean and score for the linear Gaussian bridge)**.** *Assume* (158) *holds with smooth functions α, σ such that $\sigma(t) > 0$ and*

$$D(t) := \alpha(t)\sigma'(t) - \sigma(t)\alpha'(t) \neq 0$$

for $t \in (0, 1]$. Fix $t \in (0, 1]$. Assume $x \mapsto K_{t|0}(x \mid x_0)$ is C^1 for every x_0, that differentiation with respect to x may be passed through the mixture formula (160)*, and that $p_t(x) > 0$ at the points where the score identity is evaluated. Then:*

1. *The endpoint posterior mean is determined by the velocity:*

$$m(t, x) = \frac{\sigma'(t)x - \sigma(t)b(t, x)}{D(t)}. \tag{163}$$

2. *The score can be written in terms of the endpoint mean (equivalently, in terms of b):*

$$\nabla \log p_t(x) = -\frac{1}{\sigma(t)^2}\Big(x - \alpha(t)\, m(t, x)\Big) = \frac{\alpha'(t)x - \alpha(t)b(t, x)}{\sigma(t)D(t)}. \tag{164}$$

Proof. Define

$$n(t, x) := \mathbb{E}[Z \mid X_t = x].$$

Taking conditional expectations in (158) gives

$$x = \alpha(t)\mathbb{E}[X_0 \mid X_t = x] + \sigma(t)\mathbb{E}[Z \mid X_t = x] = \alpha(t)m(t, x) + \sigma(t)n(t, x).$$

Likewise,

$$b(t, x) = \alpha'(t)m(t, x) + \sigma'(t)n(t, x).$$

Eliminating $n(t, x)$ from these two equations yields

$$D(t)m(t, x) = \sigma'(t)x - \sigma(t)b(t, x),$$

which proves (163).

For the score, conditioned on X_0, (159) gives the density

$$K_{t|0}(x \mid X_0) \propto \exp\left(-\frac{1}{2\sigma(t)^2}\|x - \alpha(t)X_0\|_2^2\right).$$

Differentiating with respect to x yields

$$\nabla_x \log K_{t|0}(x \mid X_0) = -\frac{x - \alpha(t)X_0}{\sigma(t)^2}.$$

By (160) and differentiation under the integral sign,

$$\nabla p_t(x) = \int_{\mathbb{R}^d} \nabla_x K_{t|0}(x \mid x_0)\, p_0(x_0)\, dx_0.$$

Since $p_t(x) > 0$, we may divide by $p_t(x)$ and use $\nabla_x K_{t|0} = K_{t|0}\nabla_x \log K_{t|0}$ to obtain

$$\begin{aligned}\nabla \log p_t(x) &= \frac{1}{p_t(x)} \int_{\mathbb{R}^d} \nabla_x K_{t|0}(x \mid x_0)\, p_0(x_0)\, dx_0 \\ &= \int_{\mathbb{R}^d} \nabla_x \log K_{t|0}(x \mid x_0)\, \frac{K_{t|0}(x \mid x_0)\, p_0(x_0)}{p_t(x)}\, dx_0.\end{aligned}$$

The factor multiplying $\nabla_x \log K_{t|0}$ is exactly the posterior density of X_0 given $X_t = x$, so this is the Fisher-type identity

$$\nabla \log p_t(x) = \mathbb{E}\left[\nabla_x \log K_{t|0}(x \mid X_0) \mid X_t = x\right].$$

Substituting the Gaussian score from above gives

$$\nabla \log p_t(x) = -\frac{1}{\sigma(t)^2}\,\mathbb{E}[x - \alpha(t)X_0 \mid X_t = x] = -\frac{1}{\sigma(t)^2}\Big(x - \alpha(t)m(t,x)\Big),$$

which is the first identity in (164). Using $x - \alpha(t)m(t,x) = \sigma(t)n(t,x)$ and eliminating $m(t,x)$ instead gives

$$D(t)n(t,x) = \alpha(t)b(t,x) - \alpha'(t)x,$$

hence

$$\nabla \log p_t(x) = -\frac{n(t,x)}{\sigma(t)} = \frac{\alpha'(t)x - \alpha(t)b(t,x)}{\sigma(t)D(t)}.$$

This is the second identity in (164). ■

The practical lesson is that the Gaussian bridge turns the full posterior law into a small menu of interchangeable posterior summaries. One may learn the velocity field, the endpoint posterior mean, the Gaussian innovation predictor, or the score. For this bridge these are not different conceptual objects; they are linear reparameterizations of the same posterior information.

6.1.5 Density evolution and common-marginal dynamics

The velocity field b transports the marginal density p_t. One realization of this path is the deterministic probability-flow ODE with drift b, and in flow matching this is the core marginalization trick: one can define a marginal vector field by averaging simple conditional velocities under the posterior, and this marginal vector field exactly generates the marginal probability path (Lipman et al., 2023, §3.1); see also Holderrieth and Erives (2026, Thm. 11).

Proposition 6.6 (Transport equation for the bridge marginals)**.** *Assume $t \mapsto X_t$ is differentiable almost surely and that X_t has a density p_t for every $t \in [0,1]$. Let b be defined by (161). Then p_t satisfies*

$$\partial_t p_t(x) + \nabla \cdot \big(b(t,x)p_t(x)\big) = 0. \tag{165}$$

Proof. Let $\varphi \in C_0^\infty(\mathbb{R}^d)$. Differentiate $\varphi(X_t)$ using $\dot{X}_t$ and take expectations:

$$\frac{d}{dt}\mathbb{E}[\varphi(X_t)] = \mathbb{E}\left[\langle \nabla\varphi(X_t)\rangle \dot{X}_t\right] = \mathbb{E}\left[\langle \nabla\varphi(X_t)\rangle \mathbb{E}[\dot{X}_t \mid X_t]\right] = \int_{\mathbb{R}^d} \langle \nabla\varphi(x)\rangle b(t,x)p_t(x)\,dx.$$

Integrating by parts yields the weak form of (165). ■

Once p_t and b are fixed, there is no reason to privilege a single trajectory-level dynamics. The same forward law can be realized by a whole family of reverse-time diffusions and ODEs. For sampling, the stochastic realizations are especially natural, while the probability-flow ODE appears as the zero-noise member of the same family. This common-marginal viewpoint is formalized by Albergo et al. (2025, §2) and Holderrieth and Erives (2026, Thm. 17).

Proposition 6.7 (A common-marginal family of dynamics)**.** *Assume that for each $t \in (0,1)$, the density p_t is C^2 and strictly positive on $\mathbb{R}^d$. Let $s(t,x) := \nabla \log p_t(x)$, and assume the Fokker–Planck equations below are well posed for the initial data considered. Fix any continuous schedule $\varepsilon : [0,1] \to [0,\infty)$ and define*

$$b_{\text{fwd}}(t,x) := b(t,x) + \varepsilon(t)s(t,x), \qquad b_{\text{rev}}(t,x) := b(t,x) - \varepsilon(t)s(t,x). \tag{166}$$

Consider the forward and reverse-time dynamics

$$dY_t^{\text{fwd}} = b_{\text{fwd}}(t, Y_t^{\text{fwd}})\,dt + \sqrt{2\,\varepsilon(t)}\,dB_t, \qquad dY_t^{\text{rev}} = b_{\text{rev}}(t, Y_t^{\text{rev}})\,dt + \sqrt{2\,\varepsilon(t)}\,dB_t, \tag{167}$$

where the second equation is simulated backward in time from $t = 1$ to $t = 0$. When $\varepsilon \equiv 0$, these reduce to deterministic ODEs with drift b. Then:

1. *If $Y_0^{\text{fwd}} \sim p_0$, then the time-marginal density of (Y_t^{fwd}) is p_t for every $t \in [0,1]$.*
2. *If $Y_1^{\text{rev}} \sim p_1$, then the reverse-time marginal density of (Y_t^{rev}) at time t is also p_t.*

Proof. If $\varepsilon \equiv 0$, then the first claim is exactly the transport equation (165) written in trajectory form. Assume now that ε is arbitrary. The density ρ of (Y_t^{fwd}) then satisfies the forward Fokker–Planck equation

$$\partial_t \rho + \nabla \cdot \big((b + \varepsilon s)\rho\big) = \varepsilon \Delta \rho.$$

To identify the solution, observe that p_t itself satisfies the same PDE:

$$\partial_t p_t + \nabla \cdot \big((b + \varepsilon s)p_t\big) = \partial_t p_t + \nabla \cdot (b p_t) + \varepsilon \nabla \cdot (s p_t).$$

By (165), the first two terms sum to zero, while

$$\nabla \cdot (s p_t) = \nabla \cdot \big((\nabla \log p_t) p_t\big) = \nabla \cdot (\nabla p_t) = \Delta p_t.$$

Hence p_t solves the forward Fokker–Planck equation with initial condition p_0. By well posedness, $\rho_t = p_t$ for all $t \in [0,1]$.

For the second claim, set $\tau := 1 - t$ and define $\bar{p}_\tau(x) := p_{1-\tau}(x)$. Then

$$\partial_\tau \bar{p}_\tau(x) = -\partial_t p_t(x)|_{t=1-\tau} = \nabla \cdot \big(b(1-\tau, x)\bar{p}_\tau(x)\big)$$

by (165). Now consider the forward-in-τ process associated with backward simulation in t:

$$d\bar{Y}_\tau = -b_{\text{rev}}(1-\tau, \bar{Y}_\tau)\,d\tau + \sqrt{2\,\varepsilon(1-\tau)}\,d\bar{B}_\tau.$$

Its density $\bar{\rho}_\tau$ satisfies

$$\partial_\tau \bar{\rho}_\tau + \nabla \cdot \big(-b_{\text{rev}}(1-\tau, \cdot)\bar{\rho}_\tau\big) = \varepsilon(1-\tau)\Delta \bar{\rho}_\tau.$$

Using $b_{\text{rev}} = b - \varepsilon s$, the candidate path $\bar{p}_\tau$ obeys

$$\partial_\tau \bar{p}_\tau + \nabla \cdot \big((-b + \varepsilon s)(1-\tau, \cdot)\bar{p}_\tau\big) = \varepsilon(1-\tau)\nabla \cdot \big(s(1-\tau, \cdot)\bar{p}_\tau\big) = \varepsilon(1-\tau)\Delta \bar{p}_\tau.$$

Thus $\bar{p}_\tau$ solves the forward Fokker–Planck equation for the time-reversed simulation, with initial condition $\bar{p}_0 = p_1$. By well posedness, $\bar{\rho}_\tau = \bar{p}_\tau$. Translating back to the original time variable t yields the stated reverse-time simulation property. ■

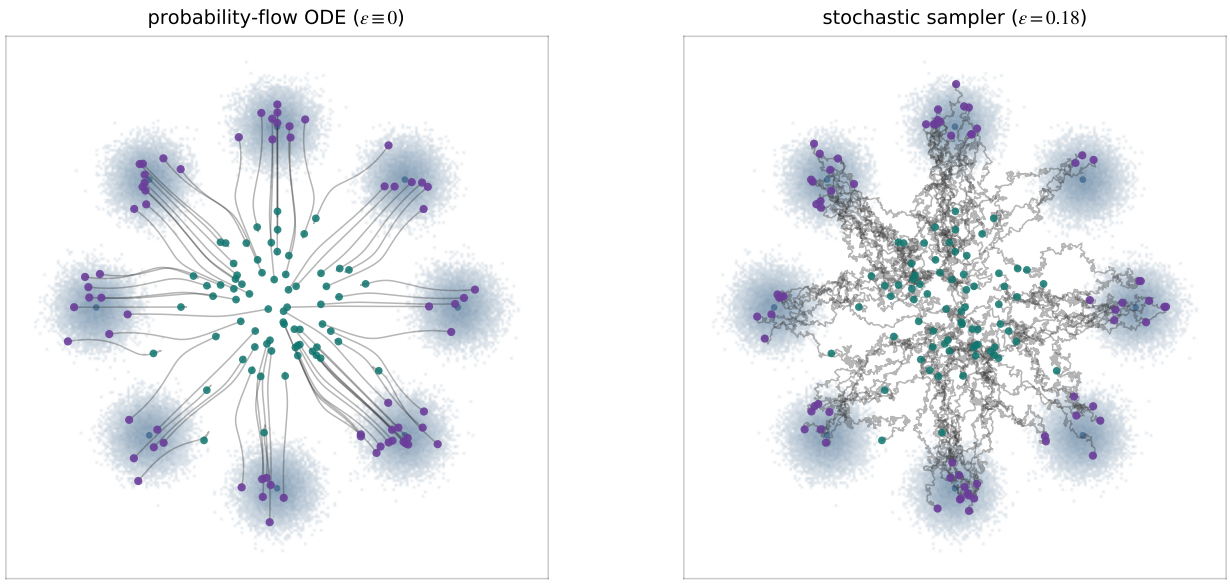

Figure 29: Reverse-time simulation from p_1 back to p_0 for a toy two-dimensional data distribution. Different choices of the noise schedule ε in (167) produce different sample trajectories while preserving the same marginal path p_t. The probability-flow ODE is the zero-noise case $\varepsilon \equiv 0$.

Corollary 6.8 (Langevin as a stationary-path special case)**.** *Suppose $p_t \equiv \pi$ is constant in time, $b \equiv 0$, and $\varepsilon(t) \equiv 1$. Then the forward member of (167) is*

$$dY_t = \nabla \log \pi(Y_t)\, dt + \sqrt{2}\, dB_t,$$

which is exactly the overdamped Langevin diffusion from Section 2.3.5, and every marginal law is π.

This is the course-level payoff of the common-marginal viewpoint. The same drift-noise template governs stationary MCMC and time-inhomogeneous diffusion samplers; what changes is whether the target law is fixed or moves along the bridge.

6.2 Regression and reverse-time simulation

Once the exact bridge picture has been reduced to tractable posterior summaries, the practical problem is to learn those summaries and discretize the resulting reverse-time dynamics. In Euclidean models this means regressing infinitesimal objects such as the velocity field and, when the bridge does not provide an explicit conversion formula, either the score itself or another summary from which the score can be recovered. After time discretization the same bridge can also be written as a latent-path model with an ELBO objective, but that is a later reformulation of the same bridge rather than the starting point.

6.2.1 Reverse-time sampling as regression

The reverse-time family in Proposition 6.7 involves both the velocity field $b(t, x)$ and the score $s(t, x) = \nabla \log p_t(x)$. In principle one could instead parameterize the full posterior law $q_{0|t}(\cdot \mid x)$ and use the exact distributional sampler from Theorem 6.4. Euclidean models usually do not do that, because the learned object would be a full conditional distribution on $\mathbb{R}^d$ at every time. This is the Euclidean tractability move

from the posterior viewpoint. Rather than parameterizing the full posterior, we ask for lower-dimensional summaries of it. One natural summary is the conditional expectation $b(t,x) = \mathbb{E}[\dot{X}_t \mid X_t = x]$, which is exactly the marginal velocity appearing in the transport equation and in flow matching. For the linear Gaussian bridge, Proposition 6.5 shows that both the score and the posterior mean of the $t = 0$ endpoint are explicit functions of that same velocity field. That special algebraic reduction is the running example in this subsection. Outside such specializations, learning b alone does not in general determine the reverse-time SDE: one also needs either a separate score model or another bridge-specific relation that recovers the score from the learned summary. Once that reduction has been made, two practical questions remain: how to learn the summary, and how to discretize the reverse-time dynamics determined by it.

For a general stochastic interpolant, the regression target is the conditional velocity $\dot{X}_t$ from (161). From this point onward we use the linear Gaussian bridge as the running example, because it makes the recovery formulas completely explicit:

$$m(t,x) = \frac{\sigma'(t)x - \sigma(t)b(t,x)}{D(t)}, \qquad \nabla \log p_t(x) = \frac{\alpha'(t)x - \alpha(t)b(t,x)}{\sigma(t)D(t)}, \qquad D(t) = \alpha(t)\sigma'(t) - \sigma(t)\alpha'(t).$$

At the level of updates, three parts of the course now line up:

$$\begin{aligned} \text{gradient descent:} \quad & x_{k+1} = x_k - \eta \nabla f(x_k), \\ \text{ULA:} \quad & x_{k+1} = x_k + \eta \nabla \log \pi(x_k) + \sqrt{2\eta}\,\xi_k, \qquad \xi_k \sim \mathcal{N}(0, I_d), \\ \text{reverse diffusion:} \quad & x_{t_k} = x_{t_{k+1}} + \Delta t \left(b(t, x_{t_{k+1}}) - \varepsilon(t) s(t, x_{t_{k+1}}) \right) + \sqrt{2\,\varepsilon(t)|\Delta t|}\,\xi_k, \end{aligned}$$

with $\Delta t = t_k - t_{k+1} < 0$. The optimization update uses only drift, Langevin adds calibrated noise to target a fixed distribution, and diffusion samplers use the same drift-noise template for a time-varying bridge. The stationary-path case in the previous subsection recovers Langevin exactly. For the present linear Gaussian bridge, the operational point is that once b is learned, the score needed by the reverse-time sampler is known as well through Proposition 6.5. The familiar straight-line bridge is recovered by $\alpha(t) = 1 - t$ and $\sigma(t) = t$. More general Gaussian bridges inherit the transformed-density and Cholesky issues from Sections 1.4.3 and 2.1.5.

6.2.2 Conditional and marginal flow matching

We learn b by regression on synthetic bridge pairs. Sample $x_0 \sim p_0$, $t \sim \mathrm{Unif}[0,1]$, and $z \sim \mathcal{N}(0, I_d)$, then form

$$x_t := \alpha(t)x_0 + \sigma(t)z, \qquad v_t := \alpha'(t)x_0 + \sigma'(t)z.$$

Since $v_t = \dot{X}_t$ for the path (158), a least-squares regression of v_t on (t, x_t) yields an estimator of $b(t,x) = \mathbb{E}[\dot{X}_t \mid X_t = x]$. This is the conditional flow matching objective specialized to the linear Gaussian bridge (Lipman et al., 2023, §3.2); the corresponding squared-loss equivalence with the marginal target is also stated in Holderrieth and Erives (2026, Thm. 12).

Proposition 6.9 (Flow matching vs. conditional flow matching (squared loss))**.** *Fix $t \in [0,1]$ and let U_t be a random vector and X_t a random variable on $\mathbb{R}^d$. Define the* marginal target $b(t,x) := \mathbb{E}[U_t \mid X_t = x]$. *Then for any measurable predictor $v_\theta(t,\cdot) : \mathbb{R}^d \to \mathbb{R}^d$,*

$$\mathbb{E}\left[\|v_\theta(t,X_t) - U_t\|_2^2\right] = \mathbb{E}\left[\|v_\theta(t,X_t) - b(t,X_t)\|_2^2\right] + \mathbb{E}\left[\|U_t - b(t,X_t)\|_2^2\right], \tag{168}$$

so the two objectives differ by a constant independent of θ.

Proof. Write $U_t = b(t, X_t) + \big(U_t - b(t, X_t)\big)$ and expand the square:

$$\|v_\theta(t, X_t) - U_t\|_2^2 = \|v_\theta(t, X_t) - b(t, X_t)\|_2^2 + \|U_t - b(t, X_t)\|_2^2 - 2\langle v_\theta(t, X_t) - b(t, X_t)\rangle U_t - b(t, X_t).$$

Taking expectations, the cross term vanishes by conditioning on X_t:

$$\mathbb{E}[\langle v_\theta(t, X_t) - b(t, X_t)\rangle U_t - b(t, X_t)] = \mathbb{E}[\langle v_\theta(t, X_t) - b(t, X_t)\rangle \mathbb{E}[U_t - b(t, X_t) \mid X_t]] = 0.$$

This yields (168). ■

In particular, minimizing the "conditional" objective $\mathbb{E}[\|v_\theta(t, X_t) - U_t\|_2^2]$ is equivalent to minimizing the "marginal" objective $\mathbb{E}[\|v_\theta(t, X_t) - b(t, X_t)\|_2^2]$. This is the key justification for conditional flow matching (Lipman et al., 2023, Thm. 2); see also Holderrieth and Erives (2026, Thm. 12). Applied to a stochastic interpolant, we simply take $U_t = \dot{X}_t$. In the linear Gaussian bridge, any endpoint-mean or score estimate is then recovered algebraically from the learned velocity field via Proposition 6.5.

6.2.3 Bridge regression training

For the linear Gaussian bridge, the training procedure is simulation-free: we only sample (X_0, Z) and evaluate (158). The regression target is the conditional path derivative, and in this one-network specialization the learned field b_θ is the only network we need.

Algorithm 27 Bridge regression training for the linear Gaussian bridge

Input: Data sampler for p_0, schedules α, σ, model $b_\theta(t, x)$, training distribution for t (e.g. Unif$[0, 1]$)

```
for training steps do
    Sample x_0 ~ p_0, t ~ Unif[0,1], z ~ N(0, I_d)
    x_t ← α(t)x_0 + σ(t)z                          ▷ bridge sample
    v_t ← α'(t)x_0 + σ'(t)z                        ▷ Ẋ_t target
    ℓ ← ‖b_θ(t, x_t) − v_t‖_2^2
    Take a gradient step on ℓ
end for
```

6.2.4 Reverse-time simulation

In the linear Gaussian bridge, the sampling procedure chooses a reverse-time dynamics from the common-marginal family in Proposition 6.7 and discretizes it using the learned velocity and the score recovered from it via Proposition 6.5. Positive ε gives the stochastic samplers, and $\varepsilon \equiv 0$ recovers the deterministic probability-flow ODE.

In this linear Gaussian setting, the same learned model supports the whole family of reverse-time dynamics through the choice of ε. In either case, once b_θ is learned, the posterior mean estimate of the data point is explicit: $\hat{x}_0(t, x) := \big(\sigma'(t)x - \sigma(t)b_\theta(t, x)\big)\big/\big(\alpha(t)\sigma'(t) - \sigma(t)\alpha'(t)\big)$. This is the promised reduction from sampling to regression: for this bridge, reverse-time simulation reduces to learning one conditional expectation, after which the score and the sampler follow from the known Gaussian bridge identities.

6.2.5 Discrete-time ELBO connection

The same bridge can also be recorded on a time grid. Then x_0 is observed, $x_{1:K}$ are latent bridge states, and the learned reverse chain $p_\theta(x_{0:K})$ is fit by the ELBO identity from Proposition 5.1. The fixed forward bridge supplies a reference law $r_{\text{fwd}}(x_{1:K} \mid x_0)$ on latent paths, so the discrete-time ELBO is the grid-level version of the same reverse-bridge learning problem (Kingma et al., 2021, §3.5, eqs. (11)–(12)). Throughout

Algorithm 28 Sampling by reverse-time dynamics

Input: Base sampler for p_1, schedules α, σ, time grid $1 = t_K > \cdots > t_0 = 0$, schedule $\varepsilon(t) \geq 0$, learned model b_θ

```
Sample X_{t_K} ~ p_1
for k = K − 1, K − 2, ..., 0 do
    t ← t_{k+1},   x ← X_{t_{k+1}}
    b ← b_θ(t, x)                                   ▷ velocity estimate
    D ← α(t)σ′(t) − σ(t)α′(t)
    s ← (α′(t)x − α(t)b) / (σ(t)D)                  ▷ score estimate, Proposition 6.5
    b_rev ← b − ε(t) s
    Δt ← t_k − t_{k+1}                              ▷ Δt < 0
    Sample ξ ~ N(0, I_d)
    X_{t_k} ← x + Δt b_rev + sqrt(2 ε(t) |Δt|) ξ    ▷ Euler–Maruyama step
end for
return X_{t_0}                                      ▷ approximate sample from p_0
```

this subsection, r_{fwd} denotes this fixed grid-level forward law and its induced conditionals on the latent path, while the earlier posterior of the $t = 0$ endpoint remains $q_{0|t}$.

Fix a grid $0 = t_0 < \cdots < t_K = 1$ and write $X_k := X_{t_k}$. Assume the discretized bridge law is Markov with kernels $r_{\text{fwd}}(x_k \mid x_{k-1})$, so that

$$r_{\text{fwd}}(x_{1:K} \mid x_0) = \prod_{k=1}^{K} r_{\text{fwd}}(x_k \mid x_{k-1}).$$

The learned reverse model is

$$p_\theta(x_{0:K}) := p_1(x_K) \prod_{k=1}^{K} p_\theta(x_{k-1} \mid x_k).$$

The bridge prescribes $r_{\text{fwd}}(x_{1:K} \mid x_0)$ in advance, and training only adjusts the reverse transitions $p_\theta(x_{k-1} \mid x_k)$.

Proposition 6.10 (ELBO for a discretized diffusion bridge)**.** *For every data point x_0, the ELBO from Proposition 5.1 with latent variables $x_{1:K}$ is*

$$\mathcal{L}_{\text{diff}}(\theta; x_0) := \mathbb{E}_{r_{\text{fwd}}(x_{1:K}|x_0)}\big[\log p_\theta(x_{0:K}) - \log r_{\text{fwd}}(x_{1:K} \mid x_0)\big] \leq \log p_\theta(x_0).$$

Moreover,

$$\begin{aligned} -\mathcal{L}_{\text{diff}}(\theta; x_0) = \mathrm{KL}\big(r_{\text{fwd}}(x_K \mid x_0) \,\|\, p_1\big) + \mathbb{E}_{r_{\text{fwd}}(x_1|x_0)}\big[-\log p_\theta(x_0 \mid x_1)\big] \\ + \sum_{k=2}^{K} \mathbb{E}_{r_{\text{fwd}}(x_k|x_0)} \mathrm{KL}\big(r_{\text{fwd}}(x_{k-1} \mid x_k, x_0) \,\|\, p_\theta(x_{k-1} \mid x_k)\big). \end{aligned} \tag{169}$$

Proof. Apply Proposition 5.1 with latent variables $z = x_{1:K}$. This gives the ELBO inequality immediately. To obtain the decomposition, factor the forward chain backward:

$$r_{\text{fwd}}(x_{1:K} \mid x_0) = r_{\text{fwd}}(x_K \mid x_0) \prod_{k=2}^{K} r_{\text{fwd}}(x_{k-1} \mid x_k, x_0).$$

Substituting the definitions of $p_\theta(x_{0:K})$ and $r_{\text{fwd}}(x_{1:K} \mid x_0)$ into $\mathcal{L}_{\text{diff}}(\theta; x_0)$ yields

$$\begin{aligned}\mathcal{L}_{\text{diff}}(\theta; x_0) &= \mathbb{E}[\log p_1(X_K) - \log r_{\text{fwd}}(X_K \mid x_0)] + \mathbb{E}[\log p_\theta(x_0 \mid X_1)] \\ &\quad + \sum_{k=2}^{K} \mathbb{E}[\log p_\theta(X_{k-1} \mid X_k) - \log r_{\text{fwd}}(X_{k-1} \mid X_k, x_0)]\,,\end{aligned}$$

where all expectations are under $r_{\text{fwd}}(x_{1:K} \mid x_0)$. The first line is the negative of the prior and reconstruction terms. For each $k \geq 2$, conditioning on X_k turns the k-th summand into the negative KL divergence

$$-\mathbb{E}_{r_{\text{fwd}}(x_k \mid x_0)} \operatorname{KL}\big(r_{\text{fwd}}(x_{k-1} \mid x_k, x_0) \,\|\, p_\theta(x_{k-1} \mid x_k)\big).$$

Collecting terms gives (169). ■

Each KL term asks the learned reverse transition $p_\theta(x_{k-1} \mid x_k)$ to match the exact grid-level bridge posterior $r_{\text{fwd}}(x_{k-1} \mid x_k, x_0)$ on that grid step. The term $\mathbb{E}[-\log p_\theta(x_0 \mid x_1)]$ is the reconstruction term, $\operatorname{KL}(r_{\text{fwd}}(x_K \mid x_0) \| p_1)$ matches the terminal latent law to the reference law, and the interior KL terms measure one-step bridge mismatch along the grid. Because the latent variables form an entire path, the posterior-matching problem unfolds into a sum of local KL terms rather than a single global one.

In the running linear Gaussian setting, the same conclusion follows directly from one-step Gaussian conditioning on the grid. Write $\alpha_k := \alpha(t_k)$ and $\sigma_k := \sigma(t_k)$, and choose Gaussian forward kernels

$$r_{\text{fwd}}(x_k \mid x_{k-1}) = \mathcal{N}\big(a_k x_{k-1},\, b_k^2 I_d\big)$$

with coefficients chosen so that the resulting marginal law has the same form as the continuous bridge,

$$X_k = \alpha_k X_0 + \sigma_k Z, \qquad Z \sim \mathcal{N}(0, I_d),$$

equivalently,

$$\alpha_k = a_k \alpha_{k-1}, \qquad \sigma_k^2 = a_k^2 \sigma_{k-1}^2 + b_k^2.$$

Conditioning the Gaussian prior $X_{k-1} \mid X_0 = x_0 \sim \mathcal{N}(\alpha_{k-1} x_0, \sigma_{k-1}^2 I_d)$ on the observation $X_k = x_k$ gives

$$r_{\text{fwd}}(x_{k-1} \mid x_k, x_0) = \mathcal{N}\big(\underline{\mu}_k(x_k, x_0),\, \underline{\Sigma}_k\big),$$

where

$$\underline{\mu}_k(x_k, x_0) = \frac{a_k \sigma_{k-1}^2}{\sigma_k^2}\, x_k + \frac{\alpha_{k-1} b_k^2}{\sigma_k^2}\, x_0, \qquad \underline{\Sigma}_k = \frac{\sigma_{k-1}^2 b_k^2}{\sigma_k^2}\, I_d.$$

If the learned reverse kernel uses that same covariance,

$$p_\theta(x_{k-1} \mid x_k) = \mathcal{N}\big(\mu_\theta(x_k, k),\, \underline{\Sigma}_k\big),$$

then the k-th KL term in (169) is

$$\operatorname{KL}\big(r_{\text{fwd}}(x_{k-1} \mid x_k, x_0) \,\|\, p_\theta(x_{k-1} \mid x_k)\big) = \frac{1}{2}\big(\underline{\mu}_k(x_k, x_0) - \mu_\theta(x_k, k)\big)^\top \underline{\Sigma}_k^{-1} \big(\underline{\mu}_k(x_k, x_0) - \mu_\theta(x_k, k)\big).$$

So in our linear Gaussian bridge, each ELBO term is literally a weighted least-squares fit to the exact one-step posterior mean. Because $x_k = \alpha_k x_0 + \sigma_k z$, that target mean is an affine function of x_0 and equivalently of the injected noise z. This is the same Gaussian posterior-mean algebra emphasized in Lai et al. (2025, §2.2.5, especially eq. (2.2.11)). This is the grid-level version of the earlier endpoint-mean algebra from Proposition 6.5: different prediction parameterizations are just different ways of writing the same conditional Gaussian mean.

The discrete-time formulas therefore encode the same reverse-bridge learning problem in grid form.

6.3 Using the sampling toolkit with diffusion models

The previous subsection answered the model-building question. It identified the exact reverse kernels and showed how, in Euclidean models, the corresponding reverse-time quantities can be learned by regression. If those exact reverse kernels were available, then by Theorem 6.4 and Proposition 6.3 iterating them would transport p_1 back to p_0. Likewise, if the exact score $s(t,x) = \nabla \log p_t(x)$ and velocity $b(t,x)$ were known and the reverse dynamics from Proposition 6.7 could be simulated accurately, then plain reverse simulation would already solve the unconditional sampling problem.

That statement is theoretically important, but it does not settle the practical sampling question. It answers an idealized problem: how do we generate one unconditional sample from p_0 if exact reverse-time quantities are available? In practice we work with learned approximations, finite time grids, and tasks that go beyond unconditional i.i.d. sampling. So once a reverse bridge is available, the question is no longer "what is diffusion?" but "which sampling problem are we solving on top of that bridge?"

No new sampling framework is needed. The earlier ideas from the course reappear with the same notation. Fix a decreasing grid $1 = t_K > \cdots > t_0 = 0$, and write

$$X_{t_{k-1}}^{\text{prop}} \sim \hat{K}_{t_{k-1}|t_k}(\cdot \mid X_{t_k})$$

for the approximate reverse kernel produced by a learned finite-step sampler such as Algorithm 28. This is meant to approximate the exact reverse kernel $K_{t_{k-1}|t_k}$, not the endpoint posterior $q_{0|t_k}$. Accordingly, $X_{t_{k-1}}^{\text{prop}}$ is a proposal for the state at the earlier time t_{k-1}, not a proposal for X_0 unless $k = 1$. The decreasing-noise family $\{p_t\}_{t\in[0,1]}$ plays the same structural role as the target ladder in Section 4.3: instead of attacking the difficult endpoint law p_0 in one step, the algorithm moves through a sequence of easier intermediate laws. Once such a bridge is on the table, modern diffusion samplers mostly modify one of three things: the proposal family, the target family, or the state space. We may want the same target law p_0, but at lower computational cost. We may want a conditional target law $p_0(\cdot \mid y)$ rather than the unconditional one. Or we may want a finite set of samples whose empirical distribution approximates the target law better than i.i.d. sampling would at the same budget. In the notation of the course, these are three different ways to modify the sampling problem: for efficient sampling, the target family $\{p_t\}$ stays fixed and we redesign the proposal or add a local correction; for guidance, the target family changes from $\{p_t\}$ to the conditional family

$$p_t^y(dx) := p_t(dx \mid y);$$

for finite-particle approximation, the state space changes from $\mathbb{R}^d$ to the product space $(\mathbb{R}^d)^N$ and the target changes from the product law $(p_t^y)^{\otimes N}$ to a coupled law on that product space. More concretely, the first task points back to gradient-based MCMC in Sections 2.3.5 and 2.3.6, the second to conditioning by likelihood reweighting in Example 2.5 and sequential reweighting in Section 4.4, and the third to the empirical-measure viewpoint of particle methods in Sections 4.4.3 and 4.4.4.

6.3.1 Efficient sampling for the same target law

The first task keeps the target law p_0 fixed and asks for a cheaper sampler. Correctness is not the issue here; computation is. With a learned model, the practical endpoint error comes from both approximation error in the learned field and discretization error along the time grid. If controlling those errors requires a very fine grid, then reverse sampling becomes slow. Because each reverse step requires one or more evaluations of the learned score or drift network, the natural cost measure is the number of network function evaluations (NFEs). So "more efficient sampling" means lower endpoint error at a fixed NFE budget, or comparable endpoint error using fewer NFEs.

The important point is that efficiency does not mean changing the target. We still want samples from the same law p_0. We are only changing how we approximate the reverse path. This is exactly the distinction that appeared earlier in gradient-based MCMC: keep the target fixed, then redesign the proposal and, if useful, locally correct it.

Two familiar levers now reappear. The first is to treat $\hat{K}_{t_{k-1}|t_k}$ as a predictor and then follow it with a short correction targeted at the new time level. Formally, one replaces the bare proposal

$$X_{t_{k-1}}^{\text{prop}} \sim \hat{K}_{t_{k-1}|t_k}(\cdot \mid X_{t_k})$$

by the composite update

$$X_{t_{k-1}} \sim C_k(\cdot \mid X_{t_{k-1}}^{\text{prop}}), \qquad X_{t_{k-1}}^{\text{prop}} \sim \hat{K}_{t_{k-1}|t_k}(\cdot \mid X_{t_k}),$$

where C_k is a short Markov kernel chosen so that its invariant law is, at least approximately, the current marginal $p_{t_{k-1}}$. The simplest example is a frozen-time Langevin corrector:

$$X^{(\ell+1)} = X^{(\ell)} + \eta_k\, s(t_{k-1}, X^{(\ell)}) + \sqrt{2\eta_k}\,\xi_\ell, \qquad \xi_\ell \sim \mathcal{N}(0, I_d), \tag{170}$$

initialized at $X^{(0)} = X_{t_{k-1}}^{\text{prop}}$. If $p_t \equiv \pi$ were fixed in time, then (170) would be exactly the ULA update from Section 2.3.5. Here the target changes with k, so the same Langevin idea is used only as a short local repair step at the frozen time t_{k-1}. If density ratios were tractable, one could similarly add an accept/reject correction as in Section 2.3.6. Such a corrector costs extra work locally, but it can still be efficient globally if it permits a coarser grid, fewer reverse steps, or a noticeably smaller endpoint error at the same total NFE budget. This is the same proposal-versus-correction tradeoff as in Sections 2.3.5 and 2.3.6, now applied along the reverse bridge.

The second lever is to keep the same continuous-time reverse dynamics but change the proposal itself by using a better discretization. When $\varepsilon \equiv 0$, Proposition 6.7 gives the probability-flow ODE

$$\dot{x}_t = b(t, x_t), \tag{171}$$

whose marginals are again $\{p_t\}$. The most basic discretization is Euler:

$$x_{t_{k-1}} \approx x_{t_k} + (t_{k-1} - t_k)\, b(t_k, x_{t_k}).$$

A more accurate two-stage rule first predicts with Euler and then averages the old and predicted drifts:

$$x^{\text{prop}} = x_{t_k} + (t_{k-1} - t_k)\, b(t_k, x_{t_k}),$$

$$x_{t_{k-1}} \approx x_{t_k} + \frac{t_{k-1} - t_k}{2}\Big(b(t_k, x_{t_k}) + b(t_{k-1}, x^{\text{prop}})\Big).$$

Both updates aim at the same path $\{p_t\}$. They differ only in the tradeoff between computation and discretization bias. So the fast-sampling question is again a proposal-design question: how much endpoint accuracy do we get from each unit of computation? In diffusion language, proposal-plus-correction appears as predictor–corrector sampling (Song et al., 2021b, Alg. 1). Low-NFE deterministic samplers such as DDIM (Song et al., 2021a, §5) and higher-order ODE solvers such as DPM-Solver (Lu et al., 2022, §3 and §5.1) fit the same template: they redesign $\hat{K}_{t_{k-1}|t_k}$ so that the same endpoint task can be solved with fewer reverse evaluations.

6.3.2 Guidance as posterior tilting

The second task changes the target law. We may want a sample that matches external information y: a class label, a text description, or a measurement. Then the target is not p_0, but the conditional law $p_0(\cdot \mid y)$. An unconditional reverse sampler can be perfectly valid for p_0 and still be solving the wrong problem.

Guidance is the name used in the diffusion literature for this target change. In the notation of these notes, the unconditional path $\{p_t\}$ is replaced by the conditional path $\{p_t^y\}$, where

$$p_t^y(dx) := p_t(dx \mid y) = \frac{p_t(y \mid x)\, p_t(dx)}{\int p_t(y \mid x')\, p_t(dx')}.$$

This is exactly the conditioning-by-likelihood-reweighting identity from Example 2.5, now indexed by time t. Relative to importance sampling and SMC, the only structural difference is where that likelihood factor enters: there it appears through weights after propagation, whereas here it is inserted directly into the reverse-time field.

To express the conditional task in the same notation, let Y be an external variable and assume the joint law of (X_t, Y) has a density. Then Bayes's rule gives the conditional score.

Proposition 6.11 (Conditional scores split into prior and likelihood terms)**.** *For every fixed t and every y with $p_t(y \mid x) > 0$,*

$$\nabla_x \log p_t(x \mid y) = \nabla_x \log p_t(x) + \nabla_x \log p_t(y \mid x). \tag{172}$$

Proof. Bayes's rule gives

$$p_t(x \mid y) = \frac{p_t(x) p_t(y \mid x)}{p_t(y)}.$$

Taking $\nabla_x \log$ and using that $p_t(y)$ does not depend on x yields (172). ■

This identity gives the sampling rule immediately. To sample from the conditional path, one replaces the unconditional score $s(t,x)$ in (166) by

$$s_y(t,x) := \nabla_x \log p_t(x \mid y) = s(t,x) + \nabla_x \log p_t(y \mid x).$$

So the reverse-time sampler is unchanged except for an added likelihood-gradient term. In the language of these notes, guidance is posterior tilting along the path $\{p_t\}$: the prior score $s(t,x)$ describes what the diffusion model prefers on its own, while the likelihood term $\nabla_x \log p_t(y \mid x)$ pulls the sample toward states compatible with the side information y.

Sometimes one intentionally strengthens that tilt and uses

$$s_w(t,x;y) := s(t,x) + w\, \nabla_x \log p_t(y \mid x), \qquad w \geq 0. \tag{173}$$

When $w = 1$, this is the exact conditional score. When $w > 1$, it is a more concentrated target:

$$s_w(t,x;y) = \nabla_x \log\big(p_t(x) p_t(y \mid x)^w\big).$$

So $w > 1$ does not leave the conditional law unchanged; it produces a tempered version that puts more mass on states that fit y especially well. This is the same power-target idea as in Section 4.3, but now it is applied only to the likelihood factor rather than to the whole target law. When $w = 1$ we are solving the exact conditional sampling problem. When $w > 1$ we are intentionally biasing the target toward stronger

agreement with y, typically to trade some faithfulness to the exact conditional law for sharper or more recognizable samples.

If a model supplies both an unconditional estimate $s_\varnothing(t,x) \approx s(t,x)$ and a conditional estimate $s_y(t,x) \approx \nabla_x \log p_t(x \mid y)$, then the common interpolation

$$s_{\mathrm{cfg}}(t,x;y) := s_\varnothing(t,x) + w\big(s_y(t,x) - s_\varnothing(t,x)\big)$$

reduces, in the exact case, to the weighted tilt (173). This is the form often used in practice.

The diffusion literature uses different names depending on how the extra likelihood information is obtained. When the likelihood-gradient term is supplied by a separate noisy-time classifier, the method is called classifier guidance (Dhariwal and Nichol, 2021, §4). When the model itself is trained to produce both unconditional and conditional outputs and the two are interpolated as above, the method is called classifier-free guidance (Ho and Salimans, 2022, §2). In our notation both methods are instances of the same posterior-tilting identity; they differ only in how $\nabla_x \log p_t(y \mid x)$ is approximated. The same formula also covers inverse problems once y is interpreted as a measurement, so we do not need a separate sampling principle for that case.

6.3.3 A sample set as a distributional approximation

The third task changes the objective again. Sometimes the output of interest is not one correct sample but a set of N samples that approximates the target law well as a whole. Then the natural object is the empirical measure

$$\hat{\mu}_N := \frac{1}{N}\sum_{i=1}^{N} \delta_{x_0^i},$$

and the practical goal is that $\hat{\mu}_N$ approximate the target law $p_0(\cdot \mid y)$ well. Equivalently, for many test functions f, we want

$$\frac{1}{N}\sum_{i=1}^{N} f(x_0^i) \approx \mathbb{E}\big[f(X_0) \mid Y = y\big].$$

This is the empirical-measure viewpoint from the particle methods earlier in the course, especially Sections 4.4.3 and 4.4.4. In particular, Proposition 4.6 shows that once the objective is empirical approximation, exact independence is sufficient but not necessary: a coupled cloud can reduce empirical-average variance for important test functions and therefore outperform an i.i.d. cloud at the same particle budget.

To write the problem cleanly in the course notation, let

$$x_t^{1:N} := (x_t^1, \dots, x_t^N) \in (\mathbb{R}^d)^N.$$

If we insist on exact i.i.d. sampling, then the natural N-particle target is just the product law

$$(p_t^y)^{\otimes N}(dx_t^{1:N}) = \prod_{i=1}^{N} p_t(dx_t^i \mid y).$$

Independent sampling gives the correct one-particle marginals, but it does not by itself solve the finite-N approximation problem. At finite N, an i.i.d. batch can cluster and leave important regions underrepresented. So if the goal is to approximate the law well with a fixed particle budget, it can be worthwhile to replace the product target by a coupled law:

$$\Pi_t(dx_t^{1:N} \mid y) \propto (p_t^y)^{\otimes N}(dx_t^{1:N}) \exp\big(\lambda R_t(x_t^{1:N})\big). \tag{174}$$

Here $\lambda \geq 0$ controls the interaction strength, and R_t is chosen to reward spread, coverage, or repulsion. When $\lambda = 0$, (174) reduces to the i.i.d. product law. When $\lambda > 0$, the target is a coupled cloud chosen to improve the empirical approximation obtained from N particles.

Now the state variable is the whole configuration $X_t^{1:N}$, not a single particle. This is an augmented-state construction in exactly the sense of Section 4: the state is enlarged from one particle to a whole interacting configuration so that local computations can produce a better global approximation. Once the problem is written this way, the earlier tools apply directly: one can run Langevin or Metropolis updates on the joint configuration, or propagate and reweight a particle population as in Section 4.4. The interaction term is not a correction applied after sampling. It is part of the target itself.

This is the viewpoint taken explicitly in Particle Guidance, which introduces a time-dependent joint-particle potential in order to obtain non-i.i.d. diverse diffusion samples (Corso et al., 2023, §3). In the notation of these notes, that paper is an interacting-particle sampler on a product-space diffusion target. The phrase "non-i.i.d." there simply means that the joint law Π_t no longer factorizes into a product of one-particle marginals.

These three tasks explain much of the modern sampling literature around diffusion models. Predictor–corrector sampling, DDIM, and DPM-Solver redesign the proposal or local correction while keeping the same target. Classifier guidance and classifier-free guidance implement posterior tilting along the path. Particle Guidance replaces the product target by an interacting law on the particle cloud. The reverse bridge provides the path $\{p_t\}$; the sampling toolkit from the earlier units does the rest.

References

Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *Journal of Machine Learning Research*, 2025.

Sivaraman Balakrishnan, Martin J. Wainwright, and Bin Yu. Statistical guarantees for the EM algorithm: From population to sample-based analysis, 2014.

Michael Betancourt. A conceptual introduction to hamiltonian monte carlo, 2017.

David M. Blei, Alp Kucukelbir, and Jon D. McAuliffe. Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518):859–877, 2017. doi: 10.1080/01621459.2017.1285773.

Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, Cambridge, UK, 2004.

Christopher P. Burgess, Irina Higgins, Arka Pal, Loic Matthey, Nick Watters, Guillaume Desjardins, and Alexander Lerchner. Understanding disentangling in beta-vae. *arXiv preprint arXiv:1804.03599*, 2018.

Gabriele Corso, Yilun Xu, Valentin De Bortoli, Regina Barzilay, and Tommi Jaakkola. Particle guidance: non-I.I.D. diverse sampling with diffusion models, 2023.

Constantinos Daskalakis, Christos Tzamos, and Manolis Zampetakis. Ten steps of EM suffice for mixtures of two gaussians, 2016.

Prafulla Dhariwal and Alexander Nichol. Diffusion models beat GANs on image synthesis, 2021.

Alain Durmus, Éric Moulines, and Eero Saksman. On the convergence of hamiltonian monte carlo, 2019.

David J. Earl and Michael W. Deem. Parallel tempering: Theory, applications, and new perspectives, 2005.

Guillaume Garrigos and Robert M. Gower. Handbook of convergence theorems for (stochastic) gradient methods, 2024.

Stuart Geman and Donald Geman. Stochastic relaxation, gibbs distributions, and the bayesian restoration of images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-6(6):721–741, 1984.

Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, MD, 4 edition, 2013.

Junxian He, Daniel Spokoyny, Graham Neubig, and Taylor Berg-Kirkpatrick. Lagging inference networks and posterior collapse in variational autoencoders. In *International Conference on Learning Representations*, 2019.

Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance, 2022.

Matthew D. Hoffman and Andrew Gelman. The no-u-turn sampler: Adaptively setting path lengths in hamiltonian monte carlo. *Journal of Machine Learning Research*, 15(47):1593–1623, 2014.

Peter Holderrieth and Ezra Erives. An introduction to flow matching and diffusion models, 2026.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *International Conference on Learning Representations*, 2014.

Diederik P. Kingma, Tim Salimans, Ben Poole, and Jonathan Ho. Variational diffusion models. In *Advances in Neural Information Processing Systems*, 2021.

Chieh-Hsin Lai, Yang Song, Dongjun Kim, Yuki Mitsufuji, and Stefano Ermon. The principles of diffusion models, 2025.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling, 2023.

Jun S. Liu. *Monte Carlo Strategies in Scientific Computing*. Springer, New York, NY, 2002.

Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps, 2022.

Stephan Mandt, Matthew D. Hoffman, and David M. Blei. Continuous-time limit of stochastic gradient descent revisited. In *NeurIPS Workshop on Optimization for Machine Learning (OPT)*, 2015.

Radford M. Neal. Mcmc using hamiltonian dynamics. In Steve Brooks, Andrew Gelman, Galin L. Jones, and Xiao-Li Meng, editors, *Handbook of Markov Chain Monte Carlo*, pages 113–162. Chapman and Hall/CRC, 2011.

Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2 edition, 2006.

Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of adam and beyond. In *International Conference on Learning Representations (ICLR)*, 2018.

Danilo J. Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic backpropagation and approximate inference in deep generative models. In *Proceedings of the 31st International Conference on Machine Learning*, pages 1278–1286, 2014.

Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021a.

Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021b.

Aki Vehtari, Andrew Gelman, Daniel Simpson, Bob Carpenter, and Paul-Christian Bürkner. Rank-normalization, folding, and localization: An improved $\hat{R}$ for assessing convergence of MCMC, 2021.

Santosh S. Vempala and Andre Wibisono. Rapid convergence of the unadjusted langevin algorithm: Isoperimetry suffices, 2019.

Max Welling and Yee Whye Teh. Bayesian learning via stochastic gradient langevin dynamics. In *Proceedings of the 28th International Conference on Machine Learning (ICML)*, 2011.

Ji Xu, Daniel Hsu, and Arian Maleki. Global analysis of expectation maximization for mixtures of two Gaussians, 2016.